Major Spark source

Student career-path app. Miami Dade College.

92 files

eslint.config.mjs
package.json
startup.sh
tsconfig.json
vite.config.ts
scripts/app-env-plugin.mjs
scripts/brand-check.mjs
scripts/brand-check.test.mjs
scripts/browser-guard.mjs
scripts/browser-smoke-verdict.mjs
scripts/browser-smoke-verdict.test.mjs
scripts/browser-smoke.mjs
scripts/check-auth-invariant.mjs
scripts/check-auth-invariant.test.mjs
scripts/grok-pwa-plugin.mjs
scripts/grok-pwa-plugin.test.mjs
scripts/grok-pwa-shared.d.mts
scripts/grok-pwa-shared.mjs
scripts/install-page.html
scripts/migrate.mjs
scripts/migration-plan.mjs
scripts/migration-plan.test.mjs
scripts/preview-thumbnail.mjs
scripts/preview.mjs
scripts/preview.test.mjs
scripts/sign-out-plan.mjs
scripts/sign-out-plan.test.mjs
scripts/with-app-env.mjs
scripts/with-app-env.test.mjs
scripts/write-atomic.mjs
scripts/write-atomic.test.mjs
server/middleware/grok-pwa.ts
server/virtual-grok-og-identity.d.ts
src/components/auth-buttons.tsx
src/components/preview-host-bridge.tsx
src/lib/app-data/app-data.test.ts
src/lib/app-data/client.server.ts
src/lib/app-data/errors.ts
src/lib/app-data/index.ts
src/lib/app-data/login.ts
src/lib/app-data/readiness-schedule.test.ts
src/lib/app-data/readiness-schedule.ts
src/lib/app-data/readiness.ts
src/lib/app-data/server-only.ts
src/lib/app-data/types.ts
src/lib/app-data/use-connector-readiness.ts
src/lib/auth/client.ts
src/lib/auth/email-password.ts
src/lib/auth/gate-identity.server.ts
src/lib/auth/gate-identity.test.ts
src/lib/auth/gate-session-marker.ts
src/lib/auth/gate-session.server.ts
src/lib/auth/gates.tsx
src/lib/auth/isolation.server.ts
src/lib/auth/middleware.ts
src/lib/auth/pglite-dialect.ts
src/lib/auth/popup.server.ts
src/lib/auth/preview.ts
src/lib/auth/provider.tsx
src/lib/auth/providers.ts

```
src/lib/auth/server.ts
src/lib/auth/sign-in-gate.test.ts
src/lib/auth/sign-in-gate.ts
src/lib/auth/use-current-user.ts
src/lib/auth/verify.server.ts
src/lib/auth0-config.ts
src/lib/db.ts
src/lib/device.ts
src/lib/env.server.ts
src/lib/error-component.tsx
src/lib/multiplayer/index.ts
src/lib/multiplayer/p2p.ts
src/lib/og/site.json
src/lib/paths.ts
src/lib/plans.ts
src/lib/preview-embedder-origin.ts
src/lib/preview-host-bridge.ts
src/lib/preview-path-prefix.test.ts
src/lib/preview-path-prefix.ts
src/lib/progress.ts
src/lib/school-email.ts
src/routeTree.gen.ts
src/router.tsx
src/routes/__root.tsx
src/routes/api.auth.ts
src/routes/api/speak.ts
src/routes/app.tsx
src/routes/auth/$.ts
src/routes/index.tsx
src/routes/login.tsx
src/start.ts
src/styles.css
```

eslint.config.mjs

```
import js from "@eslint/js";
import prettier from "eslint-config-prettier";
import reactHooks from "eslint-plugin-react-hooks";
import reactRefresh from "eslint-plugin-react-refresh";
import globals from "globals";
import tseslint from "typescript-eslint";

/** Flat ESLint config for the TanStack Start app-builder template. */
export default tseslint.config(
  {
    ignores: [
      "dist/**",
      ".output/**",
      ".vercel/**",
      ".nitro/**",
      "node_modules/**",
      "src/routeTree.gen.ts",
    ],
  },
  js.configs.recommended,
  ...tseslint.configs.recommended,
  {
    files: ["**/*.{ts,tsx,js,jsx,mjs,cjs}"],
    languageOptions: {
      ecmaVersion: 2022,
      globals: { ...globals.browser, ...globals.node },
    },
    plugins: {
      "react-hooks": reactHooks,
      "react-refresh": reactRefresh,
    },
    rules: {
      ...reactHooks.configs.recommended.rules,
      "react-refresh/only-export-components": [
        "warn",
        { allowConstantExport: true },
      ],
      "@typescript-eslint/no-unused-vars": [
        "warn",
        { argsIgnorePattern: "^_", varsIgnorePattern: "^_" },
      ],
      "@typescript-eslint/no-explicit-any": "off",
    },
  },
  // Disable rules that conflict with Prettier formatting.
  prettier,
);
```

package.json

```
{
  "name": "major-spark-student-portal",
  "private": true,
  "sideEffects": false,
  "type": "module",
  "overrides": {
    "nf3": "0.3.17"
  },
  "scripts": {
    "dev": "node scripts/with-app-env.mjs vite dev --host 0.0.0.0 --port 8080",
    "build": "rm -rf node_modules/.vite .vercel/output && node scripts/with-app-env.mjs vite build && npm ru
n db:migrate",
    "db:migrate": "node scripts/migrate.mjs",
    "build:dev": "node scripts/with-app-env.mjs vite build --mode development",
    "preview": "node scripts/with-app-env.mjs vite preview",
    "preview:restart": "node scripts/preview.mjs restart",
    "preview:stop": "node scripts/preview.mjs stop",
    "typecheck": "tsc --noEmit",
    "check:auth": "node scripts/check-auth-invariant.mjs",
    "test": "node --test 'scripts/**/*.test.mjs' && node --experimental-strip-types --test src/lib/app-data/
app-data.test.ts src/lib/app-data/readiness-schedule.test.ts src/lib/auth/gate-identity.test.ts src/lib/auth
/sign-in-gate.test.ts src/lib/preview-path-prefix.test.ts",
    "lint": "eslint .",
    "format": "prettier --write ."
  },
  "dependencies": {
    "@auth0/auth0-tanstack-start-react": "^1.0.0-beta.0",
    "@electric-sql/pglite": "^0.5.4",
    "@hookform/resolvers": "^5.7.0",
    "@radix-ui/react-accordion": "^1.2.12",
    "@radix-ui/react-alert-dialog": "^1.1.15",
    "@radix-ui/react-avatar": "^1.1.11",
    "@radix-ui/react-checkbox": "^1.3.3",
    "@radix-ui/react-collapsible": "^1.1.12",
    "@radix-ui/react-dialog": "^1.1.15",
    "@radix-ui/react-dropdown-menu": "^2.1.16",
    "@radix-ui/react-label": "^2.1.8",
    "@radix-ui/react-popover": "^1.1.15",
    "@radix-ui/react-progress": "^1.1.8",
    "@radix-ui/react-radio-group": "^1.3.8",
    "@radix-ui/react-scroll-area": "^1.2.10",
    "@radix-ui/react-select": "^2.2.6",
    "@radix-ui/react-separator": "^1.1.8",
    "@radix-ui/react-slider": "^1.3.6",
    "@radix-ui/react-slot": "^1.2.4",
    "@radix-ui/react-switch": "^1.2.6",
    "@radix-ui/react-tabs": "^1.1.13",
    "@radix-ui/react-toggle": "^1.1.10",
    "@radix-ui/react-toggle-group": "^1.1.11",
    "@radix-ui/react-tooltip": "^1.2.8",
    "@tailwindcss/vite": "^4.3.0",
    "@tanstack/react-query": "^5.101.0",
    "@tanstack/react-router": "^1.170.1",
    "@tanstack/react-start": "^1.168.61",
    "@tanstack/react-table": "^8.21.0",
    "@tanstack/router-plugin": "^1.168.1",
    "@vitejs/plugin-react": "^5.2.0",
    "better-auth": "~1.6.30",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.1.1",
    "date-fns": "^4.0.0",
    "jose": "6.2.9",
    "kysely": "^0.28.5",
    "lightningcss": "^1.28.0",
    "lucide-react": "^0.510.0",
    "nitro": "3.0.260610-beta",
    "pg": "^8.16.3",
    "react": "^19.2.0",
    "react-day-picker": "^9.14.0",
    "react-dom": "^19.2.0",
    "react-hook-form": "^7.54.0",
    "react-resizable-panels": "^4.6.5",
    "recharts": "^2.13.0",
    "sonner": "^2.0.7",
    "tailwind-merge": "^3.5.0",
    "tailwindcss": "^4.3.0",
    "tw-animate-css": "^1.3.4",
    "typescript": "^5.7.0",
    "vaul": "^1.1.2",
    "vite": "^8.2.0",
    "zod": "^4.4.0",
    "zustand": "^5.0.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.20.0",
    "@types/node": "^22.16.5",
    "@types/pg": "^8.11.10",
```

```
    "@types/react": "^19.2.0",
    "@types/react-dom": "^19.2.0",
    "eslint": "^9.20.0",
    "eslint-config-prettier": "^10.1.1",
    "eslint-plugin-prettier": "^5.2.6",
    "eslint-plugin-react-hooks": "^5.2.0",
    "eslint-plugin-react-refresh": "^0.4.20",
    "globals": "^15.15.0",
    "playwright": "^1.62.0",
    "prettier": "^3.4.0",
    "typescript-eslint": "^8.56.1"
  }
}
```

startup.sh

```
#!/bin/sh
set -eu
cd /workspace
node scripts/preview.mjs stop || true
if curl -sf -o /dev/null --max-time 2 http://127.0.0.1:8080/; then
  exit 0
fi
npm run dev >>/tmp/app-startup.log 2>&1 &
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": [
      "ES2022",
      "DOM",
      "DOM.Iterable"
    ],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "isolatedModules": true,
    "skipLibCheck": true,
    // `src/lib/db.ts` imports scripts/migration-plan.mjs; checkJs types the
    // implementation itself instead of a hand-written declaration beside it.
    "allowJs": true,
    "checkJs": true,
    "noEmit": true,
    "types": [
      "vite/client",
      "node"
    ],
    "baseUrl": ".",
    "paths": {
      "@/*": [
        "./src/*"
      ]
    },
    "allowImportingTsExtensions": true
  },
  "include": [
    "src",
    "server"
  ]
}
```

vite.config.ts

```
import { readdirSync } from "node:fs";
import { join } from "node:path";
import type { Plugin } from "vite";
import { defineConfig } from "vite";
import { tanstackStart } from "@tanstack/react-start/plugin/vite";
import viteReact from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";
import { nitro } from "nitro/vite";
// @ts-expect-error JS plugin alongside the TS vite config
import { grokPwaPlugin } from "./scripts/grok-pwa-plugin.mjs";
// @ts-expect-error JS plugin alongside the TS vite config
import { appEnvPlugin } from "./scripts/app-env-plugin.mjs";
import { isMigrationFile } from "./scripts/migration-plan.mjs";

/** The files `src/lib/db.ts` globs — same directory, same non-recursive scope. */
function hasGlobbedMigrations(root: string): boolean {
  try {
    return readdirSync(join(root, "migrations")).some(isMigrationFile);
  } catch {
    return false;
  }
}

/**
 * Finish PGLite bootstrap during dev-server setup (before traffic). Vite awaits
 * async `configureServer` hooks. Production: `src/lib/db` kicks `ensureDbReady`
 * on import.
 *
 * Vite awaiting the hook puts this on time-to-first-render, so an app with no
 * migrations — no schema to apply — skips it entirely rather than paying for a
 * PGLite instance it never queries.
 */
function pgliteBootstrapPlugin(): Plugin {
  return {
    name: "app-builder:pglite-bootstrap",
    apply: "serve",
    async configureServer(server) {
      if (!hasGlobbedMigrations(server.config.root)) return;
      try {
        const mod = (await server.ssrLoadModule("/src/lib/db.ts")) as {
          ensureDbReady?: () => Promise<void>;
        };
        if (typeof mod.ensureDbReady === "function") {
          await mod.ensureDbReady();
        }
      } catch (err) {
        console.error("[app-builder] DB bootstrap failed:", err);
        throw err;
      }
    },
  };
}

/**
 * Live-preview OAuth popup — handled HERE so the agent never has to create a
 * `/auth/popup` route (and cannot break it by scaffolding a React page that
 * paints the full app shell in the popup).
 *
 * `signIn` (client.ts) opens `/auth/popup?providerId=…` in a top-level window.
 * This middleware runs before TanStack Start, calls `handleAuthPopupRequest`,
 * and returns the 302 / completion HTML. Deployed apps do not use the popup
 * (full-page OAuth redirect), so `apply: "serve"` is enough.
 */
function authPopupPlugin(): Plugin {
  return {
    name: "app-builder:auth-popup",
    apply: "serve",
    configureServer(server) {
      // Register immediately (not in a returned post-hook) so we run BEFORE
      // TanStack Start / the SPA HTML fallback. A model-authored
      // `src/routes/auth/popup.tsx` React page must never win this path.
      server.middlewares.use(async (req, res, next) => {
        try {
          const rawUrl = req.url ?? "";
          const pathOnly = rawUrl.split("?", 1)[0] ?? "";
          if (pathOnly !== "/auth/popup") {
            next();
            return;
          }
          if ((req.method ?? "GET").toUpperCase() !== "GET") {
            res.statusCode = 405;
            res.setHeader("content-type", "text/plain; charset=utf-8");
            res.end("Method Not Allowed");
            return;
          }

          const host = String(
            req.headers["x-forwarded-host"] ?? req.headers.host ?? "localhost:8080",
```

```
          );
          const proto = String(
            req.headers["x-forwarded-proto"] ??
              ((req.socket as { encrypted?: boolean } | undefined)?.encrypted ? "https" : "http"),
          );
          const requestHeaders = new Headers();
          for (const [key, value] of Object.entries(req.headers)) {
            if (value === undefined) continue;
            if (Array.isArray(value)) {
              for (const v of value) requestHeaders.append(key, v);
            } else {
              requestHeaders.set(key, value);
            }
          }
          // Ensure Host is the public preview host so Better Auth's dynamic
          // baseURL / redirect_uri match the popup origin.
          if (!requestHeaders.has("host")) requestHeaders.set("host", host);

          const request = new Request(`${proto}://${host}${rawUrl}`, {
            method: "GET",
            headers: requestHeaders,
          });

          const mod = (await server.ssrLoadModule("/src/lib/auth/popup.server.ts")) as {
            handleAuthPopupRequest: (req: Request) => Promise<Response>;
          };
          const response = await mod.handleAuthPopupRequest(request);

          res.statusCode = response.status;
          // Preserve multiple Set-Cookie headers (OAuth state + session).
          const setCookies =
            typeof response.headers.getSetCookie === "function"
              ? response.headers.getSetCookie()
              : [];
          response.headers.forEach((value, key) => {
            if (key.toLowerCase() === "set-cookie") return;
            res.setHeader(key, value);
          });
          for (const cookie of setCookies) {
            res.appendHeader("set-cookie", cookie);
          }
          const body = Buffer.from(await response.arrayBuffer());
          res.end(body);
        } catch (err) {
          console.error("[app-builder] /auth/popup handler failed:", err);
          if (!res.headersSent) {
            res.statusCode = 500;
            res.setHeader("content-type", "text/plain; charset=utf-8");
            res.end("auth popup failed");
          }
        }
      });
    },
  };
}

// `0.0.0.0:8080` is the live-preview contract — don't change host/port.
// The dev server starts once `src/router.tsx` and `src/routes/` exist — see
// AGENTS.md § "First scaffold".
export default defineConfig(({ command, isPreview }) => ({
  server: {
    host: "0.0.0.0",
    port: 8080,
    strictPort: true,
  },
  preview: {
    host: "127.0.0.1",
    port: 8081,
    strictPort: true,
  },
  resolve: { tsconfigPaths: true },
  plugins: [
    pgliteBootstrapPlugin(),
    // Before tanstackStart so /auth/popup never falls through to the SPA.
    authPopupPlugin(),
    // Dev-only /__app-env, read by scripts/check-auth-invariant.mjs.
    appEnvPlugin(),
    // PWA head + ?install=1 tutorial page; runs before Start/Nitro.
    grokPwaPlugin(),
    tailwindcss(),
    tanstackStart(),
    ...(command === "build" || isPreview
      ? [
          nitro({
            preset: "vercel",
            // Node serverless adapter so /auth/login stays on the Vercel function.
            vercel: {
              entryFormat: "node",
            },
            // Auto-registers server/middleware/* (the PWA install page +
```

```
            // manifest + head-tag middleware). Nitro v3 defaults serverDir to
            // false, so removing this silently unwires /?install=1 on deploys.
            serverDir: "./server",
          }),
        ]
      : []),
    viteReact(),
  ],
}));
```

scripts/app-env-plugin.mjs

```
/**
 * Dev-only `/__app-env` endpoint: the client env the running Vite server
 * resolved, as JSON.
 *
 * `scripts/check-auth-invariant.mjs` reads it to compare the live dev server's
 * `VITE_AUTH_ENABLED` against the value the next build will resolve. The
 * config is loaded even when Vite is started outside `scripts/with-app-env.mjs`
 * — the case that check exists to catch. These are the same values Vite inlines
 * into the client bundle, and `apply: "serve"` keeps the route out of deployed
 * apps.
 */
export const APP_ENV_ROUTE = "/__app-env";

export function appEnvPlugin() {
  return {
    name: "app-builder:app-env",
    apply: "serve",
    configureServer(server) {
      server.middlewares.use((req, res, next) => {
        const pathOnly = (req.url ?? "").split("?", 1)[0];
        if (pathOnly !== APP_ENV_ROUTE || (req.method ?? "GET").toUpperCase() !== "GET") {
          next();
          return;
        }
        const body = Buffer.from(JSON.stringify(server.config.env), "utf8");
        res.statusCode = 200;
        res.setHeader("content-type", "application/json; charset=utf-8");
        res.setHeader("cache-control", "no-cache");
        res.setHeader("content-length", String(body.byteLength));
        res.end(body);
      });
    },
  };
}
```

scripts/brand-check.mjs

```
#!/usr/bin/env node
/**
 * Brand-asset gate shared by browser-smoke.mjs (and unit-testable without a
 * browser): a canvas app is almost always a game / visually rich app, and
 * those must ship a custom share card — the default og.grok.me placeholder is
 * not acceptable for them (see .grok/skills/og/SKILL.md).
 *
 * Games must also set type=x:game in src/lib/og/site.json so the platform
 * injector emits og:type for X game-card unfurls, and public/x-banner.jpg for
 * the 50:11 X feed card. A card file is enough for bake to emit /og.jpg, but
 * brand-check still requires site.json `"card": "custom"` so the agent-facing
 * contract stays explicit.
 *
 * Checked on the filesystem (not the served head) so preview and mid-scaffold
 * workspaces are judged the same way.
 *
 * Also runnable, so the background brand task can check its own work before it
 * reports (the parent answers without waiting for it):
 *
 *   node scripts/brand-check.mjs [--game] [--placeholder-ok] [--root <dir>]
 *
 * That run judges the files on disk even though the task is holding the
 * og-pending marker; the marker only silences what the parent's gates see. It
 * also requires the custom card: its caller is normally the pass that exists to
 * produce one, so the placeholder the parent tolerates for a plain utility is a
 * failed pass here. --placeholder-ok is for the other launch — a pass doing
 * favicon, PWA icons and title for a plain utility that keeps the og.grok.me
 * card — where no card is the expected verdict rather than a failure.
 */
import { existsSync, statSync } from "node:fs";
import { dirname, join } from "node:path";
import { fileURLToPath, pathToFileURL } from "node:url";
import { OG_SITE_REL_PATH, readOgSite, siteHasCustomCard } from "./grok-pwa-shared.mjs";

// Over this, link scrapers (X card previews included) time out or skip the
// image, so the card silently fails to unfurl. The og skill's JPEG contract
// (ffmpeg -q:v 4, ~150-300 KB) exists precisely to stay under it.
export const MAX_CARD_BYTES = 600 * 1024;

// Written by the brand task while it generates, removed when it finishes.
export const OG_PENDING_REL_PATH = ".grok/og-pending";
// A Stop mid-generation leaves the marker behind, so the demotion expires
// instead of hiding a missing card forever on that workspace.
export const OG_PENDING_MAX_AGE_MS = 10 * 60 * 1000;

export function siteDeclaresOgTypeGame(site) {
  return String(site?.type ?? "").toLowerCase() === "x:game";
}

export function ogPendingActive(workspaceRoot, now = Date.now()) {
  try {
    const { mtimeMs } = statSync(join(workspaceRoot, OG_PENDING_REL_PATH));
    return now - mtimeMs < OG_PENDING_MAX_AGE_MS;
  } catch {
    return false;
  }
}

/**
 * What the parent's gates see. The brand pass may still be generating when a
 * gate runs, and "missing" would send the agent to redo the task's work.
 *
 * In flight is silence, not a note: callers report this array as warnings, so
 * anything left in it — however it is worded — reaches the agent as one.
 */
export function computeBrandWarnings({
  hasCanvas,
  workspaceRoot = "/workspace",
  now = Date.now(),
}) {
  if (ogPendingActive(workspaceRoot, now)) return [];
  return brandWarningsOnDisk({ hasCanvas, workspaceRoot });
}

/**
 * The files on disk, marker ignored — the brand pass's own view of its work.
 * Deliberately not exported: a gate that reached for this instead of
 * `computeBrandWarnings` would warn about a card that is still generating.
 * `cardRequired` is for a caller that is itself the brand pass: for it the
 * placeholder is a failure, not the plain-utility default the parent's gate
 * leaves alone.
 */
function brandWarningsOnDisk({
  hasCanvas,
  workspaceRoot = "/workspace",
  cardRequired = false,
}) {
  const skillPath = join(workspaceRoot, ".grok/skills/og/SKILL.md");
```

```
  const sitePath = join(workspaceRoot, OG_SITE_REL_PATH);
  const site = readOgSite(workspaceRoot);
  const cardPath = [
    join(workspaceRoot, "public/og.jpg"),
    join(workspaceRoot, "public/og.png"),
  ].find(existsSync);
  const warnings = [];

  if (cardPath !== undefined) {
    if (statSync(cardPath).size > MAX_CARD_BYTES) {
      warnings.push(
        `BRAND WARNING: ${cardPath} is over 600 KB — link scrapers (X card previews included) `
          + "time out or skip images this heavy, so the card silently fails to unfurl. "
          + `Re-encode as JPEG (public/og.jpg, ffmpeg -q:v 4) per ${skillPath}.`,
      );
    }
    if (!siteHasCustomCard(site)) {
      warnings.push(
        `BRAND WARNING: ${cardPath} exists but ${sitePath} is missing "card": "custom". `
          + "Bake infers custom from the file, but set the flag in "
          + `${sitePath} per ${skillPath} so identity is explicit.`,
      );
    }
  } else if (hasCanvas) {
    warnings.push(
      `BRAND WARNING: this looks like a game/canvas app but ${workspaceRoot}/public/og.jpg `
        + "is missing. Games and visually rich apps must ship a custom 1200x630 share card "
        + "built from the app's own art — the default og.grok.me placeholder card is not "
        + `acceptable for them. You are not done: open ${skillPath} and finish the `
        + "brand-asset pass.",
    );
  } else if (cardRequired) {
    warnings.push(
      `BRAND WARNING: ${workspaceRoot}/public/og.jpg is missing and this pass exists to `
        + "produce it. Generate the 1200x630 card from the app's own art and hand it over "
        + `per ${skillPath}. If no image-generation tool is available in this session, `
        + "report that instead of reporting a pass — the app keeps the og.grok.me "
        + "placeholder.",
    );
  } else {
    warnings.push(
      "BRAND NOTE: no custom public/og.jpg — the platform will serve the og.grok.me placeholder. "
        + "Custom cards are the default for games of every kind (DOM board/word games included), "
        + "whimsical apps, creative tools, and brand-forward pages — only plain utilities "
        + "(converters, CRUD trackers, admin dashboards) keep the placeholder. If this app "
        + `is not a plain utility, finish the brand-asset pass per ${skillPath}.`,
    );
  }

  if (hasCanvas && !siteDeclaresOgTypeGame(site)) {
    warnings.push(
      'BRAND WARNING: this looks like a game/canvas app but src/lib/og/site.json is missing '
        + '"type": "x:game". X uses og:type=x:game to present the unfurl as a game card — set '
        + `it in ${sitePath} per ${skillPath}. Do not invent `
        + "x:type or overload twitter:card as the game signal.",
    );
  }

  // Games with a custom link card must also ship the 50:11 X feed card.
  // Skip while still on the og.grok.me placeholder — that pass has not started yet.
  if (hasCanvas && cardPath !== undefined) {
    const bannerPath = join(workspaceRoot, "public/x-banner.jpg");
    if (!existsSync(bannerPath)) {
      warnings.push(
        `BRAND WARNING: this looks like a game/canvas app but ${bannerPath} is missing. `
          + "Games need a 50:11 X feed card (1200×264 JPEG) at public/x-banner.jpg — "
          + `open ${skillPath} and finish the brand-asset pass.`,
      );
    } else if (statSync(bannerPath).size > MAX_CARD_BYTES) {
      warnings.push(
        `BRAND WARNING: ${bannerPath} is over 600 KB — link scrapers (X card previews `
          + "included) time out or skip images this heavy, so the feed card silently fails "
          + `to unfurl. Re-encode as JPEG (ffmpeg -q:v 4) per ${skillPath}.`,
      );
    }
  }

  return warnings;
}

export function parseBrandCheckArgs(argv) {
  const usage =
    "usage: node scripts/brand-check.mjs [--game] [--placeholder-ok] [--root <dir>]";
  let game = false;
  let placeholderOk = false;
  let root = null;
  for (let i = 0; i < argv.length; i += 1) {
    // --game runs the checks browser-smoke.mjs derives from a real <canvas>;
    // DOM board/word games need them too and have no canvas to detect.
    if (argv[i] === "--game") {
```

```
      game = true;
    } else if (argv[i] === "--placeholder-ok") {
      placeholderOk = true;
    } else if (argv[i] === "--root") {
      root = argv[++i];
      if (root === undefined) return { error: `--root needs a directory — ${usage}` };
    } else {
      return { error: `unexpected argument: ${argv[i]} — ${usage}` };
    }
  }
  return { game, placeholderOk, root };
}

function isBrandWarning(message) {
  return message.startsWith("BRAND WARNING:");
}

if (process.argv[1] && import.meta.url === pathToFileURL(process.argv[1]).href) {
  const args = parseBrandCheckArgs(process.argv.slice(2));
  if (args.error) {
    console.error(JSON.stringify({ ok: false, error: args.error }, null, 2));
    process.exit(1);
  }
  const workspaceRoot = args.root ?? join(dirname(fileURLToPath(import.meta.url)), "..");
  // The caller here is the brand task checking its own work: it is holding the
  // marker (report it, but judge the assets on disk) and it owes a custom card
  // whether or not the app draws to a canvas — unless it was launched for a
  // plain utility that keeps the placeholder, which only it knows.
  const messages = brandWarningsOnDisk({
    hasCanvas: args.game,
    workspaceRoot,
    cardRequired: !args.placeholderOk,
  });
  const warnings = messages.filter(isBrandWarning);
  console.log(
    JSON.stringify(
      {
        ok: warnings.length === 0,
        workspaceRoot,
        pending: ogPendingActive(workspaceRoot),
        warnings: warnings.length,
        messages,
      },
      null,
      2,
    ),
  );
  process.exitCode = warnings.length === 0 ? 0 : 1;
}
```

scripts/brand-check.test.mjs

```
import assert from "node:assert/strict";
import { spawnSync } from "node:child_process";
import { mkdtempSync, mkdirSync, readFileSync, utimesSync, writeFileSync } from "node:fs";
import { tmpdir } from "node:os";
import { dirname, join } from "node:path";
import { fileURLToPath } from "node:url";
import { test } from "node:test";
import {
  MAX_CARD_BYTES,
  OG_PENDING_MAX_AGE_MS,
  OG_PENDING_REL_PATH,
  computeBrandWarnings,
  ogPendingActive,
  parseBrandCheckArgs,
  siteDeclaresOgTypeGame,
} from "./brand-check.mjs";

const TEMPLATE_ROOT = join(dirname(fileURLToPath(import.meta.url)), "..");
const SCRIPT = join(TEMPLATE_ROOT, "scripts/brand-check.mjs");

const GAME_SITE = JSON.stringify({ title: "Wild Race", type: "x:game", card: "custom" });
const UTILITY_SITE = JSON.stringify({ title: "Invoice" });
const UTILITY_CUSTOM_SITE = JSON.stringify({ title: "Invoice", card: "custom" });

function makeWorkspace({
  siteJson,
  cardFile,
  narrowFile,
  cardBytes = 200 * 1024,
  narrowBytes = 200 * 1024,
  pendingAgeMs,
} = {}) {
  const root = mkdtempSync(join(tmpdir(), "brand-check-"));
  mkdirSync(join(root, "public"), { recursive: true });
  mkdirSync(join(root, "src/lib/og"), { recursive: true });
  mkdirSync(join(root, ".grok"), { recursive: true });
  if (pendingAgeMs !== undefined) {
    const marker = join(root, OG_PENDING_REL_PATH);
    writeFileSync(marker, "");
    const when = new Date(Date.now() - pendingAgeMs);
    utimesSync(marker, when, when);
  }
  if (siteJson !== undefined) {
    writeFileSync(join(root, "src/lib/og/site.json"), siteJson);
  }
  if (cardFile !== undefined) {
    writeFileSync(join(root, "public", cardFile), Buffer.alloc(cardBytes, 7));
  }
  if (narrowFile !== undefined) {
    writeFileSync(join(root, "public", narrowFile), Buffer.alloc(narrowBytes, 7));
  }
  return root;
}

test("non-canvas app with no custom card gets a soft BRAND NOTE (utility exception)", () => {
  const root = makeWorkspace({ siteJson: UTILITY_SITE });
  const warnings = computeBrandWarnings({ hasCanvas: false, workspaceRoot: root });
  assert.equal(warnings.length, 1);
  assert.match(warnings[0], /^BRAND NOTE:/);
  assert.match(warnings[0], /plain utilit/);
  assert.doesNotMatch(warnings[0], /^BRAND WARNING:/);
});

test("non-canvas app with a compliant card is silent", () => {
  const root = makeWorkspace({ siteJson: UTILITY_CUSTOM_SITE, cardFile: "og.jpg" });
  assert.deepEqual(computeBrandWarnings({ hasCanvas: false, workspaceRoot: root }), []);
});

test("card file without site.json card=custom warns", () => {
  const root = makeWorkspace({ siteJson: UTILITY_SITE, cardFile: "og.jpg" });
  const warnings = computeBrandWarnings({ hasCanvas: false, workspaceRoot: root });
  assert.equal(warnings.length, 1);
  assert.match(warnings[0], /"card": "custom"/);
});

test("card file with missing site.json warns for card=custom", () => {
  const root = makeWorkspace({ cardFile: "og.jpg" });
  const warnings = computeBrandWarnings({ hasCanvas: false, workspaceRoot: root });
  assert.equal(warnings.length, 1);
  assert.match(warnings[0], /"card": "custom"/);
});

test("card file with invalid site.json warns for card=custom", () => {
  const root = makeWorkspace({ siteJson: "{not-json", cardFile: "og.jpg" });
  const warnings = computeBrandWarnings({ hasCanvas: false, workspaceRoot: root });
  assert.equal(warnings.length, 1);
  assert.match(warnings[0], /"card": "custom"/);
});
```

```
test("oversized card warns for non-canvas apps too", () => {
  const root = makeWorkspace({
    siteJson: UTILITY_CUSTOM_SITE,
    cardFile: "og.jpg",
    cardBytes: MAX_CARD_BYTES + 1,
  });
  const warnings = computeBrandWarnings({ hasCanvas: false, workspaceRoot: root });
  assert.equal(warnings.length, 1);
  assert.match(warnings[0], /over 600 KB/);
});

test("canvas app with no card warns 'missing' and missing og:type", () => {
  const root = makeWorkspace({ siteJson: UTILITY_SITE });
  const warnings = computeBrandWarnings({ hasCanvas: true, workspaceRoot: root });
  assert.equal(warnings.length, 2);
  assert.match(warnings[0], /og\.jpg.*is missing/s);
  assert.match(warnings[0], /not done/);
  assert.match(warnings[1], /site\.json/);
  assert.match(warnings[1], /og:type|x:game/);
});

test("canvas card without type still warns for og:type", () => {
  const root = makeWorkspace({ siteJson: UTILITY_CUSTOM_SITE, cardFile: "og.jpg" });
  const warnings = computeBrandWarnings({ hasCanvas: true, workspaceRoot: root });
  assert.match(warnings.join("\n"), /x:game/);
  assert.match(warnings.join("\n"), /x-banner\.jpg/);
});

test("oversized card warns on the scraper budget (jpg and legacy png)", () => {
  for (const cardFile of ["og.jpg", "og.png"]) {
    const root = makeWorkspace({
      siteJson: GAME_SITE,
      cardFile,
      cardBytes: MAX_CARD_BYTES + 1,
      narrowFile: "x-banner.jpg",
    });
    const warnings = computeBrandWarnings({ hasCanvas: true, workspaceRoot: root });
    assert.match(warnings[0], /over 600 KB/);
  }
});

test("compliant game (custom card + site.json type + x-banner) is silent", () => {
  const root = makeWorkspace({
    siteJson: GAME_SITE,
    cardFile: "og.jpg",
    narrowFile: "x-banner.jpg",
  });
  assert.deepEqual(computeBrandWarnings({ hasCanvas: true, workspaceRoot: root }), []);
});

test("__root.tsx og:type no longer satisfies the canvas gate", () => {
  const root = makeWorkspace({
    siteJson: UTILITY_CUSTOM_SITE,
    cardFile: "og.jpg",
    narrowFile: "x-banner.jpg",
  });
  mkdirSync(join(root, "src/routes"), { recursive: true });
  writeFileSync(
    join(root, "src/routes/__root.tsx"),
    '{ property: "og:type", content: "x:game" }',
  );
  const warnings = computeBrandWarnings({ hasCanvas: true, workspaceRoot: root });
  assert.equal(warnings.length, 1);
  assert.match(warnings[0], /x:game/);
});

test("oversized x-banner warns on the same scraper budget as og.jpg", () => {
  const root = makeWorkspace({
    siteJson: GAME_SITE,
    cardFile: "og.jpg",
    narrowFile: "x-banner.jpg",
    narrowBytes: MAX_CARD_BYTES + 1,
  });
  const warnings = computeBrandWarnings({ hasCanvas: true, workspaceRoot: root });
  assert.equal(warnings.length, 1);
  assert.match(warnings[0], /x-banner\.jpg is over 600 KB/);
});

test("legacy png + site.json type still needs the X feed card", () => {
  const root = makeWorkspace({
    siteJson: GAME_SITE,
    cardFile: "og.png",
  });
  const warnings = computeBrandWarnings({ hasCanvas: true, workspaceRoot: root });
  assert.match(warnings.join("\n"), /x-banner\.jpg/);
});

test("siteDeclaresOgTypeGame reads the site contract", () => {
  assert.equal(siteDeclaresOgTypeGame({ type: "x:game" }), true);
  assert.equal(siteDeclaresOgTypeGame({ type: "website" }), false);
```

```
  assert.equal(siteDeclaresOgTypeGame({}), false);
});

test("a fresh og-pending marker silences the whole gate view", () => {
  const root = makeWorkspace({ siteJson: UTILITY_SITE, pendingAgeMs: 1000 });
  // Every caller reports this array as a count, so an in-flight card has to
  // leave nothing behind — a note would print as "brand: 1 warning(s)".
  assert.deepEqual(computeBrandWarnings({ hasCanvas: true, workspaceRoot: root }), []);
  assert.deepEqual(computeBrandWarnings({ hasCanvas: false, workspaceRoot: root }), []);
});

test("a stale og-pending marker is ignored and the warnings return", () => {
  const root = makeWorkspace({
    siteJson: UTILITY_SITE,
    pendingAgeMs: OG_PENDING_MAX_AGE_MS + 1000,
  });
  assert.equal(ogPendingActive(root), false);
  const warnings = computeBrandWarnings({ hasCanvas: true, workspaceRoot: root });
  assert.equal(warnings.length, 2);
  assert.match(warnings[0], /not done/);
});

test("the marker goes stale on the clock, not on the file", () => {
  const root = makeWorkspace({ siteJson: UTILITY_SITE, pendingAgeMs: 1000 });
  const justInside = Date.now() + OG_PENDING_MAX_AGE_MS - 5000;
  const pastIt = Date.now() + OG_PENDING_MAX_AGE_MS + 5000;
  assert.deepEqual(
    computeBrandWarnings({ hasCanvas: true, workspaceRoot: root, now: justInside }),
    [],
  );
  assert.match(
    computeBrandWarnings({ hasCanvas: true, workspaceRoot: root, now: pastIt })[0],
    /^BRAND WARNING:/,
  );
});

test("no marker means nothing is in flight", () => {
  assert.equal(ogPendingActive(makeWorkspace()), false);
});

test("parseBrandCheckArgs takes --game, --placeholder-ok and --root, and rejects the rest", () => {
  assert.deepEqual(parseBrandCheckArgs([]), { game: false, placeholderOk: false, root: null });
  assert.deepEqual(parseBrandCheckArgs(["--game", "--root", "/tmp/w"]), {
    game: true,
    placeholderOk: false,
    root: "/tmp/w",
  });
  assert.equal(parseBrandCheckArgs(["--placeholder-ok"]).placeholderOk, true);
  assert.match(parseBrandCheckArgs(["--root"]).error, /--root needs a directory/);
  assert.match(parseBrandCheckArgs(["--canvas"]).error, /unexpected argument: --canvas/);
});

const runCheck = (root, extra = []) =>
  spawnSync(process.execPath, [SCRIPT, "--root", root, ...extra], { encoding: "utf8" });

test("cli: a game missing its card reports JSON and exits 1", () => {
  const root = makeWorkspace({ siteJson: UTILITY_SITE });
  const run = runCheck(root, ["--game"]);
  assert.equal(run.status, 1, run.stdout + run.stderr);
  const result = JSON.parse(run.stdout);
  assert.equal(result.ok, false);
  assert.equal(result.pending, false);
  assert.equal(result.warnings, 2);
  assert.equal(result.workspaceRoot, root);
});

test("cli: a compliant game passes", () => {
  const root = makeWorkspace({
    siteJson: GAME_SITE,
    cardFile: "og.jpg",
    narrowFile: "x-banner.jpg",
  });
  const run = runCheck(root, ["--game"]);
  assert.equal(run.status, 0, run.stdout + run.stderr);
  assert.deepEqual(JSON.parse(run.stdout).messages, []);
});

test("cli: the task's own marker is reported, not honoured", () => {
  const root = makeWorkspace({ siteJson: UTILITY_SITE, pendingAgeMs: 1000 });
  const run = runCheck(root, ["--game"]);
  assert.equal(run.status, 1, run.stdout + run.stderr);
  const result = JSON.parse(run.stdout);
  assert.equal(result.pending, true);
  assert.equal(result.warnings, 2);
});

test("cli: a bad argument fails with JSON on stderr", () => {
  const run = runCheck(makeWorkspace(), ["--nope"]);
  assert.equal(run.status, 1);
  assert.equal(JSON.parse(run.stderr).ok, false);
```

```
});

test("cli: a non-game with no card fails too — the pass exists to produce one", () => {
  const root = makeWorkspace({ siteJson: UTILITY_SITE });
  const run = runCheck(root);
  assert.equal(run.status, 1, run.stdout + run.stderr);
  const result = JSON.parse(run.stdout);
  assert.equal(result.ok, false);
  assert.equal(result.warnings, 1);
  assert.match(result.messages[0], /^BRAND WARNING: .*og\.jpg is missing and this pass exists/);
  // The parent's gate keeps tolerating the placeholder for a plain utility.
  assert.match(computeBrandWarnings({ hasCanvas: false, workspaceRoot: root })[0], /^BRAND NOTE:/);
});

test("cli: --placeholder-ok is how a plain-utility pass reports no card as expected", () => {
  const root = makeWorkspace({ siteJson: UTILITY_SITE });
  const run = runCheck(root, ["--placeholder-ok"]);
  assert.equal(run.status, 0, run.stdout + run.stderr);
  const result = JSON.parse(run.stdout);
  assert.equal(result.ok, true);
  assert.match(result.messages[0], /^BRAND NOTE:/);
  // A game is never a placeholder app, so the flag cannot excuse one.
  const game = runCheck(root, ["--game", "--placeholder-ok"]);
  assert.equal(game.status, 1, game.stdout + game.stderr);
  assert.match(JSON.parse(game.stdout).messages[0], /^BRAND WARNING: .*not done/s);
});

test("cli: a non-game with a compliant card passes", () => {
  const root = makeWorkspace({ siteJson: UTILITY_CUSTOM_SITE, cardFile: "og.jpg" });
  const run = runCheck(root);
  assert.equal(run.status, 0, run.stdout + run.stderr);
  assert.deepEqual(JSON.parse(run.stdout).messages, []);
});

// --- the prompts are the only enforcement here, so pin them to the code ---

const readDoc = (rel) => readFileSync(join(TEMPLATE_ROOT, rel), "utf8");

test("SKILL.md and AGENTS.md name the marker path and bound this script uses", () => {
  // Prose wraps, so the minute count may straddle a line break.
  const bound = new RegExp(`${OG_PENDING_MAX_AGE_MS / 60_000}\\s+minutes`);
  for (const rel of [".grok/skills/og/SKILL.md", "AGENTS.md"]) {
    const doc = readDoc(rel);
    assert.ok(doc.includes(`/workspace/${OG_PENDING_REL_PATH}`), `${rel}: marker path`);
    assert.ok(bound.test(doc), `${rel}: staleness bound`);
  }
});

// The two places that own "never wait on the brand task". Scanning the whole
// of AGENTS.md instead would make every unrelated `wait_tasks` mention a future
// feature adds to it this test's business.
const PROHIBITION_SECTIONS = [
  {
    rel: ".grok/skills/og/SKILL.md",
    label: '§ "Brand-asset pass"',
    from: "## Brand-asset pass:",
    until: /\n## /,
  },
  {
    rel: "AGENTS.md",
    label: "execution loop step 6",
    from: "6. **Brand-asset pass",
    until: /\n7\. /,
  },
];

function prohibitionSection({ rel, label, from, until }) {
  const doc = readDoc(rel);
  const start = doc.indexOf(from);
  assert.notEqual(start, -1, `${rel}: ${label} moved — no "${from.trim()}"`);
  const rest = doc.slice(start + from.length);
  const end = rest.search(until);
  // Markdown emphasis and prose wrapping both sit between the two words.
  return (from + (end === -1 ? rest : rest.slice(0, end))).replace(/[`*]/g, "").replace(/\s+/g, " ");
}

test("the sections that own the brand-task prohibition never affirm a wait", () => {
  // Pinned on the shape of the prohibition, not on a negation being somewhere
  // nearby: "So: wait_tasks before the final verify, but never get_task_output"
  // keeps a negation in the sentence while instructing exactly the wait.
  const connectors = /(?:\s|[/,;]|\band\b|\bor\b|\bwait_tasks\b|\bget_task_output\b)+$/i;
  const negation = /\b(?:no|never|not|don[’']t)$/i;
  for (const section of PROHIBITION_SECTIONS) {
    const where = `${section.rel} ${section.label}`;
    const prose = prohibitionSection(section);
    const mentions = [...prose.matchAll(/wait_tasks|get_task_output/g)];
    assert.ok(mentions.length >= 2, `${where}: the prohibition itself went missing`);
    for (const match of mentions) {
      const before = prose.slice(0, match.index).replace(connectors, "");
      const context = prose.slice(Math.max(0, match.index - 60), match.index + 20);
```

```
      assert.ok(negation.test(before), `${where}: not a prohibition: …${context}…`);
    }
  }
});

test("SKILL.md tells the pass to self-check with the flag this CLI accepts", () => {
  const skill = readDoc(".grok/skills/og/SKILL.md");
  const invocations = skill.match(/node scripts\/brand-check\.mjs[^\n`]*/g) ?? [];
  assert.ok(invocations.length > 0);
  for (const line of invocations) {
    const argv = line.replace("node scripts/brand-check.mjs", "").trim().split(/\s+/);
    assert.equal(parseBrandCheckArgs(argv.filter(Boolean)).error, undefined, line);
  }
});
```

scripts/browser-guard.mjs

```
/**
 * Target checks shared by the Playwright capture scripts.
 *
 * Both run Chromium with `--no-sandbox` as root and take their URL and output
 * path from argv, so unchecked they will render `file:///root/.grok/auth.json`
 * into a PNG the agent can read, and write it anywhere.
 */
import { resolve, sep } from "node:path";

const LOOPBACK_HOSTNAMES = new Set(["127.0.0.1", "localhost", "::1", "[::1]"]);

/** http/https loopback only, else exit 1. `BROWSER_ALLOW_EXTERNAL_HOST=1` opts out. */
export function checkedUrl(url) {
  let parsed;
  try {
    parsed = new URL(url);
  } catch {
    fail(`not a valid URL: ${url}`);
  }
  // Rules out file:, data:, chrome:, view-source:.
  if (parsed.protocol !== "http:" && parsed.protocol !== "https:") {
    fail(`only http/https URLs are allowed, got ${parsed.protocol} in ${url}`);
  }
  if (!LOOPBACK_HOSTNAMES.has(parsed.hostname) && process.env.BROWSER_ALLOW_EXTERNAL_HOST !== "1") {
    fail(
      `${parsed.hostname} is not a loopback host; these scripts screenshot the ` +
        `local dev server. Set BROWSER_ALLOW_EXTERNAL_HOST=1 to override.`,
    );
  }
  return url;
}

/** Absolute `target` if it is strictly inside `allowedDirs`, else exit 1. */
export function checkedOutputPath(target, allowedDirs, label = "screenshot") {
  // Resolve first so `..` cannot slip past the prefix check.
  const abs = resolve(target);
  const allowed = allowedDirs.some((dir) => abs.startsWith(dir.endsWith(sep) ? dir : dir + sep));
  if (!allowed) {
    fail(`${label} path must be under ${allowedDirs.join(" or ")}, got ${abs}`);
  }
  return abs;
}

function fail(message) {
  console.error(JSON.stringify({ ok: false, error: message }, null, 2));
  process.exit(1);
}
```

scripts/browser-smoke-verdict.mjs

```
import { createHash } from "node:crypto";
export function normalizeBodyText(text) {
  return String(text ?? "")
    .replace(/\s+/g, " ")
    .trim();
}

export function normalizedBodyTextHash(text) {
  return createHash("sha256").update(normalizeBodyText(text)).digest("hex");
}

const IDENTITY_PREFIX_LEN = 64;

export function bodyTextPrefix(text) {
  return normalizeBodyText(text).slice(0, IDENTITY_PREFIX_LEN);
}
export function parseSmokeArgs(argv, env = {}) {
  const positional = [];
  let baseline = env.BROWSER_SMOKE_BASELINE || "";
  for (let i = 0; i < argv.length; i++) {
    const arg = argv[i];
    if (arg === "--baseline") {
      const value = argv[++i];
      if (!value) return { error: "--baseline requires a path to a prior verdict JSON" };
      baseline = value;
    } else if (arg.startsWith("--baseline=")) {
      const value = arg.slice("--baseline=".length);
      if (!value) return { error: "--baseline requires a path to a prior verdict JSON" };
      baseline = value;
    } else if (arg.startsWith("--")) {
      return { error: `unknown flag: ${arg}` };
    } else {
      positional.push(arg);
    }
  }
  return {
    url: positional[0] || "http://127.0.0.1:8080/",
    outPng: positional[1] || "/workspace/screenshots/app-builder-preview.png",
    baseline,
  };
}
export function derivedPaths(outPng) {
  const base = outPng.replace(/\.png$/i, "");
  return { mobilePng: `${base}-mobile.png`, verdictJson: `${base}.json` };
}

const TRIVIAL_LEN_DELTA = 20;
const TRIVIAL_LEN_RATIO = 0.1;
const COLLAPSE_RATIO = 0.5;
export function compareToBaseline(current, baseline) {
  const entries = Object.entries(current?.viewports ?? {});
  if (entries.length === 0) {
    return {
      divergesFromBaseline: true,
      reasons: ["current verdict has no viewport data"],
    };
  }
  const reasons = [];
  const baseViewports = baseline?.viewports ?? {};
  for (const [name, cur] of entries) {
    const base = baseViewports[name];
    if (!base) {
      reasons.push(`${name}: no baseline data for this viewport`);
      continue;
    }
    if (cur.status !== base.status) {
      reasons.push(`${name}: HTTP status changed ${base.status} -> ${cur.status}`);
    }
    if (base.title !== undefined && cur.title !== undefined && cur.title !== base.title) {
      reasons.push(`${name}: title changed ("${base.title}" -> "${cur.title}")`);
    }
    if (base.hasCanvas && !cur.hasCanvas) {
      reasons.push(`${name}: canvas disappeared`);
    }
    if (cur.horizontalOverflow && !base.horizontalOverflow) {
      reasons.push(`${name}: horizontal overflow appeared`);
    }
    const baseErrs = (base.consoleErrors?.length ?? 0) + (base.pageErrors?.length ?? 0);
    const curErrs = (cur.consoleErrors?.length ?? 0) + (cur.pageErrors?.length ?? 0);
    if (curErrs > 0 && baseErrs === 0) {
      reasons.push(`${name}: console/page errors appeared (${curErrs})`);
    }
    const baseLen = base.bodyTextLen ?? 0;
    const curLen = cur.bodyTextLen ?? 0;
    if (baseLen > 0 && curLen < baseLen * COLLAPSE_RATIO) {
      reasons.push(`${name}: body text collapsed (${baseLen} -> ${curLen} chars)`);
    } else if (cur.bodyTextHash !== base.bodyTextHash) {
      if (Math.abs(curLen - baseLen) > Math.max(TRIVIAL_LEN_DELTA, baseLen * TRIVIAL_LEN_RATIO)) {
```

```
        reasons.push(`${name}: body text changed (${baseLen} -> ${curLen} chars, hash mismatch)`);
      } else if (
        base.bodyTextPrefix !== undefined &&
        cur.bodyTextPrefix !== undefined &&
        cur.bodyTextPrefix !== base.bodyTextPrefix
      ) {
        reasons.push(`${name}: body text replaced (similar length, page start changed)`);
      }
    }
  }
  return { divergesFromBaseline: reasons.length > 0, reasons };
}
export function baselineComparison(current, rawText) {
  let baseline;
  try {
    baseline = JSON.parse(rawText);
  } catch {
    return { divergesFromBaseline: true, reasons: ["baseline unreadable: invalid JSON"] };
  }
  if (
    baseline === null ||
    typeof baseline !== "object" ||
    Array.isArray(baseline) ||
    baseline.viewports === null ||
    typeof baseline.viewports !== "object" ||
    Array.isArray(baseline.viewports)
  ) {
    return { divergesFromBaseline: true, reasons: ["baseline unreadable: not a verdict object"] };
  }
  return compareToBaseline(current, baseline);
}
export function exitCodeFor(viewports) {
  const list = Object.values(viewports ?? {});
  if (list.length === 0) return 1;
  if (list.some((v) => (v.status ?? 0) >= 400 || (v.status ?? 0) === 0)) return 1;
  if (list.some((v) => (v.consoleErrors?.length ?? 0) > 0 || (v.pageErrors?.length ?? 0) > 0)) {
    return 2;
  }
  return 0;
}
```

scripts/browser-smoke-verdict.test.mjs

```
import assert from "node:assert/strict";
import { readFileSync } from "node:fs";
import { dirname, join } from "node:path";
import { fileURLToPath } from "node:url";
import test from "node:test";
import {
  baselineComparison,
  bodyTextPrefix,
  compareToBaseline,
  derivedPaths,
  exitCodeFor,
  normalizeBodyText,
  normalizedBodyTextHash,
  parseSmokeArgs,
} from "./browser-smoke-verdict.mjs";

const TEMPLATE_ROOT = join(dirname(fileURLToPath(import.meta.url)), "..");

function viewport(overrides = {}) {
  return {
    status: 200,
    title: "App",
    hasCanvas: false,
    horizontalOverflow: false,
    bodyTextLen: 500,
    bodyTextHash: "aaa",
    bodyTextPrefix: "hello world",
    consoleErrors: [],
    pageErrors: [],
    ...overrides,
  };
}

function verdict(desktop = {}, mobile = {}) {
  return { viewports: { desktop: viewport(desktop), mobile: viewport(mobile) } };
}

test("normalizes whitespace before hashing", () => {
  assert.equal(normalizeBodyText("  a \n\t b  "), "a b");
  assert.equal(normalizedBodyTextHash("a \n b"), normalizedBodyTextHash("  a b "));
  assert.notEqual(normalizedBodyTextHash("a b"), normalizedBodyTextHash("a c"));
});

test("nullish and blank body text normalize to the empty string", () => {
  assert.equal(normalizeBodyText(null), "");
  assert.equal(normalizeBodyText(undefined), "");
  assert.equal(normalizedBodyTextHash(""), normalizedBodyTextHash("  \n "));
});

test("body text prefix is normalized and capped", () => {
  assert.equal(bodyTextPrefix("  a \n b  "), "a b");
  assert.equal(bodyTextPrefix("x".repeat(200)).length, 64);
});

test("identical runs do not diverge", () => {
  const { divergesFromBaseline, reasons } = compareToBaseline(verdict(), verdict());
  assert.equal(divergesFromBaseline, false);
  assert.deepEqual(reasons, []);
});

test("status change diverges", () => {
  const { divergesFromBaseline, reasons } = compareToBaseline(verdict({ status: 500 }), verdict());
  assert.equal(divergesFromBaseline, true);
  assert.match(reasons.join(";"), /desktop: HTTP status changed 200 -> 500/);
});

test("title change diverges", () => {
  const { divergesFromBaseline, reasons } = compareToBaseline(
    verdict({ title: "Error" }),
    verdict(),
  );
  assert.equal(divergesFromBaseline, true);
  assert.match(reasons.join(";"), /desktop: title changed \("App" -> "Error"\)/);
});

test("title comparison is skipped when either side lacks one", () => {
  const { divergesFromBaseline } = compareToBaseline(
    verdict({ title: undefined }),
    verdict({ title: "App" }),
  );
  assert.equal(divergesFromBaseline, false);
});

test("canvas disappearing diverges; canvas appearing does not", () => {
  const gone = compareToBaseline(verdict({ hasCanvas: false }), verdict({ hasCanvas: true }));
  assert.equal(gone.divergesFromBaseline, true);
  assert.match(gone.reasons.join(";"), /desktop: canvas disappeared/);
```

```
  const appeared = compareToBaseline(verdict({ hasCanvas: true }), verdict({ hasCanvas: false }));
  assert.equal(appeared.divergesFromBaseline, false);
});

test("horizontal overflow appearing diverges; clearing does not", () => {
  const appeared = compareToBaseline(
    verdict({}, { horizontalOverflow: true }),
    verdict({}, { horizontalOverflow: false }),
  );
  assert.equal(appeared.divergesFromBaseline, true);
  assert.match(appeared.reasons.join(";"), /mobile: horizontal overflow appeared/);

  const cleared = compareToBaseline(
    verdict({}, { horizontalOverflow: false }),
    verdict({}, { horizontalOverflow: true }),
  );
  assert.equal(cleared.divergesFromBaseline, false);
});

test("new console errors diverge", () => {
  const { divergesFromBaseline, reasons } = compareToBaseline(
    verdict({}, { consoleErrors: ["boom"] }),
    verdict(),
  );
  assert.equal(divergesFromBaseline, true);
  assert.match(reasons.join(";"), /mobile: console\/page errors appeared/);
});

test("errors already present in the baseline do not re-diverge", () => {
  const { divergesFromBaseline } = compareToBaseline(
    verdict({ consoleErrors: ["boom"] }),
    verdict({ consoleErrors: ["boom"] }),
  );
  assert.equal(divergesFromBaseline, false);
});

test("body text collapse diverges", () => {
  const { divergesFromBaseline, reasons } = compareToBaseline(
    verdict({ bodyTextLen: 10, bodyTextHash: "bbb" }),
    verdict(),
  );
  assert.equal(divergesFromBaseline, true);
  assert.match(reasons.join(";"), /desktop: body text collapsed \(500 -> 10 chars\)/);
});

test("body text dropping to zero reports collapse", () => {
  const { reasons } = compareToBaseline(
    verdict({ bodyTextLen: 0, bodyTextHash: "bbb", bodyTextPrefix: "" }),
    verdict(),
  );
  assert.match(reasons.join(";"), /desktop: body text collapsed \(500 -> 0 chars\)/);
});

test("zero-length baseline reports growth via the hash branch, not collapse", () => {
  const { reasons } = compareToBaseline(
    verdict({ bodyTextLen: 500 }),
    verdict({ bodyTextLen: 0, bodyTextHash: "bbb", bodyTextPrefix: "" }),
  );
  assert.match(reasons.join(";"), /desktop: body text changed \(0 -> 500 chars/);
});

test("hash mismatch with a trivial length delta and same page start is tolerated", () => {
  const { divergesFromBaseline } = compareToBaseline(
    verdict({ bodyTextLen: 510, bodyTextHash: "bbb" }),
    verdict(),
  );
  assert.equal(divergesFromBaseline, false);
});

test("hash mismatch with a trivial length delta but changed page start diverges", () => {
  const { divergesFromBaseline, reasons } = compareToBaseline(
    verdict({ bodyTextLen: 510, bodyTextHash: "bbb", bodyTextPrefix: "500 Internal Error" }),
    verdict(),
  );
  assert.equal(divergesFromBaseline, true);
  assert.match(
    reasons.join(";"),
    /desktop: body text replaced \(similar length, page start changed\)/,
  );
});

test("prefix check is skipped when either side lacks a prefix", () => {
  const cases = [
    [{ bodyTextPrefix: undefined }, { bodyTextPrefix: undefined }],
    [{}, { bodyTextPrefix: undefined }],
    [{ bodyTextPrefix: undefined }, {}],
  ];
  for (const [curOverrides, baseOverrides] of cases) {
    const { divergesFromBaseline } = compareToBaseline(
      verdict({ bodyTextLen: 510, bodyTextHash: "bbb", ...curOverrides }),
```

```
      verdict(baseOverrides),
    );
    assert.equal(divergesFromBaseline, false);
  }
});

test("length-delta tolerance boundary: 50 tolerated, 51 diverges (500-char baseline)", () => {
  const at = compareToBaseline(verdict({ bodyTextLen: 550, bodyTextHash: "bbb" }), verdict());
  assert.equal(at.divergesFromBaseline, false);

  const over = compareToBaseline(verdict({ bodyTextLen: 551, bodyTextHash: "bbb" }), verdict());
  assert.equal(over.divergesFromBaseline, true);
  assert.match(over.reasons.join(";"), /desktop: body text changed/);
});

test("length-delta floor dominates small baselines: 20 tolerated, 21 diverges (100-char baseline)", () => {
  const base = verdict({ bodyTextLen: 100 }, { bodyTextLen: 100 });
  const at = compareToBaseline(
    verdict({ bodyTextLen: 120, bodyTextHash: "bbb" }, { bodyTextLen: 100 }),
    base,
  );
  assert.equal(at.divergesFromBaseline, false);

  const over = compareToBaseline(
    verdict({ bodyTextLen: 121, bodyTextHash: "bbb" }, { bodyTextLen: 100 }),
    base,
  );
  assert.equal(over.divergesFromBaseline, true);
});

test("collapse boundary: exactly half is 'changed', one below is 'collapsed'", () => {
  const half = compareToBaseline(verdict({ bodyTextLen: 250, bodyTextHash: "bbb" }), verdict());
  assert.match(half.reasons.join(";"), /desktop: body text changed \(500 -> 250 chars/);

  const below = compareToBaseline(verdict({ bodyTextLen: 249, bodyTextHash: "bbb" }), verdict());
  assert.match(below.reasons.join(";"), /desktop: body text collapsed \(500 -> 249 chars\)/);
});

test("missing baseline viewport diverges", () => {
  const { divergesFromBaseline, reasons } = compareToBaseline(verdict(), {
    viewports: { desktop: viewport() },
  });
  assert.equal(divergesFromBaseline, true);
  assert.match(reasons.join(";"), /mobile: no baseline data/);
});

test("empty or null baseline diverges for every viewport", () => {
  for (const baseline of [{}, null]) {
    const { divergesFromBaseline, reasons } = compareToBaseline(verdict(), baseline);
    assert.equal(divergesFromBaseline, true);
    assert.match(reasons.join(";"), /desktop: no baseline data/);
    assert.match(reasons.join(";"), /mobile: no baseline data/);
  }
});

test("degenerate current verdict fails closed", () => {
  for (const current of [null, undefined, {}, { viewports: {} }]) {
    const { divergesFromBaseline, reasons } = compareToBaseline(current, verdict());
    assert.equal(divergesFromBaseline, true);
    assert.deepEqual(reasons, ["current verdict has no viewport data"]);
  }
});

test("multiple signals across viewports all accumulate", () => {
  const { reasons } = compareToBaseline(
    verdict({ status: 500 }, { consoleErrors: ["boom"] }),
    verdict(),
  );
  assert.match(reasons.join(";"), /desktop: HTTP status changed/);
  assert.match(reasons.join(";"), /mobile: console\/page errors appeared/);
  assert.equal(reasons.length, 2);
});

test("baselineComparison round-trips a valid baseline", () => {
  const { divergesFromBaseline } = baselineComparison(verdict(), JSON.stringify(verdict()));
  assert.equal(divergesFromBaseline, false);
});

test("baselineComparison fails closed on malformed or wrong-shape baselines", () => {
  for (const raw of ["{", "", "not json"]) {
    const { divergesFromBaseline, reasons } = baselineComparison(verdict(), raw);
    assert.equal(divergesFromBaseline, true);
    assert.deepEqual(reasons, ["baseline unreadable: invalid JSON"]);
  }
  for (const raw of [
    "null",
    "[]",
    "0",
    "false",
    '""',
```

```
    "{}",
    '{"viewports": null}',
    '{"viewports": []}',
  ]) {
    const { divergesFromBaseline, reasons } = baselineComparison(verdict(), raw);
    assert.equal(divergesFromBaseline, true);
    assert.deepEqual(reasons, ["baseline unreadable: not a verdict object"]);
  }
});

test("parseSmokeArgs defaults", () => {
  assert.deepEqual(parseSmokeArgs([], {}), {
    url: "http://127.0.0.1:8080/",
    outPng: "/workspace/screenshots/app-builder-preview.png",
    baseline: "",
  });
});

test("parseSmokeArgs consumes --baseline without shifting positionals", () => {
  assert.deepEqual(
    parseSmokeArgs(
      ["http://127.0.0.1:8081/", "--baseline", "/workspace/a.json", "/workspace/b.png"],
      {},
    ),
    {
      url: "http://127.0.0.1:8081/",
      outPng: "/workspace/b.png",
      baseline: "/workspace/a.json",
    },
  );
});

test("parseSmokeArgs rejects a trailing --baseline flag", () => {
  assert.deepEqual(parseSmokeArgs(["--baseline"], {}), {
    error: "--baseline requires a path to a prior verdict JSON",
  });
});

test("parseSmokeArgs supports the --baseline= form", () => {
  assert.equal(parseSmokeArgs(["--baseline=/workspace/a.json"], {}).baseline, "/workspace/a.json");
  assert.deepEqual(parseSmokeArgs(["--baseline="], {}), {
    error: "--baseline requires a path to a prior verdict JSON",
  });
});

test("parseSmokeArgs rejects unknown flags instead of treating them as positionals", () => {
  assert.deepEqual(parseSmokeArgs(["--nope"], {}), { error: "unknown flag: --nope" });
  assert.deepEqual(
    parseSmokeArgs(["http://127.0.0.1:8080/", "--Baseline", "/workspace/a.json"], {}),
    {
      error: "unknown flag: --Baseline",
    },
  );
});

test("parseSmokeArgs env fallback and flag precedence", () => {
  const env = { BROWSER_SMOKE_BASELINE: "/workspace/env.json" };
  assert.equal(parseSmokeArgs([], env).baseline, "/workspace/env.json");
  assert.equal(
    parseSmokeArgs(["--baseline", "/workspace/flag.json"], env).baseline,
    "/workspace/flag.json",
  );
});

test("derivedPaths handles .png, .PNG, and extensionless inputs", () => {
  assert.deepEqual(derivedPaths("/workspace/shot.png"), {
    mobilePng: "/workspace/shot-mobile.png",
    verdictJson: "/workspace/shot.json",
  });
  assert.deepEqual(derivedPaths("/workspace/shot.PNG"), {
    mobilePng: "/workspace/shot-mobile.png",
    verdictJson: "/workspace/shot.json",
  });
  assert.deepEqual(derivedPaths("/workspace/shot"), {
    mobilePng: "/workspace/shot-mobile.png",
    verdictJson: "/workspace/shot.json",
  });
});

test("exitCodeFor: clean run exits 0", () => {
  assert.equal(exitCodeFor(verdict().viewports), 0);
});

test("exitCodeFor: errors in any single viewport exit 2", () => {
  assert.equal(exitCodeFor(verdict({}, { consoleErrors: ["boom"] }).viewports), 2);
  assert.equal(exitCodeFor(verdict({ pageErrors: ["boom"] }).viewports), 2);
});

test("exitCodeFor: navigation failure in any viewport exits 1, beating errors", () => {
  assert.equal(exitCodeFor(verdict({}, { status: 500 }).viewports), 1);
```

```
  assert.equal(exitCodeFor(verdict({ status: 0 }).viewports), 1);
  assert.equal(exitCodeFor(verdict({ status: 404 }, { consoleErrors: ["boom"] }).viewports), 1);
});

test("exitCodeFor: 4xx boundary and missing status", () => {
  assert.equal(exitCodeFor(verdict({ status: 399 }).viewports), 0);
  assert.equal(exitCodeFor(verdict({ status: 400 }).viewports), 1);
  assert.equal(exitCodeFor(verdict({ status: undefined }).viewports), 1);
});

test("exitCodeFor: no viewport data is a failure", () => {
  assert.equal(exitCodeFor({}), 1);
  assert.equal(exitCodeFor(undefined), 1);
});

test("browser-smoke wires the guard and verdict helpers", () => {
  const src = readFileSync(join(TEMPLATE_ROOT, "scripts/browser-smoke.mjs"), "utf8");
  assert.match(src, /from "\.\/browser-guard\.mjs"/);
  assert.match(src, /from "\.\/browser-smoke-verdict\.mjs"/);
  assert.match(src, /const args = parseSmokeArgs\(process\.argv\.slice\(2\), process\.env\)/);
  assert.match(src, /const url = checkedUrl\(args\.url\)/);
  assert.match(src, /const outPng = checkedOutputPath\(args\.outPng, \["\/workspace"\]\)/);
  assert.match(src, /const mobilePng = checkedOutputPath\(derived\.mobilePng, \["\/workspace"\]\)/);
  assert.match(src, /const outJson = checkedOutputPath\(derived\.verdictJson, \["\/workspace"\]/);
  assert.match(src, /checkedOutputPath\(realpathSync\(args\.baseline\), \["\/workspace"\]/);
  assert.match(src, /baselinePath === outJson/);
  assert.match(src, /normalizedBodyTextHash\(/);
  assert.match(src, /bodyTextPrefix\(/);
  assert.match(src, /baselineComparison\(/);
  assert.match(src, /process\.exitCode = exitCodeFor\(viewports\)/);
  assert.match(src, /waitUntil: "domcontentloaded"/);
  assert.doesNotMatch(
    src,
    /waitUntil:\s*["']networkidle["']/,
    "Vite HMR never reaches networkidle",
  );
  assert.match(src, /} finally \{\s*await browser\?\.close\(\);/s);
  assert.doesNotMatch(
    src.slice(src.indexOf("let browser = null")),
    /process\.exit\(/,
    "process.exit after Chromium launch skips finally teardown",
  );
});

test("browser-smoke file I/O only touches guarded paths", () => {
  const src = readFileSync(join(TEMPLATE_ROOT, "scripts/browser-smoke.mjs"), "utf8");
  assert.doesNotMatch(src, /(?:writeFileSync|readFileSync|statSync|realpathSync)\(\s*["'`]/);
  const writes = [...src.matchAll(/writeFileSync\(\s*([A-Za-z_$][\w$.]*)/g)].map((m) => m[1]);
  assert.equal(writes.length, 2);
  assert.ok(
    writes.every((v) => v === "outJson"),
    `unexpected writeFileSync target: ${writes}`,
  );
  const reads = [...src.matchAll(/readFileSync\(\s*([A-Za-z_$][\w$.]*)/g)].map((m) => m[1]);
  assert.deepEqual(reads, ["baselinePath"]);
  const stats = [...src.matchAll(/statSync\(\s*([A-Za-z_$][\w$.]*)/g)].map((m) => m[1]);
  assert.deepEqual(stats, ["baselinePath"]);
});
```

scripts/browser-smoke.mjs

```
#!/usr/bin/env node
import { mkdirSync, readFileSync, realpathSync, statSync, writeFileSync } from "node:fs";
import { dirname } from "node:path";
import { chromium } from "playwright";
import { checkedOutputPath, checkedUrl } from "./browser-guard.mjs";
import { computeBrandWarnings } from "./brand-check.mjs";
import {
  authInvariantWarnings,
  buildAuthEnabled,
  compareAuthInvariant,
  probeDevAuthEnabled,
} from "./check-auth-invariant.mjs";
import {
  baselineComparison,
  bodyTextPrefix,
  derivedPaths,
  exitCodeFor,
  normalizeBodyText,
  normalizedBodyTextHash,
  parseSmokeArgs,
} from "./browser-smoke-verdict.mjs";

const args = parseSmokeArgs(process.argv.slice(2), process.env);
if (args.error) {
  console.error(JSON.stringify({ ok: false, error: args.error }, null, 2));
  process.exit(1);
}

const url = checkedUrl(args.url);
const outPng = checkedOutputPath(args.outPng, ["/workspace"]);
const derived = derivedPaths(outPng);
const mobilePng = checkedOutputPath(derived.mobilePng, ["/workspace"]);
const outJson = checkedOutputPath(derived.verdictJson, ["/workspace"], "verdict JSON");

const MAX_BASELINE_BYTES = 1024 * 1024;
const baselineRequested = Boolean(args.baseline);
let baselinePath = null;
let baselineResolveError = null;
if (baselineRequested) {
  try {
    baselinePath = checkedOutputPath(realpathSync(args.baseline), ["/workspace"], "baseline");
  } catch (err) {
    baselineResolveError = err?.code ?? "unresolvable path";
  }
  if (baselinePath === outJson) {
    console.error(
      JSON.stringify(
        {
          ok: false,
          error:
            `--baseline ${args.baseline} is this run's own verdict output; ` +
            "pass a distinct output PNG (e.g. app-builder-built.png) so the baseline is not overwritten",
        },
        null,
        2,
      ),
    );
    process.exit(1);
  }
}

const timeoutMs = Number(process.env.BROWSER_SMOKE_TIMEOUT_MS || 45000);

const VIEWPORTS = [
  { name: "desktop", width: 1280, height: 800, screenshot: outPng },
  { name: "mobile", width: 390, height: 844, screenshot: mobilePng },
];

mkdirSync(dirname(outPng), { recursive: true });

function compareAgainstBaseline(verdict) {
  if (!baselinePath) {
    return {
      divergesFromBaseline: true,
      reasons: [`baseline unreadable: ${baselineResolveError ?? "unresolvable path"}`],
    };
  }
  try {
    if (statSync(baselinePath).size > MAX_BASELINE_BYTES) {
      return { divergesFromBaseline: true, reasons: ["baseline unreadable: too large"] };
    }
    return baselineComparison(verdict, readFileSync(baselinePath, "utf8"));
  } catch (err) {
    return {
      divergesFromBaseline: true,
      reasons: [`baseline unreadable: ${err?.code ?? "read error"}`],
    };
  }
```

```
}

let browser = null;
try {
  browser = await chromium.launch({
    headless: true,
    args: ["--no-sandbox", "--disable-dev-shm-usage"],
  });

  const viewports = {};
  for (const vp of VIEWPORTS) {
    const errors = { consoleErrors: [], pageErrors: [] };
    const page = await browser.newPage({
      viewport: { width: vp.width, height: vp.height },
    });
    page.on("console", (msg) => {
      if (msg.type() === "error") errors.consoleErrors.push(msg.text());
    });
    page.on("pageerror", (err) => errors.pageErrors.push(String(err?.message || err)));
    // `domcontentloaded`, not `networkidle`: Vite keeps an HMR websocket open, so
    // networkidle never settles and would burn the whole timeout.
    const resp = await page.goto(url, { waitUntil: "domcontentloaded", timeout: timeoutMs });
    const status = resp?.status() ?? 0;
    await page.waitForTimeout(1000);

    const title = await page.title();
    const hasCanvas = (await page.locator("canvas").count()) > 0;
    const bodyText = await page
      .locator("body")
      .innerText()
      .catch(() => "");
    const horizontalOverflow = await page.evaluate(() => {
      const el = document.documentElement;
      return el.scrollWidth > el.clientWidth + 1;
    });
    await page.screenshot({ path: vp.screenshot, fullPage: false });
    await page.close();

    viewports[vp.name] = {
      width: vp.width,
      height: vp.height,
      status,
      title,
      hasCanvas,
      bodyTextLen: normalizeBodyText(bodyText).length,
      bodyTextHash: normalizedBodyTextHash(bodyText),
      bodyTextPrefix: bodyTextPrefix(bodyText),
      horizontalOverflow,
      consoleErrors: errors.consoleErrors,
      pageErrors: errors.pageErrors,
      screenshot: vp.screenshot,
    };
  }

  const brandWarnings = computeBrandWarnings({ hasCanvas: viewports.desktop.hasCanvas });
  // Only a dev server answers /__app-env, so smoking the built output reads as
  // indeterminate — report a divergence, never the absence of an observation.
  const authWarnings = authInvariantWarnings(
    compareAuthInvariant({
      devAuthEnabled: await probeDevAuthEnabled(url),
      buildAuthEnabled: buildAuthEnabled(),
    }),
  );
  const verdict = { url, viewports, brandWarnings, authWarnings, verdictFile: outJson };
  if (baselineRequested) {
    const { divergesFromBaseline, reasons } = compareAgainstBaseline(verdict);
    verdict.divergesFromBaseline = divergesFromBaseline;
    verdict.baselineReasons = reasons;
  }

  writeFileSync(outJson, JSON.stringify(verdict, null, 2));
  console.log(JSON.stringify(verdict, null, 2));
  for (const w of [...brandWarnings, ...authWarnings]) console.error(w);
  // Set the code rather than aborting the process so the `finally` browser
  // teardown always runs (agents typically smoke twice per turn; leaking
  // Chromium accumulates across retries).
  process.exitCode = exitCodeFor(viewports);
} catch (err) {
  const failure = { ok: false, url, error: String(err?.message || err) };
  try {
    writeFileSync(outJson, JSON.stringify(failure, null, 2));
  } catch (writeErr) {
    failure.verdictWriteError = String(writeErr?.message || writeErr);
  }
  console.error(JSON.stringify(failure, null, 2));
  process.exitCode = 1;
} finally {
  await browser?.close();
}
```

scripts/check-auth-invariant.mjs

```
#!/usr/bin/env node
/**
 * Fail loudly when the running dev server and the next build disagree about
 * `VITE_AUTH_ENABLED`.
 *
 * `npm run dev`, `npm run build` and `npm run preview` all get the flag from
 * `scripts/with-app-env.mjs`, so they agree by construction — but a dev server
 * started outside npm (`npx vite dev`) does not, and the result is sign-in
 * visible in the live preview and absent from the built output, or the reverse.
 *
 * The two sides compared:
 *  - **dev**: what the running server resolved, read from the `/__app-env`
 *    endpoint the template's dev-only `appEnvPlugin` serves.
 *  - **build**: what the wrapper hands `vite build` / `vite preview`.
 *
 * The built bundle is not read: Vite inlines the flag and the minifier folds
 * `"false" !== "false"` away, so the built client JS carries no marker to
 * compare against unless the app is made to emit one.
 *
 * `scripts/browser-smoke.mjs` runs the comparison on every smoke; run it
 * standalone against a live dev server with `npm run check:auth` (exit 0 agree,
 * 1 diverged, 2 could not observe). Callers comparing the flag should use
 * `compareAuthInvariant()` rather than re-deriving it.
 */
import { APP_ENV_ROUTE } from "./app-env-plugin.mjs";
import { isMainModule, mergeAppEnv, projectRoot, readAppEnv } from "./with-app-env.mjs";

const DEFAULT_DEV_URL = "http://127.0.0.1:8080";

/** The predicate `src/lib/auth/{client,server}.ts` apply to the flag. */
export function authEnabledFromEnvValue(value) {
  return value !== "false";
}

/**
 * Compare the two resolved values. `null` means "could not observe" — reported
 * as indeterminate rather than as agreement.
 */
export function compareAuthInvariant({ devAuthEnabled, buildAuthEnabled }) {
  const label = (value) => (value ? "on" : "off");
  if (devAuthEnabled === null || devAuthEnabled === undefined) {
    return {
      status: "indeterminate",
      message: "[auth-invariant] could not read the dev server's resolved VITE_AUTH_ENABLED",
    };
  }
  if (devAuthEnabled === buildAuthEnabled) {
    return {
      status: "ok",
      message: `[auth-invariant] dev and build agree: sign-in ${label(devAuthEnabled)}`,
    };
  }
  return {
    status: "diverged",
    message:
      `[auth-invariant] dev server has sign-in ${label(devAuthEnabled)} but the next ` +
      `build has it ${label(buildAuthEnabled)}. Start the app with \`npm run dev\` — ` +
      "invoking vite directly skips scripts/with-app-env.mjs, so the dev server and " +
      "the built output resolve .grok/app-env.json differently.",
  };
}

/**
 * Ask the dev server which env it resolved. Anything but a JSON object from
 * `/__app-env` (no server, a built-output preview, an older workspace without
 * the plugin) is "could not observe".
 */
export async function probeDevAuthEnabled(devUrl, fetchImpl = fetch) {
  let env;
  try {
    const response = await fetchImpl(new URL(APP_ENV_ROUTE, devUrl).href);
    if (!response.ok) return null;
    env = JSON.parse(await response.text());
  } catch {
    return null;
  }
  if (env === null || typeof env !== "object") return null;
  return authEnabledFromEnvValue(env.VITE_AUTH_ENABLED);
}

/** The smoke-verdict warnings for a comparison: a real divergence only. */
export function authInvariantWarnings(result) {
  return result.status === "diverged" ? [result.message] : [];
}

/** What `vite build` / `vite preview` will resolve, via the same wrapper. */
export function buildAuthEnabled(root = projectRoot(), processEnv = process.env) {
  const env = mergeAppEnv(readAppEnv(root), processEnv);
```

```
  return authEnabledFromEnvValue(env.VITE_AUTH_ENABLED);
}

async function main(argv) {
  const devUrlFlag = argv.indexOf("--dev-url");
  const devUrl = devUrlFlag === -1 ? DEFAULT_DEV_URL : argv[devUrlFlag + 1];
  const result = compareAuthInvariant({
    devAuthEnabled: await probeDevAuthEnabled(devUrl),
    buildAuthEnabled: buildAuthEnabled(),
  });
  if (result.status === "ok") {
    console.log(result.message);
    process.exit(0);
  }
  console.error(result.message);
  process.exit(result.status === "diverged" ? 1 : 2);
}

if (isMainModule(import.meta.url)) {
  await main(process.argv.slice(2));
}
```

scripts/check-auth-invariant.test.mjs

```
import assert from "node:assert/strict";
import { execFile } from "node:child_process";
import { mkdtempSync, symlinkSync } from "node:fs";
import { tmpdir } from "node:os";
import { join } from "node:path";
import { test } from "node:test";
import { promisify } from "node:util";
import {
  authEnabledFromEnvValue,
  authInvariantWarnings,
  buildAuthEnabled,
  compareAuthInvariant,
  probeDevAuthEnabled,
} from "./check-auth-invariant.mjs";
import { projectRoot } from "./with-app-env.mjs";

/**
 * The JSON body `/__app-env` would serve. Do not start a real Vite server —
 * `import { createServer } from "vite"` loads rolldown native bindings that
 * SIGSEGV the test worker under qemu-user (amd64 image builds).
 */
function appEnvFetch(env) {
  return async () => ({
    ok: true,
    text: async () => JSON.stringify(env),
  });
}

test("the flag predicate matches src/lib/auth", () => {
  assert.equal(authEnabledFromEnvValue("false"), false);
  assert.equal(authEnabledFromEnvValue("true"), true);
  assert.equal(authEnabledFromEnvValue(undefined), true);
});

test("reads the value a live dev server resolved", async () => {
  assert.equal(
    await probeDevAuthEnabled("http://127.0.0.1:8080", appEnvFetch({ VITE_AUTH_ENABLED: "false" })),
    false,
  );
});

test("a server started without the flag reads as sign-in on", async () => {
  assert.equal(await probeDevAuthEnabled("http://127.0.0.1:8080", appEnvFetch({})), true);
});

test("agreement passes", () => {
  assert.equal(
    compareAuthInvariant({ devAuthEnabled: false, buildAuthEnabled: false }).status,
    "ok",
  );
});

test("divergence fails in either direction", () => {
  const devOn = compareAuthInvariant({ devAuthEnabled: true, buildAuthEnabled: false });
  assert.equal(devOn.status, "diverged");
  assert.match(devOn.message, /dev server has sign-in on but the next build has it off/);
  assert.equal(
    compareAuthInvariant({ devAuthEnabled: false, buildAuthEnabled: true }).status,
    "diverged",
  );
});

test("an unobservable dev server is indeterminate, not agreement", () => {
  assert.equal(
    compareAuthInvariant({ devAuthEnabled: null, buildAuthEnabled: false }).status,
    "indeterminate",
  );
});

test("a dev server that cannot be reached probes as null", async () => {
  const unreachable = () => Promise.reject(new Error("ECONNREFUSED"));
  assert.equal(await probeDevAuthEnabled("http://127.0.0.1:1", unreachable), null);
});

test("a server without the endpoint probes as null, not as agreement", async () => {
  const notFound = async () => ({ ok: false, text: async () => "Not Found" });
  assert.equal(await probeDevAuthEnabled("http://127.0.0.1:8081", notFound), null);
  const html = async () => ({ ok: true, text: async () => "<!doctype html>" });
  assert.equal(await probeDevAuthEnabled("http://127.0.0.1:8081", html), null);
});

test("only a divergence warns the smoke verdict", () => {
  const diverged = compareAuthInvariant({ devAuthEnabled: true, buildAuthEnabled: false });
  assert.deepEqual(authInvariantWarnings(diverged), [diverged.message]);
  for (const result of [
    compareAuthInvariant({ devAuthEnabled: false, buildAuthEnabled: false }),
    compareAuthInvariant({ devAuthEnabled: null, buildAuthEnabled: false }),
  ]) {
```

```
    assert.deepEqual(authInvariantWarnings(result), []);
  }
});

test("the build side resolves the template's shipped app-env", () => {
  assert.equal(buildAuthEnabled(projectRoot(), {}), false);
  assert.equal(buildAuthEnabled(projectRoot(), { VITE_AUTH_ENABLED: "true" }), true);
});

test("the CLI reports rather than silently passing when run via a symlink", async () => {
  // A check whose exit code is the whole signal must never no-op to 0 because
  // process.argv[1] came in through a symlinked path.
  const link = join(mkdtempSync(join(tmpdir(), "auth-invariant-link-")), "scripts");
  symlinkSync(join(projectRoot(), "scripts"), link);
  const error = await promisify(execFile)(process.execPath, [
    join(link, "check-auth-invariant.mjs"),
    "--dev-url",
    "http://127.0.0.1:1",
  ]).catch((err) => err);
  assert.equal(error.code, 2);
  assert.match(error.stderr, /could not read the dev server's resolved VITE_AUTH_ENABLED/);
});
```

scripts/grok-pwa-plugin.mjs

```
/**
 * Dev/preview (Vite) half of the platform PWA chrome: serves the ?install=1
 * tutorial and the per-app manifest, and injects missing PWA head tags into
 * app documents. The deployed-app half lives in server/middleware/grok-pwa.ts;
 * both share scripts/grok-pwa-shared.mjs.
 */
import { readFileSync } from "node:fs";
import { dirname, join } from "node:path";
import { fileURLToPath } from "node:url";
import {
  acceptsHtml,
  createHeadInjector,
  injectGrokPwaHead,
  isDocumentPath,
  isInstallQuery,
  renderInstallPageHtml,
  renderWebManifest,
  snapshotOgIdentity,
} from "./grok-pwa-shared.mjs";

export const GROK_OG_IDENTITY_ID = "virtual:grok-og-identity";

const INSTALL_PAGE_PATH = join(dirname(fileURLToPath(import.meta.url)), "install-page.html");

function requestHost(req) {
  const forwarded = req.headers["x-forwarded-host"];
  const host = forwarded ?? req.headers.host ?? req.headers[":authority"];
  return Array.isArray(host) ? host[0] : host;
}

export function renderInstallPage(hostHeader, url = "/") {
  const template = readFileSync(INSTALL_PAGE_PATH, "utf8");
  return renderInstallPageHtml(template, { host: hostHeader, url });
}

function sendHtml(res, html) {
  const body = Buffer.from(html, "utf8");
  res.statusCode = 200;
  res.setHeader("content-type", "text/html; charset=utf-8");
  res.setHeader("cache-control", "no-cache");
  res.setHeader("content-length", String(body.byteLength));
  res.end(body);
}

function serveGrokPwa(middlewares) {
  middlewares.use((req, res, next) => {
    const rawUrl = req.url ?? "";
    const pathOnly = rawUrl.split("?", 1)[0] ?? "";
    const method = (req.method ?? "GET").toUpperCase();
    if (method !== "GET") {
      next();
      return;
    }

    if (pathOnly === "/__grok/manifest.webmanifest" || pathOnly === "/__grok/manifest.json") {
      const body = Buffer.from(renderWebManifest(requestHost(req)), "utf8");
      res.statusCode = 200;
      res.setHeader("content-type", "application/manifest+json; charset=utf-8");
      res.setHeader("cache-control", "no-cache");
      res.setHeader("content-length", String(body.byteLength));
      res.end(body);
      return;
    }

    if (isInstallQuery(rawUrl) && isDocumentPath(pathOnly) && acceptsHtml(req.headers.accept)) {
      try {
        sendHtml(res, renderInstallPage(requestHost(req), rawUrl));
      } catch (err) {
        console.error("[app-builder] install page missing:", err);
        res.statusCode = 500;
        res.end("install page unavailable");
      }
      return;
    }

    next();
  });
}

/**
 * Wrap res.write/res.end on app-document requests to inject missing PWA head
 * tags at the `</head>` boundary as chunks stream through (no full-document
 * buffering, so streaming SSR keeps its early flush). Skips anything already
 * content-encoded: under `vite preview` the compression middleware can hand
 * this wrapper gzipped bytes, which must pass through untouched.
 */
function wrapHtmlResponses(middlewares, cwd) {
  middlewares.use((req, res, next) => {
```

```
    const rawUrl = req.url ?? "";
    const pathOnly = rawUrl.split("?", 1)[0] ?? "";
    const method = (req.method ?? "GET").toUpperCase();
    const looksLikeDocument =
      method === "GET" &&
      String(req.headers.accept ?? "").includes("text/html") &&
      !isInstallQuery(rawUrl) &&
      isDocumentPath(pathOnly);
    if (!looksLikeDocument) {
      next();
      return;
    }

    const originalWrite = res.write.bind(res);
    const originalEnd = res.end.bind(res);
    const host = requestHost(req);
    const injector = createHeadInjector({
      host,
      cwd,
    });
    let mode = null; // null = undecided, "inject" | "passthrough"

    const decideMode = () => {
      if (mode) return mode;
      const isHtml = String(res.getHeader("content-type") ?? "").includes("text/html");
      const encoded = Boolean(res.getHeader("content-encoding"));
      mode = isHtml && !encoded ? "inject" : "passthrough";
      // Streaming SSR flushes headers before the first body chunk, so the
      // header may no longer be removable — chunked responses don't carry one.
      if (mode === "inject" && !res.headersSent) res.removeHeader("content-length");
      return mode;
    };

    const toBuffer = (chunk, encoding) => {
      if (Buffer.isBuffer(chunk)) return chunk;
      if (typeof chunk === "string") {
        return Buffer.from(chunk, typeof encoding === "string" ? encoding : "utf8");
      }
      return Buffer.from(chunk);
    };

    res.write = (chunk, encoding, cb) => {
      if (decideMode() === "passthrough") return originalWrite(chunk, encoding, cb);
      const done = typeof encoding === "function" ? encoding : cb;
      if (chunk) {
        for (const out of injector.push(toBuffer(chunk, encoding))) originalWrite(out);
      }
      if (typeof done === "function") done();
      return true;
    };

    res.end = (chunk, encoding, cb) => {
      const done = typeof encoding === "function" ? encoding : cb;
      if (decideMode() === "passthrough") return originalEnd(chunk, encoding, cb);
      if (chunk) {
        for (const out of injector.push(toBuffer(chunk, encoding))) originalWrite(out);
      }
      for (const out of injector.flush()) originalWrite(out);
      return originalEnd(undefined, undefined, done);
    };

    next();
  });
}

export function grokPwaPlugin() {
  let root = process.cwd();
  return {
    name: "app-builder:grok-pwa",
    configResolved(config) {
      root = config.root;
    },
    resolveId(id) {
      if (id === GROK_OG_IDENTITY_ID) return `\0${GROK_OG_IDENTITY_ID}`;
    },
    load(id) {
      if (id !== `\0${GROK_OG_IDENTITY_ID}`) return;
      return `export const grokOgIdentity = ${JSON.stringify(snapshotOgIdentity(root))};`;
    },
    transformIndexHtml(html) {
      return injectGrokPwaHead(html, {
        host: process.env.VITE_PUBLIC_HOSTNAME ?? "",
        cwd: root,
      });
    },
    configureServer(server) {
      // Registered directly (not in a returned post-hook) so both run BEFORE
      // TanStack Start's SSR middleware, like the auth-popup plugin.
      serveGrokPwa(server.middlewares);
      wrapHtmlResponses(server.middlewares, root);
```

```
    },
    configurePreviewServer(server) {
      serveGrokPwa(server.middlewares);
      // Post-hook: preview registers compression between the direct hooks and
      // the post-hooks, and the injector must wrap AFTER compression so it
      // sees plaintext HTML (compression then compresses the injected output).
      return () => {
        wrapHtmlResponses(server.middlewares, root);
      };
    },
  };
}
```

scripts/grok-pwa-plugin.test.mjs

```
import assert from "node:assert/strict";
import { mkdirSync, mkdtempSync, readFileSync, writeFileSync } from "node:fs";
import { tmpdir } from "node:os";
import { dirname, join } from "node:path";
import { fileURLToPath } from "node:url";
import test from "node:test";
import {
  appNameFromHost,
  createHeadInjector,
  grokXCreatorHeadTags,
  injectGrokPwaHead,
  isDocumentPath,
  isInstallQuery,
  publicAppHost,
  renderWebManifest,
  resolveOgCardAsset,
  snapshotOgIdentity,
  stripInstallParams,
} from "./grok-pwa-shared.mjs";
import { renderInstallPage } from "./grok-pwa-plugin.mjs";

const TEMPLATE_ROOT = join(dirname(fileURLToPath(import.meta.url)), "..");

test("injects before </head>", () => {
  const out = injectGrokPwaHead("<html><head><title>x</title></head><body></body></html>");
  assert.match(out, /rel="manifest"/);
  assert.match(out, /apple-touch-icon/);
  assert.match(out, /grok-app-builder\/extensions\.js/);
  assert.ok(out.indexOf("manifest") < out.indexOf("</head>"));
});

test("injects the extensions script without a project id", () => {
  const out = injectGrokPwaHead("<html><head></head></html>", {
    appName: "Demo",
    projectId: "",
  });
  assert.match(out, /src="https:\/\/grok\.com\/grok-app-builder\/extensions\.js" defer/);
  assert.doesNotMatch(out, /grok-project-id/);
  assert.doesNotMatch(out, /data-project-id/);
  assert.doesNotMatch(out, /property="grok:app_id"/);
});

test("injects project id on the script and meta when provided", () => {
  const out = injectGrokPwaHead("<html><head></head></html>", {
    appName: "Demo",
    projectId: "proj-123",
  });
  assert.match(out, /name="grok-project-id" content="proj-123"/);
  assert.match(out, /data-project-id="proj-123"/);
  assert.match(out, /property="grok:app_id" content="proj-123"/);
});

test("does not duplicate grok:app_id", () => {
  const ctx = { appName: "Demo", projectId: "proj-123" };
  const once = injectGrokPwaHead("<html><head></head></html>", ctx);
  const twice = injectGrokPwaHead(once, ctx);
  assert.equal(once, twice);
  assert.equal(twice.split('property="grok:app_id"').length - 1, 1);
});

test("omits x:creator tags without both creator values", () => {
  assert.deepEqual(grokXCreatorHeadTags("", "42"), []);
  assert.deepEqual(grokXCreatorHeadTags("@alice", ""), []);
  const out = injectGrokPwaHead("<html><head></head></html>", {
    appName: "Demo",
    projectId: "",
    creator: "@alice",
    creatorId: "",
  });
  assert.doesNotMatch(out, /property="x:creator"/);
});

test("injects x:creator tags when both creator values are set", () => {
  const out = injectGrokPwaHead("<html><head></head></html>", {
    appName: "Demo",
    projectId: "",
    creator: "@alice",
    creatorId: "42",
  });
  assert.match(out, /property="x:creator" content="@alice"/);
  assert.match(out, /property="x:creator:id" content="42"/);
});

test("escapes x:creator values", () => {
  const tags = grokXCreatorHeadTags('"><script>', '1" onclick="alert(1)');
  assert.equal(
    tags[0],
    '<meta property="x:creator" content="&quot;&gt;&lt;script&gt;">',
```

```
  );
  assert.equal(
    tags[1],
    '<meta property="x:creator:id" content="1&amp;quot; onclick=&amp;quot;alert(1)">',
  );
});

test("does not duplicate x:creator tags", () => {
  const ctx = { appName: "Demo", projectId: "", creator: "@alice", creatorId: "42" };
  const once = injectGrokPwaHead("<html><head></head></html>", ctx);
  const twice = injectGrokPwaHead(once, ctx);
  assert.equal(once, twice);
  assert.equal(twice.split('property="x:creator" content=').length - 1, 1);
  assert.equal(twice.split('property="x:creator:id"').length - 1, 1);
});

test("platform chrome overwrites share-card metas and always sets og:title", () => {
  const html =
    '<html><head><title>Hello World</title><meta property="og:title" content="Old"><meta name="twitter:card"
 content="summary"></head></html>';
  const out = injectGrokPwaHead(html, { appName: "Wild Race" });
  assert.match(out, /name="twitter:card" content="summary_large_image"/);
  assert.match(out, /property="og:title" content="Hello World"/);
  assert.doesNotMatch(out, /content="Old"/);
  assert.doesNotMatch(out, /content="summary"/);
  assert.equal(out.split('name="twitter:card"').length - 1, 1);
  assert.equal(out.split('property="og:title"').length - 1, 1);
  assert.doesNotMatch(out, /property="og:image"/);
});

test("does not duplicate twitter:card or og:title", () => {
  const once = injectGrokPwaHead("<html><head><title>Hello World</title></head></html>");
  const twice = injectGrokPwaHead(once);
  assert.equal(once, twice);
  assert.equal(twice.split('name="twitter:card"').length - 1, 1);
  assert.equal(twice.split('property="og:title"').length - 1, 1);
});

test("a baked site.image is treated as a custom card", () => {
  const out = injectGrokPwaHead("<html><head></head></html>", {
    host: "wild-race.grok.me",
    cwd: mkdtempSync(join(tmpdir(), "grok-og-image-only-")),
    site: { title: "Wild Race", image: "/og.jpg" },
  });
  assert.match(out, /property="og:image" content="https:\/\/wild-race\.grok\.me\/og\.jpg"/);
  assert.doesNotMatch(out, /og\.grok\.me/);
});

test("baked identity does not need a workspace filesystem", () => {
  const empty = mkdtempSync(join(tmpdir(), "grok-og-empty-"));
  const out = injectGrokPwaHead("<html><head></head></html>", {
    host: "wild-race.grok.me",
    cwd: empty,
    site: { title: "Pixel Nova", type: "x:game", card: "custom" },
  });
  assert.match(out, /property="og:title" content="Pixel Nova"/);
  assert.match(out, /property="og:type" content="x:game"/);
  assert.match(out, /property="og:image" content="https:\/\/wild-race\.grok\.me\/og\.jpg"/);
  assert.doesNotMatch(out, /og\.grok\.me/);
});

test("a public card file wins over a baked site without card=custom", () => {
  // Deploy middleware always passes a baked `site`. If that snapshot missed
  // the file, public/og.jpg must still beat the og.grok.me placeholder.
  const root = mkdtempSync(join(tmpdir(), "grok-og-card-"));
  mkdirSync(join(root, "public"));
  writeFileSync(join(root, "public/og.jpg"), "x");
  const out = injectGrokPwaHead("<html><head></head></html>", {
    host: "wild-race.grok.me",
    cwd: root,
    site: {},
  });
  assert.match(out, /property="og:image" content="https:\/\/wild-race\.grok\.me\/og\.jpg"/);
  assert.doesNotMatch(out, /og\.grok\.me/);
});

test("public/og.png wins when jpg is absent", () => {
  const root = mkdtempSync(join(tmpdir(), "grok-og-png-"));
  mkdirSync(join(root, "public"));
  writeFileSync(join(root, "public/og.png"), "x");
  const out = injectGrokPwaHead("<html><head></head></html>", {
    host: "wild-race.grok.me",
    cwd: root,
    site: { title: "Wild Race" },
  });
  assert.match(out, /property="og:image" content="https:\/\/wild-race\.grok\.me\/og\.png"/);
  assert.doesNotMatch(out, /og\.grok\.me/);
});

test("resolveOgCardAsset: disk file, then bake, then empty (placeholder)", () => {
```

```
  const empty = mkdtempSync(join(tmpdir(), "grok-og-none-"));
  assert.equal(resolveOgCardAsset({}, empty), "");
  assert.equal(resolveOgCardAsset({ title: "X" }, empty), "");

  const baked = resolveOgCardAsset({ card: "custom", image: "/og.jpg" }, empty);
  assert.equal(baked, "/og.jpg");

  const root = mkdtempSync(join(tmpdir(), "grok-og-disk-"));
  mkdirSync(join(root, "public"));
  writeFileSync(join(root, "public/og.jpg"), "x");
  assert.equal(resolveOgCardAsset({}, root), "/og.jpg");
  assert.equal(resolveOgCardAsset({ card: "custom", image: "/other.png" }, root), "/og.jpg");
});

test("snapshotOgIdentity stamps card=custom from a public card file", () => {
  const root = mkdtempSync(join(tmpdir(), "grok-og-snap-"));
  mkdirSync(join(root, "public"));
  writeFileSync(join(root, "public/og.jpg"), "x");
  const { site } = snapshotOgIdentity(root);
  assert.equal(site.card, "custom");
  assert.equal(site.image, "/og.jpg");
  assert.equal(site.banner, undefined);
});

test("snapshotOgIdentity stamps banner from public/x-banner.jpg", () => {
  const root = mkdtempSync(join(tmpdir(), "grok-og-banner-"));
  mkdirSync(join(root, "public"));
  writeFileSync(join(root, "public/x-banner.jpg"), "x");
  const { site } = snapshotOgIdentity(root);
  assert.equal(site.banner, "/x-banner.jpg");
});

test("emits x:game:image for a public host when site.banner is set", () => {
  const html = "<html><head><meta property=\"x:game:image\" content=\"old\"></head></html>";
  const out = injectGrokPwaHead(html, {
    host: "wild-race.grok.me",
    site: { title: "Wild Race", type: "x:game", card: "custom", banner: "/x-banner.jpg" },
  });
  assert.match(
    out,
    /property="x:game:image" content="https:\/\/wild-race\.grok\.me\/x-banner\.jpg"/,
  );
  assert.match(out, /property="x:game:image:width" content="1200"/);
  assert.match(out, /property="x:game:image:height" content="264"/);
  assert.doesNotMatch(out, /content="old"/);
  assert.equal(out.split('property="x:game:image"').length - 1, 1);
});

test("does not emit x:game:image without a public host or banner", () => {
  const noHost = injectGrokPwaHead("<html><head></head></html>", {
    site: { banner: "/x-banner.jpg" },
  });
  assert.doesNotMatch(noHost, /x:game:image/);
  const noBanner = injectGrokPwaHead("<html><head></head></html>", {
    host: "wild-race.grok.me",
    site: { type: "x:game", card: "custom" },
  });
  assert.doesNotMatch(noBanner, /x:game:image/);
});

test("site title Grok App is a real name, not a sentinel", () => {
  const out = injectGrokPwaHead("<html><head></head></html>", {
    host: "wild-race.grok.me",
    site: { title: "Grok App" },
  });
  assert.match(out, /property="og:title" content="Grok App"/);
});

test("published grok.me slug is still a title fallback", () => {
  const out = injectGrokPwaHead("<html><head></head></html>", {
    host: "wild-race.grok.me",
  });
  assert.match(out, /property="og:title" content="Wild Race"/);
});

test("rejects Vercel system hosts as og:image origins", () => {
  assert.equal(publicAppHost("01a020b6-803a-71a2-bb47-e2bec57eb9a2-662k8x1l1-xai-org.vercel.app"), "");
  assert.equal(publicAppHost("demo.vercel.app:443"), "");
  assert.equal(publicAppHost("vercel.app"), "");
  assert.equal(publicAppHost("wild-race.grok.me"), "wild-race.grok.me");
});

test("published VITE_PUBLIC_HOSTNAME wins over request Host for og:image", () => {
  const prev = process.env.VITE_PUBLIC_HOSTNAME;
  process.env.VITE_PUBLIC_HOSTNAME = "plum-plaza-reef-dream.grok.me";
  try {
    const vercelHost = injectGrokPwaHead("<html><head><title>RACK</title></head></html>", {
      host: "01a020b6-803a-71a2-bb47-e2bec57eb9a2-662k8x1l1-xai-org.vercel.app",
      site: { title: "RACK", card: "custom" },
    });
```

```
    assert.match(
      vercelHost,
      /property="og:image" content="https:\/\/plum-plaza-reef-dream\.grok\.me\/og\.jpg"/,
    );
    assert.doesNotMatch(vercelHost, /vercel\.app/);

    const otherPublicHost = injectGrokPwaHead("<html><head><title>RACK</title></head></html>", {
      host: "custom.example.com",
      site: { title: "RACK", card: "custom" },
    });
    assert.match(
      otherPublicHost,
      /property="og:image" content="https:\/\/plum-plaza-reef-dream\.grok\.me\/og\.jpg"/,
    );
    assert.doesNotMatch(otherPublicHost, /custom\.example\.com/);
  } finally {
    if (prev === undefined) delete process.env.VITE_PUBLIC_HOSTNAME;
    else process.env.VITE_PUBLIC_HOSTNAME = prev;
  }
});

test("vercel Host without a public hostname emits no og:image", () => {
  const prev = process.env.VITE_PUBLIC_HOSTNAME;
  delete process.env.VITE_PUBLIC_HOSTNAME;
  try {
    const out = injectGrokPwaHead("<html><head><title>RACK</title></head></html>", {
      host: "01a020b6-803a-71a2-bb47-e2bec57eb9a2-662k8x1l1-xai-org.vercel.app",
      site: { title: "RACK", card: "custom" },
    });
    assert.doesNotMatch(out, /property="og:image"/);
    assert.doesNotMatch(out, /vercel\.app/);
  } finally {
    if (prev === undefined) delete process.env.VITE_PUBLIC_HOSTNAME;
    else process.env.VITE_PUBLIC_HOSTNAME = prev;
  }
});

test("emits og:image for a public host and prefers a custom card", () => {
  const placeholder = injectGrokPwaHead("<html><head></head></html>", {
    appName: "Wild Race",
    host: "wild-race.grok.me",
    site: { title: "Wild Race" },
  });
  assert.match(
    placeholder,
    /property="og:image" content="https:\/\/og\.grok\.me\/v1\/card\.png\?host=wild-race\.grok\.me&amp;title=
Wild%20Race"/,
  );
  assert.match(placeholder, /property="og:image:width" content="1200"/);

  const custom = injectGrokPwaHead("<html><head></head></html>", {
    appName: "Wild Race",
    host: "wild-race.grok.me",
    site: { title: "Wild Race", card: "custom", type: "x:game" },
  });
  assert.match(custom, /property="og:image" content="https:\/\/wild-race\.grok\.me\/og\.jpg"/);
  assert.match(custom, /property="og:type" content="x:game"/);
});

test("placeholder og:image appends site.color when it is 6-digit hex", () => {
  const themed = injectGrokPwaHead("<html><head></head></html>", {
    host: "wild-race.grok.me",
    site: { title: "Wild Race", color: "#FF4D2E" },
  });
  assert.match(
    themed,
    /property="og:image" content="https:\/\/og\.grok\.me\/v1\/card\.png\?host=wild-race\.grok\.me&amp;title=
Wild%20Race&amp;color=FF4D2E"/,
  );

  const invalid = injectGrokPwaHead("<html><head></head></html>", {
    host: "wild-race.grok.me",
    site: { title: "Wild Race", color: "red" },
  });
  assert.doesNotMatch(invalid, /color=/);

  const custom = injectGrokPwaHead("<html><head></head></html>", {
    host: "wild-race.grok.me",
    site: { title: "Wild Race", card: "custom", color: "FF4D2E" },
  });
  assert.doesNotMatch(custom, /color=/);
});

test("document title entities are not double-escaped on og:title", () => {
  const out = injectGrokPwaHead(
    "<html><head><title>Cats &amp; Dogs</title></head></html>",
  );
  assert.match(out, /property="og:title" content="Cats &amp; Dogs"/);
  assert.doesNotMatch(out, /Cats &amp;amp; Dogs/);
});
```

```
test("site.json title wins over the host slug", () => {
  const out = injectGrokPwaHead("<html><head></head></html>", {
    host: "wild-race.grok.me",
    site: { title: "Pixel Nova" },
  });
  assert.match(out, /property="og:title" content="Pixel Nova"/);
});

test("injects into documents with no head element", () => {
  const out = injectGrokPwaHead("<html><body>hi</body></html>", { appName: "Solo" });
  assert.match(out, /<head>/);
  assert.match(out, /property="og:title" content="Solo"/);
  assert.match(out, /<\/head>/);
});

test("streaming injector matches </HEAD> case-insensitively", () => {
  const injector = createHeadInjector({ appName: "Wild Race" });
  const chunks = [
    ...injector.push("<html><HEAD><title>x</title></HE"),
    ...injector.push("AD><body>hello</body></html>"),
  ];
  const out = Buffer.concat(chunks).toString("utf8");
  assert.match(out, /property="og:title" content="x"/);
  assert.match(out, /<body>hello<\/body>/);
});

test("omits the extensions script when the deployer disables it", () => {
  const previous = process.env.VITE_GROK_EXTENSIONS;
  process.env.VITE_GROK_EXTENSIONS = "0";
  try {
    const baked =
      '<html><head><script src="https://grok.com/grok-app-builder/extensions.js" data-project-id="old" defer
></script></head></html>';
    const out = injectGrokPwaHead(baked, { appName: "Demo", projectId: "proj-123" });
    assert.doesNotMatch(out, /grok-app-builder\/extensions\.js/);
    assert.match(out, /name="grok-project-id" content="proj-123"/);
    assert.match(out, /property="grok:app_id" content="proj-123"/);
    assert.match(out, /rel="manifest"/);
    const twice = injectGrokPwaHead(out, { appName: "Demo", projectId: "proj-123" });
    assert.equal(out, twice);
  } finally {
    if (previous === undefined) delete process.env.VITE_GROK_EXTENSIONS;
    else process.env.VITE_GROK_EXTENSIONS = previous;
  }
});

test("keeps the extensions script on commercial when the deployer enables it", () => {
  const previous = process.env.VITE_GROK_EXTENSIONS;
  process.env.VITE_GROK_EXTENSIONS = "1";
  try {
    const out = injectGrokPwaHead("<html><head></head></html>", {
      appName: "Demo",
      projectId: "proj-123",
    });
    assert.match(out, /src="https:\/\/grok\.com\/grok-app-builder\/extensions\.js"/);
    assert.match(out, /data-project-id="proj-123"/);
  } finally {
    if (previous === undefined) delete process.env.VITE_GROK_EXTENSIONS;
    else process.env.VITE_GROK_EXTENSIONS = previous;
  }
});

test("does not duplicate the extensions script", () => {
  const ctx = { appName: "Demo", projectId: "proj-123" };
  const once = injectGrokPwaHead("<html><head></head></html>", ctx);
  const twice = injectGrokPwaHead(once, ctx);
  assert.equal(once, twice);
  assert.equal(twice.split("extensions.js").length - 1, 1);
});

test("is idempotent", () => {
  const once = injectGrokPwaHead("<html><head></head></html>");
  const twice = injectGrokPwaHead(once);
  assert.equal(once, twice);
});

test("uses the app name in the injected title tag", () => {
  const out = injectGrokPwaHead("<html><head></head></html>", { appName: "Wild Race" });
  assert.match(out, /apple-mobile-web-app-title" content="Wild Race"/);
});

test("streaming injector handles </head> split across chunks", () => {
  const injector = createHeadInjector({ appName: "Wild Race" });
  const chunks = [
    ...injector.push("<html><head><title>x</title></he"),
    ...injector.push("ad><body>hello</body></html>"),
  ];
  const out = Buffer.concat(chunks).toString("utf8");
  assert.match(out, /rel="manifest"/);
  assert.ok(out.indexOf("manifest") < out.indexOf("</head>"));
```

```
  assert.match(out, /<body>hello<\/body>/);
  assert.deepEqual(injector.flush(), []);
});

test("streaming injector passes post-head chunks through untouched", () => {
  const injector = createHeadInjector();
  injector.push("<html><head></head>");
  const [tail] = injector.push("<body>tail</body>");
  assert.equal(tail.toString("utf8"), "<body>tail</body>");
});

test("streaming injector falls back when no </head> is seen", () => {
  const injector = createHeadInjector();
  assert.deepEqual(injector.push("<html><head>"), []);
  const out = Buffer.concat(injector.flush()).toString("utf8");
  assert.match(out, /rel="manifest"/);
});

test("detects install query", () => {
  assert.equal(isInstallQuery("/?install=1&platform=ios"), true);
  assert.equal(isInstallQuery("/app?foo=1&install=true&platform=ios"), true);
  assert.equal(isInstallQuery("/?install=1"), false);
  assert.equal(isInstallQuery("/?install=1&platform=android"), false);
  assert.equal(isInstallQuery("/?install=0&platform=ios"), false);
  assert.equal(isInstallQuery("/"), false);
});

test("filters non-document paths", () => {
  assert.equal(isDocumentPath("/"), true);
  assert.equal(isDocumentPath("/app"), true);
  assert.equal(isDocumentPath("/api/thing"), false);
  assert.equal(isDocumentPath("/__grok/install/styles.css"), false);
  assert.equal(isDocumentPath("/logo.png"), false);
});

test("strips install params from the app link", () => {
  assert.equal(stripInstallParams("/?install=1&platform=ios"), "/");
  assert.equal(stripInstallParams("/app?install=1&platform=ios&tab=2"), "/app?tab=2");
});

test("names the install page from host slug", () => {
  assert.equal(appNameFromHost("localhost:8080"), "Grok App");
  assert.equal(appNameFromHost("172.17.154.217:8080"), "Grok App");
  assert.equal(appNameFromHost("wild-race.grok.me"), "Wild Race");
});

test("rejects hosts that are not plain slugs", () => {
  assert.equal(appNameFromHost("<script>alert(1)</script>"), "Grok App");
  assert.equal(appNameFromHost('"><img src=x onerror=1>.grok.me'), "Grok App");
});

test("renders install page markup", () => {
  const html = renderInstallPage("wild-race.grok.me", "/?install=1&platform=ios");
  assert.match(html, /Add Wild Race to your/);
  assert.match(html, /\/__grok\/install\/styles\.css/);
  assert.match(html, /href="\/"/);
  assert.equal(html.includes("{{APP_NAME}}"), false);
  assert.equal(html.includes("{{APP_URL}}"), false);
});

test("escapes host-derived values in the install page", () => {
  const html = renderInstallPage("<script>alert(1)</script>", "/?install=1&platform=ios");
  assert.equal(html.includes("<script>alert(1)</script>"), false);
});

test("renders the manifest with the per-app name", () => {
  const manifest = JSON.parse(renderWebManifest("wild-race.grok.me"));
  assert.equal(manifest.name, "Wild Race");
  assert.equal(manifest.short_name, "Wild Race");
  assert.equal(manifest.icons[0].src, "/__grok/icon-180.png");
});

// Tripwires: the deployed-app path only works if Nitro scans server/ — an
// accidental edit that drops serverDir or the middleware file would otherwise
// fail silently (published apps would just render the app for ?install=1).
test("vite config keeps the nitro serverDir wiring", () => {
  const viteConfig = readFileSync(join(TEMPLATE_ROOT, "vite.config.ts"), "utf8");
  assert.match(viteConfig, /serverDir:\s*"\.\/server"/);
  assert.match(viteConfig, /grokPwaPlugin\(\)/);
});

test("nitro middleware and its bundled assets exist", () => {
  const middleware = readFileSync(join(TEMPLATE_ROOT, "server/middleware/grok-pwa.ts"), "utf8");
  assert.match(middleware, /install-page\.html\?raw/);
  assert.match(middleware, /virtual:grok-og-identity/);
  readFileSync(join(TEMPLATE_ROOT, "scripts/install-page.html"));
  readFileSync(join(TEMPLATE_ROOT, "public/__grok/icon-180.png"));
  readFileSync(join(TEMPLATE_ROOT, "public/__grok/install/styles.css"));
});
```

```
test("vite plugin bakes og identity as a virtual module", () => {
  const plugin = readFileSync(join(TEMPLATE_ROOT, "scripts/grok-pwa-plugin.mjs"), "utf8");
  assert.match(plugin, /virtual:grok-og-identity/);
  assert.match(plugin, /snapshotOgIdentity/);
});
```

scripts/grok-pwa-shared.d.mts

```
export declare const DEFAULT_APP_NAME: string;
export declare const OG_SERVICE_URL_DEFAULT: string;
export declare const OG_SITE_REL_PATH: string;
export declare function escapeHtml(value: unknown): string;
export declare function appNameFromHost(hostHeader: string | null | undefined): string;
export declare function publicAppHost(hostHeader: string | null | undefined): string;
export declare function resolvePublicHost(hostHeader: string | null | undefined): string;
export declare function isInstallQuery(url: string | null | undefined): boolean;
export declare function isDocumentPath(pathname: string | null | undefined): boolean;
export declare function acceptsHtml(accept: string | null | undefined): boolean;
export declare function stripInstallParams(url: string | null | undefined): string;
export declare function renderInstallPageHtml(
  template: string,
  context?: { host?: string | null; url?: string | null },
): string;
export declare function renderWebManifest(hostHeader: string | null | undefined): string;
export declare function grokPwaHeadTags(appName?: string): Array<[string, string]>;
export declare const GROK_EXTENSIONS_SCRIPT_SRC: string;
export declare function readGrokProjectId(): string;
export declare function readGrokExtensionsEnabled(): boolean;
export declare function readXCreator(): string;
export declare function readXCreatorId(): string;
export declare function grokXCreatorHeadTags(creator?: string, creatorId?: string): string[];
export declare function grokExtensionsHeadTags(projectId?: string): string[];

export type OgSite = {
  title?: string;
  description?: string;
  type?: string;
  card?: string;
  image?: string;
  banner?: string;
  color?: string;
};

export type GrokHeadContext = {
  appName?: string;
  projectId?: string;
  creator?: string;
  creatorId?: string;
  host?: string | null;
  cwd?: string;
  site?: OgSite;
};

export declare function readOgSite(cwd?: string): OgSite;
export declare function ogCardPublicPath(cwd?: string): string;
export declare function snapshotOgIdentity(cwd?: string): { site: OgSite };
export declare function customOgAssetPath(cwd?: string): string;
export declare function resolveOgCardAsset(site?: OgSite, cwd?: string): string;
export declare function ogServiceUrl(): string;
export declare function titleFromDocument(html: string): string;
export declare function resolveOgTitle(
  site?: OgSite,
  appName?: string,
  host?: string,
  documentTitle?: string,
): string;
export declare function siteHasCustomCard(site?: OgSite): boolean;
export declare function grokOgHeadTags(ctx?: {
  host?: string;
  appName?: string;
  site?: OgSite;
  documentTitle?: string;
  cwd?: string;
}): string[];
export declare function stripShareMetaTags(html: string): string;
export declare function normalizeHeadContext(ctx?: GrokHeadContext): {
  appName: string;
  projectId: string;
  creator: string;
  creatorId: string;
  host: string;
  cwd: string;
  site: OgSite;
};
export declare function injectGrokPwaHead(html: string, ctx?: GrokHeadContext): string;
export declare function createHeadInjector(ctx?: GrokHeadContext): {
  push(chunk: Uint8Array | string): Uint8Array[];
  flush(): Uint8Array[];
};
```

scripts/grok-pwa-shared.mjs

```
/**
 * Single source of truth for platform head chrome (PWA, extensions.js, OG),
 * shared by the Vite plugin and Nitro middleware. Plain ESM so `node --test`
 * and the Nitro bundler can both consume it.
 */
import { existsSync, readFileSync } from "node:fs";
import { join } from "node:path";

export const DEFAULT_APP_NAME = "Grok App";
export const OG_SERVICE_URL_DEFAULT = "https://og.grok.me";
export const OG_SITE_REL_PATH = "src/lib/og/site.json";

const SHARE_META_KEYS = new Set([
  "og:title",
  "og:description",
  "og:image",
  "og:image:width",
  "og:image:height",
  "og:type",
  "og:url",
  "og:site_name",
  "twitter:card",
  "twitter:title",
  "twitter:image",
  "twitter:description",
  "x:game:image",
  "x:game:image:width",
  "x:game:image:height",
]);

export function escapeHtml(value) {
  return String(value)
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll('"', "&quot;")
    .replaceAll("'", "&#39;");
}

/** Inverse of escapeHtml. Decode &amp; last so a single pass undoes one encode. */
function unescapeHtml(value) {
  return String(value)
    .replaceAll("&lt;", "<")
    .replaceAll("&gt;", ">")
    .replaceAll("&quot;", '"')
    .replaceAll("&#39;", "'")
    .replaceAll("&amp;", "&");
}

/** 6-digit hex for the og.grok.me placeholder, or "" if site.color is missing/invalid. */
function placeholderCardColor(site = {}) {
  const raw = String(site.color ?? "").trim();
  const hex = raw.startsWith("#") ? raw.slice(1) : raw;
  return /^[0-9a-fA-F]{6}$/.test(hex) ? hex : "";
}

/**
 * "wild-race.grok.me" → "Wild Race". Only published app hosts encode the
 * display name in the first label. Preview / guest hosts are image origins
 * only — slugifying them produced internal names like "Hds Abc 3000 Xy".
 */
export function appNameFromHost(hostHeader) {
  const host = String(hostHeader ?? "")
    .split(",")[0]
    .trim()
    .split(":")[0]
    .toLowerCase();
  if (!host.endsWith(".grok.me")) {
    return DEFAULT_APP_NAME;
  }
  const slug = host.split(".")[0] ?? "";
  if (!slug || slug === "www" || !/^[a-z0-9-]{1,63}$/.test(slug)) {
    return DEFAULT_APP_NAME;
  }
  return (
    slug
      .split("-")
      .filter(Boolean)
      .map((part) => part.charAt(0).toUpperCase() + part.slice(1))
      .join(" ") || DEFAULT_APP_NAME
  );
}

/** True for Vercel system domains. Envoy rewrites origin Host to these; they SSO-protect `/og.jpg`. */
function isVercelSystemHost(host) {
  return (
    host === "vercel.app" ||
    host.endsWith(".vercel.app") ||
```

```
    host === "vercel.com" ||
    host.endsWith(".vercel.com")
  );
}

/** Hostname suitable for absolute og:image URLs. Preview guests (X-Forwarded-Host) are allowed. */
export function publicAppHost(hostHeader) {
  const host = String(hostHeader ?? "")
    .split(",")[0]
    .trim()
    .split(":")[0]
    .toLowerCase();
  if (!host || !/^[a-z0-9.-]+$/.test(host) || !host.includes(".")) return "";
  if (/^\d{1,3}(?:\.\d{1,3}){3}$/.test(host)) return "";
  if (isVercelSystemHost(host)) return "";
  return host;
}

/**
 * Published apps always use `VITE_PUBLIC_HOSTNAME` (the grok.me host the
 * deployer injects). Live preview has no such env, so fall back to the
 * request host / X-Forwarded-Host. Never prefer request Host on a published
 * app — Envoy rewrites it to `*.vercel.app`.
 */
export function resolvePublicHost(hostHeader) {
  return (
    publicAppHost(process.env?.VITE_PUBLIC_HOSTNAME) || publicAppHost(hostHeader)
  );
}

export function isInstallQuery(url) {
  const query = String(url ?? "").split("?", 2)[1] ?? "";
  const params = new URLSearchParams(query);
  const install = params.get("install");
  const platform = (params.get("platform") ?? "").toLowerCase();
  return (install === "1" || install === "true") && platform === "ios";
}

/** Paths that can carry an app document (vs assets / API / internals). */
export function isDocumentPath(pathname) {
  const path = String(pathname ?? "");
  return (
    !path.startsWith("/__grok/") &&
    !path.startsWith("/api/") &&
    !path.startsWith("/@") &&
    !path.startsWith("/node_modules") &&
    !/\.[a-z0-9]+$/i.test(path)
  );
}

export function acceptsHtml(accept) {
  const value = String(accept ?? "");
  return value === "" || value.includes("text/html") || value.includes("*/*");
}

/** The same URL without the install-tutorial params (used as the app link). */
export function stripInstallParams(url) {
  const [path = "/", query = ""] = String(url ?? "/").split("?", 2);
  const params = new URLSearchParams(query);
  params.delete("install");
  params.delete("platform");
  const rest = params.toString();
  return rest ? `${path}?${rest}` : path;
}

export function renderInstallPageHtml(template, { host, url } = {}) {
  return String(template)
    .replaceAll("{{APP_NAME}}", escapeHtml(appNameFromHost(host)))
    .replaceAll("{{APP_URL}}", escapeHtml(stripInstallParams(url)));
}

export function renderWebManifest(hostHeader) {
  const name = appNameFromHost(hostHeader);
  return JSON.stringify(
    {
      name,
      short_name: name,
      id: "/",
      start_url: "/",
      scope: "/",
      display: "standalone",
      background_color: "#000000",
      theme_color: "#000000",
      icons: [
        {
          src: "/__grok/icon-180.png",
          sizes: "180x180",
          type: "image/png",
        },
      ],
```

```
    },
    null,
    2,
  );
}

export function grokPwaHeadTags(appName = DEFAULT_APP_NAME) {
  return [
    // Standalone display comes from the manifest ("display": "standalone");
    // the legacy *-web-app-capable metas it replaces are deliberately absent.
    ["manifest", '<link rel="manifest" href="/__grok/manifest.webmanifest">'],
    ["apple-touch-icon", '<link rel="apple-touch-icon" href="/__grok/icon-180.png">'],
    [
      "apple-mobile-web-app-title",
      `<meta name="apple-mobile-web-app-title" content="${escapeHtml(appName)}">`,
    ],
    [
      "apple-mobile-web-app-status-bar-style",
      '<meta name="apple-mobile-web-app-status-bar-style" content="black">',
    ],
    ["theme-color", '<meta name="theme-color" content="#000000">'],
  ];
}

export const GROK_EXTENSIONS_SCRIPT_SRC = "https://grok.com/grok-app-builder/extensions.js";

export function readGrokProjectId() {
  const fromProcess = typeof process !== "undefined" ? process.env?.VITE_PROJECT_ID : "";
  return String(fromProcess ?? "").trim();
}

export function readGrokExtensionsEnabled() {
  const fromProcess = typeof process !== "undefined" ? process.env?.VITE_GROK_EXTENSIONS : "";
  return String(fromProcess ?? "").trim() !== "0";
}

export function readXCreator() {
  const fromProcess = typeof process !== "undefined" ? process.env?.X_CREATOR : "";
  return String(fromProcess ?? "").trim();
}

export function readXCreatorId() {
  const fromProcess = typeof process !== "undefined" ? process.env?.X_CREATOR_ID : "";
  return String(fromProcess ?? "").trim();
}

export function grokXCreatorHeadTags(creator = readXCreator(), creatorId = readXCreatorId()) {
  const name = String(creator ?? "").trim();
  const id = String(creatorId ?? "").trim();
  if (!name || !id) return [];
  return [
    `<meta property="x:creator" content="${escapeHtml(name)}">`,
    `<meta property="x:creator:id" content="${escapeHtml(id)}">`,
  ];
}

/** Platform "Created with Grok" banner — injected into every HTML document. */
export function grokExtensionsHeadTags(projectId = readGrokProjectId()) {
  const id = escapeHtml(projectId);
  const tags = [];
  if (projectId) {
    tags.push(`<meta name="grok-project-id" content="${id}">`);
  }
  if (!readGrokExtensionsEnabled()) return tags;
  tags.push(
    `<script src="${GROK_EXTENSIONS_SCRIPT_SRC}"${
      projectId ? ` data-project-id="${id}"` : ""
    } defer></script>`,
  );
  return tags;
}

export function readOgSite(cwd = process.cwd()) {
  try {
    const raw = readFileSync(join(cwd, OG_SITE_REL_PATH), "utf8");
    const parsed = JSON.parse(raw);
    return parsed && typeof parsed === "object" && !Array.isArray(parsed) ? parsed : {};
  } catch {
    return {};
  }
}

/** Public path of an on-disk share card, or "" if neither file exists. */
export function ogCardPublicPath(cwd = process.cwd()) {
  if (existsSync(join(cwd, "public/og.jpg"))) return "/og.jpg";
  if (existsSync(join(cwd, "public/og.png"))) return "/og.png";
  return "";
}

function detectCustomOgCard(cwd = process.cwd(), site = {}) {
```

```
  if (ogCardPublicPath(cwd)) return true;
  // Vercel runtime has no public/: trust a bake that already saw the file.
  return siteHasCustomCard(site) || Boolean(String(site.image ?? "").trim());
}

/** Snapshot for Vite/Nitro to bake into the server bundle (Vercel has no workspace FS). */
export function snapshotOgIdentity(cwd = process.cwd()) {
  const site = { ...readOgSite(cwd) };
  const disk = ogCardPublicPath(cwd);
  if (disk) {
    site.card = "custom";
    site.image = disk;
  } else {
    // site.json `card=custom` without a file must not bake a 404 /og.jpg URL.
    if (siteHasCustomCard(site)) delete site.card;
    if (site.image) delete site.image;
  }
  if (existsSync(join(cwd, "public/x-banner.jpg"))) {
    site.banner = site.banner || "/x-banner.jpg";
  }
  return { site };
}

export function customOgAssetPath(cwd = process.cwd()) {
  return ogCardPublicPath(cwd) || "/og.jpg";
}

export function ogServiceUrl() {
  const fromEnv = String(process.env?.VITE_OG_SERVICE_URL ?? "").trim();
  return (fromEnv || OG_SERVICE_URL_DEFAULT).replace(/\/+$/, "");
}

export function titleFromDocument(html) {
  const match = String(html ?? "").match(/<title\b[^>]*>([^<]*)<\/title>/i);
  return match ? unescapeHtml(match[1]).trim() : "";
}

export function resolveOgTitle(
  site = {},
  appName = DEFAULT_APP_NAME,
  host = "",
  documentTitle = "",
) {
  const fromSite = String(site.title ?? "").trim();
  if (fromSite) return fromSite;
  const fromDoc = String(documentTitle ?? "").trim();
  if (fromDoc) return fromDoc;
  const fromHost = appNameFromHost(host);
  if (fromHost && fromHost !== DEFAULT_APP_NAME) return fromHost;
  const fromArg = String(appName ?? "").trim();
  return fromArg || DEFAULT_APP_NAME;
}

export function siteHasCustomCard(site = {}) {
  return String(site.card ?? "").toLowerCase() === "custom";
}

/**
 * Preview: public/og.jpg|png on disk.
 * Vercel: the bake (`card=custom` / `image`) because the function cannot stat public/.
 * Otherwise empty — caller emits the og.grok.me placeholder.
 */
export function resolveOgCardAsset(site = {}, cwd = process.cwd()) {
  return ogCardPublicPath(cwd) || (detectCustomOgCard(cwd, site) ? String(site.image ?? "").trim() || "/og.j
pg" : "");
}

/** Stamp `card=custom` when public/og.jpg or public/og.png is on disk. */
function applyCustomCardFromFs(site, cwd) {
  const disk = ogCardPublicPath(cwd);
  if (!disk) return site;
  return { ...site, card: "custom", image: disk };
}

export function grokOgHeadTags({
  host = "",
  appName = DEFAULT_APP_NAME,
  site = {},
  documentTitle = "",
  cwd = process.cwd(),
} = {}) {
  const title = resolveOgTitle(site, appName, host, documentTitle);
  const publicHost = resolvePublicHost(host);
  const tags = [
    `<meta name="twitter:card" content="summary_large_image">`,
    `<meta property="og:title" content="${escapeHtml(title)}">`,
  ];
  const description = String(site.description ?? "").trim();
  if (description) {
    tags.push(`<meta property="og:description" content="${escapeHtml(description)}">`);
```

```
  }
  if (String(site.type ?? "").toLowerCase() === "x:game") {
    tags.push(`<meta property="og:type" content="x:game">`);
  }
  if (publicHost) {
    const asset = resolveOgCardAsset(site, cwd);
    const custom = Boolean(asset);
    let image = custom
      ? `https://${publicHost}${asset.startsWith("/") ? asset : `/${asset}`}`
      : `${ogServiceUrl()}/v1/card.png?host=${encodeURIComponent(publicHost)}&title=${encodeURIComponent(tit
le)}`;
    const color = !custom ? placeholderCardColor(site) : "";
    if (color) image += `&color=${encodeURIComponent(color)}`;
    tags.push(`<meta property="og:image" content="${escapeHtml(image)}">`);
    tags.push(`<meta property="og:image:width" content="1200">`);
    tags.push(`<meta property="og:image:height" content="630">`);
    const banner = String(site.banner ?? "").trim();
    if (banner) {
      const bannerUrl = `https://${publicHost}${banner.startsWith("/") ? banner : `/${banner}`}`;
      tags.push(`<meta property="x:game:image" content="${escapeHtml(bannerUrl)}">`);
      tags.push(`<meta property="x:game:image:width" content="1200">`);
      tags.push(`<meta property="x:game:image:height" content="264">`);
    }
  }
  return tags;
}

function stripGrokExtensionsScript(html) {
  return String(html).replace(
    /<script\b[^>]*\bsrc\s*=\s*["'][^"']*\/grok-app-builder\/extensions\.js[^"']*["'][^>]*>\s*<\/script>/gi,
    "",
  );
}

export function stripShareMetaTags(html) {
  return String(html).replace(/<meta\b[^>]*>/gi, (tag) => {
    const attrs = [...tag.matchAll(/\b(?:property|name)\s*=\s*["']([^"']+)["']/gi)];
    for (const match of attrs) {
      if (SHARE_META_KEYS.has(String(match[1]).toLowerCase())) return "";
    }
    return tag;
  });
}

function insertAfterHeadOpen(html, snippet) {
  if (/<head\b[^>]*>/i.test(html)) {
    return html.replace(/<head\b[^>]*>/i, (open) => `${open}${snippet}`);
  }
  if (/<html\b[^>]*>/i.test(html)) {
    return html.replace(/<html\b[^>]*>/i, (open) => `${open}<head>${snippet}</head>`);
  }
  return `<!doctype html><html><head>${snippet}</head>${html}`;
}

function insertBeforeHeadClose(html, snippet) {
  if (/<\/head>/i.test(html)) return html.replace(/<\/head>/i, `${snippet}</head>`);
  return insertAfterHeadOpen(html, snippet);
}

export function normalizeHeadContext(ctx = {}) {
  const cwd = ctx.cwd ?? process.cwd();
  // Middleware passes a baked `site`. Still consult the workspace so a
  // public/og.jpg generated after that snapshot (or missed by a wrong cwd)
  // wins over the og.grok.me placeholder. Vercel has no public/ to read, so
  // a correct bake is unchanged.
  const site = applyCustomCardFromFs(
    ctx.site !== undefined ? ctx.site : snapshotOgIdentity(cwd).site,
    cwd,
  );
  const appName = resolveOgTitle(site, ctx.appName ?? DEFAULT_APP_NAME, ctx.host ?? "");
  return {
    appName,
    projectId: ctx.projectId ?? readGrokProjectId(),
    creator: ctx.creator ?? readXCreator(),
    creatorId: ctx.creatorId ?? readXCreatorId(),
    host: ctx.host ?? "",
    cwd,
    site,
  };
}

export function injectGrokPwaHead(html, ctx = {}) {
  if (typeof html !== "string") return html;
  const { site, projectId, creator, creatorId, host, cwd } = normalizeHeadContext(ctx);
  const documentTitle = titleFromDocument(html);
  const appName = resolveOgTitle(
    site,
    ctx.appName ?? DEFAULT_APP_NAME,
    host,
    documentTitle,
```

```
  );
  let next = stripShareMetaTags(html);
  if (!readGrokExtensionsEnabled()) next = stripGrokExtensionsScript(next);

  const missing = grokPwaHeadTags(appName)
    .filter(([key]) => {
      if (key === "manifest") return !next.includes('href="/__grok/manifest.webmanifest"');
      if (key === "apple-touch-icon") return !next.includes('href="/__grok/icon-180.png"');
      return !next.includes(`name="${key}"`);
    })
    .map(([, tag]) => tag);

  next = insertAfterHeadOpen(
    next,
    grokOgHeadTags({ host, appName, site, documentTitle, cwd }).join(""),
  );

  if (readGrokExtensionsEnabled() && !next.includes("/grok-app-builder/extensions.js")) {
    missing.push(...grokExtensionsHeadTags(projectId));
  } else if (projectId && !next.includes('name="grok-project-id"')) {
    missing.push(`<meta name="grok-project-id" content="${escapeHtml(projectId)}">`);
  }
  if (
    projectId &&
    !next.includes('property="grok:app_id"') &&
    !next.includes("property='grok:app_id'")
  ) {
    missing.push(`<meta property="grok:app_id" content="${escapeHtml(projectId)}">`);
  }
  const creatorTags = grokXCreatorHeadTags(creator, creatorId);
  if (creatorTags.length > 0) {
    const hasCreator =
      next.includes('property="x:creator" content=') ||
      next.includes("property='x:creator' content=");
    if (!hasCreator) missing.push(creatorTags[0]);
    if (!next.includes('property="x:creator:id"')) missing.push(creatorTags[1]);
  }

  if (missing.length === 0) return next;
  return insertBeforeHeadClose(next, missing.join(""));
}

function findHeadClose(buf) {
  const at = buf.toString("latin1").search(/<\/head>/i);
  return at;
}

/**
 * Streaming head injector: buffers only until `</head>` (ASCII marker; never
 * appears inside a UTF-8 continuation byte), overwrites share-card metas,
 * then passes later chunks through so streaming SSR keeps streaming.
 */
export function createHeadInjector(ctx = {}) {
  const normalized = normalizeHeadContext(ctx);

  /** @type {Buffer[]} */
  let pending = [];
  let done = false;

  const apply = (html) =>
    injectGrokPwaHead(html, {
      appName: normalized.appName,
      projectId: normalized.projectId,
      creator: normalized.creator,
      creatorId: normalized.creatorId,
      host: normalized.host,
      cwd: normalized.cwd,
      site: normalized.site,
    });

  return {
    /** @param {Uint8Array | string} chunk @returns {Buffer[]} chunks ready to emit */
    push(chunk) {
      const buf = Buffer.isBuffer(chunk) ? chunk : Buffer.from(chunk);
      if (done) return [buf];
      pending.push(buf);
      const joined = Buffer.concat(pending);
      const at = findHeadClose(joined);
      if (at === -1) return [];
      done = true;
      pending = [];
      const closeLen = joined.toString("latin1", at).match(/^<\/head>/i)[0].length;
      const head = apply(joined.subarray(0, at + closeLen).toString("utf8"));
      return [Buffer.concat([Buffer.from(head, "utf8"), joined.subarray(at + closeLen)])];
    },
    /** @returns {Buffer[]} whatever is still buffered (no `</head>` seen) */
    flush() {
      if (done || pending.length === 0) return [];
      const rest = Buffer.concat(pending);
      pending = [];
```

```
      done = true;
      return [Buffer.from(apply(rest.toString("utf8")), "utf8")];
    },
  };
}
```

scripts/install-page.html

```
<!DOCTYPE html>
<html lang="en" class="device-desktop">
  <head>
    <meta charset="utf-8" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1, viewport-fit=cover"
    />
    <meta name="color-scheme" content="dark" />
    <meta name="theme-color" content="#000000" />
    <meta name="apple-mobile-web-app-status-bar-style" content="black" />
    <meta name="apple-mobile-web-app-title" content="{{APP_NAME}}" />
    <title>Add {{APP_NAME}} to your Home Screen</title>
    <link rel="manifest" href="/__grok/manifest.webmanifest" />
    <link rel="apple-touch-icon" href="/__grok/icon-180.png" />
    <link rel="stylesheet" href="/__grok/install/styles.css" />
    <script>
      (function () {
        var ua = navigator.userAgent || "";
        var touch = navigator.maxTouchPoints || 0;
        var isiPad = /iPad/.test(ua) || (/Macintosh/.test(ua) && touch > 1);
        var isiPhone = /iPhone|iPod/.test(ua);
        var isIOS = isiPhone || isiPad;
        var isAndroid = /Android/i.test(ua);
        var isAndroidPhone = isAndroid && /Mobile/i.test(ua);
        var isAndroidTablet = isAndroid && !/Mobile/i.test(ua);
        var minSide = Math.min(screen.width || 0, screen.height || 0);
        var maxSide = Math.max(screen.width || 0, screen.height || 0);

        var type = "desktop";
        if (isiPhone) type = "phone";
        else if (isiPad || isAndroidTablet) type = "tablet";
        else if (isAndroidPhone) type = "phone";
        else if (touch > 0 && minSide > 0 && minSide <= 500) type = "phone";
        else if (touch > 0 && minSide > 500 && maxSide <= 1400) type = "tablet";

        var iosMajor = null;
        var osToken = null;
        var safariToken = null;
        var iphoneOs = ua.match(/iPhone OS (\d+)[._]/);
        var ipadOs = ua.match(/CPU OS (\d+)[._](\d+) like Mac OS X/);
        var safariVer = ua.match(/Version\/(\d+)[._]/);
        if (iphoneOs) osToken = parseInt(iphoneOs[1], 10);
        else if (ipadOs) osToken = parseInt(ipadOs[1], 10);
        if (isIOS && safariVer) safariToken = parseInt(safariVer[1], 10);
        if (osToken != null || safariToken != null) {
          iosMajor = Math.max(osToken || 0, safariToken || 0);
        }

        var root = document.documentElement;
        var classes = ["device-" + type];
        if (iosMajor != null) {
          root.dataset.ios = String(iosMajor);
          classes.push(iosMajor >= 27 ? "ios-27-plus" : "ios-below-27");
        }
        root.className = classes.join(" ");
      })();
    </script>
  </head>
  <body>
    <div class="page">
      <header class="powered" aria-label="Powered by Grok">
        <span class="powered-by">Powered by</span>
        <span class="powered-brand">
          <img
            class="grok-logo"
            src="/__grok/install/assets/homescreen/logo-grok.svg"
            width="14"
            height="14"
            alt=""
          />
          <span class="powered-grok">Grok</span>
        </span>
      </header>

      <main class="content">
        <div class="ob" aria-hidden="true">
          <img
            class="ob-img ob-phone"
            src="/__grok/install/assets/homescreen/ob-phone.png"
            width="338"
            height="294"
            alt=""
          />
          <img
            class="ob-img ob-ipad"
            src="/__grok/install/assets/homescreen/ob-ipad.png"
            width="634"
```

```
            height="294"
            alt=""
          />
        </div>

        <section class="copy">
          <h1>Add {{APP_NAME}} to your Home Screen</h1>

          <div class="steps">
            <p class="step step-tap step-ios27">
              <span class="muted">Tap</span>
              <span class="glass glass--icon" aria-hidden="true">
                <img src="/__grok/install/assets/homescreen/glass-puzzle.svg" width="24" height="24" alt=""
/>
              </span>
              <span class="muted loc loc-phone">in the bottom bar, then</span>
              <span class="muted loc loc-ipad">in the tool bar, then</span>
              <span class="glass glass--icon" aria-hidden="true">
                <img src="/__grok/install/assets/homescreen/glass-share.svg" width="24" height="24" alt="" /
>
              </span>
            </p>

            <p class="step step-tap step-ios-legacy">
              <span class="muted">Tap</span>
              <span class="glass glass--icon" aria-hidden="true">
                <img src="/__grok/install/assets/homescreen/glass-share.svg" width="24" height="24" alt="" /
>
              </span>
              <span class="muted loc loc-phone">in the bottom bar</span>
              <span class="muted loc loc-ipad">in the tool bar</span>
            </p>

            <p class="step step-select">
              <span class="muted">Select</span>
              <span class="add-label">
                <img
                  class="plus-icon"
                  src="/__grok/install/assets/homescreen/plus.svg"
                  width="16"
                  height="16"
                  alt=""
                />
                <span class="add-text">Add to Home Screen</span>
              </span>
            </p>
          </div>
        </section>
      </main>

      <main class="content content-desktop">
        <section class="copy">
          <h1>Open this link on your iPhone or iPad</h1>
          <p class="desktop-note">
            This page shows how to add {{APP_NAME}} to an iOS Home Screen.
          </p>
          <a class="desktop-open" href="{{APP_URL}}">Open {{APP_NAME}}</a>
        </section>
      </main>
    </div>
  </body>
</html>
```

scripts/migrate.mjs

```
#!/usr/bin/env node
/**
 * Deploy-time database migrator (node-postgres, `pg`).
 *
 * Runs during `npm run build` — on every Vercel deploy — applying pending files
 * in ../migrations to DATABASE_URL. Each file is applied in one transaction and
 * recorded in a `_migrations` table, so it runs once and is safe to re-run.
 *
 * The read is non-recursive, so the opt-in auth schema under migrations/auth/
 * is not applied to an app that never asked for sign-in.
 *
 * No DATABASE_URL (local / preview builds) -> skip; the PGLite fallback applies
 * the same files at startup instead (see src/lib/db.ts).
 */
import { readdir, readFile } from "node:fs/promises";
import { fileURLToPath } from "node:url";
import { dirname, join } from "node:path";
import pg from "pg";
import { pendingMigrations } from "./migration-plan.mjs";

const databaseUrl = process.env.DATABASE_URL;
if (!databaseUrl) {
  console.log(
    "[migrate] DATABASE_URL not set — skipping (the PGLite fallback migrates itself).",
  );
  process.exit(0);
}

const migrationsDir = join(dirname(fileURLToPath(import.meta.url)), "..", "migrations");

async function main() {
  let entries;
  try {
    entries = await readdir(migrationsDir);
  } catch {
    console.log("[migrate] no migrations/ directory — nothing to do.");
    return;
  }
  // An app with no schema of its own must not pay for a database connection.
  if (pendingMigrations(entries, []).length === 0) {
    console.log("[migrate] no migrations — nothing to do.");
    return;
  }

  const pool = new pg.Pool({ connectionString: databaseUrl, max: 1 });
  const client = await pool.connect();
  try {
    await client.query(
      "CREATE TABLE IF NOT EXISTS _migrations (name TEXT PRIMARY KEY, applied_at TIMESTAMPTZ NOT NULL DEFAUL
T now())",
    );
    const applied = (await client.query("SELECT name FROM _migrations")).rows.map(
      (r) => r.name,
    );

    let count = 0;
    for (const { name } of pendingMigrations(entries, applied)) {
      const text = await readFile(join(migrationsDir, name), "utf8");
      try {
        await client.query("BEGIN");
        // pg's simple-query protocol runs a whole multi-statement file at once.
        await client.query(text);
        await client.query("INSERT INTO _migrations (name) VALUES ($1)", [name]);
        await client.query("COMMIT");
      } catch (err) {
        console.error(`[migrate] error applying ${name}`);
        try {
          await client.query("ROLLBACK");
        } catch {
          // ROLLBACK fails when the connection died — keep the original error.
        }
        throw err;
      }
      console.log(`[migrate] applied ${name}`);
      count += 1;
    }
    console.log(count ? `[migrate] done — ${count} migration(s) applied.` : "[migrate] up to date.");
  } finally {
    client.release();
    await pool.end();
  }
}

main().catch((err) => {
  console.error("[migrate] failed:", err?.message || err);
  // pg errors carry the context needed to debug a bad SQL file.
  for (const key of ["code", "detail", "hint", "position", "where"]) {
    if (err?.[key] != null) console.error(`[migrate]   ${key}: ${err[key]}`);
```

```
  }
  process.exit(1);
});
```

scripts/migration-plan.mjs

```
// @ts-check
/**
 * Migration bookkeeping shared by the two appliers — `scripts/migrate.mjs`
 * (deploy, `readdir`) and `src/lib/db.ts` (PGLite preview, `import.meta.glob`).
 *
 * Applied files are keyed by BASENAME, so the same file applies once no matter
 * which directory it is globbed from. That is what makes the auth schema safe to
 * copy from `migrations/auth/` into `migrations/` when an app turns sign-in on:
 * a database that already has `0001_auth.sql` will not re-run it.
 *
 * Neither applier descends into subdirectories, so `migrations/auth/*.sql` is
 * out of scope for both until it is copied up.
 */

/**
 * The `_migrations` key for a migration path (or bare filename).
 * @param {string} path
 * @returns {string}
 */
export function migrationName(path) {
  return path.split("/").pop() ?? path;
}

/**
 * @param {string} path
 * @returns {boolean}
 */
export function isMigrationFile(path) {
  return path.endsWith(".sql");
}

/**
 * Migrations in `paths` that are not yet in `applied`, in apply order.
 * Non-`.sql` entries (a `readdir` also yields `migrations/auth/`) are dropped.
 * @param {Iterable<string>} paths
 * @param {Iterable<string>} applied
 * @returns {Array<{ name: string, path: string }>}
 */
export function pendingMigrations(paths, applied) {
  const done = new Set(applied);
  return [...paths]
    .filter(isMigrationFile)
    .map((path) => ({ name: migrationName(path), path }))
    .sort((a, b) => a.name.localeCompare(b.name))
    .filter(({ name }) => !done.has(name));
}
```

scripts/migration-plan.test.mjs

```
import assert from "node:assert/strict";
import {
  existsSync,
  mkdirSync,
  mkdtempSync,
  readFileSync,
  readdirSync,
  writeFileSync,
} from "node:fs";
import { tmpdir } from "node:os";
import { join } from "node:path";
import { test } from "node:test";
import { isMigrationFile, migrationName, pendingMigrations } from "./migration-plan.mjs";
import { projectRoot } from "./with-app-env.mjs";

const AUTH_MIGRATION = "0001_auth.sql";

/**
 * The auth-on copy of the Better Auth schema and its source, or null when the
 * app has not turned sign-in on (the shipped state).
 */
function authSchemaCopy(root) {
  const copy = join(root, "migrations", AUTH_MIGRATION);
  const source = join(root, "migrations/auth", AUTH_MIGRATION);
  if (!existsSync(copy) || !existsSync(source)) return null;
  return { copy: readFileSync(copy, "utf8"), source: readFileSync(source, "utf8") };
}

test("_migrations keys on basename, not path", () => {
  assert.equal(migrationName("/migrations/0002_todos.sql"), "0002_todos.sql");
  assert.equal(migrationName("migrations/auth/0001_auth.sql"), "0001_auth.sql");
  assert.equal(migrationName("0001_auth.sql"), "0001_auth.sql");
});

test("a file already applied from another directory does not re-apply", () => {
  // The auth-on path copies migrations/auth/0001_auth.sql into the globbed
  // directory; a database that already has it must not run it twice.
  assert.deepEqual(pendingMigrations(["/migrations/0001_auth.sql"], ["0001_auth.sql"]), []);
});

test("pending migrations are returned in name order", () => {
  assert.deepEqual(
    pendingMigrations(
      ["/migrations/0003_c.sql", "/migrations/0001_a.sql", "/migrations/0002_b.sql"],
      ["0001_a.sql"],
    ),
    [
      { name: "0002_b.sql", path: "/migrations/0002_b.sql" },
      { name: "0003_c.sql", path: "/migrations/0003_c.sql" },
    ],
  );
});

test("non-.sql entries are dropped (readdir also yields the auth/ directory)", () => {
  assert.equal(isMigrationFile("auth"), false);
  assert.deepEqual(pendingMigrations(["auth", "README.md"], []), []);
});

test("the auth schema ships outside the globbed directory", () => {
  const migrationsDir = join(projectRoot(), "migrations");
  assert.deepEqual(pendingMigrations(readdirSync(migrationsDir), []), []);
  assert.ok(readdirSync(join(migrationsDir, "auth")).includes("0001_auth.sql"));
});

test("this workspace's auth schema copy is byte-identical to its source", () => {
  // An edited copy diverges silently: basename keying skips it on a database
  // that already ran the original, and applies it on a fresh PGLite preview.
  const pair = authSchemaCopy(projectRoot());
  if (pair === null) return; // sign-in off — nothing has been copied up
  assert.equal(
    pair.copy,
    pair.source,
    "migrations/0001_auth.sql has been edited — it must stay a verbatim copy of migrations/auth/0001_auth.sq
l",
  );
});

test("the copy check reads both files and catches an edit", () => {
  const root = mkdtempSync(join(tmpdir(), "auth-schema-"));
  mkdirSync(join(root, "migrations/auth"), { recursive: true });
  writeFileSync(join(root, "migrations/auth", AUTH_MIGRATION), "create table t ();\n");
  assert.equal(authSchemaCopy(root), null);

  writeFileSync(join(root, "migrations", AUTH_MIGRATION), "create table t ();\n");
  const same = authSchemaCopy(root);
  assert.equal(same.copy, same.source);

  writeFileSync(join(root, "migrations", AUTH_MIGRATION), "create table t (x int);\n");
```

```
  const drifted = authSchemaCopy(root);
  assert.notEqual(drifted.copy, drifted.source);
});
```

scripts/preview-thumbnail.mjs

```
#!/usr/bin/env node
// Capture a 1280x800 preview PNG of the dev server (argv[2] -> argv[3]).
// Contract with SandboxInternal.CapturePreviewThumbnail: exit 0 only after the
// PNG is written; the service treats any non-zero exit as a gated skip and does
// not download the file.
import { chromium } from "playwright";
import { checkedOutputPath, checkedUrl } from "./browser-guard.mjs";

// The service always passes a loopback URL and a /tmp path; the checks keep that
// true when the script is invoked by hand.
const url = checkedUrl(process.argv[2] || "http://127.0.0.1:8080/");
const outPng = checkedOutputPath(process.argv[3] || "/tmp/preview-thumbnail.png", [
  "/tmp",
  "/workspace",
]);
const timeoutMs = Number(process.env.PREVIEW_THUMBNAIL_TIMEOUT_MS || 45000);

const browser = await chromium.launch({
  headless: true,
  args: ["--no-sandbox", "--disable-dev-shm-usage"],
});

try {
  const page = await browser.newPage({ viewport: { width: 1280, height: 800 } });

  // `domcontentloaded`, not `networkidle`: Vite keeps an HMR websocket open, so
  // networkidle never settles and would burn the whole timeout.
  const resp = await page.goto(url, { waitUntil: "domcontentloaded", timeout: timeoutMs });
  const status = resp?.status() ?? 0;
  await page.waitForTimeout(1000);

  await page.screenshot({ path: outPng, fullPage: false });

  console.log(JSON.stringify({ url, status, screenshot: outPng }, null, 2));
} catch (err) {
  console.error(JSON.stringify({ ok: false, url, error: String(err?.message || err) }, null, 2));
  // Set the code rather than process.exit() so the `finally` browser teardown
  // always runs (avoids leaking Chromium across repeated capture calls).
  process.exitCode = 1;
} finally {
  await browser.close();
}
```

scripts/preview.mjs

```
#!/usr/bin/env node
/**
 * Owns :8081, the built-output QA preview.
 *
 * `vite preview` is strictPort, so a preview left over from an earlier turn
 * both fails the next start and keeps serving the previous build's output.
 * Every restart therefore kills the current port owner first, whoever started
 * it. Owners come from /proc, so this runs only inside the Linux sandbox.
 *
 *   node scripts/preview.mjs stop|restart
 */
import { spawn } from "node:child_process";
import {
  existsSync,
  mkdirSync,
  openSync,
  readFileSync,
  readdirSync,
  readlinkSync,
  rmSync,
  writeFileSync,
} from "node:fs";
import { dirname, join } from "node:path";
import { fileURLToPath, pathToFileURL } from "node:url";

const PREVIEW_PORT = 8081;
const PREVIEW_URL = `http://127.0.0.1:${PREVIEW_PORT}/`;

const ROOT = join(dirname(fileURLToPath(import.meta.url)), "..");
const PID_FILE = join(ROOT, ".grok/preview.pid");
const LOG_FILE = join(ROOT, ".grok/preview.log");
const READY_TIMEOUT_MS = Number(process.env.PREVIEW_READY_TIMEOUT_MS || 60000);
const GRACE_MS = 3000;
const POLL_MS = 100;

export function parsePreviewArgs(argv) {
  const [action, ...rest] = argv;
  if (!action) return { error: "usage: node scripts/preview.mjs stop|restart" };
  if (rest.length > 0) return { error: `unexpected argument: ${rest[0]}` };
  if (!["stop", "restart"].includes(action)) {
    return { error: `unknown action: ${action} (expected stop or restart)` };
  }
  return { action };
}

export function parsePid(text) {
  const pid = Number.parseInt(String(text ?? "").trim(), 10);
  // pid 1 is the sandbox init — never the preview, and dangerous to signal.
  return Number.isInteger(pid) && pid > 1 ? pid : null;
}

/** pgid of a process from its /proc/<pid>/stat line. */
export function parsePgid(stat) {
  const line = String(stat ?? "");
  // The comm field is parenthesised and may itself contain spaces; state, ppid
  // and pgrp are the three fields after it.
  const end = line.lastIndexOf(") ");
  if (end === -1) return null;
  const pgid = Number.parseInt(line.slice(end + 2).split(/\s+/)[2], 10);
  return Number.isInteger(pgid) && pgid > 0 ? pgid : null;
}

const TCP_LISTEN = "0A";

/** Socket inodes of the LISTEN sockets on `port` in a /proc/net/tcp{,6} dump. */
export function parseListenerInodes(procNetTcp, port) {
  const wanted = `:${port.toString(16).toUpperCase().padStart(4, "0")}`;
  const inodes = [];
  for (const line of String(procNetTcp ?? "").split("\n")) {
    const cols = line.trim().split(/\s+/);
    if (cols.length < 10 || cols[3] !== TCP_LISTEN || !cols[1].endsWith(wanted)) continue;
    if (/^\d+$/.test(cols[9])) inodes.push(cols[9]);
  }
  return inodes;
}

export function looksLikePreviewProcess(cmdline) {
  // /proc/<pid>/cmdline is NUL-separated.
  const argv = String(cmdline ?? "")
    .split("\0")
    .filter(Boolean)
    .join(" ");
  // The sandbox service runs scripts/preview-thumbnail.mjs in this box, and
  // this script can be running concurrently: neither is ever a target.
  if (/\bpreview[\w-]*\.mjs\b/.test(argv)) return false;
  // The `npm run preview` wrapper (`npm-cli.js run preview`) and its vite child.
  // `preview` must be the whole script name: `run preview:stop`/`preview:restart`
  // are this tooling's own wrappers, and `vite build --outDir preview-dist` is
```

```
  // not a server.
  return /\brun\s+preview(?:\s|$)/.test(argv) || /\bvite\b\s+preview\b/.test(argv);
}

/**
 * Pids to signal. Port owners are owners by definition; the pidfile pid is only
 * a claim left by an earlier run — pids are re-used across hibernate/revive, so
 * signal it only when its command line still looks like the preview.
 */
export function previewOwners({ portPids, pidFilePid, cmdlineOf }) {
  const owners = new Set(portPids);
  if (
    pidFilePid !== null &&
    !owners.has(pidFilePid) &&
    looksLikePreviewProcess(cmdlineOf(pidFilePid))
  ) {
    owners.add(pidFilePid);
  }
  return [...owners];
}

async function waitForExit(pids, { isAlive, sleep, timeoutMs, pollMs }) {
  let remaining = pids.filter((pid) => isAlive(pid));
  for (let waited = 0; remaining.length > 0 && waited < timeoutMs; waited += pollMs) {
    await sleep(pollMs);
    remaining = remaining.filter((pid) => isAlive(pid));
  }
  return remaining;
}

/**
 * SIGTERM every live pid, then SIGKILL whatever outlives the grace period.
 * Returns `{ signalled, killed, stubborn }` — `stubborn` is still alive after
 * the SIGKILL wait, which means the port is not reliably free.
 */
export async function terminatePids(
  pids,
  { kill, isAlive, sleep, graceMs = GRACE_MS, pollMs = POLL_MS },
) {
  const signalled = pids.filter((pid) => isAlive(pid));
  for (const pid of signalled) kill(pid, "SIGTERM");
  const killed = await waitForExit(signalled, { isAlive, sleep, timeoutMs: graceMs, pollMs });
  for (const pid of killed) kill(pid, "SIGKILL");
  const stubborn = await waitForExit(killed, { isAlive, sleep, timeoutMs: graceMs, pollMs });
  return { signalled, killed, stubborn };
}

/**
 * What `stop` reports. `after` is the post-kill port check: `unattributed: true`
 * means a listener exists whose pid could not be resolved, so it may not claim
 * the port is free.
 */
export function stopOutcome({ signalled, stubborn, after }) {
  const held = [...new Set([...stubborn, ...after.pids])];
  if (held.length > 0) {
    return { ok: false, error: `port ${PREVIEW_PORT} is still held by pid(s) ${held.join(", ")}` };
  }
  if (after.unattributed) {
    return {
      ok: false,
      error: `port ${PREVIEW_PORT} is held by a process this script cannot see`,
    };
  }
  const message =
    signalled.length > 0
      ? `stopped pid(s) ${signalled.join(", ")} — port ${PREVIEW_PORT} is free`
      : `nothing was listening on ${PREVIEW_PORT}`;
  return { ok: true, message };
}

const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

function isAlive(pid) {
  try {
    process.kill(pid, 0);
    return true;
  } catch (err) {
    return err?.code === "EPERM";
  }
}

function pgidOf(pid) {
  try {
    return parsePgid(readFileSync(`/proc/${pid}/stat`, "utf8"));
  } catch {
    return null;
  }
}

function killPid(pid, signal) {
```

```
  // restart() detaches the server into its own process group, so signal the
  // group to reach `vite` under the `npm` wrapper. Only for a leader: `-pid` on
  // a pid that leads no group still reaches any unrelated group numbered pid.
  if (pgidOf(pid) === pid) {
    try {
      process.kill(-pid, signal);
      return;
    } catch {
      // The group exited between the pgid read and the signal.
    }
  }
  try {
    process.kill(pid, signal);
  } catch {
    // Exited between the liveness check and the signal.
  }
}

function cmdlineOf(pid) {
  try {
    return readFileSync(`/proc/${pid}/cmdline`, "utf8");
  } catch {
    // Usually a dead pid — the stale pidfile this corroboration exists for.
    return "";
  }
}

function readPidFile() {
  try {
    return parsePid(readFileSync(PID_FILE, "utf8"));
  } catch {
    return null;
  }
}

function pidsForSocketInodes(inodes) {
  const targets = new Set([...inodes].map((inode) => `socket:[${inode}]`));
  const pids = [];
  for (const entry of readdirSync("/proc")) {
    const pid = parsePid(entry);
    if (pid === null || pid === process.pid) continue;
    let fds;
    try {
      fds = readdirSync(`/proc/${pid}/fd`);
    } catch {
      // Exited mid-scan, or owned by another user.
      continue;
    }
    for (const fd of fds) {
      try {
        if (targets.has(readlinkSync(`/proc/${pid}/fd/${fd}`))) {
          pids.push(pid);
          break;
        }
      } catch {
        // fd closed mid-scan.
      }
    }
  }
  return pids;
}

/**
 * `{ pids, unattributed }` — `unattributed: true` when the port has a listener
 * whose owning pid could not be resolved (an fd dir we may not read).
 */
function portOwners() {
  const inodes = new Set();
  for (const file of ["/proc/net/tcp", "/proc/net/tcp6"]) {
    let dump;
    try {
      dump = readFileSync(file, "utf8");
    } catch {
      // tcp6 is absent when the box has no IPv6.
      continue;
    }
    for (const inode of parseListenerInodes(dump, PREVIEW_PORT)) inodes.add(inode);
  }
  const pids = inodes.size > 0 ? pidsForSocketInodes(inodes) : [];
  return { pids, unattributed: inodes.size > 0 && pids.length === 0 };
}

async function stop(announce = true) {
  const owners = previewOwners({
    portPids: portOwners().pids,
    pidFilePid: readPidFile(),
    cmdlineOf,
  });
  const { signalled, stubborn } = await terminatePids(owners, { kill: killPid, isAlive, sleep });
```

```
  const outcome = stopOutcome({ signalled, stubborn, after: portOwners() });
  if (!outcome.ok) {
    // Keep the pidfile: a survivor the port scan cannot attribute leaves it as
    // the only record a retry could use.
    console.error(`[preview] ${outcome.error}`);
    return false;
  }
  rmSync(PID_FILE, { force: true });
  if (announce) console.log(`[preview] ${outcome.message}`);
  return true;
}

async function waitForReady(failure) {
  const deadline = Date.now() + READY_TIMEOUT_MS;
  while (Date.now() < deadline && failure() === null) {
    try {
      // Any HTTP response means the server is bound; a 404 is still ready.
      await fetch(PREVIEW_URL, { signal: AbortSignal.timeout(2000) });
      return true;
    } catch {
      await sleep(250);
    }
  }
  return false;
}

async function restart() {
  if (!(await stop())) return 1;

  mkdirSync(dirname(LOG_FILE), { recursive: true });
  const log = openSync(LOG_FILE, "a");
  const child = spawn("npm", ["run", "preview"], {
    cwd: ROOT,
    detached: true,
    stdio: ["ignore", log, log],
  });
  child.unref();
  writeFileSync(PID_FILE, `${child.pid}\n`);

  let failure = null;
  child.on("error", (err) => {
    failure = `npm run preview could not be spawned: ${err.message}`;
  });
  child.on("exit", (code, signal) => {
    failure = `npm run preview exited early (${signal ?? `code ${code}`})`;
  });

  if (!(await waitForReady(() => failure))) {
    const secs = Math.round(READY_TIMEOUT_MS / 1000);
    const why =
      failure ??
      `nothing answered on ${PREVIEW_URL} within ${secs}s — check that vite.config.ts ` +
        `still sets preview.port ${PREVIEW_PORT}`;
    console.error(`[preview] ${why} — see ${LOG_FILE}`);
    // A server that binds a few seconds later would serve a build the agent has
    // already been told to distrust.
    await stop(false);
    return 1;
  }
  console.log(`[preview] serving ${PREVIEW_URL} (pid ${child.pid}, log ${LOG_FILE})`);
  return 0;
}

if (process.argv[1] && import.meta.url === pathToFileURL(process.argv[1]).href) {
  const args = parsePreviewArgs(process.argv.slice(2));
  if (args.error) {
    console.error(`[preview] ${args.error}`);
    process.exit(1);
  }
  if (!existsSync("/proc/self")) {
    console.error("[preview] no /proc — this script only runs inside the sandbox");
    process.exit(1);
  }
  process.exitCode = args.action === "stop" ? ((await stop()) ? 0 : 1) : await restart();
}
```

scripts/preview.test.mjs

```
import assert from "node:assert/strict";
import { test } from "node:test";
import {
  looksLikePreviewProcess,
  parseListenerInodes,
  parsePgid,
  parsePid,
  parsePreviewArgs,
  previewOwners,
  stopOutcome,
  terminatePids,
} from "./preview.mjs";

test("parsePreviewArgs accepts the two actions", () => {
  for (const action of ["stop", "restart"]) {
    assert.deepEqual(parsePreviewArgs([action]), { action });
  }
});

test("parsePreviewArgs rejects missing, unknown and extra arguments", () => {
  assert.match(parsePreviewArgs([]).error, /usage:/);
  assert.match(parsePreviewArgs(["reload"]).error, /unknown action: reload/);
  assert.match(parsePreviewArgs(["start"]).error, /unknown action: start/);
  assert.match(parsePreviewArgs(["restart", "--force"]).error, /unexpected argument: --force/);
});

test("parsePid reads a pidfile and rejects junk", () => {
  assert.equal(parsePid("4321\n"), 4321);
  assert.equal(parsePid("  99  "), 99);
  assert.equal(parsePid(""), null);
  assert.equal(parsePid("not-a-pid"), null);
  assert.equal(parsePid("-7"), null);
  // pid 1 is the sandbox init, never a preview server.
  assert.equal(parsePid("1"), null);
});

test("parsePgid reads the pgrp field past a comm containing spaces", () => {
  assert.equal(parsePgid("4321 (node) S 4300 4321 4321 0 -1 4194304 1234"), 4321);
  assert.equal(parsePgid("4321 (npm run preview) S 4300 4200 4200 0 -1 0 0"), 4200);
  assert.equal(parsePgid("4321 (weird ) name) S 4300 4200 4200 0"), 4200);
  assert.equal(parsePgid("garbage"), null);
  assert.equal(parsePgid(""), null);
  assert.equal(parsePgid(undefined), null);
});

// One /proc/net/tcp row; the socket inode is column 10.
const tcpRow = (sl, local, state, inode) =>
  [
    `  ${sl}:`,
    local,
    "00000000:0000",
    state,
    "00000000:00000000",
    "00:00000000",
    "00000000",
    "1000",
    "0",
    inode,
    "1",
    "0000",
    "100",
  ].join(" ");

const PROC_NET_TCP = [
  "  sl  local_address rem_address   st tx_queue rx_queue tr tm->when retrnsmt uid timeout inode",
  tcpRow(0, "0100007F:1F91", "0A", 5551),
  tcpRow(1, "0100007F:1F90", "0A", 5552),
  tcpRow(2, "0100007F:1F91", "01", 5553),
  "",
].join("\n");

test("parseListenerInodes picks LISTEN sockets on the wanted port only", () => {
  // 0x1F91 = 8081 (LISTEN), 0x1F90 = 8080, and the third row is ESTABLISHED.
  assert.deepEqual(parseListenerInodes(PROC_NET_TCP, 8081), ["5551"]);
  assert.deepEqual(parseListenerInodes(PROC_NET_TCP, 8080), ["5552"]);
  assert.deepEqual(parseListenerInodes(PROC_NET_TCP, 9999), []);
  assert.deepEqual(parseListenerInodes("", 8081), []);
  assert.deepEqual(parseListenerInodes(undefined, 8081), []);
});

test("parseListenerInodes reads the tcp6 dump the same way", () => {
  const v6 = "00000000000000000000000000000000:1F91";
  assert.deepEqual(parseListenerInodes(tcpRow(0, v6, "0A", 7777), 8081), ["7777"]);
});

// /proc/<pid>/cmdline is NUL-separated.
const cmdline = (...argv) => argv.join("\u0000");
```

```
test("looksLikePreviewProcess matches the npm wrapper and its vite child", () => {
  const npmRun = cmdline("node", "/usr/lib/node_modules/npm/bin/npm-cli.js", "run", "preview");
  assert.equal(looksLikePreviewProcess(npmRun), true);
  assert.equal(looksLikePreviewProcess(cmdline("npm", "run", "preview")), true);
  assert.equal(
    looksLikePreviewProcess(cmdline("node", "/ws/node_modules/.bin/vite", "preview")),
    true,
  );
  // `ps -o command=` output is space-separated.
  assert.equal(looksLikePreviewProcess("node /ws/node_modules/.bin/vite preview"), true);
});

test("looksLikePreviewProcess spares the sibling scripts and re-used pids", () => {
  // The sandbox service runs this one in the same box (CapturePreviewThumbnail).
  const thumbnail = cmdline(
    "node",
    "/opt/app-template/scripts/preview-thumbnail.mjs",
    "http://127.0.0.1:8080/",
    "/tmp/preview-thumbnail.png",
  );
  assert.equal(looksLikePreviewProcess(thumbnail), false);
  assert.equal(looksLikePreviewProcess(cmdline("node", "scripts/preview.mjs", "stop")), false);
  // This tooling's own npm wrappers, which carry no `.mjs` in their cmdline.
  const npmCli = "/usr/lib/node_modules/npm/bin/npm-cli.js";
  assert.equal(looksLikePreviewProcess(cmdline("node", npmCli, "run", "preview:stop")), false);
  assert.equal(looksLikePreviewProcess(cmdline("node", npmCli, "run", "preview:restart")), false);
  assert.equal(looksLikePreviewProcess(cmdline("npm", "run", "preview:stop")), false);
  const viteBuild = cmdline("vite", "build", "--outDir", "preview-dist");
  assert.equal(looksLikePreviewProcess(viteBuild), false);
  assert.equal(looksLikePreviewProcess(cmdline("/usr/bin/preview-tool", "--x")), false);
  assert.equal(looksLikePreviewProcess(cmdline("sleep", "300")), false);
  assert.equal(looksLikePreviewProcess(cmdline("node", "server.mjs")), false);
  assert.equal(looksLikePreviewProcess(""), false);
});

test("previewOwners trusts port owners and dedupes the pidfile pid", () => {
  const owners = previewOwners({
    portPids: [50, 51],
    pidFilePid: 50,
    cmdlineOf: () => assert.fail("a port owner needs no corroboration"),
  });
  assert.deepEqual(owners, [50, 51]);
});

test("previewOwners adds a pidfile pid whose command line is still the preview", () => {
  const owners = previewOwners({
    portPids: [51],
    pidFilePid: 50,
    cmdlineOf: (pid) => (pid === 50 ? cmdline("node", "npm-cli.js", "run", "preview") : ""),
  });
  assert.deepEqual(owners, [51, 50]);
});

test("previewOwners drops a stale pidfile pid re-used by another process", () => {
  const owners = previewOwners({
    portPids: [],
    pidFilePid: 50,
    cmdlineOf: () => cmdline("sleep", "300"),
  });
  assert.deepEqual(owners, []);
});

test("previewOwners drops a pidfile pid that no longer exists", () => {
  const owners = previewOwners({ portPids: [], pidFilePid: 50, cmdlineOf: () => "" });
  assert.deepEqual(owners, []);
});

test("previewOwners on a free port with no pidfile signals nothing", () => {
  const owners = previewOwners({ portPids: [], pidFilePid: null, cmdlineOf: () => "" });
  assert.deepEqual(owners, []);
});

test("stopOutcome fails when a pid survives or the port is still held", () => {
  const stubbornOnly = stopOutcome({
    signalled: [50],
    stubborn: [50],
    after: { pids: [] },
  });
  assert.equal(stubbornOnly.ok, false);
  assert.match(stubbornOnly.error, /still held by pid\(s\) 50/);

  const newOwner = stopOutcome({
    signalled: [50],
    stubborn: [],
    after: { pids: [77] },
  });
  assert.equal(newOwner.ok, false);
  assert.match(newOwner.error, /still held by pid\(s\) 77/);
});
```

```
test("stopOutcome reports a verified free port", () => {
  const stopped = stopOutcome({
    signalled: [50],
    stubborn: [],
    after: { pids: [] },
  });
  assert.equal(stopped.ok, true);
  assert.match(stopped.message, /stopped pid\(s\) 50 — port 8081 is free/);

  const idle = stopOutcome({ signalled: [], stubborn: [], after: { pids: [] } });
  assert.equal(idle.ok, true);
  assert.match(idle.message, /nothing was listening on 8081/);
});

test("stopOutcome fails on a listener whose owner cannot be attributed", () => {
  const outcome = stopOutcome({
    signalled: [],
    stubborn: [],
    after: { pids: [], unattributed: true },
  });
  assert.equal(outcome.ok, false);
  assert.match(outcome.error, /port 8081 is held by a process this script cannot see/);
});

function fakeProcesses({ pids, ignoresTerm = [] }) {
  const alive = new Set(pids);
  const signals = [];
  return {
    signals,
    isAlive: (pid) => alive.has(pid),
    kill: (pid, signal) => {
      signals.push([pid, signal]);
      if (signal === "SIGKILL" || !ignoresTerm.includes(pid)) alive.delete(pid);
    },
    sleep: async () => {},
  };
}

test("terminatePids SIGTERMs live pids and skips dead ones", async () => {
  const fake = fakeProcesses({ pids: [11, 12] });
  const result = await terminatePids([11, 12, 13], fake);
  assert.deepEqual(result, { signalled: [11, 12], killed: [], stubborn: [] });
  assert.deepEqual(fake.signals, [
    [11, "SIGTERM"],
    [12, "SIGTERM"],
  ]);
});

test("terminatePids escalates to SIGKILL when SIGTERM is ignored", async () => {
  const fake = fakeProcesses({ pids: [11, 12], ignoresTerm: [12] });
  const result = await terminatePids([11, 12], fake);
  assert.deepEqual(result, { signalled: [11, 12], killed: [12], stubborn: [] });
  assert.deepEqual(fake.signals, [
    [11, "SIGTERM"],
    [12, "SIGTERM"],
    [12, "SIGKILL"],
  ]);
});

test("terminatePids reports a pid that survives SIGKILL as stubborn", async () => {
  const signals = [];
  const result = await terminatePids([11], {
    isAlive: () => true,
    kill: (pid, signal) => signals.push([pid, signal]),
    sleep: async () => {},
  });
  assert.deepEqual(result, { signalled: [11], killed: [11], stubborn: [11] });
  assert.deepEqual(signals, [
    [11, "SIGTERM"],
    [11, "SIGKILL"],
  ]);
});

test("terminatePids on a free port signals nothing", async () => {
  const fake = fakeProcesses({ pids: [] });
  const result = await terminatePids([], fake);
  assert.deepEqual(result, { signalled: [], killed: [], stubborn: [] });
  assert.deepEqual(fake.signals, []);
});
```

scripts/sign-out-plan.mjs

```
// @ts-check
/**
 * The sign-out sequence used by `src/lib/auth/client.ts`, kept here as a pure
 * module so its effects can be unit-tested (`node --test` only covers
 * `scripts/`), the same split `migration-plan.mjs` uses for the two appliers.
 *
 * The two environments authenticate differently, so they need different
 * answers to "the server did not reply":
 *
 * - **Live preview** — a partitioned iframe with no readable session cookie;
 *   the session rides the bearer token in `sessionStorage`. Dropping that token
 *   IS being signed out, so the server call is best effort and a wedged request
 *   must never strand the button. This is where the hang actually happens.
 * - **Deployed** — the session rides an HttpOnly `__Host-` cookie that JS
 *   cannot delete. ONLY a completed sign-out response clears it, and
 *   `server.ts` enables `session.cookieCache` (maxAge 300), so `/get-session`
 *   would keep answering from the cached cookie for minutes afterwards.
 *   Redirecting on a timeout would show the visitor "signed out" while their
 *   session is still live — so here we fail loudly instead of pretending.
 */

/**
 * Live preview: aggressive, because the local clear is what signs the user out.
 * The same-origin POST normally answers in tens of ms; lower would start
 * abandoning slow-but-working sign-outs for no gain.
 */
export const PREVIEW_SIGN_OUT_TIMEOUT_MS = 1500;

/**
 * Deployed: generous, because only the server can end this session — but still
 * bounded, so a wedged request reports failure the visitor can retry instead of
 * spinning forever. A sign-out still unanswered at 10s is not going to land.
 */
export const DEPLOYED_SIGN_OUT_TIMEOUT_MS = 10_000;

/**
 * How long to wait for a sign-out in this environment. Every sign-out network
 * call picks its bound here, so the preview/deployed split cannot drift apart
 * between callers.
 * @param {boolean} livePreview
 * @returns {number}
 */
export function signOutTimeoutMs(livePreview) {
  return livePreview ? PREVIEW_SIGN_OUT_TIMEOUT_MS : DEPLOYED_SIGN_OUT_TIMEOUT_MS;
}

/**
 * Run `start()` but give up after `timeoutMs`, reporting which happened. Never
 * rejects — callers decide what a failure means, and a `try/catch` around an
 * `await` does nothing for a promise that never settles.
 * @param {() => unknown} start
 * @param {number} timeoutMs
 * @returns {Promise<"ok" | "failed" | "timeout">}
 */
export function settleWithin(start, timeoutMs) {
  return new Promise((resolve) => {
    const timer = setTimeout(() => resolve("timeout"), timeoutMs);
    /** @param {"ok" | "failed"} outcome */
    const done = (outcome) => {
      clearTimeout(timer);
      resolve(outcome);
    };
    try {
      Promise.resolve(start()).then(
        () => done("ok"),
        () => done("failed"),
      );
    } catch {
      done("failed");
    }
  });
}

/**
 * @typedef {object} SignOutSteps
 * @property {boolean} livePreview Whether the app is the sandbox preview iframe.
 * @property {boolean} hasBearer Whether a preview bearer token is stored.
 * @property {() => unknown} requestSignOut Ask the server to end the session; must reject on a failed respo
nse.
 * @property {() => void} clearToken Drop the stored bearer token.
 * @property {() => void} redirect Leave the page.
 * @property {number} [timeoutMs]
 */

/**
 * End the session, then clear the local token and redirect.
 *
 * In the live preview those last two always run. When deployed they run only if
```

```
 * the server confirmed, because nothing else can clear the cookie — a failed or
 * timed-out sign-out throws rather than reporting a sign-out that did not
 * happen.
 * @param {SignOutSteps} steps
 * @returns {Promise<void>}
 */
export async function runSignOut({
  livePreview,
  hasBearer,
  requestSignOut,
  clearToken,
  redirect,
  timeoutMs,
}) {
  if (livePreview) {
    // No bearer means a partitioned iframe with nothing to invalidate; with one,
    // still invalidate it server-side, just don't block on the answer.
    if (hasBearer) {
      await settleWithin(requestSignOut, timeoutMs ?? signOutTimeoutMs(livePreview));
    }
    clearToken();
    redirect();
    return;
  }

  const outcome = await settleWithin(requestSignOut, timeoutMs ?? signOutTimeoutMs(livePreview));
  if (outcome !== "ok") {
    throw new Error(
      outcome === "timeout"
        ? "Sign-out timed out — you are still signed in. Please try again."
        : "Sign-out failed — you are still signed in. Please try again.",
    );
  }
  clearToken();
  redirect();
}

/**
 * @typedef {object} PreSignInSteps
 * @property {boolean} livePreview Whether the app is the sandbox preview iframe.
 * @property {boolean} hasBearer Whether a preview bearer token is stored.
 * @property {() => unknown} requestSignOut Ask the server to end any prior session.
 * @property {() => void} clearToken Drop the stored bearer token.
 * @property {number} [timeoutMs]
 */

/**
 * Drop any prior session before a new sign-in starts, so switching providers
 * actually switches identity.
 *
 * Deliberately BEST EFFORT — unlike `runSignOut` this never throws. It also
 * runs when there is no prior session at all, so treating a failure as fatal
 * would block first-time sign-in on a transport hiccup, for a visitor with no
 * session to protect. The subsequent OAuth flow issues a fresh session either
 * way. Only the wait is bounded, and by the same per-environment rule as
 * `runSignOut`: a deployed session dies server-side, so it gets the full
 * window rather than the preview's aggressive one.
 * @param {PreSignInSteps} steps
 * @returns {Promise<void>}
 */
export async function runPreSignInSignOut({
  livePreview,
  hasBearer,
  requestSignOut,
  clearToken,
  timeoutMs,
}) {
  // In the preview a missing bearer means there is nothing to clear.
  if (hasBearer || !livePreview) {
    await settleWithin(requestSignOut, timeoutMs ?? signOutTimeoutMs(livePreview));
  }
  clearToken();
}
```

scripts/sign-out-plan.test.mjs

```
import assert from "node:assert/strict";
import { test } from "node:test";
import {
  DEPLOYED_SIGN_OUT_TIMEOUT_MS,
  PREVIEW_SIGN_OUT_TIMEOUT_MS,
  runPreSignInSignOut,
  runSignOut,
  settleWithin,
  signOutTimeoutMs,
} from "./sign-out-plan.mjs";

const TEST_TIMEOUT_MS = 20;

const hangs = () => new Promise(() => {});
const rejects = () => Promise.reject(new Error("network down"));

/**
 * Drain pending microtasks without advancing mocked time, so "has it finished
 * yet?" is answered by the mocked clock rather than by wall-clock luck.
 * `setImmediate` stays real — only `setTimeout` is mocked.
 */
const flush = () => new Promise((resolve) => setImmediate(resolve));

/**
 * A `runSignOut` call with the browser effects replaced by recorders, so each
 * test asserts on what actually happened rather than on how it was written.
 */
function harness(overrides = {}) {
  /** @type {string[]} */
  const order = [];
  let requests = 0;
  const steps = {
    livePreview: false,
    hasBearer: true,
    requestSignOut: () => {
      requests += 1;
      return Promise.resolve();
    },
    clearToken: () => order.push("clear"),
    redirect: () => order.push("redirect"),
    timeoutMs: TEST_TIMEOUT_MS,
    ...overrides,
  };
  return {
    order,
    get requests() {
      return requests;
    },
    run: () => runSignOut(steps),
  };
}

/** Live preview: the bearer is the session, so the local clear always wins. */
const preview = (overrides = {}) => harness({ livePreview: true, ...overrides });

/** Deployed: only the server can clear the `__Host-` cookie. */
const deployed = (overrides = {}) => harness({ livePreview: false, ...overrides });

// ── Live preview ─────────────────────────────────────────────────────────

test("preview: a successful sign-out clears the token, then redirects", async () => {
  const h = preview();
  await h.run();
  assert.equal(h.requests, 1);
  assert.deepEqual(h.order, ["clear", "redirect"]);
});

test("preview: a rejected sign-out still clears the token and redirects", async () => {
  const h = preview({ requestSignOut: rejects });
  await h.run();
  assert.deepEqual(h.order, ["clear", "redirect"]);
});

test("preview: a sign-out that throws synchronously still clears and redirects", async () => {
  const h = preview({
    requestSignOut: () => {
      throw new Error("no fetch");
    },
  });
  await h.run();
  assert.deepEqual(h.order, ["clear", "redirect"]);
});

test("preview: a sign-out that never settles clears and redirects once the wait expires", async (t) => {
  t.mock.timers.enable({ apis: ["setTimeout"] });
  const h = preview({ requestSignOut: hangs });
  const done = h.run();
```

```
  t.mock.timers.tick(TEST_TIMEOUT_MS - 1);
  await flush();
  assert.deepEqual(h.order, [], "the server still has its window — do not give up early");

  t.mock.timers.tick(1);
  await done;
  assert.deepEqual(h.order, ["clear", "redirect"]);
});

test("preview: no bearer means nothing to invalidate, so no request is made", async () => {
  const h = preview({ hasBearer: false });
  await h.run();
  assert.equal(h.requests, 0);
  assert.deepEqual(h.order, ["clear", "redirect"]);
});

test("preview: a stored bearer is still invalidated server-side", async () => {
  const h = preview({ hasBearer: true });
  await h.run();
  assert.equal(h.requests, 1);
});

// ── Deployed ─────────────────────────────────────────────────────────
// JS cannot delete the HttpOnly `__Host-` cookie and `cookieCache` keeps
// serving the cached session, so an unconfirmed sign-out must NOT look like one.

test("deployed: a confirmed sign-out clears the token, then redirects", async () => {
  const h = deployed();
  await h.run();
  assert.deepEqual(h.order, ["clear", "redirect"]);
});

test("deployed: a sign-out that never settles throws and does NOT redirect", async () => {
  const h = deployed({ requestSignOut: hangs });
  await assert.rejects(h.run(), /still signed in/);
  assert.deepEqual(h.order, [], "no redirect may claim a sign-out the server never made");
});

test("deployed: a rejected sign-out throws and does NOT redirect", async () => {
  const h = deployed({ requestSignOut: rejects });
  await assert.rejects(h.run(), /still signed in/);
  assert.deepEqual(h.order, []);
});

test("deployed: the timeout is distinguishable from a rejection", async () => {
  await assert.rejects(deployed({ requestSignOut: hangs }).run(), /timed out/);
  await assert.rejects(deployed({ requestSignOut: rejects }).run(), /Sign-out failed/);
});

// ── Bounded wait (also used by `signIn`'s pre-sign-in session clear) ──

test("settleWithin reports the outcome and never rejects", async () => {
  assert.equal(await settleWithin(() => Promise.resolve(), TEST_TIMEOUT_MS), "ok");
  assert.equal(await settleWithin(rejects, TEST_TIMEOUT_MS), "failed");
  assert.equal(await settleWithin(hangs, TEST_TIMEOUT_MS), "timeout");
});

test("settleWithin waits its full window, then gives up rather than hanging", async (t) => {
  t.mock.timers.enable({ apis: ["setTimeout"] });
  let outcome = null;
  const done = settleWithin(hangs, TEST_TIMEOUT_MS).then((o) => (outcome = o));

  t.mock.timers.tick(TEST_TIMEOUT_MS - 1);
  await flush();
  assert.equal(outcome, null, "the request still has time left");

  t.mock.timers.tick(1);
  await done;
  assert.equal(outcome, "timeout", "the caller is never left waiting on a wedged request");
});

// ── Pre-sign-in session clear (`signIn`) ─────────────────────────────
// Same per-environment bound as sign-out, but best effort: it also runs when
// there is no prior session, so a failure must never block sign-in.

/** A pre-sign-in clear whose request never settles. */
function preSignIn(livePreview, overrides = {}) {
  let cleared = 0;
  const done = runPreSignInSignOut({
    livePreview,
    hasBearer: true,
    requestSignOut: hangs,
    clearToken: () => (cleared += 1),
    ...overrides,
  });
  return {
    done,
    get cleared() {
      return cleared;
    },
```

```
  };
}

test("pre-sign-in: the preview clear gives up at the preview bound", async (t) => {
  t.mock.timers.enable({ apis: ["setTimeout"] });
  const h = preSignIn(true);

  t.mock.timers.tick(PREVIEW_SIGN_OUT_TIMEOUT_MS - 1);
  await flush();
  assert.equal(h.cleared, 0);

  t.mock.timers.tick(1);
  await h.done;
  assert.equal(h.cleared, 1);
});

test("pre-sign-in: a deployed session gets the deployed window, not the preview one", async (t) => {
  t.mock.timers.enable({ apis: ["setTimeout"] });
  const h = preSignIn(false);

  t.mock.timers.tick(PREVIEW_SIGN_OUT_TIMEOUT_MS);
  await flush();
  assert.equal(
    h.cleared,
    0,
    "only the server can end a deployed session — do not start OAuth while it is still live",
  );

  t.mock.timers.tick(DEPLOYED_SIGN_OUT_TIMEOUT_MS - PREVIEW_SIGN_OUT_TIMEOUT_MS);
  await h.done;
  assert.equal(h.cleared, 1);
});

test("pre-sign-in: a failed clear never blocks sign-in", async () => {
  // Best effort by design: this also runs with no prior session to clear.
  await preSignIn(false, { requestSignOut: rejects, timeoutMs: TEST_TIMEOUT_MS }).done;
  await preSignIn(true, { requestSignOut: rejects, timeoutMs: TEST_TIMEOUT_MS }).done;
});

test("pre-sign-in: the preview skips the request when there is no bearer", async () => {
  let requests = 0;
  const h = preSignIn(true, {
    hasBearer: false,
    requestSignOut: () => {
      requests += 1;
      return hangs();
    },
  });
  await h.done;
  assert.equal(requests, 0);
  assert.equal(h.cleared, 1);
});

test("every sign-out bound comes from one rule", () => {
  assert.equal(signOutTimeoutMs(true), PREVIEW_SIGN_OUT_TIMEOUT_MS);
  assert.equal(signOutTimeoutMs(false), DEPLOYED_SIGN_OUT_TIMEOUT_MS);
});

test("the defaults are bounded, and deployed waits longer than preview", async (t) => {
  assert.ok(PREVIEW_SIGN_OUT_TIMEOUT_MS > 0 && PREVIEW_SIGN_OUT_TIMEOUT_MS <= 2000);
  assert.ok(DEPLOYED_SIGN_OUT_TIMEOUT_MS > PREVIEW_SIGN_OUT_TIMEOUT_MS);
  assert.ok(DEPLOYED_SIGN_OUT_TIMEOUT_MS <= 30_000);

  t.mock.timers.enable({ apis: ["setTimeout"] });
  const h = preview({ requestSignOut: hangs, timeoutMs: undefined });
  const done = h.run();
  t.mock.timers.tick(PREVIEW_SIGN_OUT_TIMEOUT_MS);
  await done;
  assert.deepEqual(h.order, ["clear", "redirect"]);
});
```

scripts/with-app-env.mjs

```
#!/usr/bin/env node
/**
 * Run a command with `.grok/app-env.json` merged into its environment.
 *
 * `dev`, `build` and `preview` all route through this wrapper, so the dev
 * server, the built bundle and the preview server can never disagree about
 * `VITE_AUTH_ENABLED` — a divergence that only shows up as a built-output
 * mismatch long after the fact. Anything that starts Vite directly bypasses it.
 *
 * Only `VITE_`-prefixed keys are honored: the file is a build flag carrier, not
 * a secret store, and only `VITE_` vars reach the browser anyway. A real
 * `process.env` entry always wins, so an explicit override still works.
 *
 * That precedence also means the file governs this workspace only. A deployed
 * build runs with the provider's project env, where the deployer sets
 * `VITE_AUTH_ENABLED` itself (today unconditionally `"true"`), so the deployed
 * flag is the platform's, not this file's.
 *
 * Vite picks the values up because `loadEnv` prefix-matches entries already in
 * `process.env`, which is why the merge has to happen before Vite starts.
 */
import { spawn } from "node:child_process";
import { readFileSync, realpathSync } from "node:fs";
import { constants as osConstants } from "node:os";
import { dirname, join } from "node:path";
import { fileURLToPath } from "node:url";

export const APP_ENV_REL_PATH = ".grok/app-env.json";

const VITE_PREFIX = "VITE_";

/**
 * Parse an app-env document, keeping only `VITE_`-prefixed string entries.
 * Anything unparseable is an empty environment — a workspace without the file
 * must behave exactly like today (auth on, no overrides).
 */
export function parseAppEnv(text) {
  let parsed;
  try {
    parsed = JSON.parse(text);
  } catch {
    return {};
  }
  if (parsed === null || typeof parsed !== "object" || Array.isArray(parsed)) return {};
  const env = {};
  for (const [key, value] of Object.entries(parsed)) {
    if (!key.startsWith(VITE_PREFIX)) continue;
    if (typeof value !== "string") continue;
    env[key] = value;
  }
  return env;
}

/** The app env recorded under `root`, or `{}` when the file is absent. */
export function readAppEnv(root) {
  try {
    return parseAppEnv(readFileSync(join(root, APP_ENV_REL_PATH), "utf8"));
  } catch {
    return {};
  }
}

/** File values under the process environment: an explicit override wins. */
export function mergeAppEnv(appEnv, processEnv) {
  return { ...appEnv, ...processEnv };
}

/**
 * Translate a child's `exit` `(code, signal)` into this process's exit status.
 *
 * Do not re-raise the signal with `process.kill(process.pid, signal)`: under
 * qemu-user (amd64 image builds on an arm host) a self-directed signal is
 * routinely delivered as SIGSEGV to the wrong process, which takes down the
 * test worker and fails the image build. `128 + signo` is what a shell reports
 * for a signal-killed command, so a cancelled `vite build` is still a failure.
 */
export function exitStatusFromChild(code, signal) {
  if (signal) {
    const signo = osConstants.signals[signal];
    return 128 + (typeof signo === "number" ? signo : 1);
  }
  return code ?? 1;
}

/** The workspace root (this file lives in `<root>/scripts/`). */
export function projectRoot() {
  return dirname(dirname(fileURLToPath(import.meta.url)));
}
```

```
/**
 * Whether `moduleUrl` is the script node was asked to run.
 *
 * Both sides are resolved through symlinks: node realpaths `import.meta.url`
 * but leaves `process.argv[1]` as typed, so comparing them raw makes a CLI
 * launched through a symlinked path (`/tmp` on macOS) a silent no-op.
 */
export function isMainModule(moduleUrl) {
  const entry = process.argv[1];
  if (!entry) return false;
  try {
    return realpathSync(entry) === fileURLToPath(moduleUrl);
  } catch {
    return false;
  }
}

function main(argv) {
  const [command, ...args] = argv;
  if (!command) {
    console.error("usage: node scripts/with-app-env.mjs <command> [args…]");
    process.exit(2);
  }
  const env = mergeAppEnv(readAppEnv(projectRoot()), process.env);
  const child = spawn(command, args, { stdio: "inherit", env });
  // The dev server is long-running and is stopped by signalling this wrapper.
  for (const signal of ["SIGINT", "SIGTERM", "SIGHUP"]) {
    process.on(signal, () => child.kill(signal));
  }
  child.on("error", (err) => {
    console.error(`[with-app-env] failed to run ${command}:`, err?.message || err);
    process.exit(127);
  });
  child.on("exit", (code, signal) => {
    process.exit(exitStatusFromChild(code, signal));
  });
}

if (isMainModule(import.meta.url)) {
  main(process.argv.slice(2));
}
```

scripts/with-app-env.test.mjs

```
import assert from "node:assert/strict";
import { execFile } from "node:child_process";
import { mkdirSync, mkdtempSync, symlinkSync, writeFileSync } from "node:fs";
import { tmpdir } from "node:os";
import { join } from "node:path";
import { test } from "node:test";
import { promisify } from "node:util";
import {
  APP_ENV_REL_PATH,
  mergeAppEnv,
  parseAppEnv,
  projectRoot,
  readAppEnv,
} from "./with-app-env.mjs";

const execFileAsync = promisify(execFile);
const WRAPPER = join(projectRoot(), "scripts/with-app-env.mjs");
const PRINT_FLAG = "process.stdout.write(String(process.env.VITE_AUTH_ENABLED));";

function makeWorkspace(appEnvJson) {
  const root = mkdtempSync(join(tmpdir(), "app-env-"));
  if (appEnvJson !== undefined) {
    mkdirSync(join(root, ".grok"), { recursive: true });
    writeFileSync(join(root, APP_ENV_REL_PATH), appEnvJson);
  }
  return root;
}

test("keeps VITE_-prefixed string entries", () => {
  assert.deepEqual(parseAppEnv('{"VITE_AUTH_ENABLED":"false"}'), {
    VITE_AUTH_ENABLED: "false",
  });
});

test("drops non-VITE keys, non-string values and malformed documents", () => {
  assert.deepEqual(parseAppEnv('{"DATABASE_URL":"postgres://x","VITE_N":1,"VITE_OK":"y"}'), {
    VITE_OK: "y",
  });
  assert.deepEqual(parseAppEnv("not json"), {});
  assert.deepEqual(parseAppEnv('["VITE_AUTH_ENABLED"]'), {});
  assert.deepEqual(parseAppEnv("null"), {});
});

test("a missing app-env.json is a clean no-op", () => {
  assert.deepEqual(readAppEnv(makeWorkspace()), {});
});

test("reads the app env from a workspace", () => {
  const root = makeWorkspace('{"VITE_AUTH_ENABLED":"false"}');
  assert.deepEqual(readAppEnv(root), { VITE_AUTH_ENABLED: "false" });
});

test("an explicit process-env override wins over the file", () => {
  const merged = mergeAppEnv(
    { VITE_AUTH_ENABLED: "false" },
    { VITE_AUTH_ENABLED: "true", PATH: "/usr/bin" },
  );
  assert.equal(merged.VITE_AUTH_ENABLED, "true");
  assert.equal(merged.PATH, "/usr/bin");
});

test("the template ships auth off", () => {
  assert.deepEqual(readAppEnv(projectRoot()), { VITE_AUTH_ENABLED: "false" });
});

test("vite loadEnv resolves the wrapped value", () => {
  // What `import.meta.env.VITE_AUTH_ENABLED` becomes: loadEnv prefix-matches
  // process.env, so the wrapper's merge has to land before Vite starts.
  // Do not `import { loadEnv } from "vite"` here — Vite 8 loads rolldown
  // native bindings that SIGSEGV the test worker under qemu-user.
  const root = makeWorkspace('{"VITE_AUTH_ENABLED":"false"}');
  const merged = mergeAppEnv(readAppEnv(root), { PATH: "/usr/bin" });
  assert.equal(merged.VITE_AUTH_ENABLED, "false");
});

test("the wrapped command runs with the app env applied", async () => {
  const { stdout } = await execFileAsync(process.execPath, [
    WRAPPER,
    process.execPath,
    "-e",
    PRINT_FLAG,
  ]);
  assert.equal(stdout, "false");
});

test("the wrapped command sees an explicit override, not the file value", async () => {
  const { stdout } = await execFileAsync(
    process.execPath,
```

```
    [WRAPPER, process.execPath, "-e", PRINT_FLAG],
    { env: { ...process.env, VITE_AUTH_ENABLED: "true" } },
  );
  assert.equal(stdout, "true");
});

test("the wrapper propagates the command's exit code", async () => {
  await assert.rejects(
    execFileAsync(process.execPath, [WRAPPER, process.execPath, "-e", "process.exit(3)"]),
    (err) => err.code === 3,
  );
});

test("a signal-killed command is never reported as success", async () => {
  // The wrapper's own SIGTERM handler must not swallow the re-raised signal:
  // a cancelled build reporting exit 0 is a silently passing gate.
  await assert.rejects(
    execFileAsync(process.execPath, [
      WRAPPER,
      process.execPath,
      "-e",
      "process.kill(process.pid, 'SIGTERM');setTimeout(() => {}, 1000);",
    ]),
    (err) => err.signal === "SIGTERM" || err.code !== 0,
  );
});

test("the CLI still runs when invoked through a symlinked path", async () => {
  // node realpaths import.meta.url but not process.argv[1], so a raw comparison
  // turns the wrapper into a no-op that exits 0 without starting anything.
  const link = join(mkdtempSync(join(tmpdir(), "app-env-link-")), "scripts");
  symlinkSync(join(projectRoot(), "scripts"), link);
  const { stdout } = await execFileAsync(process.execPath, [
    join(link, "with-app-env.mjs"),
    process.execPath,
    "-e",
    PRINT_FLAG,
  ]);
  assert.equal(stdout, "false");
});
```

scripts/write-atomic.mjs

```
#!/usr/bin/env node
/**
 * Hand a staged file over to a path another agent reads, in one step.
 *
 *   node scripts/write-atomic.mjs /workspace/.grok/og.jpg.tmp public/og.jpg
 *
 * The brand-asset task writes public/og.jpg and src/lib/og/site.json while the
 * parent may be mid-`npm run build`, so an in-place write can be read
 * half-finished. rename(2) is atomic within one filesystem: a reader sees the
 * old bytes or the new ones. /workspace is one filesystem, so a staged file
 * from anywhere else (/tmp is a separate mount) is refused rather than copied:
 * copying would have to land its temp in the target's directory, which is the
 * one thing a staged path is not allowed to do.
 */
import { existsSync, mkdirSync, renameSync } from "node:fs";
import { dirname, isAbsolute, join, relative, resolve } from "node:path";
import { fileURLToPath, pathToFileURL } from "node:url";

const ROOT = join(dirname(fileURLToPath(import.meta.url)), "..");

export function parseWriteAtomicArgs(argv) {
  const [staged, target, ...rest] = argv;
  if (!staged || !target) {
    return { error: "usage: node scripts/write-atomic.mjs <staged-file> <target>" };
  }
  if (rest.length > 0) return { error: `unexpected argument: ${rest[0]}` };
  return { staged, target };
}

function isInside(dir, file) {
  const rel = relative(dir, file);
  return rel !== "" && !rel.startsWith("..") && !isAbsolute(rel);
}

/**
 * Why a staged path can be refused: `vite build` copies public/ verbatim into
 * the deployed app, so a temp that lands there ships (and an interrupted run
 * leaves it behind).
 */
export function stagingError({ staged, target, publicDir }) {
  if (staged === target) return `staged file and target are the same path: ${target}`;
  if (isInside(publicDir, staged)) {
    return `stage outside ${publicDir} (vite build ships that directory verbatim): ${staged}`;
  }
  return null;
}

/**
 * Moves `staged` onto `target`, leaving `target` untouched if anything fails.
 * `rename` is injectable because EXDEV — the refusal the "stage under
 * /workspace/.grok/" contract rests on — cannot be provoked portably.
 */
export function handOver(staged, target, { rename = renameSync } = {}) {
  if (!existsSync(staged)) {
    throw Object.assign(new Error(`staged file is missing: ${staged}`), { code: "ENOENT" });
  }
  mkdirSync(dirname(target), { recursive: true });
  try {
    rename(staged, target);
  } catch (err) {
    if (err?.code === "EXDEV") {
      throw new Error(
        `${staged} is on another filesystem than ${target}, so the hand-over cannot be a `
          + "rename — stage under /workspace/.grok/ instead",
      );
    }
    throw err;
  }
}

if (process.argv[1] && import.meta.url === pathToFileURL(process.argv[1]).href) {
  const args = parseWriteAtomicArgs(process.argv.slice(2));
  if (args.error) {
    console.error(`[write-atomic] ${args.error}`);
    process.exit(1);
  }
  // Relative paths follow this script's root, not the caller's cwd: the brand
  // pass runs from wherever its sub-shell left it, and the public/ refusal
  // below is defined against that same root.
  const staged = resolve(ROOT, args.staged);
  const target = resolve(ROOT, args.target);
  const problem = stagingError({ staged, target, publicDir: join(ROOT, "public") });
  if (problem) {
    console.error(`[write-atomic] ${problem}`);
    process.exit(1);
  }
  try {
    handOver(staged, target);
```

```
  } catch (err) {
    console.error(`[write-atomic] ${staged} → ${target} failed: ${err?.message || err}`);
    process.exit(1);
  }
  console.log(`[write-atomic] wrote ${target}`);
}
```

scripts/write-atomic.test.mjs

```
import assert from "node:assert/strict";
import { spawnSync } from "node:child_process";
import {
  existsSync,
  mkdirSync,
  mkdtempSync,
  readFileSync,
  readdirSync,
  writeFileSync,
} from "node:fs";
import { tmpdir } from "node:os";
import { dirname, join } from "node:path";
import { fileURLToPath } from "node:url";
import { test } from "node:test";
import { handOver, parseWriteAtomicArgs, stagingError } from "./write-atomic.mjs";

const TEMPLATE_ROOT = join(dirname(fileURLToPath(import.meta.url)), "..");
const SCRIPT = join(TEMPLATE_ROOT, "scripts/write-atomic.mjs");

function makeWorkspace() {
  const root = mkdtempSync(join(tmpdir(), "write-atomic-"));
  mkdirSync(join(root, "public"), { recursive: true });
  mkdirSync(join(root, ".grok"), { recursive: true });
  return root;
}

test("parseWriteAtomicArgs needs exactly a staged file and a target", () => {
  assert.deepEqual(parseWriteAtomicArgs([".grok/og.tmp", "public/og.jpg"]), {
    staged: ".grok/og.tmp",
    target: "public/og.jpg",
  });
  assert.match(parseWriteAtomicArgs([]).error, /usage:/);
  assert.match(parseWriteAtomicArgs([".grok/og.tmp"]).error, /usage:/);
  assert.match(parseWriteAtomicArgs(["a", "b", "c"]).error, /unexpected argument: c/);
});

test("stagingError refuses a temp inside public/ and a no-op move", () => {
  const publicDir = "/workspace/public";
  assert.equal(
    stagingError({
      staged: "/workspace/.grok/og.jpg.tmp",
      target: "/workspace/public/og.jpg",
      publicDir,
    }),
    null,
  );
  assert.match(
    stagingError({
      staged: "/workspace/public/og.jpg.tmp",
      target: "/workspace/public/og.jpg",
      publicDir,
    }),
    /vite build ships that directory verbatim/,
  );
  assert.match(
    stagingError({
      staged: "/workspace/public/og.jpg",
      target: "/workspace/public/og.jpg",
      publicDir,
    }),
    /same path/,
  );
});

test("handOver replaces the target and clears the staged file", () => {
  const root = makeWorkspace();
  const staged = join(root, ".grok/og.jpg.tmp");
  const target = join(root, "public/og.jpg");
  writeFileSync(target, "old card");
  writeFileSync(staged, "new card");

  handOver(staged, target);

  assert.equal(readFileSync(target, "utf8"), "new card");
  assert.equal(existsSync(staged), false);
});

test("handOver creates a missing target directory", () => {
  const root = makeWorkspace();
  const staged = join(root, ".grok/site.json.tmp");
  writeFileSync(staged, '{"title":"Sky Strike"}');

  handOver(staged, join(root, "src/lib/og/site.json"));

  assert.equal(readFileSync(join(root, "src/lib/og/site.json"), "utf8"), '{"title":"Sky Strike"}');
});

test("an interrupted pass leaves the target on its old bytes, temp-free", () => {
```

```
  const root = makeWorkspace();
  const target = join(root, "public/og.jpg");
  writeFileSync(target, "old card");
  // The staged file a killed ffmpeg leaves behind: never handed over.
  writeFileSync(join(root, ".grok/og.jpg.tmp"), "half a JPEG");

  assert.equal(readFileSync(target, "utf8"), "old card");
  assert.throws(() => handOver(join(root, ".grok/absent.tmp"), target), { code: "ENOENT" });
  assert.equal(readFileSync(target, "utf8"), "old card");
  assert.equal(existsSync(`${target}.tmp-${process.pid}`), false);
});

test("a failed hand-over creates no directory for the target it never wrote", () => {
  const root = makeWorkspace();
  const missing = join(root, ".grok/absent.tmp");
  assert.throws(() => handOver(missing, join(root, "src/lib/og/site.json")), { code: "ENOENT" });
  assert.equal(existsSync(join(root, "src")), false);
});

test("a staged file on another filesystem is refused, not copied", () => {
  const root = makeWorkspace();
  const staged = join(root, ".grok/og.jpg.tmp");
  const target = join(root, "public/og.jpg");
  writeFileSync(staged, "new card");
  writeFileSync(target, "old card");
  const crossDevice = () => {
    throw Object.assign(new Error("EXDEV"), { code: "EXDEV" });
  };

  assert.throws(() => handOver(staged, target, { rename: crossDevice }), {
    message: /stage under \/workspace\/\.grok\//,
  });
  // Copying would have had to stage its own temp inside public/, which is the
  // one place stagingError refuses.
  assert.deepEqual(readdirSync(join(root, "public")), ["og.jpg"]);
  assert.equal(readFileSync(target, "utf8"), "old card");
});

test("cli: hands the file over, and refuses a temp staged in public/", () => {
  const root = makeWorkspace();
  writeFileSync(join(root, ".grok/og.jpg.tmp"), "new card");
  const ok = spawnSync(
    process.execPath,
    [SCRIPT, join(root, ".grok/og.jpg.tmp"), join(root, "public/og.jpg")],
    { encoding: "utf8" },
  );
  assert.equal(ok.status, 0, ok.stdout + ok.stderr);
  assert.equal(readFileSync(join(root, "public/og.jpg"), "utf8"), "new card");

  // publicDir comes from the script's own root, so refusal is checked there.
  const templateRoot = join(dirname(SCRIPT), "..");
  const staged = join(templateRoot, "public/og.jpg.tmp");
  const refused = spawnSync(
    process.execPath,
    [SCRIPT, staged, join(templateRoot, "public/og.jpg")],
    { encoding: "utf8" },
  );
  assert.equal(refused.status, 1);
  assert.match(refused.stderr, /stage outside/);
  assert.equal(existsSync(staged), false);
});

test("cli: relative paths follow the script's root, not the caller's cwd", () => {
  // Same relative pair the skill documents, run from a workspace that has its
  // own public/: resolving against cwd would take the staged temp out of the
  // directory the refusal is defined against and move it.
  const root = makeWorkspace();
  writeFileSync(join(root, "public/og.jpg.tmp"), "half a JPEG");
  const run = spawnSync(process.execPath, [SCRIPT, "public/og.jpg.tmp", "public/og.jpg"], {
    cwd: root,
    encoding: "utf8",
  });
  assert.equal(run.status, 1, run.stdout + run.stderr);
  assert.match(run.stderr, /stage outside/);
  assert.equal(readFileSync(join(root, "public/og.jpg.tmp"), "utf8"), "half a JPEG");
  assert.equal(existsSync(join(root, "public/og.jpg")), false);
});

test("every hand-over the og skill prints is one this script accepts", () => {
  // The card and banner recipes live in the skill's references/, not SKILL.md.
  const skillDir = join(TEMPLATE_ROOT, ".grok/skills/og");
  const docs = [
    join(skillDir, "SKILL.md"),
    ...readdirSync(join(skillDir, "references")).map((f) => join(skillDir, "references", f)),
  ];
  const invocations = docs.flatMap(
    (path) => readFileSync(path, "utf8").match(/node scripts\/write-atomic\.mjs[^\n`]*/g) ?? [],
  );
  assert.ok(invocations.length >= 3, "og.jpg, x-banner.jpg and site.json each hand over");
  for (const line of invocations) {
```

```
    const argv = line.replace("node scripts/write-atomic.mjs", "").trim().split(/\s+/);
    const args = parseWriteAtomicArgs(argv);
    assert.equal(args.error, undefined, line);
    assert.equal(
      stagingError({ staged: args.staged, target: args.target, publicDir: "/workspace/public" }),
      null,
      line,
    );
  }
});

test("cli: a missing staged file fails without touching the target", () => {
  const root = makeWorkspace();
  writeFileSync(join(root, "public/og.jpg"), "old card");
  const run = spawnSync(
    process.execPath,
    [SCRIPT, join(root, ".grok/absent.tmp"), join(root, "public/og.jpg")],
    { encoding: "utf8" },
  );
  assert.equal(run.status, 1);
  assert.match(run.stderr, /\[write-atomic\]/);
  assert.equal(readFileSync(join(root, "public/og.jpg"), "utf8"), "old card");
});
```

server/middleware/grok-pwa.ts

```
/**
 * Deployed-app (Nitro) half of the platform PWA chrome. Auto-registered as
 * global h3 middleware because vite.config.ts sets `serverDir: "./server"` —
 * without that option Nitro v3 never scans this directory.
 *
 * - `?install=1&platform=ios` on a document path → the Home Screen tutorial,
 *   bundled into the server build via `?raw` (the public/ directory is CDN
 *   static output on Vercel and not readable from the function).
 * - `/__grok/manifest.webmanifest` → per-app-named manifest (kept out of
 *   public/ so this dynamic response is the only one).
 * - Other HTML documents → stream-inject PWA + OG head tags at `</head>`.
 *   OG identity is baked via `virtual:grok-og-identity` at `vite build`
 *   (this function cannot read `src/lib/og/site.json` or `public/og.jpg`).
 *   This must be a middleware transforming `next()`: h3 discards the `response`
 *   runtime hook's return value, and `render:html` does not exist in Nitro v3.
 */
import installPageTemplate from "../../scripts/install-page.html?raw";
import { grokOgIdentity } from "virtual:grok-og-identity";
import {
  acceptsHtml,
  createHeadInjector,
  isDocumentPath,
  isInstallQuery,
  renderInstallPageHtml,
  renderWebManifest,
} from "../../scripts/grok-pwa-shared.mjs";

interface GrokPwaEvent {
  url: URL;
  req: { method: string; headers: Headers };
}

function requestHost(event: GrokPwaEvent): string {
  return (
    event.req.headers.get("x-forwarded-host") ?? event.req.headers.get("host") ?? event.url.host
  );
}

function injectHeadStreaming(response: Response, host: string): Response {
  const injector = createHeadInjector({
    host,
    site: grokOgIdentity.site,
  });
  const transformed = response.body!.pipeThrough(
    new TransformStream<Uint8Array, Uint8Array>({
      transform(chunk, controller) {
        for (const out of injector.push(chunk)) controller.enqueue(out);
      },
      flush(controller) {
        for (const out of injector.flush()) controller.enqueue(out);
      },
    }),
  );
  const headers = new Headers(response.headers);
  headers.delete("content-length");
  return new Response(transformed, {
    status: response.status,
    statusText: response.statusText,
    headers,
  });
}

export default async function grokPwaMiddleware(
  event: GrokPwaEvent,
  next: () => unknown | Promise<unknown>,
): Promise<unknown> {
  const method = (event.req.method ?? "GET").toUpperCase();
  if (method !== "GET") return next();

  const path = event.url.pathname;
  const urlWithQuery = path + event.url.search;

  if (path === "/__grok/manifest.webmanifest" || path === "/__grok/manifest.json") {
    return new Response(renderWebManifest(requestHost(event)), {
      headers: {
        "content-type": "application/manifest+json; charset=utf-8",
        "cache-control": "no-cache",
      },
    });
  }

  if (
    isInstallQuery(urlWithQuery) &&
    isDocumentPath(path) &&
    acceptsHtml(event.req.headers.get("accept"))
  ) {
    const html = renderInstallPageHtml(installPageTemplate, {
      host: requestHost(event),
```

```
      url: urlWithQuery,
    });
    return new Response(html, {
      headers: {
        "content-type": "text/html; charset=utf-8",
        "cache-control": "no-cache",
      },
    });
  }

  if (!isDocumentPath(path)) return next();

  const result = await next();
  if (
    result instanceof Response &&
    result.body &&
    String(result.headers.get("content-type") ?? "").includes("text/html") &&
    !result.headers.get("content-encoding")
  ) {
    return injectHeadStreaming(result, requestHost(event));
  }
  return result;
}
```

server/virtual-grok-og-identity.d.ts

```
declare module "virtual:grok-og-identity" {
  export const grokOgIdentity: {
    site: {
      title?: string;
      description?: string;
      type?: string;
      card?: string;
      image?: string;
      banner?: string;
      color?: string;
    };
  };
}
```

src/components/auth-buttons.tsx

```
import { useAuth0, useLogin, useLogout } from "@auth0/auth0-tanstack-start-react/client";

export function AuthButtons({ bar = false }: { bar?: boolean }) {
  const login = useLogin();
  const logout = useLogout();
  const { isAuthenticated, user } = useAuth0();

  if (isAuthenticated) {
    return bar ? (
      <button
        type="button"
        onClick={() => logout()}
        className="h-10 rounded-full border border-rule bg-paper px-4 text-sm font-semibold text-sage"
      >
        Log out
      </button>
    ) : (
      <div className="flex flex-col gap-3">
        {user?.email ? <p className="text-sm text-ink">{user.email}</p> : null}
        <button
          type="button"
          onClick={() => logout()}
          className="h-12 w-full rounded-full border border-neon text-base font-semibold text-sage"
        >
          Log out
        </button>
      </div>
    );
  }

  const loginButton = (
    <button
      type="button"
      onClick={() => login("/")}
      className={
        bar
          ? "h-10 rounded-full bg-neon px-4 text-sm font-semibold text-paper shadow-neon"
          : "h-12 w-full rounded-full bg-neon text-base font-semibold text-paper shadow-neon"
      }
    >
      Log in
    </button>
  );
  const signUpButton = (
    <button
      type="button"
      onClick={() => login("/", { authorizationParams: { screen_hint: "signup" } })}
      className={
        bar
          ? "h-10 rounded-full border border-neon bg-paper px-4 text-sm font-semibold text-sage"
          : "h-12 w-full rounded-full border border-neon text-base font-semibold text-sage"
      }
    >
      Sign up
    </button>
  );

  if (bar) {
    return (
      <div className="flex gap-2">
        {loginButton}
        {signUpButton}
      </div>
    );
  }

  return (
    <div className="flex flex-col gap-3">
      {loginButton}
      {signUpButton}
    </div>
  );
}
```

src/components/preview-host-bridge.tsx

```
/**
 * Mount once in `__root.tsx` so the Grok preview chrome can drive navigation
 * (and later receive registered routes). Noops when the app is not embedded.
 */

import { useEffect } from "react";
import { useRouter } from "@tanstack/react-router";
import {
  collectRoutePathsFromTree,
  installPreviewHostBridge,
} from "@/lib/preview-host-bridge";

export function PreviewHostBridge() {
  const router = useRouter();

  useEffect(() => {
    return installPreviewHostBridge({
      navigate: (path) => {
        router.history.push(path);
      },
      getRoutePaths: () => collectRoutePathsFromTree(router.routeTree),
    });
  }, [router]);

  return null;
}
```

src/lib/app-data/app-data.test.ts

```
import { afterEach, beforeEach, describe, it, mock } from "node:test";
import assert from "node:assert/strict";
import {
  callTool,
  failureMemoSize,
  isConnectorTokenReady,
} from "./client.server.ts";
import { ConnectorType, GoogleCalendarTools } from "./types.ts";
import type { ToolArgs } from "./types.ts";
import { isLoginRequired, redirectToLoginIfRequired } from "./login.ts";
import { classifyCallToolError } from "./errors.ts";
import type { CallToolResult } from "./types.ts";

type WindowStub = {
  location: { assign: (url: string) => void; href: string };
  self?: unknown;
  top?: unknown;
  open?: (url: string, target: string) => unknown;
};

function withWindow<T>(stub: WindowStub, fn: () => T): T {
  (globalThis as { window?: unknown }).window = stub;
  try {
    return fn();
  } finally {
    delete (globalThis as { window?: unknown }).window;
  }
}

function fakeJwt(claims: Record<string, unknown>): string {
  const encode = (value: unknown) =>
    Buffer.from(JSON.stringify(value)).toString("base64url");
  return `${encode({ alg: "HS256", typ: "JWT" })}.${encode(claims)}.sig`;
}

async function withStubbedGate(
  status: number,
  body: Record<string, unknown>,
  run: (calls: () => number) => Promise<void>,
): Promise<void> {
  process.env.GROK_CONNECTORS_URL = "https://connectors.invalid.example";
  const realFetch = globalThis.fetch;
  let calls = 0;
  globalThis.fetch = (async () => {
    calls += 1;
    return new Response(JSON.stringify(body), { status });
  }) as typeof fetch;
  try {
    await run(() => calls);
  } finally {
    globalThis.fetch = realFetch;
    delete process.env.GROK_CONNECTORS_URL;
  }
}

describe("callTool failure memo", () => {
  it("replays an identical failure within the memo window without refetching", async () => {
    await withStubbedGate(500, { ok: false, errorMessage: "boom" }, async (calls) => {
      const options = {
        connectorType: ConnectorType.GoogleDrive,
        token: fakeJwt({ sub: "memo-user-1", iat: 1000, exp: 2000 }),
      };
      const first = await callTool("google_drive_search", { q: "memo" }, options);
      const second = await callTool("google_drive_search", { q: "memo" }, options);
      assert.equal(first.ok, false);
      assert.equal(first.errorMessage, "boom");
      assert.deepEqual(second, first);
      assert.equal(calls(), 1);
    });
  });

  it("keys by token identity so a reminted token shares the memo entry", async () => {
    await withStubbedGate(500, { ok: false, errorMessage: "boom" }, async (calls) => {
      const first = await callTool(
        "google_drive_search",
        { q: "remint" },
        {
          connectorType: ConnectorType.GoogleDrive,
          token: fakeJwt({ sub: "memo-user-2", iat: 1000, exp: 2000 }),
        },
      );
      const second = await callTool(
        "google_drive_search",
        { q: "remint" },
        {
          connectorType: ConnectorType.GoogleDrive,
          token: fakeJwt({ sub: "memo-user-2", iat: 1001, exp: 2001 }),
        },
```

```
      );
      assert.equal(first.ok, false);
      assert.deepEqual(second, first);
      assert.equal(calls(), 1);

      const other = await callTool(
        "google_drive_search",
        { q: "remint" },
        {
          connectorType: ConnectorType.GoogleDrive,
          token: fakeJwt({ sub: "memo-user-3", iat: 1000, exp: 2000 }),
        },
      );
      assert.equal(other.ok, false);
      assert.equal(calls(), 2);
    });
  });

  it("sweeps expired entries on write so unique keys do not accumulate", async () => {
    await withStubbedGate(500, { ok: false, errorMessage: "boom" }, async () => {
      mock.timers.enable({ apis: ["Date"], now: 1_000_000 });
      try {
        const sizeBefore = failureMemoSize();
        const options = {
          connectorType: ConnectorType.GoogleDrive,
          token: fakeJwt({ sub: "memo-user-5", iat: 1000, exp: 2000 }),
        };
        await callTool("google_drive_search", { q: "sweep-a" }, options);
        await callTool("google_drive_search", { q: "sweep-b" }, options);
        assert.equal(failureMemoSize(), sizeBefore + 2);

        mock.timers.setTime(1_000_000 + 5_001);
        await callTool("google_drive_search", { q: "sweep-c" }, options);
        assert.equal(
          failureMemoSize(),
          sizeBefore + 1,
          "expired sweep-a/sweep-b entries must be removed on the sweep-c write",
        );
      } finally {
        mock.timers.reset();
      }
    });
  });

  it("never memoizes login-required 401s", async () => {
    process.env.GROK_PROJECT_ID = "proj-1";
    try {
      await withStubbedGate(401, { errorMessage: "login required" }, async (calls) => {
        const options = {
          connectorType: ConnectorType.GoogleDrive,
          token: fakeJwt({ sub: "memo-user-4", iat: 1000, exp: 2000 }),
        };
        const first = await callTool("google_drive_search", { q: "auth" }, options);
        const second = await callTool("google_drive_search", { q: "auth" }, options);
        assert.equal(first.ok, false);
        assert.equal(first.loginRequired, true);
        assert.equal(second.loginRequired, true);
        assert.equal(calls(), 2);
      });
    } finally {
      delete process.env.GROK_PROJECT_ID;
    }
  });
});

describe("callTool in the workspace preview vs deployed", () => {
  const options = { connectorType: ConnectorType.GoogleDrive };
  const savedEnvToken = process.env.GROK_CONNECTOR_ACCESS_TOKEN;

  beforeEach(() => {
    delete process.env.GROK_CONNECTOR_ACCESS_TOKEN;
    delete process.env.GROK_PROJECT_ID;
  });
  afterEach(() => {
    if (savedEnvToken === undefined) {
      delete process.env.GROK_CONNECTOR_ACCESS_TOKEN;
    } else {
      process.env.GROK_CONNECTOR_ACCESS_TOKEN = savedEnvToken;
    }
    delete process.env.GROK_PROJECT_ID;
  });

  it("returns pending (no loginRequired) when the preview has no token yet", async () => {
    const result = await callTool("google_drive_search", {}, options);
    assert.equal(result.ok, false);
    assert.equal(result.pending, true);
    assert.equal(result.loginRequired, undefined);
    assert.match(result.errorMessage ?? "", /^connector_token_pending/);
  });
```

```
  it("returns a plain error (no sign-in CTA) when a deployed app has no token", async () => {
    process.env.GROK_PROJECT_ID = "proj-1";
    const result = await callTool("google_drive_search", {}, options);
    assert.equal(result.ok, false);
    assert.equal(result.loginRequired, undefined);
    assert.equal(result.loginUrl, undefined);
    assert.equal(result.pending, undefined);
    assert.match(result.errorMessage ?? "", /^missing_connector_token/);
  });

  it("treats a gate 401 in the preview as pending and parks the rejected token", async () => {
    await withStubbedGate(401, { errorMessage: "login required" }, async (calls) => {
      process.env.GROK_CONNECTOR_ACCESS_TOKEN = fakeJwt({ sub: "p", iat: 1, exp: 2 });
      assert.equal(isConnectorTokenReady(), true);

      const result = await callTool("google_drive_search", {}, options);
      assert.equal(result.pending, true);
      assert.equal(result.loginRequired, undefined);
      assert.match(result.errorMessage ?? "", /^connector_token_pending/);
      assert.equal(calls(), 1);
      assert.equal(isConnectorTokenReady(), false);

      process.env.GROK_CONNECTOR_ACCESS_TOKEN = fakeJwt({ sub: "p", iat: 3, exp: 4 });
      assert.equal(isConnectorTokenReady(), true);
    });
  });

  it("keeps loginRequired for a gate 401 on a deployed app", async () => {
    process.env.GROK_PROJECT_ID = "proj-1";
    await withStubbedGate(401, { errorMessage: "login required" }, async () => {
      const result = await callTool("google_drive_search", {}, {
        ...options,
        token: fakeJwt({ sub: "d", iat: 1, exp: 2 }),
      });
      assert.equal(result.loginRequired, true);
      assert.equal(result.pending, undefined);
    });
  });
});

describe("callTool", () => {
  it("resolves ok:false for non-serializable args instead of rejecting", async () => {
    process.env.GROK_CONNECTORS_URL = "https://connectors.invalid.example";
    try {
      const circular: Record<string, unknown> = {};
      circular.self = circular;
      const result = await callTool(
        "google_drive_search",
        circular as ToolArgs,
        {
          connectorType: ConnectorType.GoogleDrive,
          token: "opaque-token",
        },
      );
      assert.equal(result.ok, false);
      assert.match(result.errorMessage ?? "", /circular/i);
    } finally {
      delete process.env.GROK_CONNECTORS_URL;
    }
  });
});

describe("isLoginRequired", () => {
  it("is true only for login-required failures", () => {
    assert.equal(
      isLoginRequired({ ok: false, data: null, loginRequired: true }),
      true,
    );
    assert.equal(isLoginRequired({ ok: false, data: null }), false);
    assert.equal(
      isLoginRequired({ ok: false, data: null, errorMessage: "access_denied" }),
      false,
    );
    assert.equal(
      isLoginRequired({
        ok: true,
        data: {},
        loginRequired: true,
      } as CallToolResult),
      false,
    );
  });
});

describe("redirectToLoginIfRequired", () => {
  it("no-ops when login is not required", () => {
    let target = "";
    const did = withWindow(
      {
        location: {
```

```
        assign: (u) => {
          target = u;
        },
        href: "https://my-app.grok.me/current",
      },
    },
    () =>
      redirectToLoginIfRequired({
        ok: false,
        data: null,
        errorMessage: "tool error",
      }),
  );
  assert.equal(did, false);
  assert.equal(target, "");
});

it("navigates to the server-built loginUrl in the browser", () => {
  let target = "";
  const did = withWindow(
    {
      location: {
        assign: (u) => {
          target = u;
        },
        href: "https://my-app.grok.me/current",
      },
    },
    () =>
      redirectToLoginIfRequired({
        ok: false,
        data: null,
        loginRequired: true,
        loginUrl: "https://gate.grok.me/__gate/signin?return_to=x",
      }),
  );
  assert.equal(did, true);
  assert.equal(target, "https://gate.grok.me/__gate/signin?return_to=x");
});

it("opens a new tab instead of navigating when framed", () => {
  let assigned = "";
  let opened = "";
  const openedTab: { opener?: unknown } = { opener: "parent" };
  const did = withWindow(
    {
      self: "frame",
      top: "host",
      open: (url) => {
        opened = url;
        return openedTab;
      },
      location: {
        assign: (u) => {
          assigned = u;
        },
        href: "https://my-app.grok.me/current",
      },
    },
    () =>
      redirectToLoginIfRequired({
        ok: false,
        data: null,
        loginRequired: true,
        loginUrl: "https://gate.grok.me/__gate/signin?return_to=x",
      }),
  );
  assert.equal(did, true);
  assert.equal(opened, "https://gate.grok.me/__gate/signin?return_to=x");
  assert.equal(openedTab.opener, null);
  assert.equal(assigned, "");
});

it("falls back to navigation when framed and the popup is blocked", () => {
  let assigned = "";
  const did = withWindow(
    {
      self: "frame",
      top: "host",
      open: () => null,
      location: {
        assign: (u) => {
          assigned = u;
        },
        href: "https://my-app.grok.me/current",
      },
    },
    () =>
      redirectToLoginIfRequired({
        ok: false,
```

```
          data: null,
          loginRequired: true,
          loginUrl: "https://gate.grok.me/__gate/signin?return_to=x",
        }),
    );
    assert.equal(did, true);
    assert.equal(assigned, "https://gate.grok.me/__gate/signin?return_to=x");
  });

  it("returns false when the result has no loginUrl", () => {
    let target = "";
    const did = withWindow(
      {
        location: {
          assign: (u) => {
            target = u;
          },
          href: "https://my-app.grok.me/current",
        },
      },
      () =>
        redirectToLoginIfRequired({ ok: false, data: null, loginRequired: true }),
    );
    assert.equal(did, false);
    assert.equal(target, "");
  });

  it("returns false on the server (no window)", () => {
    const did = redirectToLoginIfRequired({
      ok: false,
      data: null,
      loginRequired: true,
      loginUrl: "https://gate.grok.me/__gate/signin?return_to=x",
    });
    assert.equal(did, false);
  });
});

describe("GoogleCalendarTools", () => {
  it("does not expose a list_events name", () => {
    assert.equal(GoogleCalendarTools.search, "google_calendar_search");
    assert.equal(GoogleCalendarTools.listCalendars, "google_calendar_list_calendars");
    const toolNames: readonly string[] = Object.values(GoogleCalendarTools);
    assert.equal(toolNames.includes("google_calendar_list_events"), false);
  });
});

describe("classifyCallToolError", () => {
  it("returns null for successful results", () => {
    assert.equal(classifyCallToolError({ ok: true, data: {} }), null);
  });

  it("classifies a pending preview token as pending before any other kind", () => {
    const state = classifyCallToolError({
      ok: false,
      data: null,
      pending: true,
      errorMessage: "connector_token_pending: the preview has not received the connector token yet",
    });
    assert.equal(state?.kind, "pending");
    assert.match(state?.message ?? "", /Connecting/);
    assert.match(state?.detail ?? "", /connector_token_pending/);
  });

  it("classifies a deployed missing token as error, not a login CTA", () => {
    const state = classifyCallToolError({
      ok: false,
      data: null,
      errorMessage: "missing_connector_token: open this app through the edge gate",
    });
    assert.equal(state?.kind, "error");
    assert.match(state?.detail ?? "", /missing_connector_token/);
  });

  it("classifies gate loginRequired without a missing-token message as login", () => {
    const state = classifyCallToolError({
      ok: false,
      data: null,
      loginRequired: true,
      errorMessage: "login required",
    });
    assert.equal(state?.kind, "login");
    assert.equal(state?.detail, "login required");
  });

  it("classifies not_connected and failed_precondition", () => {
    assert.equal(
      classifyCallToolError({
        ok: false,
        data: null,
```

```
        errorMessage: "connector_not_connected: Notion",
      })?.kind,
      "not_connected",
    );
    assert.equal(
      classifyCallToolError({
        ok: false,
        data: null,
        errorMessage: "FAILED_PRECONDITION: no connector",
      })?.kind,
      "not_connected",
    );
  });

  it("classifies scope_denied without claiming a missing grant", () => {
    const state = classifyCallToolError({
      ok: false,
      data: null,
      errorMessage: "scope_denied: tool notion-list-recent-pages not in grant scopes",
    });
    assert.equal(state?.kind, "scope_denied");
    assert.match(state?.message ?? "", /tool outside its grant/);
    assert.match(state?.detail ?? "", /notion-list-recent-pages/);
  });

  it("classifies access_denied", () => {
    assert.equal(
      classifyCallToolError({
        ok: false,
        data: null,
        errorMessage: "access_denied",
      })?.kind,
      "access_denied",
    );
  });

  it("falls back to a generic error with the raw message", () => {
    const state = classifyCallToolError({
      ok: false,
      data: null,
      errorMessage: "boom",
    });
    assert.equal(state?.kind, "error");
    assert.equal(state?.message, "boom");
    const empty = classifyCallToolError({ ok: false, data: null });
    assert.equal(empty?.kind, "error");
    assert.equal(empty?.message, "Something went wrong. Try again.");
    assert.equal(empty?.detail, undefined);
  });
});
```

src/lib/app-data/client.server.ts

```
import { createHash } from "node:crypto";
import { getRequest } from "@tanstack/react-start/server";
import {
  assertSameSiteRequest,
  CrossSiteRequestError,
} from "../auth/isolation.server.ts";
import { env, isWorkspacePreview } from "../env.server.ts";
import { assertAppDataServerOnly } from "./server-only.ts";
import {
  CONNECTOR_TOKEN_HEADER,
  CONNECTOR_TOKEN_PENDING_CODE,
  ConnectorType,
  type CallToolOptions,
  type CallToolResult,
  type ToolArgs,
} from "./types.ts";

assertAppDataServerOnly("app-data/client.server");

export const CONNECTORS_HOST_STAGING = "connectors.app-builder-testing.com";
export const CONNECTORS_HOST_PROD = "connectors.grok.me";

function isLoopbackHost(host: string): boolean {
  return host === "localhost" || host === "127.0.0.1" || host === "[::1]";
}

type InboundContext = {
  token: string | null;
  publicHost: string | null;
  connectorsBase: string | null;
};

function connectorsBaseFor(publicHost: string | null): string | null {
  const explicit = env("GROK_CONNECTORS_URL");
  if (explicit) return explicit.replace(/\/+$/, "");

  const host = publicHost?.toLowerCase();
  if (!host || isLoopbackHost(host)) return null;
  if (
    host === "app-builder-testing.com" ||
    host.endsWith(".app-builder-testing.com")
  ) {
    return `https://${CONNECTORS_HOST_STAGING}`;
  }
  if (host === "grok.me" || host.endsWith(".grok.me")) {
    return `https://${CONNECTORS_HOST_PROD}`;
  }
  return null;
}

function tryGetRequest(): Request | null {
  try {
    return getRequest() ?? null;
  } catch {
    return null;
  }
}

function inboundContext(): InboundContext {
  const req = tryGetRequest();
  const xf = req?.headers.get("x-forwarded-host")?.split(",")[0]?.trim();
  const publicHost =
    (xf || req?.headers.get("host") || "").split(":")[0]?.trim() || null;
  const headerToken = req?.headers.get(CONNECTOR_TOKEN_HEADER)?.trim() || null;
  const envToken =
    process.env.NODE_ENV === "production"
      ? null
      : (env("GROK_CONNECTOR_ACCESS_TOKEN") ?? null);
  return {
    token: headerToken ?? envToken,
    publicHost,
    connectorsBase: connectorsBaseFor(publicHost),
  };
}

export function resolveGateAppDataBase(): string | null {
  return inboundContext().connectorsBase;
}

export function getConnectorAccessToken(): string | null {
  return inboundContext().token;
}

export { isWorkspacePreview } from "../env.server.ts";

// Digest of the last preview token the gate answered 401 for. The readiness
// probe reports "not ready" while that exact token is still the one on the
// request, so the client waits for the preview panel to push a fresh token
```

```
// instead of re-calling the gate with a token already known to be rejected.
let rejectedTokenDigest: string | null = null;

function tokenDigest(token: string): string {
  return createHash("sha256").update(token).digest("base64url");
}

function noteTokenRejected(token: string): void {
  rejectedTokenDigest = tokenDigest(token);
}

function noteTokenAccepted(token: string): void {
  if (rejectedTokenDigest === tokenDigest(token)) rejectedTokenDigest = null;
}

/**
 * True when the inbound request carries a connector token the gate has not
 * rejected. This is what the preview readiness probe reports; it never calls
 * the gate.
 */
export function isConnectorTokenReady(): boolean {
  const token = inboundContext().token;
  return token !== null && tokenDigest(token) !== rejectedTokenDigest;
}

type GateJson = {
  ok?: boolean;
  data?: unknown;
  errorMessage?: string;
  loginUrl?: string;
};

async function gatePost(
  ctx: InboundContext,
  body: Record<string, unknown>,
  token: string,
): Promise<{ status: number; json: GateJson }> {
  const base = ctx.connectorsBase;
  if (!base) {
    throw new Error(
      "cannot resolve gate host (missing x-forwarded-host/host on the server request); " +
        "open the app through the gated public URL so the gate can proxy and inject credentials",
    );
  }

  if (!/^https?:\/\//i.test(base)) {
    throw new Error(
      `gate base must be absolute http(s) URL (got ${base}); refusing relative fetch`,
    );
  }

  const headers: Record<string, string> = {
    "content-type": "application/json",
    accept: "application/json",
    authorization: `Bearer ${token}`,
  };
  if (ctx.publicHost) {
    headers["x-forwarded-host"] = ctx.publicHost;
  }

  const res = await fetch(`${base}/call-tool`, {
    method: "POST",
    headers,
    body: JSON.stringify(body),
    redirect: "manual",
  });

  let json: GateJson = {};
  const text = await res.text();
  if (text) {
    try {
      json = JSON.parse(text) as GateJson;
    } catch {
      json = {
        ok: false,
        errorMessage: `gate non-JSON response (HTTP ${res.status}): ${text.slice(0, 200)}`,
      };
    }
  }

  return { status: res.status, json };
}

function gateSigninUrl(ctx: InboundContext): string | undefined {
  const base = ctx.connectorsBase;
  if (!base) return undefined;
  try {
    const connectorsHost = new URL(base).host.toLowerCase();
    const gateHost = connectorsHost.replace(/^connectors\./, "gate.");
    if (gateHost === connectorsHost) return undefined;
```

```
    const publicHost = ctx.publicHost?.toLowerCase();
    const gated =
      publicHost && !isLoopbackHost(publicHost)
        ? `https://${publicHost}`
        : undefined;
    const signin = `https://${gateHost}/__gate/signin`;
    return gated
      ? `${signin}?return_to=${encodeURIComponent(gated)}`
      : signin;
  } catch {
    return undefined;
  }
}

const PENDING_TOKEN_MISSING =
  "the preview has not received the connector token yet; it arrives once " +
  "the connector grant is approved";
const PENDING_TOKEN_REJECTED =
  "the gate rejected the current preview token; the preview panel pushes a " +
  "fresh one on its own schedule";

function pendingTokenResult(reason: string): CallToolResult {
  return {
    ok: false,
    data: null,
    pending: true,
    errorMessage: `${CONNECTOR_TOKEN_PENDING_CODE}: ${reason}`,
  };
}

// Deployed apps only reach here when the request bypassed the gate (the gate
// injects the token on every proxied request), so a sign-in redirect cannot
// fix it: no loginRequired / loginUrl.
function missingAuthResult(): CallToolResult {
  if (isWorkspacePreview()) return pendingTokenResult(PENDING_TOKEN_MISSING);
  return {
    ok: false,
    data: null,
    errorMessage:
      "missing_connector_token: open this app through the edge gate " +
      "(the server must receive x-connector-access-token on the inbound request)",
  };
}

function unauthorizedResult(
  ctx: InboundContext,
  json: GateJson,
  token: string,
): CallToolResult {
  if (isWorkspacePreview()) {
    noteTokenRejected(token);
    return pendingTokenResult(PENDING_TOKEN_REJECTED);
  }
  const loginUrl =
    gateSigninUrl(ctx) ??
    (typeof json.loginUrl === "string" && json.loginUrl
      ? json.loginUrl
      : undefined);
  return {
    ok: false,
    data: null,
    loginRequired: true,
    errorMessage: json.errorMessage ?? "login required",
    ...(loginUrl ? { loginUrl } : {}),
  };
}

function crossSiteBlockedResult(): CallToolResult | null {
  try {
    assertSameSiteRequest();
    return null;
  } catch (e) {
    if (e instanceof CrossSiteRequestError) {
      return { ok: false, data: null, errorMessage: e.message };
    }
    return null;
  }
}

const FAILURE_MEMO_TTL_MS = 5_000;
const failureMemo = new Map<string, { at: number; result: CallToolResult }>();

function tokenIdentityKey(token: string): string {
  const payload = token.split(".")[1];
  if (payload) {
    try {
      const claims: unknown = JSON.parse(
        Buffer.from(payload, "base64url").toString("utf8"),
      );
      if (claims && typeof claims === "object" && !Array.isArray(claims)) {
```

```
        const { sub, team_id: teamId } = claims as {
          sub?: unknown;
          team_id?: unknown;
        };
        if (typeof sub === "string" && sub) {
          return createHash("sha256")
            .update(
              JSON.stringify([sub, typeof teamId === "string" ? teamId : null]),
            )
            .digest("base64url");
        }
      }
    } catch {}
  }
  return createHash("sha256").update(token).digest("base64url");
}

function memoizedFailure(key: string | null): CallToolResult | null {
  if (!key) return null;
  const hit = failureMemo.get(key);
  if (!hit) return null;
  if (Date.now() - hit.at > FAILURE_MEMO_TTL_MS) {
    failureMemo.delete(key);
    return null;
  }
  return hit.result;
}

function memoizeFailure(
  key: string | null,
  result: CallToolResult,
): CallToolResult {
  if (!key) return result;
  const now = Date.now();
  for (const [staleKey, entry] of failureMemo) {
    if (now - entry.at > FAILURE_MEMO_TTL_MS) failureMemo.delete(staleKey);
  }
  failureMemo.set(key, { at: now, result });
  return result;
}

export function failureMemoSize(): number {
  return failureMemo.size;
}

function safeMemoKey(parts: unknown[]): string | null {
  try {
    return JSON.stringify(parts);
  } catch {
    return null;
  }
}

function nonPostBlockedResult(): CallToolResult | null {
  const req = tryGetRequest();
  if (!req || req.method === "POST") return null;
  return {
    ok: false,
    data: null,
    errorMessage:
      `blocked ${req.method} inbound request: connector calls must run inside ` +
      'a createServerFn({ method: "POST" }) handler',
  };
}

export async function callTool(
  toolName: string,
  args: ToolArgs,
  options: CallToolOptions,
): Promise<CallToolResult> {
  const blocked = crossSiteBlockedResult() ?? nonPostBlockedResult();
  if (blocked) return blocked;

  const ctx = inboundContext();
  const token = options.token ?? ctx.token;
  if (!token) {
    return missingAuthResult();
  }

  const connectorType = options.connectorType;
  if (!connectorType) {
    return {
      ok: false,
      data: null,
      errorMessage:
        "connectorType is required: pass the connector type granted to this app " +
        "(e.g. { connectorType: ConnectorType.GoogleDrive })",
    };
  }
```

```
  const memoKey = safeMemoKey([
    toolName,
    args,
    connectorType,
    options?.connectorCatalogId ?? null,
    tokenIdentityKey(token),
  ]);
  const memoized = memoizedFailure(memoKey);
  if (memoized) {
    return memoized;
  }
  const fail = (errorMessage: string): CallToolResult =>
    memoizeFailure(memoKey, { ok: false, data: null, errorMessage });
  if (connectorType === ConnectorType.Mcp && !options?.connectorCatalogId) {
    return {
      ok: false,
      data: null,
      errorMessage: "connectorCatalogId is required when connectorType is Mcp",
    };
  }

  try {
    const { status, json } = await gatePost(
      ctx,
      {
        host: ctx.publicHost ?? undefined,
        connector_type: connectorType,
        tool_name: toolName,
        arguments: args,
        connector_catalog_id: options.connectorCatalogId,
      },
      token,
    );

    if (status === 401) {
      return unauthorizedResult(ctx, json, token);
    }
    noteTokenAccepted(token);
    if (status === 403) {
      return fail(json.errorMessage ?? "access_denied");
    }
    if (json.errorMessage && json.ok === false) {
      return fail(json.errorMessage);
    }
    if (status >= 400 && json.ok !== true) {
      return fail(json.errorMessage ?? `HTTP ${status}`);
    }
    if (json.ok === false) {
      return fail(json.errorMessage ?? "tool error");
    }
    return { ok: true, data: json.data ?? null };
  } catch (e) {
    return fail(e instanceof Error ? e.message : String(e));
  }
}

export {
  ConnectorType,
  GoogleCalendarTools,
  GoogleDriveTools,
  CONNECTOR_TOKEN_HEADER,
} from "./types.ts";
export type {
  CallToolResult,
  CallToolOptions,
  ToolArgs,
  ConnectorTypeName,
} from "./types.ts";
export { isLoginRequired, redirectToLoginIfRequired } from "./login.ts";
```

src/lib/app-data/errors.ts

```ts
import type { CallToolResult } from "./types.ts";
import { isConnectorPending, isLoginRequired } from "./login.ts";

export type CallToolErrorKind =
  | "pending"
  | "login"
  | "not_connected"
  | "scope_denied"
  | "access_denied"
  | "error";

export type CallToolErrorState = {
  kind: CallToolErrorKind;
  message: string;
  detail?: string;
};

type MessageRule = {
  needles: readonly string[];
  kind: CallToolErrorKind;
  message: string;
};

const MESSAGE_RULES: readonly MessageRule[] = [
  {
    needles: ["not_connected", "failed_precondition"],
    kind: "not_connected",
    message: "Connect this connector in Grok to load your data.",
  },
  {
    needles: ["scope_denied"],
    kind: "scope_denied",
    message: "This view isn't available — the app requested a tool outside its grant.",
  },
  {
    needles: ["access_denied"],
    kind: "access_denied",
    message: "You don't have access to this data.",
  },
];

function matchMessageRule(raw: string): MessageRule | undefined {
  return MESSAGE_RULES.find((rule) =>
    rule.needles.some((needle) => raw.includes(needle)),
  );
}

export function classifyCallToolError(
  result: CallToolResult,
): CallToolErrorState | null {
  if (result.ok) return null;
  const detail = result.errorMessage || undefined;
  const raw = (result.errorMessage ?? "").toLowerCase();
  if (isConnectorPending(result)) {
    return { kind: "pending", message: "Connecting to your data…", detail };
  }
  if (raw.includes("missing_connector_token")) {
    return {
      kind: "error",
      message: "Open this app from Grok to load your data.",
      detail,
    };
  }
  if (isLoginRequired(result)) {
    return {
      kind: "login",
      message: "Continue with Grok to load your data.",
      detail,
    };
  }
  const rule = matchMessageRule(raw);
  if (rule) return { kind: rule.kind, message: rule.message, detail };
  return {
    kind: "error",
    message: detail ?? "Something went wrong. Try again.",
    detail,
  };
}
```

src/lib/app-data/index.ts

```
export {
  CONNECTOR_TOKEN_HEADER,
  ConnectorType,
  GoogleCalendarTools,
  GoogleDriveTools,
} from "./types.ts";
export type {
  CallToolOptions,
  CallToolResult,
  ConnectorTypeName,
  ToolArgs,
} from "./types.ts";
export {
  isConnectorPending,
  isLoginRequired,
  redirectToLoginIfRequired,
} from "./login.ts";
export { classifyCallToolError } from "./errors.ts";
export type { CallToolErrorKind, CallToolErrorState } from "./errors.ts";
export { useRefetchWhenConnectorReady } from "./use-connector-readiness.ts";
export type { ConnectorWaitStatus } from "./use-connector-readiness.ts";
```

src/lib/app-data/login.ts

```
import type { CallToolResult } from "./types.ts";

export function isLoginRequired(result: CallToolResult): boolean {
  return result.ok === false && result.loginRequired === true;
}

export function isConnectorPending(result: CallToolResult): boolean {
  return result.ok === false && result.pending === true;
}

export function isFramed(): boolean {
  try {
    return window.self !== window.top;
  } catch {
    return true;
  }
}

export function redirectToLoginIfRequired(result: CallToolResult): boolean {
  if (!isLoginRequired(result)) return false;
  const url = result.loginUrl;
  if (!url) return false;
  if (typeof window === "undefined") return false;
  if (isFramed()) {
    const opened = window.open(url, "_blank");
    if (opened) {
      opened.opener = null;
      return true;
    }
  }
  window.location.assign(url);
  return true;
}
```

src/lib/app-data/readiness-schedule.test.ts

```
import { describe, it } from "node:test";
import assert from "node:assert/strict";
import {
  READINESS_PROBE_DELAYS_MS,
  READINESS_PROBE_MAX_TOTAL_MS,
  readinessProbeDelayMs,
  readinessProbeExhausted,
} from "./readiness-schedule.ts";

describe("readinessProbeDelayMs", () => {
  it("backs off through the schedule and then holds the last delay", () => {
    const last = READINESS_PROBE_DELAYS_MS[READINESS_PROBE_DELAYS_MS.length - 1];
    assert.deepEqual(
      [0, 1, 2, 3, 4, 50].map(readinessProbeDelayMs),
      [...READINESS_PROBE_DELAYS_MS, last, last],
    );
  });

  it("clamps a negative attempt to the first delay", () => {
    assert.equal(readinessProbeDelayMs(-3), READINESS_PROBE_DELAYS_MS[0]);
  });
});

describe("readinessProbeExhausted", () => {
  it("gives up only once the total wait budget has elapsed", () => {
    assert.equal(readinessProbeExhausted(1_000, 1_000), false);
    assert.equal(
      readinessProbeExhausted(1_000, 1_000 + READINESS_PROBE_MAX_TOTAL_MS - 1),
      false,
    );
    assert.equal(
      readinessProbeExhausted(1_000, 1_000 + READINESS_PROBE_MAX_TOTAL_MS),
      true,
    );
  });
});
```

src/lib/app-data/readiness-schedule.ts

```
export const READINESS_PROBE_DELAYS_MS = [1_000, 2_000, 3_000, 5_000] as const;

export const READINESS_PROBE_MAX_TOTAL_MS = 3 * 60_000;

export function readinessProbeDelayMs(attempt: number): number {
  const index = Math.min(
    Math.max(attempt, 0),
    READINESS_PROBE_DELAYS_MS.length - 1,
  );
  return READINESS_PROBE_DELAYS_MS[index];
}

export function readinessProbeExhausted(
  startedAtMs: number,
  nowMs: number,
): boolean {
  return nowMs - startedAtMs >= READINESS_PROBE_MAX_TOTAL_MS;
}
```

src/lib/app-data/readiness.ts

```
import { createServerFn } from "@tanstack/react-start";

export type ConnectorReadiness = { ready: boolean };

export const getConnectorReadiness = createServerFn({ method: "POST" }).handler(
  async (): Promise<ConnectorReadiness> => {
    const { isConnectorTokenReady } = await import("./client.server.ts");
    return { ready: isConnectorTokenReady() };
  },
);
```

src/lib/app-data/server-only.ts

```
export function assertAppDataServerOnly(
  context = "app-data/client.server",
): void {
  if (typeof window !== "undefined") {
    throw new Error(
      `@/lib/${context} is server-only. Call connector tools from a createServerFn handler (dynamic import o
f @/lib/app-data/client.server), never from a React component, useEffect, or browser fetch. Types and login
helpers are client-safe via @/lib/app-data.`,
    );
  }
}

assertAppDataServerOnly("app-data/client.server");
```

src/lib/app-data/types.ts

```
export const CONNECTOR_TOKEN_HEADER = "x-connector-access-token";

export const CONNECTOR_TOKEN_PENDING_CODE = "connector_token_pending";

export const CONNECTOR_TOKEN_READY_EVENT = "grok:connector-token-ready";

export const ConnectorType = {
  GoogleDrive: "GoogleDrive",
  Gmail: "Gmail",
  GoogleCalendar: "GoogleCalendar",
  Outlook: "Outlook",
  OutlookCalendar: "OutlookCalendar",
  MicrosoftTeams: "MicrosoftTeams",
  Mcp: "Mcp",
} as const;

export type ConnectorTypeName =
  (typeof ConnectorType)[keyof typeof ConnectorType];

export const GoogleDriveTools = {
  search: "google_drive_search",
  readFile: "google_drive_read_file",
  listFolder: "google_drive_list_folder",
  createFolder: "google_drive_create_folder",
  trashFile: "google_drive_trash_file",
} as const;

export const GoogleCalendarTools = {
  listCalendars: "google_calendar_list_calendars",
  search: "google_calendar_search",
  getEvent: "google_calendar_get_event",
  availability: "google_calendar_availability",
} as const;

export type CallToolResult<T = unknown> = {
  ok: boolean;
  data: T | null;
  errorMessage?: string;
  loginRequired?: boolean;
  loginUrl?: string;
  pending?: boolean;
};

export type CallToolOptions = {
  connectorType: ConnectorTypeName;
  connectorCatalogId?: string;
  token?: string | null;
};

export type ToolArgs = Record<string, unknown>;
```

src/lib/app-data/use-connector-readiness.ts

```
import { useEffect, useRef, useState } from "react";
import { isFramed } from "./login.ts";
import { getConnectorReadiness } from "./readiness.ts";
import {
  READINESS_PROBE_MAX_TOTAL_MS,
  readinessProbeDelayMs,
  readinessProbeExhausted,
} from "./readiness-schedule.ts";
import { CONNECTOR_TOKEN_READY_EVENT } from "./types.ts";

export type ConnectorWaitStatus =
  | "idle"
  | "waiting"
  | "timed_out"
  | "not_embedded";

export const READINESS_PROBE_TIMEOUT_MS = 10_000;

function withTimeout<T>(promise: Promise<T>, ms: number): Promise<T | null> {
  return new Promise((resolve) => {
    const timer = setTimeout(() => resolve(null), ms);
    const settle = (value: T | null) => {
      clearTimeout(timer);
      resolve(value);
    };
    promise.then(settle, () => settle(null));
  });
}

async function isConnectorReady(): Promise<boolean> {
  const result = await withTimeout(
    getConnectorReadiness(),
    READINESS_PROBE_TIMEOUT_MS,
  );
  return result?.ready === true;
}

/**
 * While `waiting` is true (a connector call returned `pending`), probes the
 * server for the connector token and calls `refetch` once it is present. The
 * probe is a header check on the app's own server — it never reaches the gate.
 * A `connector-token-ready` bridge event from the Grok preview chrome triggers
 * `refetch` immediately. A top-level page (download/export, local dev, the
 * sandbox's own `npm run preview`) is not framed by any preview, so no token
 * can ever arrive: the hook reports `not_embedded` without probing. Any framed
 * page probes, even when the parent origin cannot be resolved (empty referrer,
 * no `ancestorOrigins`): the token comes through the preview proxy, and the
 * bridge event is only the faster signal.
 */
export function useRefetchWhenConnectorReady(
  waiting: boolean,
  refetch: () => unknown,
): ConnectorWaitStatus {
  const refetchRef = useRef(refetch);
  const [timedOut, setTimedOut] = useState(false);
  const [notEmbedded, setNotEmbedded] = useState(false);

  useEffect(() => {
    refetchRef.current = refetch;
  }, [refetch]);

  useEffect(() => {
    if (!waiting) return;
    if (!isFramed()) {
      setNotEmbedded(true);
      return () => setNotEmbedded(false);
    }
    let cancelled = false;
    let refetching = false;
    let attempt = 0;
    let timer: ReturnType<typeof setTimeout> | undefined;
    const startedAt = Date.now();

    const runRefetch = async () => {
      if (refetching) return;
      refetching = true;
      try {
        await refetchRef.current();
      } catch {
        // The query owns its own error state; a failed refetch must not stop
        // the probe loop.
      } finally {
        refetching = false;
      }
    };
    const schedule = () => {
      timer = setTimeout(probe, readinessProbeDelayMs(attempt));
      attempt += 1;
```

```
    };
    const probe = async () => {
      if (cancelled || readinessProbeExhausted(startedAt, Date.now())) return;
      const ready = await isConnectorReady();
      if (cancelled) return;
      if (ready) await runRefetch();
      if (!cancelled) schedule();
    };
    const onTokenReady = () => {
      void runRefetch();
    };
    // Time-based terminal state: a probe that never settles or a refetch that
    // throws cannot leave the UI on "waiting" forever.
    const deadline = setTimeout(() => {
      if (!cancelled) setTimedOut(true);
    }, READINESS_PROBE_MAX_TOTAL_MS);

    window.addEventListener(CONNECTOR_TOKEN_READY_EVENT, onTokenReady);
    schedule();
    return () => {
      cancelled = true;
      clearTimeout(deadline);
      if (timer !== undefined) clearTimeout(timer);
      window.removeEventListener(CONNECTOR_TOKEN_READY_EVENT, onTokenReady);
      setTimedOut(false);
    };
  }, [waiting]);

  if (!waiting) return "idle";
  if (notEmbedded) return "not_embedded";
  return timedOut ? "timed_out" : "waiting";
}
```

src/lib/auth/client.ts

```
import { genericOAuthClient } from "better-auth/client/plugins";
import { createAuthClient } from "better-auth/react";
import { runPreSignInSignOut, runSignOut } from "../../../scripts/sign-out-plan.mjs";
import { GROK_PROVIDERS } from "./providers";

/**
 * Better Auth client for this React SPA (browser-side).
 *
 * Talks to this app's OWN Better Auth at same-origin `/api/auth/*`. In the live
 * preview the app is an embedded iframe with PARTITIONED cookies, so after a
 * popup sign-in it can't read the session cookie — it authenticates with a
 * bearer token instead (captured from the popup, see `signIn`). The `onRequest`
 * hook attaches that token when present; when deployed (cookie auth) no token
 * is stored, so nothing changes.
 *
 * To sign out call `signOut()` below, NOT `authClient.signOut()`: the raw call
 * leaves the bearer token in place, and `onRequest` keeps re-attaching it, so
 * the visitor stays signed in.
 */
export const authClient = createAuthClient({
  plugins: [genericOAuthClient()],
  fetchOptions: {
    onRequest(ctx) {
      const token = getBearerToken();
      if (token) ctx.headers.set("Authorization", `Bearer ${token}`);
      return ctx;
    },
  },
});

/**
 * True when sign-in UI should be shown — i.e. whenever `VITE_AUTH_ENABLED` is
 * not `"false"`. The shipped template sets it to `"false"`
 * (`.grok/app-env.json`), which selects the dev user (see `use-current-user`);
 * with the key removed, sign-in is real in preview (baked preview client) and
 * when deployed (injected per-app client).
 */
export const authEnabled = import.meta.env.VITE_AUTH_ENABLED !== "false";

/** The upstream providers to render sign-in buttons for. */
export { GROK_PROVIDERS };

// ── Live-preview bearer token ──────────────────────────────────────────
// The embedded preview iframe has partitioned cookies, so we keep the session's
// bearer token in sessionStorage and attach it to every Better Auth request (and
// to server functions, via `@/lib/auth/middleware`). Empty everywhere except the
// preview after a popup sign-in, so the cookie path is untouched elsewhere.
const BEARER_KEY = "grok-auth.bearer-token";

/** The stored preview bearer token, or null. */
export function getBearerToken(): string | null {
  if (typeof window === "undefined") return null;
  try {
    return window.sessionStorage.getItem(BEARER_KEY);
  } catch {
    return null;
  }
}

function setBearerToken(token: string | null): void {
  if (typeof window === "undefined") return;
  try {
    if (token) window.sessionStorage.setItem(BEARER_KEY, token);
    else window.sessionStorage.removeItem(BEARER_KEY);
  } catch {
    /* storage unavailable — ignore */
  }
}

/**
 * The sandbox live preview runs this app inside an iframe on a `*.grok-sandbox.com`
 * host, where a full-page redirect to the broker can't work — so sign-in uses a
 * popup there and a normal redirect everywhere else.
 */
function inLivePreview(): boolean {
  return (
    typeof window !== "undefined" &&
    window.location.hostname.endsWith(".grok-sandbox.com")
  );
}

/** Message the popup posts back to the opener once sign-in completes. */
type PopupMessage = { source: "grok-auth-popup"; token: string | null; error?: string };

/**
 * Start sign-in with one upstream provider (`providerId` from `GROK_PROVIDERS`),
 * federating through the Grok auth broker.
 *
```

```
 * - **Live preview** (`*.grok-sandbox.com` iframe): opens a POPUP to
 *   `/auth/popup`, served by the template Vite plugin (see `vite.config.ts` +
 *   `popup.server.ts`) — 302s to the broker/upstream login (no app chrome) and,
 *   on return, posts the session bearer token back. We store it and refresh the
 *   session; no top-level navigation of the iframe to the broker.
 * - **Deployed** (and local non-iframe): a normal full-page redirect into the broker.
 *
 * Either way it clears any existing local session FIRST so switching providers
 * actually switches identity.
 */
export async function signIn(
  providerId: string,
  opts: { callbackURL?: string; errorCallbackURL?: string } = {},
): Promise<void> {
  const callbackURL = opts.callbackURL ?? "/";
  const errorCallbackURL = opts.errorCallbackURL ?? "/";

  // Open the popup SYNCHRONOUSLY on the user gesture — before any await
  // (including signOut). Awaiting first drops user-gesture privilege in some
  // browsers when the opener is a cross-origin live-preview iframe.
  const popup = inLivePreview() ? openSignInPopup(providerId) : null;

  // Clear any prior session so switching providers actually switches identity.
  // Bounded because the popup is already open — a request that never settles
  // would leave it hanging — but bounded PER ENVIRONMENT: only the server can
  // end a deployed session, so cutting it short at the preview's 1.5s would
  // start OAuth with the old session still live.
  await runPreSignInSignOut({
    livePreview: inLivePreview(),
    hasBearer: Boolean(getBearerToken()),
    requestSignOut: () => authClient.signOut(),
    clearToken: () => setBearerToken(null),
  });

  if (inLivePreview()) {
    if (!popup) throw new Error("Pop-up blocked — allow pop-ups for sign-in");
    const token = await waitForPopupToken(popup);
    if (!token) throw new Error("Sign-in was cancelled or failed");
    setBearerToken(token);
    // Refresh the client session store with the bearer attached (onRequest).
    // Avoid a full iframe reload when we're already on the destination — that
    // reload was the slow "still loading after the popup closed" feeling.
    try {
      await authClient.getSession();
    } catch {
      /* session store will recover on next useSession fetch */
    }
    if (typeof window !== "undefined") {
      const dest = new URL(callbackURL, window.location.origin);
      const here = window.location;
      if (dest.origin !== here.origin || dest.pathname !== here.pathname || dest.search !== here.search) {
        window.location.href = callbackURL;
      }
    }
    return;
  }

  const { data, error } = await authClient.signIn.oauth2({
    providerId,
    callbackURL,
    errorCallbackURL,
  });
  if (error) throw new Error(error.message ?? "Sign-in failed");
  if (data?.url) window.location.href = data.url;
}

/**
 * Open `/auth/popup` in a new window. Must run synchronously inside the click
 * handler (no await before this). The path is served by the template Vite
 * plugin (`authPopupPlugin` in vite.config.ts) — NOT by a React route.
 *
 * Opens the real URL directly (not about:blank → assign). From a cross-origin
 * iframe the about:blank dance often fails on the first click and the window
 * ends up showing the app shell.
 */
function openSignInPopup(providerId: string): Window | null {
  const origin = window.location.origin;
  const url = `${origin}/auth/popup?providerId=${encodeURIComponent(providerId)}`;
  // Unique name per attempt so a prior attempt stuck on the SPA is not reused.
  const name = `grok-signin-${Date.now()}`;
  return window.open(url, name, "popup,width=500,height=650");
}

/**
 * Wait for the popup's completion page to postMessage the session bearer (or
 * for the user to dismiss the popup).
 */
function waitForPopupToken(popup: Window): Promise<string | null> {
  return new Promise((resolve) => {
    const origin = window.location.origin;
```

```
    let settled = false;
    let closeTimer: number | undefined;
    const settle = (token: string | null) => {
      if (settled) return;
      settled = true;
      cleanup();
      resolve(token);
    };
    const onMessage = (event: MessageEvent) => {
      if (event.origin !== origin) return;
      const data = event.data as PopupMessage | undefined;
      if (!data || data.source !== "grok-auth-popup") return;
      settle(data.token ?? null);
    };
    // Fallback when the user dismisses the popup. Grace period lets the
    // completion page's postMessage win over a racing `popup.closed`.
    const pollTimer = window.setInterval(() => {
      if (!popup.closed) return;
      window.clearInterval(pollTimer);
      closeTimer = window.setTimeout(() => settle(null), 400);
    }, 300);
    function cleanup() {
      window.clearInterval(pollTimer);
      if (closeTimer !== undefined) window.clearTimeout(closeTimer);
      window.removeEventListener("message", onMessage);
    }
    window.addEventListener("message", onMessage);
  });
}

/**
 * Sign out of THIS app's local session, clear the preview token, then redirect.
 *
 * Use this, never `authClient.signOut()` — see the note on `authClient`.
 * Sequencing lives in `scripts/sign-out-plan.mjs` so it can be unit-tested.
 *
 * **Rejects when deployed if the server never confirms.** There the session is
 * an HttpOnly cookie only the server can clear, so redirecting anyway would
 * report a sign-out that did not happen. `<UserButton />` handles that for you;
 * a hand-rolled control must catch it and let the visitor retry. In the live
 * preview the local clear is sufficient, so it always resolves.
 */
export async function signOut(redirectTo = "/"): Promise<void> {
  await runSignOut({
    livePreview: inLivePreview(),
    hasBearer: Boolean(getBearerToken()),
    // Better Auth resolves with `{ error }` instead of rejecting, so surface a
    // failed response as a rejection for the sequence to act on.
    requestSignOut: async () => {
      const { error } = await authClient.signOut();
      if (error) throw new Error(error.message ?? "Sign-out failed");
    },
    clearToken: () => setBearerToken(null),
    redirect: () => {
      window.location.href = redirectTo;
    },
  });
}
```

src/lib/auth/email-password.ts

```
/**
 * Local email/password sign-in (this app's Better Auth DB — not the broker).
 *
 * Off by default. To enable: set `emailAndPasswordEnabled` to `true` below,
 * then build sign-up / sign-in forms with `authClient.signUp.email` /
 * `authClient.signIn.email` from `@/lib/auth/client` (see the auth skill).
 *
 * Do NOT edit `server.ts` for this — that file is frozen pre-wired config.
 */
export const emailAndPasswordEnabled = false;
```

src/lib/auth/gate-identity.server.ts

```
import {
  importJWK,
  jwtVerify,
  type JWK,
  type JWTVerifyGetKey,
} from "jose";
import { env, isWorkspacePreview } from "../env.server.ts";

export const GATE_IDENTITY_HEADER = "x-grok-identity";
export const GATE_JWKS_PATH = "/__gate/identity-key";

const JWKS_CACHE_TTL_MS = 300_000;
const PREVIEW_AUDIENCE = "preview";
export const PREVIEW_GATE_ORIGIN = "http://127.0.0.1:6014";
const FALLBACK_EMAIL_DOMAIN = "viewer.grok.invalid";
const FALLBACK_NAME = "Grok user";

export type GateIdentity = {
  sub: string;
  email: string | null;
  name: string | null;
  teamId: string | null;
};

export type GateJwks = { keys: JWK[] };

export type JwksFetch = (url: string) => Promise<GateJwks | null>;

export function gateIdentityEnabled(): boolean {
  return env("VITE_AUTH_ENABLED") !== "false";
}

export function gateTokenAudience(): string {
  if (isWorkspacePreview()) return PREVIEW_AUDIENCE;
  return `app:${env("GROK_PROJECT_ID")}`;
}

async function defaultJwksFetch(url: string): Promise<GateJwks | null> {
  try {
    const res = await fetch(url, {
      headers: { accept: "application/json" },
      redirect: "manual",
    });
    if (!res.ok) return null;
    const body = (await res.json()) as GateJwks;
    return Array.isArray(body?.keys) ? body : null;
  } catch {
    return null;
  }
}

const jwksCache = new Map<string, { jwks: GateJwks; fetchedAt: number }>();

export function gateKeyResolver(
  url: string,
  jwksFetch: JwksFetch = defaultJwksFetch,
): JWTVerifyGetKey {
  return async (protectedHeader) => {
    const kid =
      typeof protectedHeader.kid === "string" ? protectedHeader.kid : undefined;
    const findKey = (jwks: GateJwks): JWK | undefined =>
      jwks.keys.find(
        (k) =>
          k.kty === "OKP" && k.crv === "Ed25519" && (!kid || k.kid === kid),
      );

    let entry = jwksCache.get(url);
    if (!entry || Date.now() - entry.fetchedAt > JWKS_CACHE_TTL_MS) {
      const jwks = await jwksFetch(url);
      if (jwks) {
        entry = { jwks, fetchedAt: Date.now() };
        jwksCache.set(url, entry);
      }
    }

    let key = entry ? findKey(entry.jwks) : undefined;
    if (!key) {
      const jwks = await jwksFetch(url);
      if (jwks) {
        entry = { jwks, fetchedAt: Date.now() };
        jwksCache.set(url, entry);
        key = findKey(jwks);
      }
    }
    if (!key) {
      throw new Error("no gate identity key matches the token kid");
    }
    return importJWK(key, "EdDSA");
```

```
  };
}

export type VerifyGateIdentityTokenOptions = {
  issuer: string;
  audience: string;
  getKey: JWTVerifyGetKey;
};

export async function verifyGateIdentityToken(
  token: string,
  options: VerifyGateIdentityTokenOptions,
): Promise<GateIdentity | null> {
  try {
    const { payload } = await jwtVerify(token, options.getKey, {
      algorithms: ["EdDSA"],
      issuer: options.issuer,
      audience: options.audience,
      requiredClaims: ["sub", "iat", "exp"],
      maxTokenAge: "10 minutes",
    });
    const sub = typeof payload.sub === "string" ? payload.sub.trim() : "";
    if (!sub) return null;
    return {
      sub,
      email: typeof payload.email === "string" ? payload.email : null,
      name: typeof payload.name === "string" ? payload.name : null,
      teamId: typeof payload.team_id === "string" ? payload.team_id : null,
    };
  } catch {
    return null;
  }
}

type GateEndpoints = { issuer: string; jwksUrl: string };

export function resolveGateEndpoints(headers: Headers): GateEndpoints | null {
  const explicit = env("GROK_GATE_ORIGIN");
  if (explicit) {
    const origin = explicit.replace(/\/+$/, "");
    return { issuer: origin, jwksUrl: `${origin}${GATE_JWKS_PATH}` };
  }

  if (isWorkspacePreview()) {
    return {
      issuer: PREVIEW_GATE_ORIGIN,
      jwksUrl: `${PREVIEW_GATE_ORIGIN}${GATE_JWKS_PATH}`,
    };
  }

  const xf = headers.get("x-forwarded-host")?.split(",")[0]?.trim();
  const host = (xf || headers.get("host") || "")
    .split(":")[0]
    ?.trim()
    .toLowerCase();
  if (!host) return null;

  let issuer: string | null = null;
  if (
    host === "app-builder-testing.com" ||
    host.endsWith(".app-builder-testing.com")
  ) {
    issuer = "https://gate.app-builder-testing.com";
  } else if (host === "grok.me" || host.endsWith(".grok.me")) {
    issuer = "https://gate.grok.me";
  }
  if (!issuer) return null;

  return { issuer, jwksUrl: `${issuer}${GATE_JWKS_PATH}` };
}

export type GateLinkedAccount = { providerId: string; accountId: string };

export function sessionBoundToGateIdentity(
  accounts: readonly GateLinkedAccount[],
  identitySub: string,
  gateProviderId: string,
): boolean {
  return accounts.some(
    (account) =>
      account.providerId === gateProviderId &&
      account.accountId === identitySub,
  );
}

export async function gateIdentityFromHeaders(
  headers: Headers,
  jwksFetch?: JwksFetch,
): Promise<GateIdentity | null> {
  if (!gateIdentityEnabled()) return null;
```

```
  const token = headers.get(GATE_IDENTITY_HEADER)?.trim();
  if (!token) return null;
  const endpoints = resolveGateEndpoints(headers);
  if (!endpoints) return null;
  return verifyGateIdentityToken(token, {
    issuer: endpoints.issuer,
    audience: gateTokenAudience(),
    getKey: gateKeyResolver(endpoints.jwksUrl, jwksFetch),
  });
}

export type GateUserInfo = {
  id: string;
  email: string;
  emailVerified: boolean;
  name: string;
};

export function gateIdentityUserInfo(identity: GateIdentity): GateUserInfo {
  return {
    id: identity.sub,
    email: (
      identity.email ?? `${identity.sub}@${FALLBACK_EMAIL_DOMAIN}`
    ).toLowerCase(),
    emailVerified: Boolean(identity.email),
    name: identity.name ?? FALLBACK_NAME,
  };
}
```

src/lib/auth/gate-identity.test.ts

```
import { describe, it } from "node:test";
import assert from "node:assert/strict";
import { generateKeyPairSync, type KeyObject } from "node:crypto";
import { SignJWT, exportJWK, type JWK } from "jose";
import {
  gateIdentityEnabled,
  gateIdentityFromHeaders,
  gateIdentityUserInfo,
  gateKeyResolver,
  gateTokenAudience,
  sessionBoundToGateIdentity,
  verifyGateIdentityToken,
  type GateJwks,
} from "./gate-identity.server.ts";

const ISSUER = "https://gate.app-builder-testing.com";
const AUDIENCE = "app:proj-123";

type TestKey = { privateKey: KeyObject; jwk: JWK; kid: string };

async function makeKey(kid: string): Promise<TestKey> {
  const { publicKey, privateKey } = generateKeyPairSync("ed25519");
  const jwk = await exportJWK(publicKey);
  return {
    privateKey,
    kid,
    jwk: { ...jwk, alg: "EdDSA", use: "sig", kid },
  };
}

type SignOptions = {
  issuer?: string;
  audience?: string;
  expiresIn?: number;
  issuedAt?: number;
  omitExp?: boolean;
};

async function signToken(
  key: TestKey,
  claims: Record<string, unknown>,
  options: SignOptions = {},
): Promise<string> {
  const now = Math.floor(Date.now() / 1000);
  const jwt = new SignJWT(claims)
    .setProtectedHeader({ alg: "EdDSA", kid: key.kid })
    .setIssuer(options.issuer ?? ISSUER)
    .setAudience(options.audience ?? AUDIENCE)
    .setIssuedAt(options.issuedAt ?? now);
  if (!options.omitExp) {
    jwt.setExpirationTime((options.issuedAt ?? now) + (options.expiresIn ?? 300));
  }
  return jwt.sign(key.privateKey);
}

function staticJwks(keys: JWK[]): {
  fetchImpl: (url: string) => Promise<GateJwks | null>;
  calls: () => number;
} {
  let count = 0;
  return {
    fetchImpl: async () => {
      count += 1;
      return { keys };
    },
    calls: () => count,
  };
}

let urlCounter = 0;
function uniqueUrl(): string {
  urlCounter += 1;
  return `https://test-${urlCounter}.invalid/__gate/identity-key`;
}

describe("verifyGateIdentityToken", () => {
  it("returns the identity for a valid token", async () => {
    const key = await makeKey("k1");
    const token = await signToken(key, {
      sub: "user-1",
      email: "viewer@example.com",
      name: "Viewer",
      team_id: "team-9",
      jti: "j1",
    });
    const identity = await verifyGateIdentityToken(token, {
      issuer: ISSUER,
      audience: AUDIENCE,
```

```
    getKey: gateKeyResolver(uniqueUrl(), staticJwks([key.jwk]).fetchImpl),
  });
  assert.deepEqual(identity, {
    sub: "user-1",
    email: "viewer@example.com",
    name: "Viewer",
    teamId: "team-9",
  });
});

it("omits optional claims as nulls", async () => {
  const key = await makeKey("k1");
  const token = await signToken(key, { sub: "user-2", jti: "j2" });
  const identity = await verifyGateIdentityToken(token, {
    issuer: ISSUER,
    audience: AUDIENCE,
    getKey: gateKeyResolver(uniqueUrl(), staticJwks([key.jwk]).fetchImpl),
  });
  assert.deepEqual(identity, {
    sub: "user-2",
    email: null,
    name: null,
    teamId: null,
  });
});

it("rejects a wrong audience", async () => {
  const key = await makeKey("k1");
  const token = await signToken(
    key,
    { sub: "user-1" },
    { audience: "app:other-project" },
  );
  const identity = await verifyGateIdentityToken(token, {
    issuer: ISSUER,
    audience: AUDIENCE,
    getKey: gateKeyResolver(uniqueUrl(), staticJwks([key.jwk]).fetchImpl),
  });
  assert.equal(identity, null);
});

it("rejects a wrong issuer", async () => {
  const key = await makeKey("k1");
  const token = await signToken(
    key,
    { sub: "user-1" },
    { issuer: "https://evil.example.com" },
  );
  const identity = await verifyGateIdentityToken(token, {
    issuer: ISSUER,
    audience: AUDIENCE,
    getKey: gateKeyResolver(uniqueUrl(), staticJwks([key.jwk]).fetchImpl),
  });
  assert.equal(identity, null);
});

it("rejects an expired token", async () => {
  const key = await makeKey("k1");
  const past = Math.floor(Date.now() / 1000) - 3600;
  const token = await signToken(
    key,
    { sub: "user-1" },
    { issuedAt: past, expiresIn: 300 },
  );
  const identity = await verifyGateIdentityToken(token, {
    issuer: ISSUER,
    audience: AUDIENCE,
    getKey: gateKeyResolver(uniqueUrl(), staticJwks([key.jwk]).fetchImpl),
  });
  assert.equal(identity, null);
});

it("rejects a token without exp", async () => {
  const key = await makeKey("k1");
  const token = await signToken(key, { sub: "user-1" }, { omitExp: true });
  const identity = await verifyGateIdentityToken(token, {
    issuer: ISSUER,
    audience: AUDIENCE,
    getKey: gateKeyResolver(uniqueUrl(), staticJwks([key.jwk]).fetchImpl),
  });
  assert.equal(identity, null);
});

it("rejects a token signed by a different key with the same kid", async () => {
  const trusted = await makeKey("k1");
  const attacker = await makeKey("k1");
  const token = await signToken(attacker, { sub: "user-1" });
  const identity = await verifyGateIdentityToken(token, {
    issuer: ISSUER,
    audience: AUDIENCE,
```

```
      getKey: gateKeyResolver(uniqueUrl(), staticJwks([trusted.jwk]).fetchImpl),
    });
    assert.equal(identity, null);
  });

  it("refetches the JWKS when the kid rotates", async () => {
    const oldKey = await makeKey("k-old");
    const newKey = await makeKey("k-new");
    const url = uniqueUrl();
    let calls = 0;
    let published: JWK[] = [oldKey.jwk];
    const fetchImpl = async (): Promise<GateJwks> => {
      calls += 1;
      return { keys: published };
    };
    const getKey = gateKeyResolver(url, fetchImpl);

    const first = await verifyGateIdentityToken(
      await signToken(oldKey, { sub: "user-1" }),
      { issuer: ISSUER, audience: AUDIENCE, getKey },
    );
    assert.equal(first?.sub, "user-1");
    assert.equal(calls, 1);

    published = [newKey.jwk];
    const second = await verifyGateIdentityToken(
      await signToken(newKey, { sub: "user-1" }),
      { issuer: ISSUER, audience: AUDIENCE, getKey },
    );
    assert.equal(second?.sub, "user-1");
    assert.equal(calls, 2);
  });
});

describe("gateIdentityFromHeaders", () => {
  it("verifies the header token end to end and fails closed without it", async () => {
    const key = await makeKey("k1");
    const { fetchImpl } = staticJwks([key.jwk]);
    process.env.GROK_PROJECT_ID = "proj-123";
    process.env.GROK_GATE_ORIGIN = ISSUER;
    try {
      const token = await signToken(key, {
        sub: "user-1",
        email: "viewer@example.com",
      });
      const withToken = await gateIdentityFromHeaders(
        new Headers({ "x-grok-identity": token }),
        fetchImpl,
      );
      assert.equal(withToken?.sub, "user-1");

      const withoutToken = await gateIdentityFromHeaders(
        new Headers(),
        fetchImpl,
      );
      assert.equal(withoutToken, null);
    } finally {
      delete process.env.GROK_PROJECT_ID;
      delete process.env.GROK_GATE_ORIGIN;
    }
  });

  it("activates on a deployed-shaped request without GROK_GATE_ORIGIN", async () => {
    const key = await makeKey("k-deployed");
    const fetchedFrom: string[] = [];
    const fetchImpl = async (url: string): Promise<GateJwks> => {
      fetchedFrom.push(url);
      return { keys: [key.jwk] };
    };
    process.env.GROK_PROJECT_ID = "proj-123";
    delete process.env.GROK_GATE_ORIGIN;
    try {
      const token = await signToken(key, {
        sub: "user-1",
        email: "viewer@example.com",
      });
      const identity = await gateIdentityFromHeaders(
        new Headers({
          host: "my-app.app-builder-testing.com",
          "x-grok-identity": token,
        }),
        fetchImpl,
      );
      assert.equal(identity?.sub, "user-1");
      assert.equal(
        fetchedFrom[0],
        "https://gate.app-builder-testing.com/__gate/identity-key",
      );
    } finally {
      delete process.env.GROK_PROJECT_ID;
```

```
  }
});

it("verifies a preview-audience token with no gate env vars via the loopback default", async () => {
  const key = await makeKey("k-preview");
  const fetchedFrom: string[] = [];
  const fetchImpl = async (url: string): Promise<GateJwks> => {
    fetchedFrom.push(url);
    return { keys: [key.jwk] };
  };
  delete process.env.GROK_PROJECT_ID;
  delete process.env.GROK_GATE_ORIGIN;
  const token = await signToken(
    key,
    { sub: "user-1" },
    { issuer: "http://127.0.0.1:6014", audience: "preview" },
  );
  const identity = await gateIdentityFromHeaders(
    new Headers({
      host: "my-session.grok-sandbox.com",
      "x-grok-identity": token,
    }),
    fetchImpl,
  );
  assert.equal(identity?.sub, "user-1");
  assert.equal(fetchedFrom[0], "http://127.0.0.1:6014/__gate/identity-key");
});

it("rejects a wrong-issuer token in the loopback default mode", async () => {
  const key = await makeKey("k-preview-iss");
  const { fetchImpl } = staticJwks([key.jwk]);
  delete process.env.GROK_PROJECT_ID;
  delete process.env.GROK_GATE_ORIGIN;
  const token = await signToken(
    key,
    { sub: "user-1" },
    { issuer: ISSUER, audience: "preview" },
  );
  const identity = await gateIdentityFromHeaders(
    new Headers({ "x-grok-identity": token }),
    fetchImpl,
  );
  assert.equal(identity, null);
});

it("verifies a preview-audience token when only GROK_GATE_ORIGIN is set", async () => {
  const key = await makeKey("k1");
  const { fetchImpl } = staticJwks([key.jwk]);
  delete process.env.GROK_PROJECT_ID;
  process.env.GROK_GATE_ORIGIN = ISSUER;
  try {
    const token = await signToken(
      key,
      { sub: "user-1" },
      { audience: "preview" },
    );
    const identity = await gateIdentityFromHeaders(
      new Headers({ "x-grok-identity": token }),
      fetchImpl,
    );
    assert.deepEqual(identity, {
      sub: "user-1",
      email: null,
      name: null,
      teamId: null,
    });
  } finally {
    delete process.env.GROK_GATE_ORIGIN;
  }
});

it("rejects a preview-audience token when GROK_PROJECT_ID is set", async () => {
  const key = await makeKey("k1");
  const { fetchImpl } = staticJwks([key.jwk]);
  process.env.GROK_PROJECT_ID = "proj-123";
  process.env.GROK_GATE_ORIGIN = ISSUER;
  try {
    const token = await signToken(
      key,
      { sub: "user-1" },
      { audience: "preview" },
    );
    const identity = await gateIdentityFromHeaders(
      new Headers({ "x-grok-identity": token }),
      fetchImpl,
    );
    assert.equal(identity, null);
  } finally {
    delete process.env.GROK_PROJECT_ID;
    delete process.env.GROK_GATE_ORIGIN;
```

```
    }
  });

  it("rejects an app-audience token in preview mode", async () => {
    const key = await makeKey("k1");
    const { fetchImpl } = staticJwks([key.jwk]);
    delete process.env.GROK_PROJECT_ID;
    process.env.GROK_GATE_ORIGIN = ISSUER;
    try {
      const token = await signToken(key, { sub: "user-1" });
      const identity = await gateIdentityFromHeaders(
        new Headers({ "x-grok-identity": token }),
        fetchImpl,
      );
      assert.equal(identity, null);
    } finally {
      delete process.env.GROK_GATE_ORIGIN;
    }
  });
});

describe("gateIdentityEnabled", () => {
  it("is enabled by default with no gate env vars", () => {
    delete process.env.GROK_PROJECT_ID;
    delete process.env.GROK_GATE_ORIGIN;
    assert.equal(gateIdentityEnabled(), true);
  });

  it("is disabled when VITE_AUTH_ENABLED is false", () => {
    process.env.VITE_AUTH_ENABLED = "false";
    try {
      assert.equal(gateIdentityEnabled(), false);
    } finally {
      delete process.env.VITE_AUTH_ENABLED;
    }
  });
});

describe("gateTokenAudience", () => {
  it("pins app:<id> when GROK_PROJECT_ID is set, even alongside GROK_GATE_ORIGIN", () => {
    process.env.GROK_PROJECT_ID = "proj-123";
    process.env.GROK_GATE_ORIGIN = ISSUER;
    try {
      assert.equal(gateTokenAudience(), "app:proj-123");
    } finally {
      delete process.env.GROK_PROJECT_ID;
      delete process.env.GROK_GATE_ORIGIN;
    }
  });

  it("pins preview when GROK_PROJECT_ID is unset", () => {
    delete process.env.GROK_PROJECT_ID;
    process.env.GROK_GATE_ORIGIN = ISSUER;
    try {
      assert.equal(gateTokenAudience(), "preview");
    } finally {
      delete process.env.GROK_GATE_ORIGIN;
    }
  });
});

describe("gateIdentityUserInfo", () => {
  it("falls back to a synthetic email and name for sub-only claims", () => {
    assert.deepEqual(
      gateIdentityUserInfo({
        sub: "User-1",
        email: null,
        name: null,
        teamId: null,
      }),
      {
        id: "User-1",
        email: "user-1@viewer.grok.invalid",
        emailVerified: false,
        name: "Grok user",
      },
    );
  });

  it("keeps real claims, lowercasing the email and marking it verified", () => {
    assert.deepEqual(
      gateIdentityUserInfo({
        sub: "user-1",
        email: "Viewer@Example.com",
        name: "Viewer",
        teamId: "team-9",
      }),
      {
        id: "user-1",
        email: "viewer@example.com",
```

```
        emailVerified: true,
        name: "Viewer",
      },
    );
  });
});

describe("sessionBoundToGateIdentity", () => {
  const provider = "grok-gate";

  it("keeps the session when it is bound to the same gate sub", () => {
    assert.equal(
      sessionBoundToGateIdentity(
        [
          { providerId: "google", accountId: "g-1" },
          { providerId: provider, accountId: "user-1" },
        ],
        "user-1",
        provider,
      ),
      true,
    );
  });

  it("rotates when the session belongs to a different gate sub", () => {
    assert.equal(
      sessionBoundToGateIdentity(
        [{ providerId: provider, accountId: "user-1" }],
        "user-2",
        provider,
      ),
      false,
    );
  });

  it("rotates when the session user has no gate-bound account", () => {
    assert.equal(
      sessionBoundToGateIdentity(
        [{ providerId: "google", accountId: "user-1" }],
        "user-1",
        provider,
      ),
      false,
    );
    assert.equal(sessionBoundToGateIdentity([], "user-1", provider), false);
  });
});
```

src/lib/auth/gate-session-marker.ts

```
/**
 * Client-readable marker for gate-materialized sessions ("Sign in with Grok"
 * zero-click sessions minted by `gate-session.server.ts`). Signing out of a
 * gate session is a no-op — the next request re-materializes it from
 * `x-grok-identity` — so `UserButton` uses this to hide its sign-out control.
 * `__Host-` prefixed like the other auth cookies: browsers reject a `__Host-`
 * cookie carrying a `Domain`, so an untrusted sibling `*.grok.me` app cannot
 * plant a parent-domain copy that the host-only clear could never expire.
 * Client-safe: no server imports.
 */
export const GATE_SESSION_MARKER_COOKIE = "__Host-grok_gate_session";

export function hasGateSessionMarker(): boolean {
  if (typeof document === "undefined") return false;
  return document.cookie
    .split(";")
    .some((pair) => pair.trim().startsWith(`${GATE_SESSION_MARKER_COOKIE}=`));
}
```

src/lib/auth/gate-session.server.ts

```
import type { BetterAuthPlugin } from "better-auth";
import { createAuthMiddleware, getSessionFromCtx } from "better-auth/api";
import {
  parseSetCookieHeader,
  setRequestCookie,
  setSessionCookie,
} from "better-auth/cookies";
import { handleOAuthUserInfo } from "better-auth/oauth2";
import {
  GATE_IDENTITY_HEADER,
  gateIdentityEnabled,
  gateIdentityFromHeaders,
  gateIdentityUserInfo,
  sessionBoundToGateIdentity,
} from "./gate-identity.server";
import { GATE_SESSION_MARKER_COOKIE } from "./gate-session-marker";

export const GATE_PROVIDER_ID = "grok-gate";
const GATE_ACCOUNT_ISSUER = "https://grok.com";
const LOG = "[gate-identity]";

type GateAccount = Parameters<typeof handleOAuthUserInfo>[1]["account"];

/**
 * Emit the signed session cookie so the browser actually receives it.
 *
 * `setSessionCookie` writes into the Better Auth middleware header bag, but on
 * TanStack Start that bag is not always copied onto the final HTTP response
 * (the response can end up with no `Set-Cookie`). Sign the token ourselves and
 * push it through TanStack's `setCookie` + `responseHeaders` so both the
 * framework cookie store and any after-hooks see it.
 */
async function emitSessionCookie(
  ctx: Parameters<Parameters<typeof createAuthMiddleware>[0]>[0],
  sessionTokenName: string,
  sessionToken: string,
): Promise<string | null> {
  const attributes = ctx.context.authCookies.sessionToken.attributes;
  const maxAge = ctx.context.sessionConfig.expiresIn;
  const cookieOptions = {
    ...attributes,
    maxAge,
  };

  let signedCookie: string;
  try {
    signedCookie = await ctx.setSignedCookie(
      sessionTokenName,
      sessionToken,
      ctx.context.secret,
      cookieOptions,
    );
  } catch (err) {
    console.error(`${LOG} setSignedCookie failed`, err);
    return null;
  }

  const sessionValue = parseSetCookieHeader(signedCookie).get(
    sessionTokenName,
  )?.value;
  if (!sessionValue) {
    console.error(`${LOG} signed Set-Cookie missing session token value`, {
      cookiePreview: signedCookie.slice(0, 120),
    });
    return null;
  }

  // Primary path: TanStack Start's response cookie store (reaches the browser).
  try {
    const { setCookie } = await import("@tanstack/react-start/server");
    setCookie(sessionTokenName, sessionValue, {
      path: cookieOptions.path ?? "/",
      httpOnly: cookieOptions.httpOnly ?? true,
      secure: cookieOptions.secure ?? true,
      sameSite: (cookieOptions.sameSite as "lax" | "strict" | "none") ?? "lax",
      maxAge: typeof maxAge === "number" ? maxAge : undefined,
      domain: cookieOptions.domain,
    });
  } catch (err) {
    console.error(`${LOG} TanStack setCookie failed`, err);
  }

  // Also stash on Better Auth responseHeaders so after-hooks (tanstackStartCookies)
  // can forward it if they run.
  try {
    const responseHeaders = ctx.context.responseHeaders;
    if (responseHeaders) {
      responseHeaders.append("set-cookie", signedCookie);
```

```
    } else {
      console.error(`${LOG} ctx.context.responseHeaders is missing`);
    }
  } catch (err) {
    console.error(`${LOG} responseHeaders.append(set-cookie) failed`, err);
  }

  return sessionValue;
}

/**
 * Expire the previous user's `session_data` cookie cache after an identity
 * swap. The cache is signed against the old session and outlives it (5-min
 * TTL), so without this `/get-session` keeps serving the replaced user.
 * Mirrors `emitSessionCookie`'s dual-path delivery: TanStack's response
 * cookie store plus Better Auth's `responseHeaders` bag.
 */
async function expireSessionDataCookie(
  ctx: Parameters<Parameters<typeof createAuthMiddleware>[0]>[0],
  cookie: { name: string; attributes: { path?: string; secure?: boolean } },
): Promise<void> {
  const path = cookie.attributes.path ?? "/";
  const secure = cookie.attributes.secure ?? true;
  try {
    const { setCookie } = await import("@tanstack/react-start/server");
    setCookie(cookie.name, "", {
      path,
      httpOnly: true,
      secure,
      sameSite: "lax",
      maxAge: 0,
    });
  } catch (err) {
    console.error(`${LOG} TanStack setCookie (expire session_data) failed`, err);
  }
  try {
    ctx.context.responseHeaders?.append(
      "set-cookie",
      `${cookie.name}=; Path=${path}; HttpOnly; ` +
        `${secure ? "Secure; " : ""}SameSite=Lax; Max-Age=0`,
    );
  } catch (err) {
    console.error(
      `${LOG} responseHeaders.append (expire session_data) failed`,
      err,
    );
  }
}

/**
 * Write or clear the client-readable gate-session marker
 * (`gate-session-marker.ts`), through the same dual-path delivery as
 * `emitSessionCookie`. Not HttpOnly by design: `UserButton` reads it to hide
 * sign-out for gate sessions (signing out would re-materialize instantly).
 */
async function writeGateMarkerCookie(
  ctx: Parameters<Parameters<typeof createAuthMiddleware>[0]>[0],
  clear: boolean,
): Promise<void> {
  const sessionMaxAge = ctx.context.sessionConfig.expiresIn;
  const maxAge = clear
    ? 0
    : typeof sessionMaxAge === "number"
      ? sessionMaxAge
      : undefined;
  const value = clear ? "" : "1";
  try {
    const { setCookie } = await import("@tanstack/react-start/server");
    setCookie(GATE_SESSION_MARKER_COOKIE, value, {
      path: "/",
      httpOnly: false,
      secure: true,
      sameSite: "lax",
      maxAge,
    });
  } catch (err) {
    console.error(`${LOG} TanStack setCookie (gate marker) failed`, err);
  }
  try {
    ctx.context.responseHeaders?.append(
      "set-cookie",
      `${GATE_SESSION_MARKER_COOKIE}=${value}; Path=/; Secure; SameSite=Lax` +
        (maxAge === undefined ? "" : `; Max-Age=${maxAge}`),
    );
  } catch (err) {
    console.error(`${LOG} responseHeaders.append (gate marker) failed`, err);
  }
}

/**
```

```
 * Clear a stale marker when a `/get-session` arrives without `x-grok-identity`:
 * the browser is no longer behind a gate viewer (returned anonymously, or the
 * session is a broker one), so sign-out must not stay hidden. Emits the
 * Max-Age=0 clear only when the marker is actually on the request.
 */
async function clearGateMarkerIfPresent(
  ctx: Parameters<Parameters<typeof createAuthMiddleware>[0]>[0],
  inbound: Headers,
): Promise<void> {
  const cookieHeader = inbound.get("cookie") ?? "";
  if (!cookieHeader.includes(`${GATE_SESSION_MARKER_COOKIE}=`)) return;
  await writeGateMarkerCookie(ctx, true);
}

/** Drop a cookie from the request `Cookie` header (inverse of `setRequestCookie`). */
function removeRequestCookie(headers: Headers, name: string): void {
  const cookieHeader = headers.get("cookie");
  if (!cookieHeader) return;
  const kept = cookieHeader
    .split(";")
    .map((pair) => pair.trim())
    .filter((pair) => pair && !pair.startsWith(`${name}=`));
  if (kept.length > 0) {
    headers.set("cookie", kept.join("; "));
  } else {
    headers.delete("cookie");
  }
}

export function gateIdentitySessions() {
  return {
    id: "grok-gate-identity",
    hooks: {
      before: [
        {
          matcher: (ctx: { path?: string }) => ctx.path === "/get-session",
          handler: createAuthMiddleware(async (ctx) => {
            if (!gateIdentityEnabled()) return;
            const inbound = ctx.request?.headers ?? ctx.headers;
            if (!inbound) {
              console.error(`${LOG} no request headers on /get-session`);
              return;
            }
            // Bearer auth (live-preview popup) already carries a session — leave it alone.
            if (inbound.get("authorization")) return;
            if (!inbound.get(GATE_IDENTITY_HEADER)) {
              await clearGateMarkerIfPresent(ctx, inbound);
              return;
            }

            const identity = await gateIdentityFromHeaders(inbound);
            if (!identity) {
              console.error(
                `${LOG} ${GATE_IDENTITY_HEADER} present but verification failed`,
              );
              return;
            }

            const sessionCookieName = ctx.context.authCookies.sessionToken.name;
            const cookieHeader = inbound.get("cookie") ?? "";
            if (cookieHeader.includes(`${sessionCookieName}=`)) {
              const existing = await getSessionFromCtx(ctx).catch((err) => {
                console.error(`${LOG} getSessionFromCtx failed`, err);
                return null;
              });
              if (existing?.session && existing.user) {
                const accounts = await ctx.context.internalAdapter
                  .findAccounts(existing.user.id)
                  .catch((err) => {
                    console.error(`${LOG} findAccounts failed`, err);
                    return null;
                  });
                if (!accounts) {
                  console.error(
                    `${LOG} could not load accounts for existing session user`,
                    { userId: existing.user.id },
                  );
                  return;
                }
                if (
                  sessionBoundToGateIdentity(
                    accounts,
                    identity.sub,
                    GATE_PROVIDER_ID,
                  )
                ) {
                  await writeGateMarkerCookie(ctx, false);
                  return;
                }
                await ctx.context.internalAdapter
```

```
                .deleteSession(existing.session.token)
                .catch((err) => {
                  console.error(
                    `${LOG} deleteSession (stale non-gate session) failed`,
                    err,
                  );
                  return null;
                });
            }
          }

          try {
            const result = await handleOAuthUserInfo(ctx, {
              userInfo: gateIdentityUserInfo(identity),
              account: {
                providerId: GATE_PROVIDER_ID,
                issuer: GATE_ACCOUNT_ISSUER,
                accountId: identity.sub,
              } as GateAccount,
            });
            if (result.error || !result.data) {
              console.error(`${LOG} handleOAuthUserInfo failed`, {
                error: result.error,
                hasData: Boolean(result.data),
                sub: identity.sub,
              });
              return;
            }

            // Persist session rows + internal newSession state.
            await setSessionCookie(ctx, result.data);

            // Explicitly sign the token and emit Set-Cookie — do NOT rely on
            // reading it back from ctx.context.responseHeaders (often empty
            // here, which previously caused a silent signed-out render).
            const sessionValue = await emitSessionCookie(
              ctx,
              sessionCookieName,
              result.data.session.token,
            );
            if (!sessionValue) {
              console.error(
                `${LOG} session created in DB but cookie was not emitted`,
                { userId: result.data.user.id },
              );
              return;
            }

            await writeGateMarkerCookie(ctx, false);

            const sessionDataCookie = ctx.context.authCookies.sessionData;
            await expireSessionDataCookie(ctx, sessionDataCookie);

            // Inject the cookie into this request so the rest of /get-session
            // resolves the newly created session in the same round-trip.
            const headers = new Headers(
              Object.fromEntries(inbound.entries()),
            );
            setRequestCookie(headers, sessionCookieName, sessionValue);
            removeRequestCookie(headers, sessionDataCookie.name);
            return { context: { headers } };
          } catch (err) {
            console.error(`${LOG} gate identity session hook threw`, err);
            return;
          }
        }),
      },
    ],
  },
 } satisfies BetterAuthPlugin;
}
```

src/lib/auth/gates.tsx

```
import { useState, useSyncExternalStore, type ReactNode } from "react";
import { Navigate } from "@tanstack/react-router";
import { GROK_PROVIDERS, authEnabled, signIn, signOut } from "./client";
import { hasGateSessionMarker } from "./gate-session-marker";
import { resolveSignInGateState } from "./sign-in-gate";
import { useCurrentUser, useCurrentUserState } from "./use-current-user";

const subscribeToNothing = () => () => {};
const noGateSessionOnServer = () => false;

/**
 * Auth state components — plain wrappers around `useCurrentUserState()`.
 *
 * With auth on, visitors are signed out until they authenticate — in the sandbox
 * live preview too, which does real sign-in. The shared dev user appears only
 * when auth is disabled (`VITE_AUTH_ENABLED=false`, the shipped default).
 * While the session is still resolving, gates that care about signed-out state
 * render nothing so there's no signed-out flash on hard reload.
 */

/** Where `RedirectToSignIn` sends signed-out visitors. Create this route. */
export const SIGN_IN_PATH = "/login";

/** Render children only when a user is present (real session, or the disabled-auth dev user). */
export function SignedIn({ children }: { children: ReactNode }) {
  const { user } = useCurrentUserState();
  return user ? <>{children}</> : null;
}

/**
 * Render children only once we KNOW the visitor is signed out (`isPending` has
 * cleared and there is no user). Hidden while the session is still loading.
 */
export function SignedOut({ children }: { children: ReactNode }) {
  const { user, isPending } = useCurrentUserState();
  if (isPending || user) return null;
  return <>{children}</>;
}

/**
 * Client-side redirect to the sign-in route (TanStack `<Navigate>` — NOT a full
 * `window.location` reload). A hard navigation re-bootstraps the SPA and re-runs
 * session loading, which feels like a second "Loading…" on /login.
 *
 * Guard routes by waiting out `isPending` first (see `use-current-user`), then
 * render this.
 */
export function RedirectToSignIn({ to = SIGN_IN_PATH }: { to?: string }) {
  return <Navigate to={to} />;
}

export function SignInGate({
  children,
  fallback,
}: {
  children: ReactNode;
  fallback?: ReactNode;
}) {
  const { user, isPending } = useCurrentUserState();
  const state = resolveSignInGateState({ isPending, hasUser: user !== null });
  if (state === "pending") return null;
  if (state === "signed_in") return <>{children}</>;
  return <>{fallback ?? <SignInButtons />}</>;
}

export function SignInButtons() {
  return (
    <div className="flex w-full max-w-sm flex-col gap-2">
      {GROK_PROVIDERS.map((p) => (
        <button
          key={p.providerId}
          type="button"
          onClick={() => signIn(p.providerId, { callbackURL: "/" })}
          className="w-full cursor-pointer rounded-md border border-neutral-300 px-4 py-2 hover:bg-neutral-1
00 dark:border-neutral-700 dark:hover:bg-neutral-900"
        >
          Continue with {p.label}
        </button>
      ))}
    </div>
  );
}

/**
 * Minimal signed-in identity chip + sign-out. Restyle freely (see the
 * `design-ui` skill). Sign-out is only shown when auth is enabled (the
 * disabled-auth dev user has nothing to sign out of) and the session is not
 * gate-materialized — behind the gate the next request signs the viewer
```

```
 * straight back in, so a sign-out control there is a broken loop.
 */
export function UserButton() {
  const user = useCurrentUser();
  // Sign-out can take a moment (and can fail when deployed), so the control
  // shows it is working and cannot be fired twice.
  const [signingOut, setSigningOut] = useState(false);
  const gateSession = useSyncExternalStore(
    subscribeToNothing,
    hasGateSessionMarker,
    noGateSessionOnServer,
  );
  if (!user) return null;
  const label = user.displayName ?? user.primaryEmail ?? "Account";
  return (
    <div className="flex items-center gap-2">
      {user.profileImageUrl ? (
        <img
          src={user.profileImageUrl}
          alt=""
          className="h-8 w-8 rounded-full object-cover"
        />
      ) : (
        <span className="grid h-8 w-8 place-items-center rounded-full bg-black/10 text-sm font-medium dark:b
g-white/20">
          {label.charAt(0).toUpperCase()}
        </span>
      )}
      <span className="text-sm font-medium">{label}</span>
      {authEnabled && !gateSession && (
        <button
          type="button"
          disabled={signingOut}
          onClick={() => {
            setSigningOut(true);
            // Success navigates away; on failure re-enable so it can be retried.
            void signOut().catch(() => setSigningOut(false));
          }}
          className="cursor-pointer text-sm underline-offset-4 opacity-70 hover:underline disabled:cursor-wa
it disabled:no-underline"
        >
          {signingOut ? "Signing out…" : "Sign out"}
        </button>
      )}
    </div>
  );
}
```

src/lib/auth/isolation.server.ts

```
import { getRequest } from "@tanstack/react-start/server";

/**
 * Fetch-Metadata sibling isolation — **server-only** (`.server.ts` suffix).
 *
 * MUST keep the `.server` suffix: this file imports `@tanstack/react-start/server`
 * (`getRequest` → Node `AsyncLocalStorage`). If it is imported from a dual
 * client/server module under a non-`.server` name, Vite ships it to the browser
 * and the app dies with: `AsyncLocalStorage is not a constructor`.
 *
 * Apps deployed on `*.grok.me` are "same-site" to each other but MUTUALLY
 * UNTRUSTED, and a `SameSite=Lax` session cookie IS sent on same-site
 * subrequests — so without this, a malicious sibling could make a SCRIPTED
 * (fetch/XHR/form-POST) request to this app's server functions and ride this
 * app's session cookie.
 *
 * We allow only: same-origin requests (this app's own client), non-browser
 * requests (SSR / server-to-server, which send no `Sec-Fetch-Site`), and
 * top-level GET navigations (how the OAuth callback and normal page loads
 * arrive). Every cross-site / same-site *scripted* request is rejected.
 * Together with `__Host-` cookies and Better Auth's `trustedOrigins`, this
 * closes the sibling-tenant attack surface. Enforced at the `authMiddleware`
 * chokepoint (see `middleware.ts`).
 */
export class CrossSiteRequestError extends Error {
  readonly status = 403;
  constructor() {
    super("Forbidden: cross-site request blocked");
    this.name = "CrossSiteRequestError";
  }
}

/** Throw `CrossSiteRequestError` for a scripted cross-site/sibling request. */
export function assertSameSiteRequest(): void {
  const request = getRequest();
  if (!request) return; // no request context (e.g. build) — nothing to guard
  const h = request.headers;
  const site = h.get("sec-fetch-site");
  // Non-browser client (no header), the app's own origin, or a direct
  // (address-bar/bookmark) load are all fine.
  if (!site || site === "same-origin" || site === "none") return;
  // A top-level GET navigation (e.g. the broker's OAuth callback redirect) is
  // fine even when it's cross-site; scripted requests never set navigate mode.
  const dest = h.get("sec-fetch-dest");
  const isTopLevelGet =
    h.get("sec-fetch-mode") === "navigate" &&
    request.method === "GET" &&
    dest !== "object" &&
    dest !== "embed";
  if (isTopLevelGet) return;
  throw new CrossSiteRequestError();
}
```

src/lib/auth/middleware.ts

```
import { createMiddleware } from "@tanstack/react-start";

/**
 * Auth middleware for server functions — the standard way to get the caller's
 * verified user id. When deployed the session cookie is same-origin and rides
 * along automatically. In the live preview the client also forwards the bearer
 * token (partitioned cookies) via the `.client` hook below — call sites do not
 * thread it themselves.
 *
 *   import { createServerFn } from "@tanstack/react-start";
 *   import { getSql } from "@/lib/db";
 *   import { authMiddleware } from "@/lib/auth/middleware";
 *
 *   export const listTodos = createServerFn({ method: "GET" })
 *     .middleware([authMiddleware])
 *     .handler(async ({ context }) => {
 *       const sql = await getSql();
 *       return sql`select * from todos where user_id = ${context.userId}`;
 *     });
 *
 * Signed out with auth on (live preview included) -> throws `UnauthorizedError`
 * (see `verify.server.ts`). With auth disabled (`VITE_AUTH_ENABLED=false`, the
 * shipped default) it resolves the shared dev user — but throws instead when a
 * `DATABASE_URL` is also set, so an app without sign-in must not use this at
 * all. On the auth-on path, use it on every server function that touches
 * per-user data and scope every query by `context.userId`.
 */
export const authMiddleware = createMiddleware({ type: "function" })
  .client(async ({ next }) => {
    // Live preview (partitioned iframe): the session rides a bearer token, not a
    // cookie, so forward it to the server. Null when deployed (cookie auth), so
    // this is a no-op there.
    const { getBearerToken } = await import("./client");
    return next({ sendContext: { bearerToken: getBearerToken() ?? undefined } });
  })
  .server(async ({ next, context }) => {
    // ONLY import `*.server` modules here. This file is dual client/server
    // (bearer hook on the client). A plain `./isolation` path was renamed to
    // `isolation.server.ts` — keep this import in sync so image `tsc` resolves
    // it, and so Vite does not ship `@tanstack/react-start/server` to the browser.
    const { assertSameSiteRequest } = await import("./isolation.server");
    const { requireUserId } = await import("./verify.server");
    // Reject scripted cross-site/sibling requests before touching per-user data.
    assertSameSiteRequest();
    const userId = await requireUserId(context.bearerToken);
    return next({ context: { userId } });
  });
```

src/lib/auth/pglite-dialect.ts

```
/**
 * Kysely dialect for Better Auth over the app's embedded PGLite instance.
 * Lazy: resolves `getClient` on first connection so migrations can finish first.
 */
import type { PGlite } from "@electric-sql/pglite";
import {
  CompiledQuery,
  type DatabaseConnection,
  type DatabaseIntrospector,
  type Dialect,
  type Driver,
  type Kysely,
  PostgresAdapter,
  PostgresIntrospector,
  PostgresQueryCompiler,
  type QueryCompiler,
  type QueryResult,
  type TransactionSettings,
} from "kysely";

type Client = PGlite;

/** Factory used by `auth/server.ts`: `pgliteDialect(() => getPglite())`. */
export function pgliteDialect(
  getClient: () => Promise<Client> | Client,
): Dialect {
  return {
    createAdapter: () => new PostgresAdapter(),
    createDriver: () => new LazyPGliteDriver(getClient),
    createQueryCompiler: (): QueryCompiler => new PostgresQueryCompiler(),
    createIntrospector: (db: Kysely<unknown>): DatabaseIntrospector =>
      new PostgresIntrospector(db),
  };
}

class LazyPGliteDriver implements Driver {
  private client: Client | undefined;
  private connection: PGliteConnection | undefined;
  private queue: Array<(con: PGliteConnection) => void> = [];

  constructor(private readonly getClient: () => Promise<Client> | Client) {}

  async init(): Promise<void> {
    this.client = await this.getClient();
  }

  async acquireConnection(): Promise<DatabaseConnection> {
    if (this.client === undefined) {
      this.client = await this.getClient();
    }
    if (this.connection !== undefined) {
      return new Promise((resolve) => {
        this.queue.push(resolve);
      });
    }
    this.connection = new PGliteConnection(this.client);
    return this.connection;
  }

  async releaseConnection(connection: DatabaseConnection): Promise<void> {
    if (connection !== this.connection) {
      throw new Error("Invalid connection");
    }
    const next = this.queue.shift();
    if (next === undefined) {
      this.connection = undefined;
      return;
    }
    next(this.connection);
  }

  async beginTransaction(
    conn: DatabaseConnection,
    settings: TransactionSettings,
  ): Promise<void> {
    const c = conn as PGliteConnection;
    if (settings.isolationLevel) {
      await c.executeQuery(
        CompiledQuery.raw(
          `start transaction isolation level ${settings.isolationLevel}`,
        ),
      );
    } else {
      await c.executeQuery(CompiledQuery.raw("begin"));
    }
  }

  async commitTransaction(conn: DatabaseConnection): Promise<void> {
```

```
    await (conn as PGliteConnection).executeQuery(CompiledQuery.raw("commit"));
  }

  async rollbackTransaction(conn: DatabaseConnection): Promise<void> {
    await (conn as PGliteConnection).executeQuery(
      CompiledQuery.raw("rollback"),
    );
  }

  async destroy(): Promise<void> {
    // Do not close the client: it is the shared getPglite() singleton used by
    // app SQL (getSql). Only drop our local handle so auth teardown cannot
    // poison the rest of the process.
    this.client = undefined;
    this.connection = undefined;
    this.queue = [];
  }
}

class PGliteConnection implements DatabaseConnection {
  constructor(private readonly client: Client) {}

  async executeQuery<O>(compiledQuery: CompiledQuery): Promise<QueryResult<O>> {
    const result = await this.client.query(compiledQuery.sql, [
      ...compiledQuery.parameters,
    ]);
    if (result.affectedRows) {
      return {
        numAffectedRows: BigInt(result.affectedRows),
        rows: result.rows as O[],
      };
    }
    return { rows: result.rows as O[] };
  }

  async *streamQuery<O>(
    compiledQuery: CompiledQuery,
    chunkSize: number,
  ): AsyncIterableIterator<QueryResult<O>> {
    if (!Number.isInteger(chunkSize) || chunkSize <= 0) {
      throw new Error("chunkSize must be a positive integer");
    }
    const result = await this.client.query(compiledQuery.sql, [
      ...compiledQuery.parameters,
    ]);
    for (let i = 0; i < result.rows.length; i += chunkSize) {
      yield { rows: result.rows.slice(i, i + chunkSize) as O[] };
    }
  }
}
```

src/lib/auth/popup.server.ts

```
/**
 * Live-preview sign-in popup — server-only (NEVER import from the client).
 *
 * The sandbox preview runs the app in a partitioned iframe, so OAuth must happen
 * in a top-level popup (first-party cookies). This handler is the ENTIRE popup
 * document — no React shell:
 *
 *   Phase 1 (`?providerId=…`): start OAuth server-side and 302 straight to the
 *     broker / upstream login page. The popup never paints the app.
 *   Phase 2 (`?done=1`): after the broker round-trip, emit a tiny HTML page that
 *     posts the session token to the opener and closes. No SPA hydrate, no
 *     server-fn round-trip.
 *
 * Wired automatically by the Vite `authPopupPlugin` in `vite.config.ts` during
 * `npm run dev` (live preview). Do NOT create `src/routes/auth/popup.tsx` — a
 * React route here paints the full app shell in the popup. The opener lives in
 * `client.ts` (`signIn` → `openSignInPopup`).
 */
import { auth, SESSION_TOKEN_COOKIE } from "./server";

/** Message shape the popup posts to the opener (must match `client.ts`). */
type PopupMessage = {
  source: "grok-auth-popup";
  token: string | null;
  error?: string;
};

/**
 * Handle `GET /auth/popup`. Invoked by the Vite `authPopupPlugin` (dev / live
 * preview). Do not re-export this from a React route file.
 */
export async function handleAuthPopupRequest(request: Request): Promise<Response> {
  const url = new URL(request.url);
  const done = url.searchParams.get("done") === "1";

  if (done) {
    const errored = url.searchParams.has("error");
    const token = errored ? null : readCookie(request, SESSION_TOKEN_COOKIE);
    const message: PopupMessage = {
      source: "grok-auth-popup",
      token,
      ...(errored ? { error: url.searchParams.get("error") ?? "sign_in_failed" } : {}),
    };
    return new Response(completionHtml(message), {
      status: 200,
      headers: {
        "content-type": "text/html; charset=utf-8",
        // Never cache a page that embeds a session token.
        "cache-control": "no-store",
      },
    });
  }

  const providerId = url.searchParams.get("providerId")?.trim();
  if (!providerId) {
    return new Response("Missing providerId", {
      status: 400,
      headers: { "content-type": "text/plain; charset=utf-8" },
    });
  }

  // Stay first-party for the callback so the session cookie lands in THIS popup.
  const back = `${url.origin}/auth/popup?done=1`;
  try {
    const apiRes = await auth.api.signInWithOAuth2({
      body: {
        providerId,
        callbackURL: back,
        errorCallbackURL: `${back}&error=1`,
      },
      // Forward the preview host so Better Auth derives the correct baseURL /
      // redirect_uri for the dynamic `*.grok-sandbox.com` origin.
      headers: request.headers,
      asResponse: true,
    });

    if (!apiRes.ok) {
      const detail = await apiRes.text().catch(() => "");
      return completionResponse({
        source: "grok-auth-popup",
        token: null,
        error: detail || `oauth_init_failed_${apiRes.status}`,
      });
    }

    const body = (await apiRes.json().catch(() => null)) as {
      url?: string;
    } | null;
```

```
    const location = body?.url;
    if (!location) {
      return completionResponse({
        source: "grok-auth-popup",
        token: null,
        error: "oauth_init_missing_url",
      });
    }

    // 302 to the broker (which headlessly forwards to Google/X). Forward any
    // Set-Cookie (OAuth state / PKCE) so the callback can complete in this popup.
    const headers = new Headers({ location, "cache-control": "no-store" });
    for (const cookie of apiRes.headers.getSetCookie()) {
      headers.append("set-cookie", cookie);
    }
    return new Response(null, { status: 302, headers });
  } catch (err) {
    const message = err instanceof Error ? err.message : "oauth_init_threw";
    return completionResponse({
      source: "grok-auth-popup",
      token: null,
      error: message,
    });
  }
}

function completionResponse(message: PopupMessage): Response {
  return new Response(completionHtml(message), {
    status: 200,
    headers: {
      "content-type": "text/html; charset=utf-8",
      "cache-control": "no-store",
    },
  });
}

/** Minimal HTML: postMessage the token to the opener and close. No React. */
function completionHtml(message: PopupMessage): string {
  // JSON is safe inside a <script type="application/json"> block; the inline
  // script only reads it. Avoids escaping pitfalls of embedding in JS source.
  const payload = JSON.stringify(message).replace(/</g, "\\u003c");
  return `<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Signing in…</title>
<style>
  html,body{margin:0;min-height:100%;background:#0b0b0c;color:#a1a1aa;
    font:14px/1.5 -apple-system,BlinkMacSystemFont,"Segoe UI",Roboto,Helvetica,Arial,sans-serif}
  main{min-height:100vh;display:grid;place-items:center;padding:1.5rem;text-align:center}
</style>
</head>
<body>
<main><p>Signing you in…</p></main>
<script type="application/json" id="grok-auth-popup-msg">${payload}</script>
<script>
(function () {
  var el = document.getElementById("grok-auth-popup-msg");
  var msg = { source: "grok-auth-popup", token: null };
  try { if (el && el.textContent) msg = JSON.parse(el.textContent); } catch (e) {}
  try {
    if (window.opener) window.opener.postMessage(msg, window.location.origin);
  } catch (e) {}
  try { window.close(); } catch (e) {}
})();
</script>
</body>
</html>`;
}

/** Read a single cookie value from the request (handles `=` inside values). */
function readCookie(request: Request, name: string): string | null {
  const header = request.headers.get("cookie");
  if (!header) return null;
  for (const part of header.split(";")) {
    const trimmed = part.trim();
    if (!trimmed) continue;
    const eq = trimmed.indexOf("=");
    if (eq <= 0) continue;
    if (trimmed.slice(0, eq) !== name) continue;
    const raw = trimmed.slice(eq + 1);
    try {
      return decodeURIComponent(raw);
    } catch {
      return raw;
    }
  }
  return null;
}
```

src/lib/auth/preview.ts

```
/**
 * Shared LIVE-PREVIEW OAuth client (server-only — NEVER import from the client).
 *
 * The sandbox serves each live preview on a dynamic `https://*.grok-sandbox.com`
 * URL, which can't be pre-registered per app. The broker instead exposes ONE
 * shared "preview" client that accepts any
 * `https://*.grok-sandbox.com/api/auth/oauth2/callback/*`
 * (broker: `app-builder-deployer/auth/src/preview-oauth.ts`). Baking it here lets
 * the live preview do REAL sign-in — no demo/mock users — with no platform
 * injection. When deployed the deployer injects a per-app
 * `GROK_AUTH_*` that overrides these (see `server.ts`).
 *
 * These MUST equal the broker's `GROK_PREVIEW_CLIENT_ID` /
 * `GROK_PREVIEW_CLIENT_SECRET` (set in the broker's Vercel env; the broker stores
 * only the secret's `base64url(SHA-256)` hash). This is a dedicated, low-privilege
 * client (preview-only, `*.grok-sandbox.com`) — rotate it by regenerating the
 * broker env var and this constant together.
 */
export const PREVIEW_CLIENT_ID = "grok_preview";
export const PREVIEW_CLIENT_SECRET =
  "8bcdb7fc5a33874ad933ca568918d5790388a0795e44c4d1dea691f801b17ec5";

/** The shared auth broker issuer (OIDC discovery lives under it). */
export const GROK_ISSUER_DEFAULT = "https://auth.grok.me";

/**
 * Host patterns whose callbacks the preview client accepts. Better Auth derives
 * the live preview's real origin from the request host and validates it against
 * this list (wildcard-matched), so the OAuth `redirect_uri` becomes the concrete
 * `https://<preview-host>/api/auth/oauth2/callback/...` the broker allows.
 */
export const PREVIEW_ALLOWED_HOSTS = ["*.grok-sandbox.com"] as const;
```

src/lib/auth/provider.tsx

```
import type { ReactNode } from "react";

/**
 * App-wide client provider mounted once near the root (in `src/routes/__root.tsx`):
 *
 *   <AuthProvider><Outlet /></AuthProvider>
 *
 * Better Auth's React client (`@/lib/auth/client`) needs NO context provider —
 * its `useSession()` works standalone — so this is a passthrough today. It's
 * kept as the single, stable mount point for any future client-side providers
 * (e.g. a toast or theme provider) without churning the root shell.
 */
export function AuthProvider({ children }: { children: ReactNode }) {
  return <>{children}</>;
}
```

src/lib/auth/providers.ts

```
/**
 * The upstream identity providers this app offers for sign-in (via the broker).
 *
 * Source of truth for BOTH the server (`server.ts`, one `genericOAuth` provider
 * per entry) and the client (`client.ts` / sign-in buttons). Kept in its own
 * dependency-free module so the client can import it without pulling the
 * server-only Better Auth instance (and `pg`) into the browser bundle.
 *
 * Each app federates to the shared **auth broker** (`GROK_AUTH_ISSUER`), which
 * holds the real Google/X secrets. The app never sees them — it only knows its
 * own per-app client id/secret and which upstream to ask the broker for (`idp`).
 *
 * To add an upstream (e.g. GitHub) once the broker supports it: add one entry
 * here (`{ providerId: "grok-github", idp: "github", label: "GitHub" }`). The
 * `providerId` is this app's local id and the OAuth callback path segment
 * (`/api/auth/oauth2/callback/<providerId>`); `idp` is the hint the broker reads
 * to pick the upstream (Better Auth's id for X is still `twitter`).
 */
export type GrokProvider = {
  /** This app's local provider id; also the callback path segment. */
  providerId: string;
  /** Upstream hint the broker forwards to (Better Auth social id). */
  idp: string;
  /** Human label for the sign-in button. */
  label: string;
};

export const GROK_PROVIDERS: readonly GrokProvider[] = [
  { providerId: "grok-google", idp: "google", label: "Google" },
  { providerId: "grok-x", idp: "twitter", label: "X" },
];
```

src/lib/auth/server.ts

```
/**
 * Self-hosted Better Auth for THIS app (server-only).
 *
 * Pre-wired for live preview + deploy — do not rewrite this file. To enable
 * local email/password, flip the flag in `./email-password` only (see auth skill).
 *
 * The app runs its own Better Auth at `/api/auth/*`, so the session cookie stays
 * on this app's own origin. Sign-in federates to the shared **Grok auth broker**
 * (`GROK_AUTH_ISSUER`) via the `genericOAuth` plugin — the broker brokers the
 * upstream sign-in methods (Google, X, …) and holds their shared secrets; this
 * app only holds its own client id/secret and names the upstream it wants via
 * each provider's `idp` hint.
 *
 * Tri-mode:
 *   - Deployed: the deployer injects a per-app `GROK_AUTH_*` + `BETTER_AUTH_URL`
 *     + `DATABASE_URL`, so real federated auth is persisted in Postgres.
 *   - Sandbox live preview: no injection -> falls back to the shared **preview
 *     client** (`./preview`) and derives the preview's `https://*.grok-sandbox.com`
 *     origin from the request, so real sign-in works (no demo users). Sessions
 *     and identities persist in the embedded PGLite DB (same DB as app data);
 *     the process restart wipes both. Live-preview iframe clients use a bearer
 *     token (partitioned cookies) — see `client.ts`.
 *   - Off (`VITE_AUTH_ENABLED=false`, the shipped default): no providers;
 *     `requireUserId` resolves a dev user with no database configured, and
 *     throws fail-closed once `DATABASE_URL` is set (see `verify.server.ts`).
 *
 * NEVER import this from client code — it pulls in `pg` + the preview secret +
 * server-only Better Auth internals. The client uses `@/lib/auth/client`;
 * components read the user via `@/lib/auth/use-current-user`; server functions get
 * a verified id via `@/lib/auth/middleware`.
 */
import { betterAuth } from "better-auth";
import { bearer, genericOAuth } from "better-auth/plugins";
import { tanstackStartCookies } from "better-auth/tanstack-start";
import { getCookie } from "@tanstack/react-start/server";
import { randomBytes } from "node:crypto";
import { Pool } from "pg";
import { ensureDbReady, getPglite } from "../db";
import { emailAndPasswordEnabled } from "./email-password";
import { GATE_PROVIDER_ID, gateIdentitySessions } from "./gate-session.server";
import { GROK_PROVIDERS } from "./providers";
import { pgliteDialect } from "./pglite-dialect";
import {
  GROK_ISSUER_DEFAULT,
  PREVIEW_ALLOWED_HOSTS,
  PREVIEW_CLIENT_ID,
  PREVIEW_CLIENT_SECRET,
} from "./preview";

// Kick (and share) PGLite bootstrap as soon as the auth server module loads.
void ensureDbReady();

/**
 * Preview secret must outlive module reloads: PGLite (and its session rows) is
 * stored on `globalThis`, so an HMR re-eval of this file must NOT mint a new
 * signing secret or every existing session becomes invalid mid-dev. Process
 * restart clears both the secret and PGLite together.
 */
const globalAuthRef = globalThis as typeof globalThis & {
  __grokAuthPreviewSecret__?: string;
};
function previewAuthSecret(): string {
  globalAuthRef.__grokAuthPreviewSecret__ ??= randomBytes(32).toString("hex");
  return globalAuthRef.__grokAuthPreviewSecret__;
}

/** Read an env var, treating empty/whitespace as unset. */
const env = (key: string): string | undefined => {
  const value = process.env[key]?.trim();
  return value ? value : undefined;
};

// Explicit off-switch. The deployer sets `VITE_AUTH_ENABLED=true` when it
// provisions auth; set it to "false" to force auth off everywhere (dev user).
const authDisabled = env("VITE_AUTH_ENABLED") === "false";

// Broker federation creds: the deployer injects a per-app client when deployed;
// otherwise fall back to the shared live-preview client, which the broker accepts
// for any `*.grok-sandbox.com` callback (see `./preview`).
const grokIssuer = env("GROK_AUTH_ISSUER") ?? GROK_ISSUER_DEFAULT;
const grokClientId = env("GROK_AUTH_CLIENT_ID") ?? PREVIEW_CLIENT_ID;
const grokClientSecret = env("GROK_AUTH_CLIENT_SECRET") ?? PREVIEW_CLIENT_SECRET;

/** True when federated sign-in is active (real auth is enforced). */
export const authConfigured =
  !authDisabled && Boolean(grokClientId && grokClientSecret);

// This app's own Better Auth origin. When deployed the deployer injects the
```

```
// public URL. In the sandbox live preview there's no fixed URL (each preview gets
// a dynamic `*.grok-sandbox.com` host), so we hand Better Auth a dynamic baseURL:
// it derives the origin per-request from the (proxied) host, validated against the
// preview allowlist, which makes the OAuth `redirect_uri` the concrete preview URL
// the broker's preview client accepts.
const explicitBaseURL = env("BETTER_AUTH_URL");
// Explicit `string[]` (not a readonly tuple) — Better Auth's DynamicBaseURLConfig
// requires a mutable `allowedHosts: string[]`.
const previewAllowedHosts: string[] = [...PREVIEW_ALLOWED_HOSTS];
// Local `npm run dev` (port 8080 contract). Browsers may send Origin as any of
// these for the same server — trusting only `localhost` rejects `127.0.0.1` and
// breaks email/password with "Invalid origin".
const LOCAL_DEV_ORIGINS: string[] = [
  "http://localhost:8080",
  "http://127.0.0.1:8080",
  "http://[::1]:8080",
];
const baseURL = explicitBaseURL ?? {
  // Include loopback hosts so dynamic baseURL resolves for local email/password
  // (not only the preview wildcard).
  allowedHosts: [...previewAllowedHosts, "localhost", "127.0.0.1", "[::1]"],
  // `auto` → trust both http:// and https:// expansions of allowedHosts
  // (preview is https; local dev is http).
  protocol: "auto" as const,
  fallback: "http://localhost:8080",
};

// Origins Better Auth accepts on credentialed POSTs (sign-up/sign-in, etc.).
// Missing entries here surface as FORBIDDEN "Invalid origin".
const trustedOrigins: string[] = explicitBaseURL
  ? [explicitBaseURL, ...LOCAL_DEV_ORIGINS]
  : [
      // Host wildcards (matched against Origin's host)
      ...previewAllowedHosts,
      // Full-origin wildcards (matched against Origin)
      ...previewAllowedHosts.flatMap((host) => [`https://${host}`, `http://${host}`]),
      ...LOCAL_DEV_ORIGINS,
    ];

const databaseUrl = env("DATABASE_URL");

// Static broker OAuth endpoints (skip OIDC discovery on every sign-in / callback).
// Discovery would cost an extra network hop to the broker before the popup can
// even redirect to Google/X — the live-preview popup felt stuck on the app for
// that whole round-trip. These paths match the broker's discovery document.
const issuerBase = grokIssuer.replace(/\/+$/, "");
const grokAuthorizationUrl = `${issuerBase}/api/auth/oauth2/authorize`;
const grokTokenUrl = `${issuerBase}/api/auth/oauth2/token`;
const grokUserInfoUrl = `${issuerBase}/api/auth/oauth2/userinfo`;

// Real Postgres when `DATABASE_URL` is set (deployed apps), else the app's
// embedded PGLite (preview) via a Kysely dialect — so Better Auth persists to the
// SAME DB as app data, including email/password users. Both use the Better Auth
// schema from `migrations/auth/0001_auth.sql`, copied into `migrations/` when
// the app turns sign-in on.
const database = databaseUrl
  ? new Pool({ connectionString: databaseUrl })
  : { dialect: pgliteDialect(() => getPglite()), type: "postgres" as const };

/** Session token cookie name — also read by the live-preview popup completion page. */
export const SESSION_TOKEN_COOKIE = "__Host-grok-auth.session_token";

// Built separately so the `betterAuth({...})` call stays easy to edit without
// breaking brackets (models often trip on the conditional plugin spread).
const grokOAuthPlugin = authConfigured
  ? genericOAuth({
      config: GROK_PROVIDERS.map(({ providerId, idp }) => ({
        providerId,
        clientId: grokClientId as string,
        clientSecret: grokClientSecret as string,
        // Prefer static endpoints over `discoveryUrl` so initiating (and
        // completing) OAuth does not wait on a broker discovery fetch.
        authorizationUrl: grokAuthorizationUrl,
        tokenUrl: grokTokenUrl,
        userInfoUrl: grokUserInfoUrl,
        scopes: ["openid", "profile", "email"],
        // `prompt: "login"` forces the broker to re-authenticate against the
        // upstream on every sign-in instead of silently reusing an existing
        // broker session. Combined with the broker sending Google
        // `prompt=select_account`, the user always gets the account chooser
        // and can pick (or switch) which account to sign in with.
        authorizationUrlParams: { idp, prompt: "login" },
      })),
    })
  : null;

export const auth = betterAuth({
  baseURL,
  // Deployed apps inject BETTER_AUTH_SECRET. Preview: process-stable secret on
  // globalThis so HMR doesn't invalidate PGLite-backed sessions (see above).
```

```
  secret: env("BETTER_AUTH_SECRET") ?? previewAuthSecret(),
  database,

  // CSRF / origin check for credentialed auth POSTs (email sign-up/sign-in, …).
  // See `trustedOrigins` construction above — must cover live preview hosts AND
  // local loopback variants, or clients get "Invalid origin".
  trustedOrigins,

  // Encrypt broker-issued OAuth tokens at rest, and treat the broker's upstreams
  // as trusted first-party identities. The broker owns identity and X emails are
  // synthetic/unverified, so WITHOUT this a login can fail with
  // `account_not_linked` (Better Auth refuses to attach an untrusted, unverified
  // identity to an existing user). Google and X carry DISTINCT emails, so this
  // never merges them into one user — they stay separate identities.
  account: {
    encryptOAuthTokens: true,
    accountLinking: {
      enabled: true,
      trustedProviders: [
        ...GROK_PROVIDERS.map((p) => p.providerId),
        GATE_PROVIDER_ID,
      ],
      // X's synthetic email is never "verified", so don't gate linking on the
      // local user's email-verified state.
      requireLocalEmailVerified: false,
    },
  },

  // Cache the session in the short-lived signed `session_data` cookie so reads
  // (incl. the client's `/get-session`) skip the DB — this shrinks the "loading"
  // window and reduces auth flicker. See the `auth` skill for the full
  // flicker-prevention guidance (gate on `isPending`; SSR the session).
  session: { cookieCache: { enabled: true, maxAge: 300 } },

  // Local email/password — toggled only via `./email-password` (not a plugin).
  ...(emailAndPasswordEnabled ? { emailAndPassword: { enabled: true } } : {}),

  // `__Host-` prefixed cookies: the browser REFUSES any same-named cookie that
  // carries a `Domain` attribute, so a sibling `*.grok.me` app cannot "toss" a
  // `Domain=.grok.me` session cookie onto this app. `__Host-` requires Secure +
  // Path=/ + no Domain; Better Auth otherwise uses `__Secure-` (which permits
  // Domain), so we drop its auto prefix (`useSecureCookies: false`) and set
  // Secure + the names ourselves. (Browsers allow Secure cookies on
  // `http://localhost`, so local dev still works.)
  advanced: {
    useSecureCookies: false,
    defaultCookieAttributes: { secure: true, sameSite: "lax", path: "/" },
    cookies: {
      session_token: { name: SESSION_TOKEN_COOKIE },
      session_data: { name: "__Host-grok-auth.session_data" },
      account_data: { name: "__Host-grok-auth.account_data" },
      dont_remember: { name: "__Host-grok-auth.dont_remember" },
    },
  },

  plugins: [
    gateIdentitySessions(),

    // One genericOAuth provider per upstream (when auth is on), all federating
    // to the broker with the SAME client and differing only by the `idp` hint.
    ...(grokOAuthPlugin ? [grokOAuthPlugin] : []),

    // Accept `Authorization: Bearer <session-token>` as an alternative to the
    // cookie. Needed for the LIVE PREVIEW: the app runs in an embedded iframe
    // where cookies are partitioned, so after popup sign-in it authenticates with
    // a bearer token instead (see `client.ts` / the `auth` skill). The hook only
    // fires when an Authorization header is present, so the cookie path
    // (deployed apps) is unaffected.
    bearer(),

    // Bridges Better Auth's Set-Cookie into TanStack Start responses. MUST be
    // last so it runs after every other plugin's hooks.
    tanstackStartCookies(),
  ],
});

export function readSessionToken(): string | null {
  return getCookie(SESSION_TOKEN_COOKIE) ?? null;
}

// Re-exported for convenience; the array lives in the dependency-free
// `providers.ts` so the client can import it too.
export { GROK_PROVIDERS } from "./providers";
```

src/lib/auth/sign-in-gate.test.ts

```
import { describe, it } from "node:test";
import assert from "node:assert/strict";
import { resolveSignInGateState } from "./sign-in-gate.ts";

describe("resolveSignInGateState", () => {
  it("is pending while the session check is in flight, user or not", () => {
    assert.equal(
      resolveSignInGateState({ isPending: true, hasUser: false }),
      "pending",
    );
    assert.equal(
      resolveSignInGateState({ isPending: true, hasUser: true }),
      "pending",
    );
  });

  it("is signed_in once a user is present", () => {
    assert.equal(
      resolveSignInGateState({ isPending: false, hasUser: true }),
      "signed_in",
    );
  });

  it("is signed_out only after the check resolved with no user", () => {
    assert.equal(
      resolveSignInGateState({ isPending: false, hasUser: false }),
      "signed_out",
    );
  });
});
```

src/lib/auth/sign-in-gate.ts

```
export type SignInGateState = "pending" | "signed_in" | "signed_out";

export type SignInGateInput = {
  isPending: boolean;
  hasUser: boolean;
};

export function resolveSignInGateState(
  input: SignInGateInput,
): SignInGateState {
  if (input.isPending) return "pending";
  return input.hasUser ? "signed_in" : "signed_out";
}
```

src/lib/auth/use-current-user.ts

```
import { useMemo } from "react";
import { authClient, authEnabled } from "./client";

/** Normalized user shape used across the app, auth on or off. */
export type AppUser = {
  id: string;
  displayName: string | null;
  primaryEmail: string | null;
  profileImageUrl: string | null;
  /** True when this is the sandbox/dev fallback (auth not configured). */
  isDevFallback: boolean;
};

/**
 * Stable fallback user, used ONLY when auth is disabled
 * (`VITE_AUTH_ENABLED=false`, the shipped default). With auth on, the sandbox
 * live preview does real sign-in via the baked preview client. Its id is
 * `"dev-user"` — the SAME id `verify.server.ts` returns server-side — so per-user
 * rows written in that mode belong to one consistent owner.
 */
export const DEV_USER: AppUser = {
  id: "dev-user",
  displayName: "Dev User",
  primaryEmail: "dev@example.com",
  profileImageUrl: null,
  isDevFallback: true,
};

/** `useCurrentUserState()` result: the user plus the session-loading flag. */
export type CurrentUserState = {
  /** The user — `null` BOTH while the session loads and when signed out. */
  user: AppUser | null;
  /** True while the session is still resolving — don't treat `user: null` as signed out yet. */
  isPending: boolean;
};

const DEV_STATE: CurrentUserState = { user: DEV_USER, isPending: false };

/**
 * Current user + loading state. Same behavior in live preview and when deployed:
 *   - Auth enabled -> the real signed-in user; `user` is `null` while
 *                             the session resolves (`isPending: true`) and when
 *                             signed out (`isPending: false`). Session comes from
 *                             Better Auth `useSession()` → `/api/auth/get-session`
 *                             (cookie when deployed; bearer in live preview).
 *   - Auth disabled (`VITE_AUTH_ENABLED=false`) -> `DEV_USER`, never pending.
 *
 * Protect a route by waiting out `isPending` before acting on `user` —
 * redirecting on `user: null` alone bounces signed-in visitors to sign-in on
 * every hard reload:
 *
 *   import { RedirectToSignIn } from "@/lib/auth/gates";
 *   const { user, isPending } = useCurrentUserState();
 *   if (isPending) return null;              // still resolving — don't redirect yet
 *   if (!user) return <RedirectToSignIn />;  // definitely signed out
 *
 * `authEnabled` is a module-level constant fixed at load, so the guarded hook
 * call keeps a stable hook order across every render of a given component.
 */
export function useCurrentUserState(): CurrentUserState {
  if (!authEnabled) return DEV_STATE;
  // eslint-disable-next-line react-hooks/rules-of-hooks -- authEnabled is constant for the app's lifetime
  return useSessionUserState();
}

function useSessionUserState(): CurrentUserState {
  const { data, isPending } = authClient.useSession();
  const id = data?.user.id;
  const displayName = data?.user.name ?? null;
  const primaryEmail = data?.user.email ?? null;
  const profileImageUrl = data?.user.image ?? null;
  const user = useMemo<AppUser | null>(
    () => (id ? { id, displayName, primaryEmail, profileImageUrl, isDevFallback: false } : null),
    [id, displayName, primaryEmail, profileImageUrl],
  );
  return useMemo(() => ({ user, isPending }), [user, isPending]);
}

/**
 * Convenience view of `useCurrentUserState().user` for display (e.g.
 * `user?.displayName ?? "Guest"`). NOTE: `null` means *loading OR signed out* —
 * for redirects/guards use `useCurrentUserState()` and check `isPending`.
 */
export function useCurrentUser(): AppUser | null {
  return useCurrentUserState().user;
}
```

src/lib/auth/verify.server.ts

```
import { getRequest } from "@tanstack/react-start/server";
import { gateIdentityEnabled } from "./gate-identity.server";
import { auth, authConfigured } from "./server";

/**
 * Server-side session resolution (server-only).
 *
 * Because this app runs its OWN Better Auth at same-origin `/api/auth/*`, the
 * session cookie is sent with every request to this app — server functions AND
 * SSR loaders included. So we resolve the user straight from the request cookies
 * via `auth.api.getSession` (no client-minted JWT needed). Never trust a
 * client-supplied user id — only the result of this verification.
 */

/** True when a real database is configured server-side. */
const databaseConfigured = Boolean(process.env.DATABASE_URL?.trim());

/** Re-export so callers can branch on it without importing `server.ts`. */
export { authConfigured };

if (databaseConfigured && !authConfigured) {
  console.error(
    "[auth] DATABASE_URL is set but auth is disabled (VITE_AUTH_ENABLED=false) " +
      "— requireUserId() will reject every request (fail closed) rather than " +
      "share one dev user on a real database.",
  );
}

/** Dev fallback user id, used only when auth is disabled (VITE_AUTH_ENABLED=false). */
export const DEV_USER_ID = "dev-user";

/**
 * Thrown by `requireUserId` when the caller has no valid session. Carries
 * `status: 401`; the message is a stable contract — match
 * `err.message === "Unauthorized"` client-side to send the visitor to sign-in.
 */
export class UnauthorizedError extends Error {
  readonly status = 401;
  constructor() {
    super("Unauthorized");
    this.name = "UnauthorizedError";
  }
}

export type VerifiedUser = { id: string; email: string | null };

/**
 * Resolve the signed-in user from the current request, or `null` when auth isn't
 * configured / nobody is signed in. Safe to call from server functions and SSR
 * loaders.
 *
 * `bearerToken` is for the LIVE PREVIEW: the app runs in a partitioned iframe
 * whose cookies don't reach the server, so `authMiddleware` forwards the session
 * as a bearer token, which we present as `Authorization: Bearer …` (the `bearer`
 * plugin resolves it). When deployed no token is passed and the cookie is used.
 */
export async function getSessionUser(
  bearerToken?: string,
): Promise<VerifiedUser | null> {
  if (!authConfigured && !gateIdentityEnabled()) return null;
  const request = getRequest();
  if (!request) return null;
  let headers = request.headers;
  if (bearerToken) {
    headers = new Headers(request.headers);
    headers.set("Authorization", `Bearer ${bearerToken}`);
  }
  const session = await auth.api.getSession({ headers });
  if (!session?.user) return null;
  return { id: session.user.id, email: session.user.email ?? null };
}

/**
 * Resolve the current user id for a server function, or throw when unauthorized.
 * Prefer `authMiddleware` (`./middleware`), which calls this for you.
 * - Auth enabled -> the verified session user id; throws
 *   `UnauthorizedError` when signed out. Works in the sandbox preview too (real
 *   sign-in via the baked preview client).
 * - Auth disabled (`VITE_AUTH_ENABLED=false`) + `DATABASE_URL` set -> throw (fail
 *   closed): one shared dev user on a real database would let every visitor
 *   read/write everyone's rows.
 * - Auth disabled + no database -> the shared dev user id.
 */
export async function requireUserId(bearerToken?: string): Promise<string> {
  if (!authConfigured && !gateIdentityEnabled()) {
    if (databaseConfigured) {
      throw new Error(
        "Auth is disabled (VITE_AUTH_ENABLED=false) but DATABASE_URL is set — " +
```

```
          "refusing to fall back to the shared dev user against a real database.",
      );
    }
    return DEV_USER_ID;
  }
  const user = await getSessionUser(bearerToken);
  if (!user) throw new UnauthorizedError();
  return user.id;
}
```

src/lib/auth0-config.ts

```
import type { Auth0ServerOptions } from "@auth0/auth0-tanstack-start-react/types";

function read(name: string) {
  const value = process.env[name]?.trim();
  return value || undefined;
}

/** Public values fall back so the build can finish when Vercel has not injected them yet. */
export function auth0ServerOptions(): Auth0ServerOptions {
  const domain = process.env.VITE_AUTH0_DOMAIN || process.env.AUTH0_DOMAIN || "placeholder.auth0.com";
  const clientId = process.env.VITE_AUTH0_CLIENT_ID || process.env.AUTH0_CLIENT_ID || "placeholder-client-id
";
  const clientSecret = read("AUTH0_CLIENT_SECRET");
  const secret = read("AUTH0_SECRET");
  const appBaseUrl = read("APP_BASE_URL");
  return {
    domain: () => domain,
    clientId,
    ...(clientSecret ? { clientSecret } : {}),
    ...(secret ? { secret } : {}),
    ...(appBaseUrl ? { appBaseUrl } : {}),
  };
}
```

src/lib/db.ts

```
import { pendingMigrations } from "../../scripts/migration-plan.mjs";

/** Which database backend is active. */
export type DbSource = "neon" | "pglite";

// An empty/whitespace DATABASE_URL (an easy misconfig in deploy UIs) must mean
// "unset" — otherwise production would silently run on the PGLite fallback.
const rawDatabaseUrl =
  typeof process !== "undefined" ? process.env.DATABASE_URL : undefined;
const databaseUrl =
  rawDatabaseUrl && rawDatabaseUrl.trim() ? rawDatabaseUrl : undefined;

/**
 * Active backend: real **Neon** when `DATABASE_URL` is set (deployed / configured
 * sandbox), otherwise a local embedded **PGLite** (Postgres compiled to WASM) so
 * the app has a working database even with nothing configured — the live preview
 * included. Swap in Neon later by just setting `DATABASE_URL`; no code changes.
 */
export const dbSource: DbSource = databaseUrl ? "neon" : "pglite";

/**
 * Minimal shared SQL surface, satisfied by both Neon and PGLite. Both the
 * tagged-template and `.query()` forms resolve to an array of row objects:
 *
 *   const sql = await getSql();
 *   const rows = await sql`select * from todos where id = ${id}`; // parameterized
 *   const rows2 = await sql.query("select * from todos where id = $1", [id]);
 */
export interface Sql {
  <T = Record<string, unknown>>(
    strings: TemplateStringsArray,
    ...values: unknown[]
  ): Promise<T[]>;
  query<T = Record<string, unknown>>(
    text: string,
    params?: unknown[],
  ): Promise<T[]>;
}

/**
 * Init state lives on globalThis as promises: dev HMR creates new instances of
 * this module, and two instances racing module-level state would open a second
 * pool or run two concurrent PGLite migration passes (whose duplicate
 * `_migrations` insert rejects — and would get memoized, poisoning every later
 * `getSql()`). A failed init clears its slot so the next call retries.
 */
const globalRef = globalThis as typeof globalThis & {
  __pgSqlPromise__?: Promise<Sql>;
  __pgliteInstance__?: Promise<import("@electric-sql/pglite").PGlite>;
  __pgliteMigrateChain__?: Promise<void>;
};

/**
 * Result-type parity: Postgres sends every value as text plus a type OID — the
 * JS value is the DRIVER's parsing choice, and pg and PGLite disagree (pg:
 * int8 -> string, date -> local-midnight Date; PGLite: int8 -> BigInt, which
 * JSON.stringify rejects, date -> UTC Date). Normalize both so preview and
 * production return identical, JSON-safe shapes:
 *   int8/bigint (incl. count(*)) -> number (past 2^53 loses precision — cast
 *                                   `::text` if you ever need huge integers)
 *   date                         -> 'YYYY-MM-DD' string
 *   interval                     -> Postgres interval text
 * numeric already comes back as a string on both (arbitrary precision).
 */
const OID_INT8 = 20;
const OID_DATE = 1082;
const OID_INTERVAL = 1186;
const identity = (v: string) => v;

type Run = <T>(text: string, params: unknown[]) => Promise<T[]>;

/** Wrap a query runner in the tagged-template + `.query()` `Sql` surface. */
function toSql(run: Run): Sql {
  const sql = (async <T = Record<string, unknown>>(
    strings: TemplateStringsArray,
    ...values: unknown[]
  ): Promise<T[]> => {
    // Rebuild with $1, $2, … placeholders so values stay parameterized.
    let text = strings[0];
    for (let i = 0; i < values.length; i += 1) text += `$${i + 1}${strings[i + 1]}`;
    return run<T>(text, values);
  }) as unknown as Sql;
  sql.query = <T = Record<string, unknown>>(text: string, params: unknown[] = []) =>
    run<T>(text, params);
  return sql;
}

function createNeonSql(): Promise<Sql> {
```

```
  globalRef.__pgSqlPromise__ ??= (async () => {
    // Regular Postgres driver: node-postgres (`pg`) — works directly with Neon's
    // pooled endpoint. One pool per process; warm serverless instances reuse it.
    const { Pool, types } = await import("pg");
    types.setTypeParser(OID_INT8, Number);
    types.setTypeParser(OID_DATE, identity);
    types.setTypeParser(OID_INTERVAL, identity);
    const pool = new Pool({ connectionString: databaseUrl });
    return toSql(async <T>(text: string, params: unknown[]) => {
      const res = await pool.query(text, params);
      return res.rows as T[];
    });
  })().catch((err) => {
    globalRef.__pgSqlPromise__ = undefined;
    throw err;
  });
  return globalRef.__pgSqlPromise__;
}

async function createPgliteSql(): Promise<Sql> {
  // Embedded Postgres, imported on demand so it never loads on the Neon path.
  // One in-memory instance per process, shared across HMR module instances, so
  // data survives source edits (it resets on dev-server restart).
  globalRef.__pgliteInstance__ ??= (async () => {
    const { PGlite } = await import("@electric-sql/pglite");
    const pg = new PGlite({
      parsers: {
        [OID_INT8]: Number,
        [OID_DATE]: identity,
        [OID_INTERVAL]: identity,
      },
    });
    await pg.waitReady;
    await pg.exec(
      "create table if not exists _migrations (name text primary key, applied_at timestamptz not null defaul
t now())",
    );
    return pg;
  })().catch((err) => {
    globalRef.__pgliteInstance__ = undefined;
    throw err;
  });
  const pg = await globalRef.__pgliteInstance__;

  // Apply migrations/ (the single schema source) so preview matches production.
  // SQL is inlined by the bundler via import.meta.glob (no runtime fs); applied
  // files are tracked in _migrations. The glob does not descend, so the opt-in
  // auth schema under migrations/auth/ stays out. Runs once per module instance
  // — so an HMR reload after adding a migration file applies it live — with
  // passes serialized on a global chain so concurrent callers never
  // double-apply.
  const migrate = async (): Promise<void> => {
    const migrations = import.meta.glob("/migrations/*.sql", {
      query: "?raw",
      import: "default",
      eager: true,
    }) as Record<string, string>;
    const doneRows = await pg.query<{ name: string }>(
      "select name from _migrations",
    );
    const done = doneRows.rows.map((r) => r.name);
    for (const { name, path } of pendingMigrations(Object.keys(migrations), done)) {
      // Apply + record atomically (parity with scripts/migrate.mjs) so a failed
      // statement can't leave a file half-applied but untracked.
      await pg.transaction(async (tx) => {
        await tx.exec(migrations[path]);
        await tx.query("insert into _migrations (name) values ($1)", [name]);
      });
    }
  };
  const pass = (globalRef.__pgliteMigrateChain__ ?? Promise.resolve())
    .catch(() => undefined) // an earlier failed pass must not wedge the chain
    .then(migrate);
  globalRef.__pgliteMigrateChain__ = pass;
  await pass;

  return toSql(async <T>(text: string, params: unknown[]) => {
    const result = await pg.query<T>(text, params);
    return result.rows;
  });
}

let sqlPromise: Promise<Sql> | null = null;

async function createSql(): Promise<Sql> {
  if (typeof window !== "undefined") {
    throw new Error(
      "@/lib/db is server-only — call getSql() from a createServerFn handler " +
        "or a server route loader, never from client code.",
    );
```

```
  }
  return dbSource === "neon" ? createNeonSql() : createPgliteSql();
}

/**
 * Get the shared, **server-only** SQL client. Neon when `DATABASE_URL` is set,
 * otherwise the local PGLite fallback. Memoized — safe to call per request.
 *
 * Schema comes from `migrations/*.sql`, auto-applied before the first query on
 * both backends — define tables there, never inline in server functions.
 */
export function getSql(): Promise<Sql> {
  sqlPromise ??= createSql().catch((err) => {
    sqlPromise = null; // don't memoize failures — let the next call retry
    throw err;
  });
  return sqlPromise;
}

/**
 * The shared PGLite instance (preview only), with `migrations/*.sql` applied.
 * Lets Better Auth persist to the SAME embedded DB as app data in preview (via a
 * Kysely dialect). Throws when `DATABASE_URL` is set (that path uses Neon).
 */
export async function getPglite(): Promise<import("@electric-sql/pglite").PGlite> {
  if (dbSource !== "pglite") {
    throw new Error("getPglite() is only available on the PGLite fallback (no DATABASE_URL)");
  }
  await getSql();
  const pg = await globalRef.__pgliteInstance__;
  if (!pg) throw new Error("PGLite instance failed to initialize");
  return pg;
}

/**
 * Finish DB bootstrap before the server handles traffic.
 *
 * - **PGLite** (preview / no `DATABASE_URL`): open the in-memory DB and apply
 *   `migrations/*.sql`. Idempotent — concurrent callers share one promise.
 * - **Neon**: no-op (pool is created lazily on first query).
 *
 * Vite `configureServer` awaits this at dev startup; production imports of this
 * module kick it off immediately (see bottom of file).
 */
export function ensureDbReady(): Promise<void> {
  if (dbSource !== "pglite") return Promise.resolve();
  return getSql().then(() => undefined);
}

// Server-only eager start: kick PGLite bootstrap as soon as this module loads in
// Node. Client bundles never hit this path (`getSql` throws in the browser).
const globalBoot = globalThis as typeof globalThis & {
  __pgBootstrapPromise__?: Promise<void>;
};
if (typeof window === "undefined" && dbSource === "pglite") {
  globalBoot.__pgBootstrapPromise__ ??= ensureDbReady().catch((err) => {
    globalBoot.__pgBootstrapPromise__ = undefined;
    console.error("[db] PGLite bootstrap failed:", err);
    throw err;
  });
}
```

src/lib/device.ts

```
import { useEffect, useState } from "react";

export type Device = "phone" | "tablet" | "computer";

export function detectDevice(): Device {
  if (typeof window === "undefined") return "phone";
  const width = window.innerWidth;
  const ua = navigator.userAgent;
  const iPad = /iPad/.test(ua) || (navigator.platform === "MacIntel" && navigator.maxTouchPoints > 1);
  const coarse = window.matchMedia("(pointer: coarse)").matches;
  if (width < 700) return "phone";
  if (iPad || (coarse && width < 1200)) return "tablet";
  return "computer";
}

export function useDevice(): Device {
  const [device, setDevice] = useState<Device>("phone");

  useEffect(() => {
    const apply = () => {
      const next = detectDevice();
      setDevice(next);
      document.documentElement.dataset.device = next;
    };
    apply();
    const pointer = window.matchMedia("(pointer: coarse)");
    window.addEventListener("resize", apply);
    pointer.addEventListener("change", apply);
    return () => {
      window.removeEventListener("resize", apply);
      pointer.removeEventListener("change", apply);
    };
  }, []);

  return device;
}

export function pageFrame(device: Device) {
  if (device === "tablet") {
    return {
      main: "min-h-screen bg-desk px-6 py-8 text-ink",
      sheet: "relative mx-auto flex min-h-[calc(100vh-4rem)] w-full max-w-2xl flex-col rounded-page border b
order-rule bg-paper px-8 pt-16 pb-10 shadow-[0_18px_50px_rgba(60,51,40,0.12)]",
    };
  }
  if (device === "computer") {
    return {
      main: "min-h-screen bg-desk px-8 py-10 text-ink",
      sheet: "relative mx-auto flex min-h-[calc(100vh-5rem)] w-full max-w-3xl flex-col rounded-page border b
order-rule bg-paper px-10 pt-16 pb-10 shadow-[0_18px_50px_rgba(60,51,40,0.12)]",
    };
  }
  return {
    main: "min-h-screen bg-desk text-ink",
    sheet: "relative mx-auto flex min-h-screen w-full max-w-none flex-col bg-paper px-5 pt-16 pb-8",
  };
}
```

src/lib/env.server.ts

```
export function env(key: string): string | undefined {
  const v = process.env[key]?.trim();
  return v || undefined;
}

/**
 * Workspace preview vs deployed app. The deployer writes GROK_PROJECT_ID on
 * every publish; the sandbox preview never has it. Single source of truth for
 * the split — gate audience, gate endpoints and connector-token semantics all
 * key off this predicate.
 */
export function isWorkspacePreview(): boolean {
  return !env("GROK_PROJECT_ID");
}
```

src/lib/error-component.tsx

```
import type { ErrorComponentProps } from "@tanstack/react-router";
import { TriangleAlert } from "lucide-react";

const FALLBACK_MESSAGE = "An unexpected error occurred. Try reloading the page.";

function errorMessage(error: unknown): string {
  if (error instanceof Error && error.message) return error.message;
  if (typeof error === "string" && error) return error;
  return FALLBACK_MESSAGE;
}

export function AppErrorComponent({ error }: ErrorComponentProps) {
  return (
    <main
      className={
        "flex min-h-screen flex-col items-center justify-center gap-3 px-6 text-center " +
        "bg-zinc-50 text-zinc-900 dark:bg-zinc-950 dark:text-zinc-50"
      }
    >
      <span className="text-red-500" aria-hidden="true">
        <TriangleAlert className="size-10" strokeWidth={2} />
      </span>
      <h1 className="text-lg font-semibold">Something went wrong</h1>
      <p className="max-w-md text-sm break-words text-zinc-500 dark:text-zinc-400">
        {errorMessage(error)}
      </p>
    </main>
  );
}
```

src/lib/multiplayer/index.ts

```
export { P2PRoom, defaultIceServers } from "./p2p";
export type {
  PeerInfo,
  P2PRoomOptions,
  SignalKind,
  PeerRow,
  SignalRow,
  RtcPollResponse,
} from "./p2p";
```

src/lib/multiplayer/p2p.ts

```
/**
 * Full-mesh WebRTC rooms: one RTCPeerConnection per remote peer, signaled
 * through /api/rtc (see signaling.server.ts), game data flowing directly
 * browser-to-browser afterwards. Client-authoritative by construction — see
 * the multiplayer-p2p skill for when NOT to use this.
 *
 * Negotiation follows the "perfect negotiation" pattern: on a glare (both
 * sides offering at once) the polite peer — the lexicographically smaller id —
 * rolls back and accepts, so pairs converge without wedging.
 */

export type SignalKind = "offer" | "answer" | "ice";

/**
 * Wire contract between this client and the signaling relay the app provides
 * at /api/rtc (see the multiplayer-p2p skill for a reference implementation).
 * The client only needs these shapes — the relay's storage is the app's choice.
 */
export interface PeerRow {
  id: string;
  name: string;
}
export interface SignalRow {
  id: number;
  from: string;
  kind: SignalKind;
  payload: unknown;
}
export interface RtcPollResponse {
  peers: PeerRow[];
  signals: SignalRow[];
}

export interface PeerInfo {
  id: string;
  name: string;
  connectionState: RTCPeerConnectionState;
  /** Selected local ICE candidate type: host | srflx | prflx | relay. */
  candidateType: string | null;
  /** Data-channel ping RTT (ms), measured every 2s once connected. */
  rttMs: number | null;
}

export interface P2PRoomOptions {
  room: string;
  selfId: string;
  name?: string;
  /** Defaults to VITE_STUN_URLS (comma-separated) or Google public STUN. */
  iceServers?: RTCIceServer[];
  onPeersChanged?: (peers: PeerInfo[]) => void;
  /** Fires for both the unreliable "state" and reliable "reliable" channels. */
  onMessage?: (from: string, data: unknown, channel: "state" | "reliable") => void;
  /** Fires once, on the first successful signaling poll (registration). */
  onConnected?: () => void;
}

interface PeerSlot {
  pc: RTCPeerConnection;
  state?: RTCDataChannel;
  reliable?: RTCDataChannel;
  makingOffer: boolean;
  ignoreOffer: boolean;
  /** ICE candidates that arrived before the remote description (buffered). */
  pendingCandidates: RTCIceCandidateInit[];
  /** Last time this pair made observable progress toward connected. */
  lastProgressAt: number;
  /** Watchdog recreations (dialer) / stall windows (receiver) so far. */
  recoveryAttempts: number;
  /** Gave up after MAX_RECOVERY_ATTEMPTS — excluded from fast-poll pressure. */
  terminal?: boolean;
  /** One-shot: pc was already recreated to absorb a failing remote offer. */
  recreatedForOffer?: boolean;
  info: PeerInfo;
  pingSentAt?: number;
}

const FAST_POLL_MS = 400;
const IDLE_POLL_MS = 2000;
const PING_INTERVAL_MS = 2000;
const STALL_MS = 10_000;
const MAX_RECOVERY_ATTEMPTS = 3;
const SIGNAL_RETRY_DELAYS_MS = [250, 750];

export function defaultIceServers(): RTCIceServer[] {
  const urls = (import.meta.env.VITE_STUN_URLS as string | undefined)
    ?.split(",")
    .map((u) => u.trim())
    .filter(Boolean);
```

```
  // Two independent providers: ICE queries all of them in parallel during
  // gathering, so either one being unreachable costs nothing.
  return [
    {
      urls: urls?.length ? urls : ["stun:stun.l.google.com:19302", "stun:stun.cloudflare.com:3478"],
    },
  ];
}

export class P2PRoom {
  private readonly opts: P2PRoomOptions;
  private readonly peers = new Map<string, PeerSlot>();
  /** Per-remote-peer signal delivery chains (order-preserving). */
  private readonly signalQueues = new Map<string, Promise<void>>();
  private cursor = 0;
  private pollTimer: ReturnType<typeof setTimeout> | null = null;
  private pingTimer: ReturnType<typeof setInterval> | null = null;
  private closed = false;
  private everPolled = false;
  private lastPeersFingerprint = "";

  constructor(opts: P2PRoomOptions) {
    this.opts = opts;
  }

  /**
   * The first poll IS the join: it registers this peer and returns the
   * roster. A failed first poll (cold DB, offline tab) must not strand the
   * room: the loop and timers start regardless and the next poll retries.
   */
  async join(): Promise<void> {
    try {
      await this.pollOnce();
    } catch {
      // First poll can fail transiently; the scheduled loop below retries.
    }
    if (this.closed) return;
    this.schedulePoll(this.anyPairConnecting() ? FAST_POLL_MS : IDLE_POLL_MS);
    this.pingTimer = setInterval(() => {
      this.pingAll();
      this.watchdog();
    }, PING_INTERVAL_MS);
  }

  close(): void {
    this.closed = true;
    if (this.pollTimer) clearTimeout(this.pollTimer);
    if (this.pingTimer) clearInterval(this.pingTimer);
    for (const slot of this.peers.values()) slot.pc.close();
    this.peers.clear();
    // Leaving the roster is the teardown broadcast: everyone's next poll
    // drops this peer and closes their side of the pair.
    void fetch("/api/rtc", {
      method: "POST",
      headers: { "content-type": "application/json" },
      body: JSON.stringify({ op: "leave", room: this.opts.room, peer: this.opts.selfId }),
      keepalive: true,
    }).catch(() => {});
  }

  /** Send on the unreliable game-state channel (drops stale packets). */
  broadcast(data: unknown): void {
    const wire = JSON.stringify({ t: "d", d: data });
    for (const slot of this.peers.values()) {
      if (slot.state?.readyState === "open") slot.state.send(wire);
    }
  }

  /** Send reliably (ordered) to one peer, or to all when peerId is omitted. */
  send(data: unknown, peerId?: string): void {
    const wire = JSON.stringify({ t: "d", d: data });
    const targets = peerId ? [this.peers.get(peerId)] : [...this.peers.values()];
    for (const slot of targets) {
      if (slot?.reliable?.readyState === "open") slot.reliable.send(wire);
    }
  }

  peerList(): PeerInfo[] {
    return [...this.peers.values()].map((s) => ({ ...s.info }));
  }

  // ── signaling loop ──────────────────────────────────────────────

  private schedulePoll(delay: number): void {
    if (this.closed) return;
    if (this.pollTimer) clearTimeout(this.pollTimer);
    this.pollTimer = setTimeout(() => void this.poll(), delay);
  }

  private anyPairConnecting(): boolean {
```

```
  for (const s of this.peers.values()) {
    // Terminal pairs (NAT-blocked after all recovery attempts) must not pin
    // the session at the 400ms fast-poll rate.
    if (s.terminal) continue;
    if (s.info.connectionState !== "connected") return true;
  }
  return false;
}

private async pollOnce(): Promise<void> {
  const params = new URLSearchParams({
    room: this.opts.room,
    peer: this.opts.selfId,
    name: this.opts.name ?? "",
    since: String(this.cursor),
  });
  const res = await fetch(`/api/rtc?${params}`);
  if (this.closed) return;
  if (!res.ok) throw new Error(`signaling poll failed: ${res.status}`);
  const body = (await res.json()) as RtcPollResponse;
  if (this.closed) return;
  if (!this.everPolled) {
    this.everPolled = true;
    this.opts.onConnected?.();
  }
  this.reconcileRoster(body.peers);
  const roster = new Set(body.peers.map((p) => p.id));
  for (const sig of body.signals) {
    this.cursor = Math.max(this.cursor, sig.id);
    await this.onSignal(sig.from, sig.kind, sig.payload, roster);
    if (this.closed) return;
  }
}

private async poll(): Promise<void> {
  if (this.closed) return;
  try {
    await this.pollOnce();
  } catch {
    // Transient poll failures are expected (tab sleep, deploy roll); retry.
  }
  this.schedulePoll(this.anyPairConnecting() ? FAST_POLL_MS : IDLE_POLL_MS);
}

private reconcileRoster(peers: { id: string; name: string }[]): void {
  const alive = new Set(peers.map((p) => p.id));
  for (const p of peers) {
    if (p.id === this.opts.selfId) continue;
    const existing = this.peers.get(p.id);
    if (existing) {
      existing.info.name = p.name;
    } else {
      // Exactly one side dials each pair; the other waits for the offer.
      this.connectTo(p.id, p.name, this.opts.selfId > p.id);
    }
  }
  for (const [id, slot] of this.peers) {
    if (!alive.has(id)) {
      slot.pc.close();
      this.peers.delete(id);
    }
  }
  this.emitPeers();
}

// ── per-pair connection ──────────────────────────────────────────

private connectTo(peerId: string, name: string, initiator: boolean): PeerSlot | null {
  if (this.closed) return null;
  const pc = new RTCPeerConnection({
    iceServers: this.opts.iceServers ?? defaultIceServers(),
  });
  const slot: PeerSlot = {
    pc,
    makingOffer: false,
    ignoreOffer: false,
    pendingCandidates: [],
    lastProgressAt: Date.now(),
    recoveryAttempts: 0,
    info: {
      id: peerId,
      name,
      connectionState: pc.connectionState,
      candidateType: null,
      rttMs: null,
    },
  };
  this.peers.set(peerId, slot);

  pc.onicecandidate = (e) => {
```

```
    if (e.candidate) void this.sendSignal(peerId, "ice", e.candidate.toJSON());
  };
  pc.onconnectionstatechange = () => {
    slot.info.connectionState = pc.connectionState;
    if (pc.connectionState === "connecting" || pc.connectionState === "connected") {
      slot.lastProgressAt = Date.now();
    }
    if (pc.connectionState === "connected") {
      slot.recoveryAttempts = 0;
      slot.terminal = false;
      void this.readCandidateType(slot);
    }
    this.emitPeers();
    if (pc.connectionState === "failed") {
      // Refires negotiationneeded → a fresh offer through signaling, so a
      // lost offer or dead path cannot wedge the pair (glare-safe).
      pc.restartIce();
    }
    if (pc.connectionState === "failed" || pc.connectionState === "disconnected") {
      this.schedulePoll(FAST_POLL_MS);
    }
  };
  pc.onnegotiationneeded = async () => {
    try {
      slot.makingOffer = true;
      await pc.setLocalDescription();
      await this.sendSignal(peerId, "offer", pc.localDescription!.toJSON());
    } catch {
      // A failed offer is retried on the next negotiationneeded.
    } finally {
      slot.makingOffer = false;
    }
  };
  pc.ondatachannel = (e) => this.attachChannel(slot, e.channel);

  if (initiator) {
    // Creating the channels triggers negotiationneeded → the offer.
    this.attachChannel(
      slot,
      pc.createDataChannel("state", { ordered: false, maxRetransmits: 0 }),
    );
    this.attachChannel(slot, pc.createDataChannel("reliable", { ordered: true }));
  }
  return slot;
}

private attachChannel(slot: PeerSlot, channel: RTCDataChannel): void {
  if (channel.label === "state") slot.state = channel;
  else slot.reliable = channel;
  channel.onopen = () => {
    slot.lastProgressAt = Date.now();
  };
  channel.onmessage = (e) => {
    let msg: { t: string; d?: unknown };
    try {
      msg = JSON.parse(e.data as string) as { t: string; d?: unknown };
    } catch {
      return;
    }
    if (msg.t === "ping") {
      if (slot.state?.readyState === "open") {
        slot.state.send(JSON.stringify({ t: "pong" }));
      }
    } else if (msg.t === "pong") {
      if (slot.pingSentAt) {
        slot.info.rttMs = Math.round(performance.now() - slot.pingSentAt);
        slot.pingSentAt = undefined;
        this.emitPeers();
      }
    } else {
      this.opts.onMessage?.(
        slot.info.id,
        msg.d,
        channel.label === "state" ? "state" : "reliable",
      );
    }
  };
}

/** Apply buffered ICE candidates once a remote description is in place. */
private async flushPendingCandidates(slot: PeerSlot): Promise<void> {
  while (slot.pendingCandidates.length > 0) {
    const candidate = slot.pendingCandidates.shift()!;
    try {
      await slot.pc.addIceCandidate(candidate);
    } catch (err) {
      if (!slot.ignoreOffer) console.warn("[p2p] addIceCandidate failed:", err);
    }
    if (this.closed) return;
  }
```

```
}

private async onSignal(
  from: string,
  kind: SignalKind,
  payload: unknown,
  roster: Set<string>,
): Promise<void> {
  if (this.closed) return;
  let slot = this.peers.get(from);
  if (!slot) {
    // New peers dial us in the same poll that adds them to the roster.
    // Signals outlive membership, so drop senders the roster doesn't vouch for.
    if (!roster.has(from)) return;
    const created = this.connectTo(from, "", false);
    if (!created) return;
    slot = created;
  }
  const polite = this.opts.selfId < from;

  try {
    if (kind === "offer" || kind === "answer") {
      const description = payload as RTCSessionDescriptionInit;
      const collision =
        kind === "offer" && (slot.makingOffer || slot.pc.signalingState !== "stable");
      slot.ignoreOffer = !polite && collision;
      if (slot.ignoreOffer) return;
      try {
        await slot.pc.setRemoteDescription(description); // implicit rollback when polite
      } catch (err) {
        // A pc resumed from suspend can be unable to take any new remote
        // offer (stale DTLS fingerprint). Rebuild the pair once and apply
        // the same offer to the fresh pc before giving up.
        if (kind !== "offer" || slot.recreatedForOffer) throw err;
        const attempts = slot.recoveryAttempts;
        const name = slot.info.name;
        slot.pc.close();
        this.peers.delete(from);
        const fresh = this.connectTo(from, name, false);
        if (!fresh) return;
        fresh.recoveryAttempts = attempts;
        fresh.recreatedForOffer = true;
        slot = fresh;
        await slot.pc.setRemoteDescription(description);
      }
      if (this.closed) return;
      await this.flushPendingCandidates(slot);
      if (this.closed) return;
      if (kind === "offer") {
        await slot.pc.setLocalDescription();
        if (this.closed) return;
        await this.sendSignal(from, "answer", slot.pc.localDescription!.toJSON());
      }
    } else if (kind === "ice") {
      const candidate = payload as RTCIceCandidateInit;
      if (!slot.pc.remoteDescription) {
        // Candidate raced ahead of its SDP — hold it until the description
        // lands (flushed after every successful setRemoteDescription).
        slot.pendingCandidates.push(candidate);
        return;
      }
      try {
        await slot.pc.addIceCandidate(candidate);
      } catch (err) {
        // The enclosing catch would swallow a rethrow; log the real signal.
        if (!slot.ignoreOffer) console.warn("[p2p] addIceCandidate failed:", err);
      }
    }
  } catch {
    // Negotiation errors resolve on the next offer cycle; state is visible
    // to the app via connectionState.
  }
}

/**
 * Signals are serialized per remote peer (a candidate must never overtake
 * its SDP into the DB) and retried on failure with short backoff.
 */
private sendSignal(to: string, kind: SignalKind, payload: unknown): Promise<void> {
  const prev = this.signalQueues.get(to) ?? Promise.resolve();
  const next = prev.then(() => this.postSignal(to, kind, payload));
  this.signalQueues.set(
    to,
    next.catch(() => {}),
  );
  return next;
}

private async postSignal(to: string, kind: SignalKind, payload: unknown): Promise<void> {
  for (let attempt = 0; ; attempt++) {
```

```
    if (this.closed) return;
    try {
      const res = await fetch("/api/rtc", {
        method: "POST",
        headers: { "content-type": "application/json" },
        body: JSON.stringify({
          op: "signal",
          room: this.opts.room,
          from: this.opts.selfId,
          to,
          kind,
          payload,
        }),
      });
      if (res.ok) return;
      throw new Error(`signal POST failed: ${res.status}`);
    } catch (err) {
      if (attempt >= SIGNAL_RETRY_DELAYS_MS.length) {
        // Delivery gave up; the pair converges on the next offer cycle (or
        // the watchdog rebuilds it). Logged once so failures are visible.
        console.warn(`[p2p] signal ${kind} to ${to} failed after retries`, err);
        return;
      }
      await new Promise((r) => setTimeout(r, SIGNAL_RETRY_DELAYS_MS[attempt]));
    }
  }
}

// ── diagnostics + recovery ──────────────────────────────────────────

private pingAll(): void {
  const wire = JSON.stringify({ t: "ping" });
  for (const slot of this.peers.values()) {
    if (slot.state?.readyState !== "open") continue;
    const stale =
      slot.pingSentAt !== undefined && performance.now() - slot.pingSentAt > 2 * PING_INTERVAL_MS;
    if (slot.pingSentAt === undefined || stale) {
      // A lost pong must not freeze rttMs forever: expire and re-ping.
      slot.pingSentAt = performance.now();
      slot.state.send(wire);
    }
  }
}

/**
 * Stuck-pair recovery, piggybacked on the ping interval. A pair that has
 * made no progress for STALL_MS gets rebuilt by the dialer with a FRESH
 * RTCPeerConnection (new DTLS identity — fixes the suspend/resume
 * fingerprint wedge). After MAX_RECOVERY_ATTEMPTS the pair is terminal:
 * visible to the app as its last connectionState, ignored by fast-poll.
 */
private watchdog(): void {
  if (this.closed) return;
  const now = Date.now();
  for (const [peerId, slot] of this.peers) {
    // pc.close() and some suspend/resume wedges never fire
    // connectionstatechange — read the LIVE state so a silently-dead pc
    // still trips the stall timer instead of hiding behind a cached
    // "connected". Only live progress states refresh the stall clock.
    const live = slot.pc.connectionState;
    if (live !== slot.info.connectionState) {
      slot.info.connectionState = live;
      if (live === "connecting" || live === "connected") slot.lastProgressAt = now;
      this.emitPeers();
    }
    if (slot.terminal || live === "connected") continue;
    if (now - slot.lastProgressAt <= STALL_MS) continue;
    if (slot.recoveryAttempts >= MAX_RECOVERY_ATTEMPTS) {
      slot.terminal = true;
      this.emitPeers();
      continue;
    }
    slot.recoveryAttempts += 1;
    slot.lastProgressAt = now; // re-arm the stall window
    if (this.opts.selfId > peerId) {
      // We are the dialer: rebuild the pair from scratch.
      const { name } = slot.info;
      const attempts = slot.recoveryAttempts;
      slot.pc.close();
      this.peers.delete(peerId);
      const fresh = this.connectTo(peerId, name, true);
      if (fresh) fresh.recoveryAttempts = attempts;
      this.schedulePoll(FAST_POLL_MS);
    }
    // Receiver side: count the stall window and wait for the dialer's
    // fresh offer (onSignal absorbs it, recreating our pc if needed).
  }
}

private async readCandidateType(slot: PeerSlot): Promise<void> {
```

```
    // relay = TURN (none configured by default); srflx/host = direct path.
    try {
      const stats = await slot.pc.getStats();
      let selected: RTCIceCandidatePairStats | undefined;
      stats.forEach((s) => {
        if (s.type === "candidate-pair" && (s as RTCIceCandidatePairStats).nominated) {
          selected = s as RTCIceCandidatePairStats;
        }
      });
      const localId = selected?.localCandidateId;
      if (localId) {
        const local = stats.get(localId) as { candidateType?: string } | undefined;
        slot.info.candidateType = local?.candidateType ?? null;
        this.emitPeers();
      }
    } catch {
      // getStats is best-effort diagnostics only.
    }
  }

  private emitPeers(): void {
    // Only notify when something observable actually changed — React state
    // setters otherwise re-render consumers on every poll/ping.
    const list = this.peerList();
    const fingerprint = JSON.stringify(
      list.map((p) => [p.id, p.name, p.connectionState, p.candidateType, p.rttMs]),
    );
    if (fingerprint === this.lastPeersFingerprint) return;
    this.lastPeersFingerprint = fingerprint;
    this.opts.onPeersChanged?.(list);
  }
}
```

src/lib/og/site.json

```
{
  "title": "Major Spark",
  "card": "custom"
}
```

src/lib/paths.ts

```
export type CareerId = "architect" | "mechanical" | "chef" | "cyber" | "it";

export type Step = {
  action: string;
  short: string;
  explain: string;
  done: string;
  time: string;
  alongside: string;
  support?: string;
};

export type Career = {
  id: CareerId;
  name: string;
  line: string;
  start: string;
  steps: Step[];
};

export const ACCESS_STEP: Step = {
  action: "Contact ACCESS at your campus and ask them to put your accommodations in place.",
  short: "Ask your campus ACCESS office to set up your accommodations.",
  explain:
    "ACCESS is the Miami Dade College office that sets up accommodations. Email the office at the campus whe
re you take classes. Ask what they need from you before the term starts, so the accommodations are ready whe
n classes begin.",
  done: "ACCESS has your request, and you know what they still need from you.",
  time: "Before the first day of class. Start as soon as you register.",
  alongside: "Write down the campus office you emailed and the date you sent it.",
};

export const CAREERS: Career[] = [
  {
    id: "architect",
    name: "Architect",
    line: "Drawings, buildings, and a long license path.",
    start: "Starts with design classes at Miami Dade College.",
    steps: [
      {
        action: "Enroll in Architectural Design and Construction Technology.",
        short: "Start the architecture associate degree at MDC.",
        explain:
          "Start the Associate in Science in Architectural Design and Construction Technology. The program c
ode is 26034. An advisor sets the first term: Architectural Communications, Architectural Drawing 1, History
 of Architecture, and College Algebra. This degree lets you work as a drafter. It does not make you a licens
ed architect.",
        done: "You are registered for that first term, and the program name is written down.",
        time: "See an advisor first. Then register before the last day listed on this page.",
        alongside: "Save the program map before you leave campus.",
        support:
          "Tell the advisor about classroom support you have used before, so the first studio can be set up
ahead of time.",
      },
      {
        action: "Finish drawing, design, and the first CAD class.",
        short: "Finish the first drawing, design, and CAD classes.",
        explain:
          "The next terms are Architectural Design 1, Materials and Construction, and Computer Aided Draftin
g 1. The portfolio starts here, not later. Save one finished drawing from each class in the same term you tu
rn it in. A firm asks for that folder. The transcript does not show the drawings.",
        done: "Those classes are passed, and you can show one drawing from each.",
        time: "The next terms on the sequence guide.",
        alongside: "Photograph each finished drawing the week it is graded.",
        support:
          "If studio hours run long, ask about a lab time or a quieter station before the term fills up.",
      },
      {
        action: "Take building code and presentation when the sequence reaches them.",
        short: "Take code and presentation in their terms.",
        explain:
          "Building Code Regulations and the computer presentation classes are already on the degree sequenc
e. Take them in the term the guide lists. Save the presentation file that term. You do not add a separate so
ftware class later. These are the classes that let you talk about code and show work on a screen.",
        done: "The code class and a computer presentation class are passed, and the file is saved.",
        time: "The later terms of the associate degree.",
        alongside: "Put the presentation file in the same folder as the drawings.",
      },
      {
        action: "The portfolio is finished when the studios are finished.",
        short: "The portfolio closes with the last studio.",
        explain:
          "There is no extra portfolio semester. When the last studio is graded, the folder should already h
old one piece from each class plus the digital presentation. That is what you send with a transfer applicati
on or a drafter application.",
        done: "The folder has one piece from each finished studio, plus the digital presentation.",
        time: "A few minutes after each critique, then one review before you apply.",
        alongside: "Add the piece the same week the project is graded.",
```

```
      },
      {
        action: "Finish the associate degree, then the professional degree.",
        short: "Finish the A.S., then transfer for the architecture degree.",
        explain:
          "Finish the Associate in Science at MDC. A licensed architect still needs a professional degree af
ter that. In Miami, Florida International University is a common next school. Ask which of these credits tra
nsfer before you take extras. You can work in a firm as a drafter while that degree is still ahead.",
        done: "The associate degree is posted, and you have applied to the next architecture program.",
        time: "About two years full time at MDC, then the bachelor’s program.",
        alongside: "Order an official transcript the week you apply.",
        support:
          "Ask the next school’s access office to receive your accommodation history before classes start.",
      },
      {
        action: "After the professional degree, open the license record.",
        short: "Start the license record after the professional degree.",
        explain:
          "MDC does not give the architect exam. After the professional degree, open an NCARB record at ncar
b.org. That is where hours are logged and exam sections are scheduled. Starting the record early is the edge
. Classmates who wait have the degree and no hour log.",
        done: "The NCARB record is open, or you have a written date to open it.",
        time: "After the professional degree. The license itself takes several more years.",
        alongside: "Save the login and the next exam date in the same folder as the portfolio.",
      },
    ],
  },
  {
    id: "mechanical",
    name: "Mechanical engineer",
    line: "Machines, heat, motion — math first, then a university degree.",
    start: "Starts with math and science at Miami Dade College.",
    steps: [
      {
        action: "Start the mechanical engineering pathway at MDC.",
        short: "Enroll in the engineering associate pathway.",
        explain:
          "Start the Associate in Arts pathway for mechanical engineering. The program code is 10911. Miami
Dade College does not give the bachelor’s degree. An advisor sets the first term: English, Introduction to E
ngineering, College Algebra, and the first-year seminar.",
        done: "You are registered on that pathway, not in a random set of classes.",
        time: "See an advisor first. Then register before the last day listed on this page.",
        alongside: "Save the pathway name and the transfer school you are aiming at.",
        support:
          "Mention accommodations you have used, especially for timed math tests, before the term starts.",
      },
      {
        action: "Finish chemistry, precalculus, and trigonometry.",
        short: "Finish chemistry and the math before calculus.",
        explain:
          "The sequence next takes General Chemistry with the lab, Pre-Calculus Algebra, and Trigonometry. I
f placement puts you in an earlier math class, take it. It adds a term. It is still the school path. Enginee
ring schools will not skip these.",
        done: "Chemistry, precalculus, and trigonometry are passed.",
        time: "The terms before calculus on your sequence guide.",
        alongside: "After each math exam, write the one topic to review before the next class.",
      },
      {
        action: "Finish calculus and physics with calculus.",
        short: "Finish the calculus and physics sequence.",
        explain:
          "The pathway then runs Calculus 1, 2, and 3, Physics with Calculus 1 and 2, and both labs. These a
re the classes a university uses to decide if you are ready. A graduate who transfers without them repeats a
 year.",
        done: "Calculus 3 and Physics with Calculus 2 are passed, including the labs.",
        time: "Several terms. Earlier math makes it longer.",
        alongside: "Check the transfer guide once a term so a class is not left off.",
      },
      {
        action: "The last MDC terms include statics, differential equations, and C.",
        short: "Finish statics, differential equations, and programming.",
        explain:
          "Engineering Mechanics — Statics, Differential Equations, and C for Engineers are on the MDC seque
nce for this pathway. They are the edge at transfer. Classmates who stop at the general education classes ar
rive with less of the first engineering year already done.",
        done: "Those three are passed, and you know which ones the next school accepts.",
        time: "The last terms of the associate pathway.",
        alongside: "Write down which credits the university accepted the week the list arrives.",
        support:
          "Ask the university access office to set up support before the first engineering exam.",
      },
      {
        action: "Finish the associate pathway and transfer.",
        short: "Finish at MDC, then start the bachelor’s degree.",
        explain:
          "Finish the Associate in Arts on this pathway, then transfer into a Bachelor of Science in mechani
cal engineering. Florida International University is a common South Florida next school. Apply while one ter
m is still left so there is no empty gap.",
        done: "The associate degree is posted, and you have an admission decision.",
        time: "About two years full time if math starts at College Algebra. Longer if it starts earlier.",
```

```
        alongside: "Order the official transcript the week you apply.",
      },
      {
        action: "The last year of the bachelor’s degree includes the FE exam.",
        short: "The bachelor’s ends with the FE exam.",
        explain:
          "The Fundamentals of Engineering exam is not an MDC class. It belongs to the last year of the bach
elor’s degree, before you apply for the first engineering job. Register at ncees.org and take it at Pearson
VUE in that year. You do not wait until after you are hired to decide. Passing it lets you apply in Florida
as an engineer intern.",
        done: "You have a test date, or a passing result.",
        time: "A few months of review in the last year of the bachelor’s degree.",
        alongside: "Put the test date on a calendar the day you register.",
      },
    ],
  },
  {
    id: "chef",
    name: "Chef",
    line: "Knives, heat, and a kitchen that stays usable while you cook.",
    start: "Starts in Culinary Arts at Miami Dade College.",
    steps: [
      {
        action: "Enroll in Culinary Arts Management at MDC.",
        short: "Start the culinary associate degree.",
        explain:
          "Apply for the Associate in Science in Culinary Arts Management. The program code is 22031. The fi
rst semester is three classes: Culinary Terminology, Food Production 1, and Safety and Sanitation. In the Fo
od Production lab, clean while the food is still cooking. Wipe the board and wash the knife before the food
goes on the plate.",
        done: "You are registered for that first term.",
        time: "First you apply and get accepted. Then register before the last day listed on this page. Thos
e three classes take one semester.",
        alongside: "Save the acceptance email and the class schedule in one place.",
        support:
          "Ask about lab support before the first knife class: extra time, a written recipe, or a quieter st
ation.",
      },
      {
        action: "The first term includes the food-handler card.",
        short: "The first term includes the food-handler card.",
        explain:
          "Safety and Sanitation, HFT 1212, is a first-term class. The food-handler card belongs to that sam
e term. The class teaches the rules and does not hand you the card. Buy ServSafe Food Handler at servsafe.co
m and take it while you are in HFT 1212, unless the advisor says the class includes the exam. You do not pic
k another time later.",
        done: "HFT 1212 is on your schedule, and the card is saved with the date it expires.",
        time: "The first term. The test itself is a few hours.",
        alongside: "Ask the culinary advisor in the first week whether the class includes the exam.",
      },
      {
        action: "Production, baking, and garde manger post the Chef Apprentice certificate.",
        short: "Those courses post the Chef Apprentice certificate.",
        explain:
          "Food Production 2, Basic Baking, and Garde Manger are the next terms in the sequence. When a reci
pe says sauté, cook in a little fat until the food is soft and light gold. Finishing the marked courses post
s the Chef Apprentice college credit certificate on your MDC transcript. There is no separate test and no ex
tra term. Ask for it to be posted when those courses are done.",
        done: "Those classes are passed, and you have asked if the Chef Apprentice certificate is posted.",
        time: "The middle terms of the program.",
        alongside: "In each lab, clear the board before you plate.",
      },
      {
        action: "The last term includes ServSafe Manager.",
        short: "The last term includes ServSafe Manager.",
        explain:
          "The last part of the program is Supervision, Food and Beverage Cost Controls, the internship, and
 the externship. ServSafe Manager belongs in that same stretch, not after you graduate. Register at servsafe
.com and book the proctored exam while those classes are on your schedule. Ask the culinary advisor if MDC h
as a sitting that term. Passing it makes you a Certified Food Protection Manager.",
        done: "The internship is on your transcript, and ServSafe Manager is passed.",
        time: "The last terms, including the exam in that same stretch.",
        alongside: "After each kitchen week, write one station you can now run.",
        support:
          "If the noise or pace is hard, ask for the same station for the first few shifts.",
      },
      {
        action: "Finishing the degree posts the operations certificate.",
        short: "The degree posts the operations certificate.",
        explain:
          "The last step is finishing the Associate in Science. The Culinary Arts Management Operations coll
ege credit certificate posts from the marked courses. It is not another exam and not another semester. You l
eave with the degree, that certificate, the food-handler card from the first term, and ServSafe Manager from
 the last term.",
        done: "The degree is posted, the operations certificate shows, and both cards are saved with their d
ates.",
        time: "The end of the program. Full time is about two years.",
        alongside: "Check that the degree and both cards are in one folder.",
      },
    ],
```

```
  },
  {
    id: "cyber",
    name: "Cybersecurity specialist",
    line: "Protect systems. Start with how computers talk.",
    start: "Starts in cybersecurity classes at Miami Dade College.",
    steps: [
      {
        action: "Start the cybersecurity associate degree at MDC.",
        short: "Enroll in the cybersecurity associate degree.",
        explain:
          "Start the Associate in Science in Cybersecurity. The first term is Introduction to Computers, Cyb
ersecurity Fundamentals, Networking Technologies, and math. Miami Dade College also offers a bachelor's degr
ee in cybersecurity after this one, so you can stay to finish.",
        done: "You are registered, and you know you are in the associate degree first.",
        time: "See an advisor first. Then register before the last day listed on this page.",
        alongside: "Write the degree name on the first page of your notes.",
        support:
          "Ask about exam time and a quiet room before the first quiz, not after a low score.",
      },
      {
        action: "The networking and fundamentals terms include those exams.",
        short: "Those terms include Network+ and Security+.",
        explain:
          "Networking Technologies is the term for CompTIA Network+. Cybersecurity Fundamentals is the term
for CompTIA Security+. Schedule each exam at Pearson VUE while you are in that class. The class does not han
d you the card, and you do not save the exam for after graduation.",
        done: "Both classes are passed, and both exams are scheduled or passed.",
        time: "The early terms, then a few weeks of review for each exam.",
        alongside: "Schedule the exam date before you start the review.",
        support:
          "Request the testing accommodation when you book the exam. Testing centers need notice.",
      },
      {
        action: "The Linux class term includes the Linux+ exam.",
        short: "The Linux term includes Linux+.",
        explain:
          "The next terms are operating systems, secure scripting, and Linux. CompTIA Linux+ belongs to the
Linux class term, CTS 1111. Buy it from CompTIA and schedule it at Pearson VUE while that class is on your s
chedule.",
        done: "Those classes are passed, and Linux+ is scheduled or passed.",
        time: "The middle terms of the associate degree.",
        alongside: "Save each passing score in the same folder as your schedule.",
      },
      {
        action: "The later security classes include their exams.",
        short: "Those later classes include their exams.",
        explain:
          "Cybersecurity Analysis is the term for CompTIA CySA+. Network Defense and Countermeasures is the
term for the EC-Council Certified Network Defender exam. Schedule each one during that class. Ask if the ter
m includes a voucher before you pay. You are not choosing a someday certificate. The class term is the time.
",
        done: "Both classes are passed, and you have a date for at least one of the two exams.",
        time: "The later terms, plus review time for each exam.",
        alongside: "Ask the program if this term includes an exam voucher before you pay.",
      },
      {
        action: "Finish the associate degree, then the bachelor's if you stay.",
        short: "Finish the A.S., then the B.S. at MDC if you continue.",
        explain:
          "Finish the Associate in Science. The Bachelor of Science in Cybersecurity is offered at MDC after
 it. A first job beside the degree, such as help desk or a security operations internship, is the other edge
. The certificates plus real tickets put you ahead of a graduate who only has classes.",
        done: "The associate degree is posted, and you know whether you are continuing to the bachelor's.",
        time: "About two years full time for the associate degree. The bachelor's is longer.",
        alongside: "Keep a private list of problems you solved. No customer names.",
      },
    ],
  },
  {
    id: "it",
    name: "IT consultant",
    line: "Solve a tech problem and explain it in plain language.",
    start: "Starts in information technology at Miami Dade College.",
    steps: [
      {
        action: "Start Computer Information Technology at MDC.",
        short: "Enroll in the IT associate degree.",
        explain:
          "Start the Associate in Science in Computer Information Technology. The program code is 25055. The
 first term is English, Introduction to Computers, Networking Technologies, and math. You can stay afterward
 for the Bachelor of Science in Information Systems Technology.",
        done: "You are registered in that associate program.",
        time: "See an advisor first. Then register before the last day listed on this page.",
        alongside: "Save the program map with your advisor's name on it.",
        support: "Tell the advisor which classroom supports have helped you before.",
      },
      {
        action: "The A+ classes include the A+ exam.",
        short: "Those classes include the A+ exam.",
```

```
        explain:
          "A+ Computer Operating Systems and A+ Computer Essentials are the terms for CompTIA A+. Schedule t
he exam at Pearson VUE while you are in the second of those classes. The classes do not hand you the card. T
he term is already on the plan, so you do not pick a later month.",
        done: "Both classes are passed, and A+ is scheduled or passed.",
        time: "The terms those classes appear, then a few weeks of review.",
        alongside: "Schedule the exam before you start the review.",
        support: "Request extra time or a quiet room when you book the exam.",
      },
      {
        action: "Each later class includes its own exam.",
        short: "Each later class includes its exam.",
        explain:
          "Networking Technologies is the term for CompTIA Network+. IT Project Management is the term for C
ompTIA Project+. The two Windows support classes are the term for the Microsoft Modern Desktop Administrator
 exam. Schedule each exam during its class. CompTIA exams are bought from CompTIA. The Microsoft exam starts
 at Microsoft, then Pearson VUE.",
        done: "Each of those classes is passed, and each exam is scheduled in that class’s term.",
        time: "Across the middle and later terms.",
        alongside: "Keep each score with your class schedule.",
      },
      {
        action: "The marked courses post the college certificates.",
        short: "Those courses post the college certificates.",
        explain:
          "The Help Desk Support Technician certificate and the Information Technology Support certificate a
re part of this degree. MDC posts them when the marked courses are finished. There is no extra exam and no e
xtra term. Ask which of your classes count, and check that each certificate shows before the last term ends.
",
        done: "You have asked which courses count, and you know whether each certificate is posted.",
        time: "The same terms as the degree. Confirm it before the last term ends.",
        alongside: "Write the certificate names next to the courses that earn them.",
      },
      {
        action: "Finish the associate degree ready to be hired.",
        short: "Graduate with the degree and the certificates.",
        explain:
          "Finish the Associate in Science. You can continue into the Bachelor of Science in Information Sys
tems Technology at MDC. By then the path has already included A+, Network+, Project+, and the Microsoft exam
, each in the term of its class, plus the college certificates the courses post.",
        done: "The degree is posted, and the exam results are saved with it.",
        time: "About five terms full time, on the sequence guide.",
        alongside: "Read your two-sentence offer, or the job you want, to someone outside tech.",
      },
    ],
  },
];

export function careerById(id: CareerId): Career {
  const found = CAREERS.find((career) => career.id === id);
  if (!found) throw new Error("Unknown career");
  return found;
}

export type Interest = "make" | "fix" | "help";
export type SchoolPlan = "mdc" | "transfer";
export type WorkTiming = "during" | "after";

export type Suggestion = {
  id: CareerId;
  reason: string;
};

/** Three answers, only the five careers already in the book. Not a personality test. */
export function suggestCareers(
  interest: Interest,
  school: SchoolPlan,
  work: WorkTiming,
): Suggestion[] {
  if (interest === "make" && school === "transfer") {
    return [
      {
        id: "architect",
        reason:
          work === "during"
            ? "You want to make things people can see, and you can start that at MDC, then transfer. Office
work can begin before the degree is finished."
            : "You want to make things people can see. Architecture starts at MDC and continues at a univers
ity.",
      },
    ];
  }

  if (interest === "make") {
    return [
      {
        id: "chef",
        reason:
          "You want to make something real, and culinary arts can be finished at MDC. Kitchen shifts can sta
rt while you are still in class.",
```

```
      },
    ];
  }

  if (interest === "fix" && school === "transfer") {
    return [
      {
        id: "mechanical",
        reason:
          work === "during"
            ? "You like figuring out how things work. Mechanical engineering starts with math and science at
 MDC, then a university degree, with a summer of hands-on work along the way."
            : "You like figuring out how things work. Mechanical engineering starts at MDC and needs a unive
rsity degree after that.",
      },
    ];
  }

  if (interest === "fix") {
    return [
      {
        id: "cyber",
        reason:
          "You like how systems work, and cybersecurity can be finished at MDC — including a bachelor's — wi
th a first tech job while you study.",
      },
    ];
  }

  const it: Suggestion = {
    id: "it",
    reason:
      work === "during"
        ? "You want to help people use technology. An IT degree at MDC can include a support job while you a
re still in school."
        : "You want to help people use technology. An IT degree at MDC leads to one clear first service afte
r you finish.",
  };

  if (school === "transfer") return [it];

  return [
    it,
    {
      id: "cyber",
      reason:
        "Cybersecurity is the other path you can finish at MDC if you would rather protect systems than expl
ain them.",
    },
  ];
}
```

src/lib/plans.ts

```
import type { CareerId } from "@/lib/paths";

export type PlanCourse = {
  code: string;
  name: string;
};

export type PlanTerm = {
  label: string;
  credits: string;
  courses: PlanCourse[];
};

export type PlanCredential = {
  name: string;
  when: string;
};

export type Plan = {
  program: string;
  pace: string;
  terms: PlanTerm[];
  entry: PlanCredential[];
  credentials: PlanCredential[];
  struggles: string[];
};

export const PLANS: Record<CareerId, Plan> = {
  architect: {
    program: "A.S. Architectural Design and Construction Technology · code 26034 · 66 credits",
    pace: "Six terms if you study full time, on the Fall 2024 sequence guide. A licensed architect still nee
ds a professional degree after MDC.",
    terms: [
      {
        label: "Term 1",
        credits: "12 credits",
        courses: [
          { code: "ARC 1115", name: "Architectural Communications 1" },
          { code: "ARC 1126C", name: "Architectural Drawing 1" },
          { code: "ARC 2701", name: "History of Architecture 1" },
          { code: "MAC 1105", name: "College Algebra" },
        ],
      },
      {
        label: "Term 2",
        credits: "15 credits",
        courses: [
          { code: "ARC 1301", name: "Architectural Design 1" },
          { code: "ARC 2461", name: "Architectural Materials and Construction 1" },
          { code: "ARC 2171", name: "Computer Aided Drafting 1" },
          { code: "ENC 1101", name: "English Composition 1" },
        ],
      },
      {
        label: "Term 3",
        credits: "7 credits",
        courses: [
          { code: "AMH or POS", name: "One social science: AMH 2010, AMH 2020, or POS 2041" },
          { code: "Elective", name: "One ARC, BCT, BCN, or CGS 1060C course" },
        ],
      },
      {
        label: "Term 4",
        credits: "12 credits",
        courses: [
          { code: "ARC 1128", name: "Architectural Drawing 2" },
          { code: "ARC 2056", name: "Computer Aided Architectural Presentation" },
          { code: "ARC 2172", name: "Computer Aided Drafting 2" },
        ],
      },
      {
        label: "Term 5",
        credits: "14 credits",
        courses: [
          { code: "ARC 2053", name: "Architectural Computer Applications" },
          { code: "ARC 2681", name: "Environmental Technology" },
          { code: "BCT 2760", name: "Building Code Regulations" },
          { code: "Elective", name: "One more ARC, BCT, BCN, or CGS 1060C course" },
        ],
      },
      {
        label: "Term 6",
        credits: "6 credits",
        courses: [
          { code: "Humanities", name: "One humanities course, such as ARH 1000 or HUM 1020" },
          { code: "Science", name: "One natural science, such as PHY 2053" },
          { code: "CSP or CGS 1060C", name: "Computer Skills Placement, or Introduction to Computer Technolo
gy" },
```

```
        ],
      },
    ],
    entry: [
      {
        name: "No architect license yet",
        when: "You can work as a drafter or designer in a firm with this degree. You may not call yourself a
n architect until you are licensed.",
      },
      {
        name: "Florida architect license",
        when: "Not an MDC exam. After a professional degree, open a record at ncarb.org, log your hours, and
 sign up for the exam sections there.",
      },
    ],
    credentials: [
      {
        name: "Design portfolio",
        when: "Build it during the drawing and design courses. Transfer schools ask for it.",
      },
    ],
    struggles: [
      "Studio classes run long, and the room can be loud, bright, and crowded.",
      "Critiques put your work in front of the class. Feedback can feel personal even when it is about the d
rawing.",
      "Deadlines stack. A project that looks almost done can still need many quiet hours.",
      "Switching from a detail to the whole building, and back, is constant.",
      "Group work depends on shared files and meetings that move fast.",
    ],
  },
  mechanical: {
    program: "A.A. Engineering — Mechanical pathway · code 10911 · 60 credits",
    pace: "Six terms if you start in College Algebra and study full time, on the Fall 2024 sequence guide. E
arlier math adds terms.",
    terms: [
      {
        label: "Term 1",
        credits: "13 credits",
        courses: [
          { code: "ENC 1101", name: "English Composition 1" },
          { code: "EGN 1008C", name: "Introduction to Engineering" },
          { code: "Humanities", name: "One humanities course" },
          { code: "MAC 1105", name: "College Algebra" },
          { code: "SLS 1106", name: "First Year Experience Seminar" },
        ],
      },
      {
        label: "Term 2",
        credits: "14 credits",
        courses: [
          { code: "ENC 1102", name: "English Composition 2" },
          { code: "Social science", name: "AMH 2010, AMH 2020, or POS 2041" },
          { code: "CHM 1045", name: "General Chemistry and Qualitative Analysis" },
          { code: "CHM 1045L", name: "General Chemistry lab" },
          { code: "MAC 1140", name: "Pre-Calculus Algebra" },
        ],
      },
      {
        label: "Term 3",
        credits: "3 credits",
        courses: [{ code: "MAC 1114", name: "Trigonometry" }],
      },
      {
        label: "Term 4",
        credits: "14 credits",
        courses: [
          { code: "MAC 2311", name: "Calculus and Analytical Geometry 1" },
          { code: "Social science", name: "A second social science" },
          { code: "Speech", name: "One oral communication course, such as SPC 2608" },
          { code: "PHY 1025", name: "Basic Physics" },
        ],
      },
      {
        label: "Term 5",
        credits: "12 credits",
        courses: [
          { code: "Humanities", name: "A second humanities course" },
          { code: "MAC 2312", name: "Calculus and Analytical Geometry 2" },
          { code: "PHY 2048", name: "Physics with Calculus 1" },
          { code: "PHY 2048L", name: "Physics with Calculus 1 lab" },
        ],
      },
      {
        label: "Term 6",
        credits: "4 credits",
        courses: [{ code: "MAC 2313", name: "Calculus and Analytical Geometry 3" }],
      },
      {
        label: "Before you transfer",
        credits: "Ask an advisor",
```

```
        courses: [
          { code: "MAP 2302", name: "Differential Equations" },
          { code: "PHY 2049", name: "Physics with Calculus 2" },
          { code: "PHY 2049L", name: "Physics with Calculus 2 lab" },
          { code: "COP 2270", name: "C for Engineers" },
          { code: "EGN 2312", name: "Engineering Mechanics — Statics" },
        ],
      },
    ],
    entry: [
      {
        name: "No license required for a first engineering job",
        when: "You can work under a licensed engineer before you have your own license.",
      },
      {
        name: "Fundamentals of Engineering exam",
        when: "Not an MDC exam. Near the end of the bachelor’s degree, register at ncees.org and take it at
Pearson VUE.",
      },
      {
        name: "Professional Engineer exam",
        when: "Years after the FE exam, if you want to sign and seal your own work.",
      },
    ],
    credentials: [
      {
        name: "Computer Skills Placement, or CGS 1060C",
        when: "Take CGS 1060C at MDC, or take the Computer Skills Placement test at an MDC testing center. A
sk advising which one your plan uses.",
      },
    ],
    struggles: [
      "Math is every term, and a timed test can hide what you actually know.",
      "Labs have several steps at once. Missing one early step breaks the later ones.",
      "Problem sets are long. Starting is often harder than the math itself.",
      "Group projects assume you can jump into someone else’s half-finished work.",
      "Small calculation errors matter, and catching them takes a second, slower pass.",
    ],
  },
  chef: {
    program: "A.S. Culinary Arts Management · code 22031 · 60 credits",
    pace: "Six terms if you study full time, on the Fall 2024 sequence guide.",
    terms: [
      {
        label: "Term 1",
        credits: "12 credits",
        courses: [
          { code: "ENC 1101", name: "English Composition 1" },
          { code: "FSS 1200", name: "Culinary Terminology and Procedures" },
          { code: "FSS 1202C", name: "Food Production 1 — Fundamental Skills" },
          { code: "HFT 1212", name: "Safety and Sanitation" },
        ],
      },
      {
        label: "Term 2",
        credits: "12 credits",
        courses: [
          { code: "Social science", name: "AMH 2010, AMH 2020, or POS 2041" },
          { code: "FSS 1204C", name: "Food Production 2 — American Regional Cuisine" },
          { code: "FSS 1246C", name: "Basic Baking — Foundational Skills" },
          { code: "MGF 1130", name: "Mathematical Thinking" },
        ],
      },
      {
        label: "Term 3",
        credits: "9 credits",
        courses: [
          { code: "CCT or CGS 1060C", name: "Computer Competency Test, or Introduction to Computer Technolog
y" },
          { code: "HFT 1220", name: "Supervision and Personnel Management" },
          { code: "FSS 2248C", name: "Garde Manger" },
          { code: "HUN 1201", name: "Essentials of Human Nutrition" },
        ],
      },
      {
        label: "Term 4",
        credits: "12 credits",
        courses: [
          { code: "BSC 1005", name: "General Education Biology" },
          { code: "FSS 1801", name: "Culinary Sustainability and Practices" },
          { code: "FSS 2242C", name: "International Cuisines" },
          { code: "HFT 1454", name: "Food and Beverage Cost Controls" },
        ],
      },
      {
        label: "Term 5",
        credits: "8 credits",
        courses: [
          { code: "ACG 1403", name: "Excel for Business" },
          { code: "FSS 2205C", name: "Food Production 3 — Contemporary Cuisine" },
```

```
          { code: "FSS 2381", name: "Culinary Management Internship" },
        ],
      },
      {
        label: "Term 6",
        credits: "7 credits",
        courses: [
          { code: "PHI 2010", name: "Introduction to Philosophy" },
          { code: "FSS 2940", name: "Culinary Management Externship" },
        ],
      },
    ],
    entry: [
      {
        name: "Florida food-handler training",
        when: "HFT 1212 at MDC teaches this. It does not give you the certificate. Buy ServSafe Food Handler
 at servsafe.com and take it on your own, unless the advisor says this term’s class includes the exam.",
      },
      {
        name: "ServSafe Manager",
        when: "The higher certificate. Register yourself at servsafe.com and book a proctored exam. Ask the
culinary advisor if MDC has a sitting. If not, you do it on your own. Take it before you graduate.",
      },
      {
        name: "No state chef license",
        when: "Florida does not license chefs. The American Culinary Federation Certified Culinarian credent
ial is optional, not required to get hired.",
      },
    ],
    credentials: [
      {
        name: "College Credit Certificate: Chef Apprentice",
        when: "Not a separate exam. You earn it by finishing the marked courses at MDC.",
      },
      {
        name: "College Credit Certificate: Culinary Arts Management Operations",
        when: "Not a separate exam. You earn it by finishing the marked courses at MDC.",
      },
    ],
    struggles: [
      "A working kitchen is hot, loud, and full of smells and timers at the same time.",
      "Service does not pause. Directions are often short and shouted.",
      "Shifts start early, run late, or both. Sleep and meals get irregular.",
      "You are on your feet, and the pace can change in a minute.",
      "One ticket has many steps. Dropping one step is easy when the line speeds up.",
    ],
  },
  cyber: {
    program: "A.S. Cybersecurity · code 25079 · 60 credits",
    pace: "Five terms if you study full time, on the Spring 2025 sequence guide. It continues into the Bache
lor of Science in Cybersecurity at MDC.",
    terms: [
      {
        label: "Term 1",
        credits: "15 credits",
        courses: [
          { code: "CGS 1060C", name: "Introduction to Computer Technology and Applications" },
          { code: "CTS 1120", name: "Cybersecurity Fundamentals" },
          { code: "CTS 1134", name: "Networking Technologies" },
          { code: "MAC 1105 or STA 2023", name: "College Algebra or Statistical Methods" },
        ],
      },
      {
        label: "Term 2",
        credits: "15 credits",
        courses: [
          { code: "CGS 1700", name: "Introduction to Operating Systems" },
          { code: "CIS 1531", name: "Introduction to Secure Scripting" },
          { code: "CTS 1111", name: "Linux+" },
          { code: "ENC 1101", name: "English Composition 1" },
        ],
      },
      {
        label: "Term 3",
        credits: "6 credits",
        courses: [
          { code: "Social science", name: "AMH 2010, AMH 2020, or POS 2041" },
          { code: "Science", name: "One natural science" },
        ],
      },
      {
        label: "Term 4",
        credits: "12 credits",
        courses: [
          { code: "CET 2880C", name: "Digital Forensics" },
          { code: "CIS 2350", name: "Cybersecurity Analysis" },
          { code: "CTS 2314", name: "Network Defense and Countermeasures" },
        ],
      },
      {
```

```
        label: "Term 5",
        credits: "12 credits",
        courses: [
          { code: "Electives", name: "9 credits chosen with an advisor" },
          { code: "Humanities", name: "One humanities course" },
        ],
      },
    ],
    entry: [
      {
        name: "No Florida license",
        when: "You do not need a state license to work in cybersecurity.",
      },
      {
        name: "CompTIA Security+",
        when: "The exam employers most often ask for first. CTS 1120 prepares you. Buy it from CompTIA and s
chedule it at Pearson VUE.",
      },
      {
        name: "CompTIA Network+",
        when: "Often asked with Security+. CTS 1134 prepares you. Buy it from CompTIA and schedule it at Pea
rson VUE.",
      },
    ],
    credentials: [
      { name: "CompTIA Network+", when: "CTS 1134 prepares you. Buy the exam from CompTIA and schedule it at
 Pearson VUE." },
      { name: "CompTIA Security+", when: "CTS 1120 prepares you. Buy the exam from CompTIA and schedule it a
t Pearson VUE." },
      { name: "CompTIA Linux+", when: "CGS 1700 and CTS 1111 prepare you. Buy the exam from CompTIA and sche
dule it at Pearson VUE." },
      { name: "CompTIA CySA+", when: "CIS 2350 prepares you. Buy the exam from CompTIA and schedule it at Pe
arson VUE." },
      { name: "Certified Network Defender", when: "CTS 2314 prepares you. Buy the exam from EC-Council and s
chedule it at a test center." },
    ],
    struggles: [
      "Screens, logs, and alerts can run for hours with very little change of scene.",
      "An incident can demand a fast switch from quiet reading to urgent decisions.",
      "Certification exams are timed. Knowing the material is not the same as finishing the clock.",
      "The tools and the threats change, so last term’s notes do not stay finished.",
      "On-call work can interrupt sleep and plans with little warning.",
    ],
  },
  it: {
    program: "A.S. Computer Information Technology · code 25055 · 60 credits",
    pace: "Five terms if you study full time, on the Fall 2024 sequence guide. It can continue into the Bach
elor of Science in Information Systems Technology.",
    terms: [
      {
        label: "Term 1",
        credits: "14 credits",
        courses: [
          { code: "ENC 1101", name: "English Composition 1" },
          { code: "CTS 1134 or CTS 1650", name: "Networking Technologies, or CCNA 1" },
          { code: "MAC 1105 or STA 2023", name: "College Algebra or Statistical Methods" },
          { code: "CGS 1060C", name: "Introduction to Computer Technology and Applications" },
        ],
      },
      {
        label: "Term 2",
        credits: "15 credits",
        courses: [
          { code: "Humanities", name: "One humanities course" },
          { code: "CGS 1560", name: "A+ Computer Operating Systems" },
          { code: "CTS 1328", name: "Supporting Microsoft Clients" },
          { code: "Elective", name: "4 credits chosen with an advisor" },
        ],
      },
      {
        label: "Term 3",
        credits: "6 credits",
        courses: [
          { code: "Social science", name: "AMH 2010, AMH 2020, or POS 2041" },
          { code: "Science", name: "One natural science" },
        ],
      },
      {
        label: "Term 4",
        credits: "12 credits",
        courses: [
          { code: "CTS 1131", name: "A+ Computer Essentials and Support" },
          { code: "CTS 2148C", name: "IT Project Management" },
          { code: "CTS 2153", name: "Supporting Windows Users and Applications" },
        ],
      },
      {
        label: "Term 5",
        credits: "13 credits",
        courses: [
```

```
          { code: "CGS 2108", name: "Advanced Desktop Applications" },
          { code: "Electives", name: "9 credits chosen with an advisor" },
        ],
      },
    ],
    entry: [
      {
        name: "No Florida license",
        when: "You do not need a state license to work in IT support or consulting.",
      },
      {
        name: "CompTIA A+",
        when: "The usual first exam for help desk. CGS 1560 and CTS 1131 prepare you. Buy it from CompTIA an
d schedule it at Pearson VUE.",
      },
      {
        name: "CompTIA Network+",
        when: "Add this for a network job. CTS 1134 prepares you. Buy it from CompTIA and schedule it at Pea
rson VUE.",
      },
    ],
    credentials: [
      { name: "CompTIA Network+", when: "CTS 1134 prepares you. Buy the exam from CompTIA and schedule it at
 Pearson VUE." },
      { name: "CompTIA A+", when: "CGS 1560 and CTS 1131 prepare you. Buy the exam from CompTIA and schedule
 it at Pearson VUE." },
      { name: "CompTIA Project+", when: "CTS 2148C prepares you. Buy the exam from CompTIA and schedule it a
t Pearson VUE." },
      {
        name: "Microsoft Modern Desktop Administrator Associate",
        when: "CTS 1328 and CTS 2153 prepare you. Sign up through Microsoft, then schedule at Pearson VUE.",
      },
      {
        name: "College Credit Certificate: Help Desk Support Technician",
        when: "Not a separate exam. You earn it by finishing the marked courses at MDC.",
      },
      {
        name: "College Credit Certificate: Information Technology Support",
        when: "Not a separate exam. You earn it by finishing the marked courses at MDC.",
      },
    ],
    struggles: [
      "Tickets arrive unfinished, and the next one starts before the last one feels done.",
      "You explain the same problem to a frustrated person, often more than once.",
      "The work jumps from a password reset to a network outage in one afternoon.",
      "Meetings and messages interrupt the part of the job that needs quiet focus.",
      "Some problems have no clear end, so done has to be defined out loud.",
    ],
  },
};

export type StudyPace = "full" | "three-quarter" | "part";

const CREDITS: Record<string, number> = {
  "ARC 1115": 2,
  "ARC 1126C": 4,
  "ARC 1128": 4,
  "ARC 1301": 4,
  "ARC 2053": 4,
  "ARC 2056": 4,
  "ARC 2171": 4,
  "ARC 2172": 4,
  "ARC 2461": 4,
  "ARC 2681": 3,
  "ARC 2701": 3,
  "BCT 2760": 3,
  "MAC 1105": 3,
  "MAC 1114": 3,
  "MAC 1140": 3,
  "MAC 2311": 5,
  "MAC 2312": 4,
  "MAC 2313": 4,
  "ENC 1101": 3,
  "ENC 1102": 3,
  "EGN 1008C": 3,
  "EGN 2312": 3,
  "SLS 1106": 1,
  "CHM 1045": 3,
  "CHM 1045L": 2,
  "PHY 1025": 3,
  "PHY 2048": 4,
  "PHY 2048L": 1,
  "PHY 2049": 4,
  "PHY 2049L": 1,
  "MAP 2302": 3,
  "COP 2270": 4,
  "FSS 1200": 3,
  "FSS 1202C": 3,
  "FSS 1204C": 3,
  "FSS 1246C": 3,
```

```
  "FSS 1801": 3,
  "FSS 2205C": 3,
  "FSS 2242C": 3,
  "FSS 2248C": 3,
  "FSS 2381": 4,
  "FSS 2940": 4,
  "HFT 1212": 3,
  "HFT 1220": 3,
  "HFT 1454": 3,
  "HUN 1201": 3,
  "MGF 1130": 3,
  "BSC 1005": 3,
  "PHI 2010": 3,
  "ACG 1403": 1,
  "CGS 1060C": 4,
  "CGS 1560": 4,
  "CGS 1700": 4,
  "CGS 2108": 4,
  "CIS 1531": 4,
  "CIS 2350": 4,
  "CTS 1111": 4,
  "CTS 1120": 4,
  "CTS 1131": 4,
  "CTS 1134": 4,
  "CTS 1328": 4,
  "CTS 2148C": 4,
  "CTS 2153": 4,
  "CTS 2314": 4,
  "CET 2880C": 4,
  "CTS 1134 or CTS 1650": 4,
  "MAC 1105 or STA 2023": 3,
};

const STAY_TOGETHER: Array<[string, string]> = [
  ["FSS 1200", "FSS 1202C"],
  ["CHM 1045", "CHM 1045L"],
  ["PHY 2048", "PHY 2048L"],
  ["PHY 2049", "PHY 2049L"],
];

const MUST_FINISH_FIRST: Record<string, string[][]> = {
  "ENC 1102": [["ENC 1101"]],
  "MAC 1140": [["MAC 1105"]],
  "MAC 1114": [["MAC 1105"]],
  "MAC 2311": [["MAC 1140"], ["MAC 1114"]],
  "MAC 2312": [["MAC 2311"]],
  "MAC 2313": [["MAC 2312"]],
  "MAP 2302": [["MAC 2312"]],
  "CHM 1045": [["MAC 1105"]],
  "PHY 1025": [["MAC 1105"]],
  "PHY 2048": [["MAC 2311"], ["PHY 1025"]],
  "PHY 2049": [["PHY 2048"], ["PHY 2048L"]],
  "ARC 2461": [["ARC 1126C"]],
  "ARC 2171": [["ARC 1126C", "ARC 2461"]],
  "ARC 1128": [["ARC 1126C"]],
  "ARC 2172": [["ARC 2171"]],
  "ARC 2681": [["ARC 1126C"]],
  "PHI 2010": [["ENC 1101"]],
  "FSS 1204C": [["FSS 1200"], ["FSS 1202C"]],
  "FSS 1246C": [["FSS 1200"], ["FSS 1202C"]],
  "FSS 2248C": [["FSS 1200"], ["FSS 1202C"], ["FSS 1246C"]],
  "FSS 1801": [["FSS 2248C"]],
  "FSS 2242C": [["FSS 1204C"], ["FSS 1246C"], ["FSS 2248C"]],
  "FSS 2205C": [["FSS 2242C"]],
  "FSS 2381": [["FSS 2205C"]],
  "CGS 1560": [["CGS 1060C"]],
  "CTS 1328": [["CGS 1060C"]],
  "CTS 1131": [["CGS 1560"]],
  "CTS 2153": [["CTS 1328"]],
  "CGS 2108": [["CGS 1060C"]],
  "CIS 2350": [["CTS 1120"], ["CTS 1134"]],
  "CTS 2314": [["CTS 1120"], ["CTS 1134"]],
};

export function blockingPrereqs(course: PlanCourse, doneKeys: string[], onPlan: Set<string>): string[] {
  const groups = MUST_FINISH_FIRST[course.code];
  if (!groups) return [];
  const doneCodes = new Set(doneKeys.map((key) => key.split("|")[0]));
  const missing: string[] = [];
  for (const group of groups) {
    const present = group.filter((code) => onPlan.has(code));
    if (present.length === 0) continue;
    if (!present.some((code) => doneCodes.has(code))) missing.push(present.join(" or "));
  }
  return missing;
}

export function courseNeeds(course: PlanCourse): string | null {
  const needs: Record<string, string> = {
    "ENC 1101": "May need a placement class before you can start.",
```

```
    "ENC 1102": "After ENC 1101.",
    "MAC 1105": "May need MAT 1033 or a placement class first.",
    "MAC 1105 or STA 2023": "College Algebra may need MAT 1033 first. Statistics may need MAT 1033 or MGF 11
31.",
    "MAC 1140": "After MAC 1105.",
    "MAC 1114": "After MAC 1105.",
    "MAC 2311": "After precalculus and trigonometry.",
    "MAC 2312": "After MAC 2311.",
    "MAC 2313": "After MAC 2312.",
    "MAP 2302": "After MAC 2312.",
    "MGF 1130": "May need a placement class first.",
    "CHM 1045": "After MAC 1105, and CHM 1025 or a chemistry placement. Take with the lab.",
    "CHM 1045L": "Take with CHM 1045.",
    "PHY 1025": "After MAC 1105.",
    "PHY 2048": "After MAC 2311 and a first physics class. Take with the lab.",
    "PHY 2048L": "Take with PHY 2048.",
    "PHY 2049": "After PHY 2048 and its lab. Take with PHY 2049L.",
    "PHY 2049L": "Take with PHY 2049.",
    "ARC 1301": "Take with or after ARC 1115.",
    "ARC 2461": "After ARC 1126C.",
    "ARC 2171": "After ARC 1126C or ARC 2461.",
    "ARC 1128": "After ARC 1126C.",
    "ARC 2172": "After ARC 2171.",
    "ARC 2681": "After ARC 1126C.",
    "PHI 2010": "After ENC 1101.",
    "FSS 1200": "Take with FSS 1202C.",
    "FSS 1202C": "Take with FSS 1200.",
    "FSS 1204C": "After FSS 1200 and FSS 1202C.",
    "FSS 1246C": "After FSS 1200 and FSS 1202C.",
    "FSS 2248C": "After FSS 1200, FSS 1202C, and FSS 1246C.",
    "FSS 1801": "After FSS 2248C.",
    "FSS 2242C": "After the earlier food-production, baking, and garde manger classes.",
    "FSS 2205C": "After FSS 2242C and the earlier production classes.",
    "FSS 2381": "After FSS 2205C.",
    "CGS 1560": "After CGS 1060C, or if you already have computer experience.",
    "CTS 1328": "After CGS 1060C, or if you already have computer experience.",
    "CTS 1131": "After CGS 1560.",
    "CTS 2153": "After CTS 1328.",
    "CGS 2108": "After CGS 1060C.",
    "CIS 2350": "After CTS 1120 and CTS 1134.",
    "CTS 2314": "After CTS 1120 and CTS 1134.",
    "CTS 1120": "A networking class first is recommended, not required.",
  };
  return needs[course.code] ?? null;
}

export function courseCredits(course: PlanCourse): number {
  if (course.name.startsWith("Advisor-chosen")) return 3;
  if (CREDITS[course.code] != null) return CREDITS[course.code];
  if (course.code === "Electives") return 9;
  if (course.code === "Elective") return 4;
  if (course.code.startsWith("CCT") || course.code.startsWith("CSP")) return 0;
  return 3;
}

function expand(course: PlanCourse): PlanCourse[] {
  if (course.code !== "Electives") return [course];
  return [1, 2, 3].map((part) => ({
    code: "Elective",
    name: `Advisor-chosen elective, ${part} of 3`,
  }));
}

export function termsForPace(plan: Plan, pace: StudyPace, transferring: boolean): PlanTerm[] {
  const source = transferring
    ? plan.terms.filter((term) => term.label !== "Before you transfer")
    : plan.terms;
  if (pace === "full") return source;

  const cap = pace === "three-quarter" ? 9 : 6;
  const flat = source.flatMap((term) => term.courses).flatMap(expand);
  const bins: PlanCourse[][] = [];
  let current: PlanCourse[] = [];
  let credits = 0;

  for (let index = 0; index < flat.length; index += 1) {
    const course = flat[index];
    const next = flat[index + 1];
    const pair = next != null && STAY_TOGETHER.some(([first, second]) => course.code === first && next.code
=== second);
    const block = pair ? [course, next] : [course];
    const blockCredits = block.reduce((sum, item) => sum + courseCredits(item), 0);
    if (current.length > 0 && credits + blockCredits > cap) {
      bins.push(current);
      current = [];
      credits = 0;
    }
    current.push(...block);
    credits += blockCredits;
    if (pair) index += 1;
```

```
  }
  if (current.length > 0) bins.push(current);

  return bins.map((courses, index) => ({
    label: `Term ${index + 1}`,
    credits: `${courses.reduce((sum, course) => sum + courseCredits(course), 0)} credits`,
    courses,
  }));
}

export function graduationLine(termCount: number): string {
  const months = termCount * 4;
  const years = Math.floor(months / 12);
  const extra = months % 12;
  const span =
    years === 0
      ? `about ${extra} months`
      : extra === 0
        ? `about ${years} ${years === 1 ? "year" : "years"}`
        : `about ${years} ${years === 1 ? "year" : "years"} and ${extra} months`;
  return `${termCount} terms, ${span}, if you start in the fall and keep going through fall, spring, and sum
mer.`;
}

export function afterDegree(careerId: CareerId): string | null {
  if (careerId === "architect") {
    return "That finishes the MDC associate degree. A professional architecture degree comes after it.";
  }
  if (careerId === "mechanical") {
    return "That finishes the MDC associate degree. The engineering bachelor’s comes after you transfer.";
  }
  if (careerId === "cyber") {
    return "That finishes the associate degree. The bachelor’s in cybersecurity at MDC comes after it.";
  }
  if (careerId === "it") {
    return "That finishes the associate degree. A bachelor’s in information systems comes after it if you co
ntinue.";
  }
  return null;
}
```

src/lib/preview-embedder-origin.ts

```
export function isGrokEmbedderOrigin(origin: string): boolean {
  try {
    const url = new URL(origin);
    if (url.protocol !== "https:" && url.protocol !== "http:") return false;
    const host = url.hostname.toLowerCase();
    if (host === "grok.com" || host.endsWith(".grok.com")) return true;
    if (host === "localhost" || host === "127.0.0.1" || host === "[::1]") return true;
    return false;
  } catch {
    return false;
  }
}

export function isSandboxPreviewGuestHost(hostname: string): boolean {
  const host = hostname.toLowerCase();
  return host === "grok-sandbox.com" || host.endsWith(".grok-sandbox.com");
}

function isRemintPreviewPair(guestHost: string, parentHost: string): boolean {
  const guest = guestHost.toLowerCase();
  const parent = parentHost.toLowerCase();
  const sep = ".preview.";
  const i = guest.indexOf(sep);
  if (i <= 0) return false;
  const label = guest.slice(0, i);
  const rest = guest.slice(i + sep.length);
  if (label.includes(".") || !rest.includes(".")) return false;
  return parent === rest || parent === `grok.${rest}`;
}

export function resolveParentEmbedderOrigin(
  parentIsSelf: boolean,
  referrer: string,
  ancestorOrigin?: string | null,
  guestHostname: string = "",
): string | null {
  if (parentIsSelf) return null;
  for (const candidate of [referrer, ancestorOrigin ?? ""].filter(Boolean)) {
    try {
      const url = new URL(
        candidate.includes("://") ? candidate : `https://${candidate}`,
      );
      if (url.protocol !== "https:" && url.protocol !== "http:") continue;
      if (isGrokEmbedderOrigin(url.origin)) return url.origin;
      if (
        isSandboxPreviewGuestHost(guestHostname) ||
        isRemintPreviewPair(guestHostname, url.hostname)
      ) {
        return url.origin;
      }
    } catch {
      // try next candidate
    }
  }
  return null;
}
```

src/lib/preview-host-bridge.ts

```
/**
 * Guest side of the grok-web ↔ sandbox preview postMessage bridge.
 *
 * Activates only when this page is framed by an allowlisted Grok embedder.
 * Top-level runs (download/export, local `npm run dev`, deployed sites) noop.
 */

import { z } from "zod";
import { CONNECTOR_TOKEN_READY_EVENT } from "./app-data/types";
import { resolveParentEmbedderOrigin } from "./preview-embedder-origin";

export {
  isGrokEmbedderOrigin,
  isSandboxPreviewGuestHost,
  resolveParentEmbedderOrigin,
} from "./preview-embedder-origin";

export const PREVIEW_BRIDGE_CHANNEL = "grok-preview-bridge" as const;
export const PREVIEW_BRIDGE_VERSION = 1 as const;

const EnvelopeSchema = z.object({
  channel: z.literal(PREVIEW_BRIDGE_CHANNEL),
  version: z.number().int().positive(),
  type: z.string().min(1),
});

const HelloSchema = EnvelopeSchema.extend({
  type: z.literal("hello"),
});

const NavigateSchema = EnvelopeSchema.extend({
  type: z.literal("navigate"),
  path: z.string().min(1),
});

const HistorySchema = EnvelopeSchema.extend({
  type: z.literal("history"),
  delta: z.union([z.literal(-1), z.literal(1)]),
});

const ConnectorTokenReadySchema = EnvelopeSchema.extend({
  type: z.literal("connector-token-ready"),
});

export type PreviewHostBridgeOptions = {
  /** Prefer the app router when available; falls back to history.pushState. */
  navigate?: (path: string) => void;
  /** Best-effort registered paths for host autosuggest (may be empty). */
  getRoutePaths?: () => string[];
};

export function isSafeBridgePath(path: string): boolean {
  if (!path.startsWith("/") || path.startsWith("//") || path.includes("\\")) {
    return false;
  }
  try {
    const resolved = new URL(path, "https://preview.invalid");
    return resolved.origin === "https://preview.invalid";
  } catch {
    return false;
  }
}

/**
 * Origin of the Grok embedder framing this page, or null when the page runs
 * top-level (download/export, local `npm run dev`, deployed sites) or under a
 * non-Grok parent. Client-only; null during SSR.
 */
export function resolveCurrentEmbedderOrigin(): string | null {
  if (typeof window === "undefined") return null;
  const ancestorOrigin =
    typeof location.ancestorOrigins !== 'undefined' && location.ancestorOrigins.length > 0
      ? location.ancestorOrigins[0]
      : null;
  return resolveParentEmbedderOrigin(
    window.parent === window,
    document.referrer,
    ancestorOrigin,
    window.location.hostname,
  );
}

/**
 * Install host↔guest messaging. Returns a dispose function.
 * Noops (returns a no-op dispose) when not embedded under a Grok parent.
 */
export function installPreviewHostBridge(
  options: PreviewHostBridgeOptions = {},
```

```
): () => void {
  const parentOrigin = resolveCurrentEmbedderOrigin();
  if (parentOrigin === null) return () => {};

  const ROOT_STATE_KEY = "__grokPreviewBridgeRoot";
  const originalPushState = window.history.pushState.bind(window.history);
  const originalReplaceState = window.history.replaceState.bind(window.history);

  const isAtHistoryRoot = () => {
    const state = window.history.state;
    return Boolean(
      state && typeof state === "object" && state[ROOT_STATE_KEY] === true,
    );
  };

  // Floor for chrome Back: only the first install in a fresh history stack is
  // root. Full document navigations reinstall this bridge on a deep URL; if we
  // re-stamped that entry as root, Back would no-op while earlier in-preview
  // history still exists. Preserve an existing tag (bfcache / SPA remount).
  try {
    const current = window.history.state;
    const alreadyTagged =
      current !== null &&
      typeof current === "object" &&
      Object.prototype.hasOwnProperty.call(current, ROOT_STATE_KEY);
    if (!alreadyTagged) {
      const isRoot = window.history.length <= 1;
      const marked =
        current && typeof current === "object"
          ? { ...current, [ROOT_STATE_KEY]: isRoot }
          : { [ROOT_STATE_KEY]: isRoot };
      originalReplaceState(marked, "", window.location.href);
    }
  } catch {
    // ignore if the document cannot be marked
  }

  const post = (message: object) => {
    window.parent.postMessage(message, parentOrigin);
  };

  const reportLocation = () => {
    post({
      channel: PREVIEW_BRIDGE_CHANNEL,
      version: PREVIEW_BRIDGE_VERSION,
      type: "location",
      path: window.location.pathname || "/",
      search: window.location.search,
      hash: window.location.hash,
    });
  };

  const reportRoutes = () => {
    const paths = options.getRoutePaths?.() ?? [];
    post({
      channel: PREVIEW_BRIDGE_CHANNEL,
      version: PREVIEW_BRIDGE_VERSION,
      type: "routes",
      paths,
    });
  };

  const defaultNavigate = (path: string) => {
    if (!isSafeBridgePath(path)) return;
    try {
      const url = new URL(path, window.location.origin);
      if (url.origin !== window.location.origin) return;
      const next = `${url.pathname}${url.search}${url.hash}`;
      window.history.pushState(window.history.state, "", next);
      window.dispatchEvent(new PopStateEvent("popstate", { state: window.history.state }));
    } catch {
      // ignore malformed paths
    }
  };

  const navigate = (path: string) => {
    if (!isSafeBridgePath(path)) return;
    if (options.navigate) {
      options.navigate(path);
      return;
    }
    defaultNavigate(path);
  };

  const announce = () => {
    reportLocation();
    reportRoutes();
    post({
      channel: PREVIEW_BRIDGE_CHANNEL,
      version: PREVIEW_BRIDGE_VERSION,
```

```
    type: "ready",
  });
};

// Host re-handshake: it may have (re)mounted after our install-time
// announce, or asked before we hydrated. Announce again.
const onHello = (data: unknown) => {
  if (!HelloSchema.safeParse(data).success) return;
  announce();
};

const onNavigate = (data: unknown) => {
  const parsed = NavigateSchema.safeParse(data);
  if (!parsed.success) return;
  navigate(parsed.data.path);
  // Router navigations often update location asynchronously; report after a tick.
  queueMicrotask(reportLocation);
};

const onHistory = (data: unknown) => {
  const parsed = HistorySchema.safeParse(data);
  if (!parsed.success) return;
  // Do not history.go(-1) off the first entry — that leaves the preview.
  if (parsed.data.delta === -1 && isAtHistoryRoot()) return;
  // Location sync comes from the popstate listener once history settles.
  window.history.go(parsed.data.delta);
};

const onConnectorTokenReady = (data: unknown) => {
  if (!ConnectorTokenReadySchema.safeParse(data).success) return;
  window.dispatchEvent(new Event(CONNECTOR_TOKEN_READY_EVENT));
};

const hostMessageHandlers = new Map<string, (data: unknown) => void>([
  ["hello", onHello],
  ["navigate", onNavigate],
  ["history", onHistory],
  ["connector-token-ready", onConnectorTokenReady],
]);

const onMessage = (event: MessageEvent) => {
  if (event.source !== window.parent) return;
  if (event.origin !== parentOrigin) return;

  const envelope = EnvelopeSchema.safeParse(event.data);
  if (!envelope.success || envelope.data.version !== PREVIEW_BRIDGE_VERSION) return;
  hostMessageHandlers.get(envelope.data.type)?.(event.data);
};

const onPopState = () => {
  reportLocation();
};

// Same-document `#` navigations fire hashchange, not popstate / pushState.
const onHashChange = () => {
  reportLocation();
};

// Patch history so in-app SPA navigations sync the host address bar.
window.history.pushState = (data, unused, url) => {
  const next =
    data && typeof data === "object"
      ? { ...data, [ROOT_STATE_KEY]: false }
      : data;
  originalPushState(next, unused, url);
  reportLocation();
};
window.history.replaceState = (data, unused, url) => {
  const next =
    isAtHistoryRoot()
      ? {
          ...(data && typeof data === "object" ? data : {}),
          [ROOT_STATE_KEY]: true,
        }
      : data;
  originalReplaceState(next, unused, url);
  reportLocation();
};

window.addEventListener("message", onMessage);
window.addEventListener("popstate", onPopState);
window.addEventListener("hashchange", onHashChange);

announce();

return () => {
  window.removeEventListener("message", onMessage);
  window.removeEventListener("popstate", onPopState);
  window.removeEventListener("hashchange", onHashChange);
  window.history.pushState = originalPushState;
```

```
    window.history.replaceState = originalReplaceState;
  };
}

/** Collect static path patterns from a TanStack route tree (best-effort). */
export function collectRoutePathsFromTree(routeTree: unknown): string[] {
  const paths = new Set<string>();

  const walk = (node: unknown) => {
    if (!node || typeof node !== "object") return;
    const record = node as {
      fullPath?: unknown;
      path?: unknown;
      children?: unknown;
    };
    const full =
      typeof record.fullPath === "string"
        ? record.fullPath
        : typeof record.path === "string"
          ? record.path
          : null;
    if (full !== null && full !== "") {
      paths.add(full.startsWith("/") ? full : `/${full}`);
    } else if (full === "") {
      paths.add("/");
    }
    const children = record.children;
    if (Array.isArray(children)) {
      for (const child of children) walk(child);
    } else if (children && typeof children === "object") {
      for (const child of Object.values(children as Record<string, unknown>)) {
        walk(child);
      }
    }
  };

  walk(routeTree);
  return [...paths];
}
```

src/lib/preview-path-prefix.test.ts

```
import { describe, it } from "node:test";
import assert from "node:assert/strict";
import { prefixRewrite, previewPathPrefix } from "./preview-path-prefix.ts";

const PORT = "/hds-up68yqz8egcm-6014-7b575";

describe("previewPathPrefix", () => {
  it("reads the port id segment on path-based preview hosts", () => {
    assert.equal(previewPathPrefix("preview.grokgsac.com", `${PORT}/`), PORT);
    assert.equal(previewPathPrefix("Preview.Grok.GenAI.mil", `${PORT}/about`), PORT);
    assert.equal(previewPathPrefix("preview.grok.mil", PORT), PORT);
  });

  it("ignores other hosts and paths without a port id", () => {
    assert.equal(previewPathPrefix("abc.grok-sandbox.com", `${PORT}/`), undefined);
    assert.equal(previewPathPrefix("localhost", `${PORT}/`), undefined);
    assert.equal(previewPathPrefix("preview.grokgsac.com", "/about"), undefined);
    assert.equal(previewPathPrefix("preview.grokgsac.com", "/hds-abc-web-x1/"), undefined);
    assert.equal(previewPathPrefix("preview.grokgsac.com", `${PORT}.js`), undefined);
  });
});

describe("prefixRewrite", () => {
  const rewrite = prefixRewrite(PORT);
  const run = (fn: typeof rewrite.input, href: string) =>
    String(fn?.({ url: new URL(href, "https://preview.grokgsac.com") }));

  it("strips the prefix on input", () => {
    assert.equal(run(rewrite.input, PORT), "https://preview.grokgsac.com/");
    assert.equal(run(rewrite.input, `${PORT}/a?b=1#c`), "https://preview.grokgsac.com/a?b=1#c");
    assert.equal(run(rewrite.input, "/other"), "https://preview.grokgsac.com/other");
  });

  it("adds the prefix on output", () => {
    assert.equal(run(rewrite.output, "/a?b=1"), `https://preview.grokgsac.com${PORT}/a?b=1`);
    assert.equal(run(rewrite.output, "/"), `https://preview.grokgsac.com${PORT}/`);
  });
});
```

src/lib/preview-path-prefix.ts

```
import type { LocationRewrite } from "@tanstack/react-router";

/**
 * Path-based preview hosts serve the dev server under `/{port_id}/` and strip
 * that prefix before forwarding, so SSR sees `/` while the browser URL keeps the
 * prefix. TanStack Start resets `basepath` on hydration, so a `rewrite` keeps
 * client routing and generated links aligned with the public URL.
 */
const PATH_BASED_PREVIEW_HOSTS = new Set([
  "preview.grok.genai.mil",
  "preview.grok.mil",
  "preview.grokgsac.com",
]);

const PORT_ID_PREFIX = /^\/hds-[a-z0-9]+-\d+-[a-z0-9]+(?=\/|$)/i;

export function previewPathPrefix(hostname: string, pathname: string): string | undefined {
  if (!PATH_BASED_PREVIEW_HOSTS.has(hostname.toLowerCase())) return undefined;
  return PORT_ID_PREFIX.exec(pathname)?.[0];
}

export function prefixRewrite(prefix: string): LocationRewrite {
  return {
    input: ({ url }) => {
      if (url.pathname === prefix) url.pathname = "/";
      else if (url.pathname.startsWith(`${prefix}/`))
        url.pathname = url.pathname.slice(prefix.length);
      return url;
    },
    output: ({ url }) => {
      url.pathname = `${prefix}${url.pathname}`;
      return url;
    },
  };
}

export function previewPathRewrite(): LocationRewrite | undefined {
  if (typeof window === "undefined") return undefined;
  const prefix = previewPathPrefix(window.location.hostname, window.location.pathname);
  return prefix ? prefixRewrite(prefix) : undefined;
}
```

src/lib/progress.ts

```
export type PathSnapshot = {
  careerId: string | null;
  usedAccommodations: boolean;
  style: string;
  showExplain: boolean;
  pace: string;
  campus: string;
  stepIndex: number;
  doneByCareer: Record<string, string[]>;
};

const CAREER_IDS = new Set(["architect", "mechanical", "chef", "cyber", "it"]);
const STYLES = new Set(["see", "try", "read", "short"]);
const PACES = new Set(["full", "three-quarter", "part"]);
const KEY = "pathbook-snapshot";

export function cleanSnapshot(input: unknown): PathSnapshot {
  const raw = input && typeof input === "object" ? (input as Record<string, unknown>) : {};
  const careerId = typeof raw.careerId === "string" && CAREER_IDS.has(raw.careerId) ? raw.careerId : null;
  const doneByCareer: Record<string, string[]> = {};
  if (raw.doneByCareer && typeof raw.doneByCareer === "object") {
    for (const [key, value] of Object.entries(raw.doneByCareer as Record<string, unknown>)) {
      if (!CAREER_IDS.has(key) || !Array.isArray(value)) continue;
      doneByCareer[key] = value.filter((item): item is string => typeof item === "string").slice(0, 200);
    }
  }
  const stepIndex = typeof raw.stepIndex === "number" && raw.stepIndex >= 0 ? Math.floor(raw.stepIndex) : 0;
  return {
    careerId,
    usedAccommodations: raw.usedAccommodations === true,
    style: typeof raw.style === "string" && STYLES.has(raw.style) ? raw.style : "read",
    showExplain: raw.showExplain !== false,
    pace: typeof raw.pace === "string" && PACES.has(raw.pace) ? raw.pace : "full",
    campus: typeof raw.campus === "string" ? raw.campus.slice(0, 40) : "Kendall",
    stepIndex: Math.min(stepIndex, 40),
    doneByCareer,
  };
}

export function loadLocalProgress(): PathSnapshot | null {
  if (typeof window === "undefined") return null;
  try {
    const raw = localStorage.getItem(KEY);
    if (!raw) return null;
    return cleanSnapshot(JSON.parse(raw));
  } catch {
    return null;
  }
}

export function saveLocalProgress(input: unknown) {
  const snap = cleanSnapshot(input);
  localStorage.setItem(KEY, JSON.stringify(snap));
}
```

src/lib/school-email.ts

```
const SCHOOL_DOMAINS = ["mymdc.net"];

export function isSchoolEmail(email: string | undefined) {
  const domain = email?.split("@")[1]?.toLowerCase();
  return Boolean(domain && SCHOOL_DOMAINS.includes(domain));
}
```

src/routeTree.gen.ts

```
/* eslint-disable */

// @ts-nocheck

// noinspection JSUnusedGlobalSymbols

// This file was automatically generated by TanStack Router.
// You should NOT make any changes in this file as it will be overwritten.
// Additionally, you should also exclude this file from your linter and/or formatter to prevent it from bein
g checked or modified.

import { Route as rootRouteImport } from './routes/__root'
import { Route as IndexRouteImport } from './routes/index'
import { Route as AppRouteImport } from './routes/app'
import { Route as LoginRouteImport } from './routes/login'
import { Route as ApiAuthRouteImport } from './routes/api.auth'
import { Route as ApiSpeakRouteImport } from './routes/api/speak'
import { Route as AuthSplatRouteImport } from './routes/auth/$'

const IndexRoute = IndexRouteImport.update({
  id: '/',
  path: '/',
  getParentRoute: () => rootRouteImport,
} as any)
const AppRoute = AppRouteImport.update({
  id: '/app',
  path: '/app',
  getParentRoute: () => rootRouteImport,
} as any)
const LoginRoute = LoginRouteImport.update({
  id: '/login',
  path: '/login',
  getParentRoute: () => rootRouteImport,
} as any)
const ApiAuthRoute = ApiAuthRouteImport.update({
  id: '/api/auth',
  path: '/api/auth',
  getParentRoute: () => rootRouteImport,
} as any)
const ApiSpeakRoute = ApiSpeakRouteImport.update({
  id: '/api/speak',
  path: '/api/speak',
  getParentRoute: () => rootRouteImport,
} as any)
const AuthSplatRoute = AuthSplatRouteImport.update({
  id: '/auth/$',
  path: '/auth/$',
  getParentRoute: () => rootRouteImport,
} as any)

export interface FileRoutesByFullPath {
  '/': typeof IndexRoute
  '/app': typeof AppRoute
  '/login': typeof LoginRoute
  '/api/auth': typeof ApiAuthRoute
  '/api/speak': typeof ApiSpeakRoute
  '/auth/$': typeof AuthSplatRoute
}
export interface FileRoutesByTo {
  '/': typeof IndexRoute
  '/app': typeof AppRoute
  '/login': typeof LoginRoute
  '/api/auth': typeof ApiAuthRoute
  '/api/speak': typeof ApiSpeakRoute
  '/auth/$': typeof AuthSplatRoute
}
export interface FileRoutesById {
  __root__: typeof rootRouteImport
  '/': typeof IndexRoute
  '/app': typeof AppRoute
  '/login': typeof LoginRoute
  '/api/auth': typeof ApiAuthRoute
  '/api/speak': typeof ApiSpeakRoute
  '/auth/$': typeof AuthSplatRoute
}
export interface FileRouteTypes {
  fileRoutesByFullPath: FileRoutesByFullPath
  fullPaths: '/' | '/app' | '/login' | '/api/auth' | '/api/speak' | '/auth/$'
  fileRoutesByTo: FileRoutesByTo
  to: '/' | '/app' | '/login' | '/api/auth' | '/api/speak' | '/auth/$'
  id:
    | '__root__'
    | '/'
    | '/app'
    | '/login'
    | '/api/auth'
    | '/api/speak'
    | '/auth/$'
```

```
  fileRoutesById: FileRoutesById
}
export interface RootRouteChildren {
  IndexRoute: typeof IndexRoute
  AppRoute: typeof AppRoute
  LoginRoute: typeof LoginRoute
  ApiAuthRoute: typeof ApiAuthRoute
  ApiSpeakRoute: typeof ApiSpeakRoute
  AuthSplatRoute: typeof AuthSplatRoute
}

declare module '@tanstack/react-router' {
  interface FileRoutesByPath {
    '/': {
      id: '/'
      path: '/'
      fullPath: '/'
      preLoaderRoute: typeof IndexRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/app': {
      id: '/app'
      path: '/app'
      fullPath: '/app'
      preLoaderRoute: typeof AppRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/login': {
      id: '/login'
      path: '/login'
      fullPath: '/login'
      preLoaderRoute: typeof LoginRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/api/auth': {
      id: '/api/auth'
      path: '/api/auth'
      fullPath: '/api/auth'
      preLoaderRoute: typeof ApiAuthRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/api/speak': {
      id: '/api/speak'
      path: '/api/speak'
      fullPath: '/api/speak'
      preLoaderRoute: typeof ApiSpeakRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/auth/$': {
      id: '/auth/$'
      path: '/auth/$'
      fullPath: '/auth/$'
      preLoaderRoute: typeof AuthSplatRouteImport
      parentRoute: typeof rootRouteImport
    }
  }
}

const rootRouteChildren: RootRouteChildren = {
  IndexRoute: IndexRoute,
  AppRoute: AppRoute,
  LoginRoute: LoginRoute,
  ApiAuthRoute: ApiAuthRoute,
  ApiSpeakRoute: ApiSpeakRoute,
  AuthSplatRoute: AuthSplatRoute,
}
export const routeTree = rootRouteImport
  ._addFileChildren(rootRouteChildren)
  ._addFileTypes<FileRouteTypes>()

import type { getRouter } from './router.tsx'
import type { startInstance } from './start.ts'
declare module '@tanstack/react-start' {
  interface Register {
    ssr: true
    router: Awaited<ReturnType<typeof getRouter>>
    config: Awaited<ReturnType<typeof startInstance.getOptions>>
  }
}
```

src/router.tsx

```
import { auth0RouterContext } from "@auth0/auth0-tanstack-start-react/client";
import type { Auth0RouterContext } from "@auth0/auth0-tanstack-start-react/types";
import { createRouter } from "@tanstack/react-router";
import { AppErrorComponent } from "@/lib/error-component";
import { previewPathRewrite } from "@/lib/preview-path-prefix";
import { routeTree } from "./routeTree.gen";

export function getRouter() {
  return createRouter({
    routeTree,
    rewrite: previewPathRewrite(),
    defaultErrorComponent: AppErrorComponent,
    context: { auth0: auth0RouterContext } as { auth0: Auth0RouterContext },
  });
}
```

src/routes/__root.tsx

```
import { Auth0Provider, auth0BeforeLoad } from "@auth0/auth0-tanstack-start-react/client";
import type { Auth0RouterContext } from "@auth0/auth0-tanstack-start-react/types";
import { createRootRouteWithContext, HeadContent, Outlet, Scripts } from "@tanstack/react-router";
import { PreviewHostBridge } from "@/components/preview-host-bridge";
import { AuthProvider } from "@/lib/auth/provider";
import appCss from "../styles.css?url";

const APP_NAME = "Major Spark";

export const Route = createRootRouteWithContext<{ auth0: Auth0RouterContext }>()({
  beforeLoad: auth0BeforeLoad(),
  head: () => ({
    meta: [
      { charSet: "utf-8" },
      { name: "viewport", content: "width=device-width, initial-scale=1" },
      { title: APP_NAME },
      {
        name: "description",
        content:
          "A clear, step-by-step career path that starts at Miami Dade College.",
      },
      { name: "theme-color", content: "#2a2f93" },
    ],
    links: [
      { rel: "icon", type: "image/svg+xml", href: "/favicon.svg" },
      { rel: "stylesheet", href: appCss },
      { rel: "manifest", href: "/__grok/manifest.webmanifest" },
      { rel: "apple-touch-icon", href: "/__grok/icon-180.png" },
      { rel: "preconnect", href: "https://fonts.googleapis.com" },
      {
        rel: "preconnect",
        href: "https://fonts.gstatic.com",
        crossOrigin: "anonymous",
      },
      {
        rel: "stylesheet",
        href: "https://fonts.googleapis.com/css2?family=Source+Serif+4:ital,opsz,wght@0,8..60,500;0,8..60,60
0;1,8..60,600&family=Open+Sans:ital,wght@0,400;0,600;0,700;1,400;1,600&family=Source+Sans+3:ital,wght@0,400;
0,560;0,650;1,400;1,600&display=swap",
      },
    ],
  }),
  component: () => (
    <html lang="en" suppressHydrationWarning>
      <head>
        <HeadContent />
      </head>
      <body>
        <PreviewHostBridge />
        <AuthProvider>
          <Auth0Provider>
            <Outlet />
          </Auth0Provider>
        </AuthProvider>
        <Scripts />
      </body>
    </html>
  ),
});
```

src/routes/api.auth.ts

```
import { createFileRoute } from "@tanstack/react-router";

export function authFallback() {
  return { safeBuild: true, user: null };
}

export const Route = createFileRoute("/api/auth")({
  loader: async () => authFallback(),
  server: {
    handlers: {
      GET: async () => Response.json(authFallback()),
      POST: async () => Response.json(authFallback()),
    },
  },
});
```

src/routes/api/speak.ts

```
import { createFileRoute } from "@tanstack/react-router";
import { env } from "@/lib/env.server";

const VOICE = "EXAVITQu4vr4xnSDxMaL";

export const Route = createFileRoute("/api/speak")({
  server: {
    handlers: {
      POST: async ({ request }) => {
        const key = env("ELEVENLABS_API_KEY");
        if (!key) return Response.json({ error: "missing-key" }, { status: 503 });
        const body = await request.json().catch(() => null);
        const text = typeof body?.text === "string" ? body.text.replace(/\s+/g, " ").trim().slice(0, 800) :
"";
        if (!text) return Response.json({ error: "empty" }, { status: 400 });
        const upstream = await fetch(`https://api.elevenlabs.io/v1/text-to-speech/${VOICE}`, {
          method: "POST",
          headers: {
            "xi-api-key": key,
            "Content-Type": "application/json",
            Accept: "audio/mpeg",
          },
          body: JSON.stringify({
            text,
            model_id: "eleven_multilingual_v2",
            voice_settings: {
              stability: 0.4,
              similarity_boost: 0.8,
              style: 0,
              use_speaker_boost: true,
              speed: 0.94,
            },
          }),
        });
        if (!upstream.ok || !upstream.body) return Response.json({ error: "upstream" }, { status: 502 });
        return new Response(upstream.body, { headers: { "Content-Type": "audio/mpeg" } });
      },
    },
  },
});
```

src/routes/app.tsx

```
import { requireAuth } from "@auth0/auth0-tanstack-start-react/client";
import type { Auth0RouterContext } from "@auth0/auth0-tanstack-start-react/types";
import { createFileRoute, redirect } from "@tanstack/react-router";
import { isSchoolEmail } from "@/lib/school-email";
import { Check, Settings } from "lucide-react";
import { AuthButtons } from "@/components/auth-buttons";
import { loadLocalProgress, saveLocalProgress, type PathSnapshot } from "@/lib/progress";
import { pageFrame, useDevice } from "@/lib/device";
import { PLANS, afterDegree, blockingPrereqs, courseNeeds, graduationLine, termsForPace, type StudyPace } fr
om "@/lib/plans";
import { useEffect, useMemo, useState } from "react";
import {
  CAREERS,
  careerById,
  ACCESS_STEP,
  suggestCareers,
  type CareerId,
  type Interest,
  type SchoolPlan,
  type WorkTiming,
} from "@/lib/paths";

export const Route = createFileRoute("/app")({
  beforeLoad: ({ context }: { context: { auth0: Auth0RouterContext } }) => {
    requireAuth({ loginPath: "/", returnTo: "/app" })({ context });
    if (!isSchoolEmail(context.auth0.user?.email)) {
      throw redirect({ to: "/", search: { denied: "school" } });
    }
  },
  component: Home,
});

type Screen = "cover" | "careers" | "fit" | "matches" | "question" | "learn" | "path" | "list" | "plan";
type TextSize = "small" | "regular" | "large";
type Style = "see" | "try" | "read" | "short";

const STYLE_ORDER: Style[] = ["see", "try", "read", "short"];
const STYLE_LABEL: Record<Style, string> = {
  see: "See it",
  try: "Try it",
  read: "Read it",
  short: "Short",
};

const ADVISORS = [
  ["Kendall", "Kadvise@mdc.edu"],
  ["Hialeah", "Hiadvisement@mdc.edu"],
  ["Homestead", "Hadvisement@mdc.edu"],
  ["Medical", "Madvisement@mdc.edu"],
  ["Meek", "meekstudentservices@mdc.edu"],
  ["North", "Anorth@mdc.edu"],
  ["Padrón", "Padvisement@mdc.edu"],
  ["West", "Westadvisement@mdc.edu"],
  ["Wolfson", "wadvisement@mdc.edu"],
] as const;

const CAMPUS_FIT: Record<
  CareerId,
  { campus: (typeof ADVISORS)[number][0]; why: string; person: string }
> = {
  architect: {
    campus: "Kendall",
    why: "Kendall has the School of Architecture and Interior Design. The drawing and design labs are based
there, so that is the better chance of a seat.",
    person: "MDC Works at Kendall lists Bernard Krogh, room R243, 305-237-2128.",
  },
  mechanical: {
    campus: "Kendall",
    why: "Kendall houses the School of Engineering, Technology and Design. The later engineering classes are
 more likely to be open there. Early English and math can be taken closer to home.",
    person: "MDC Works at Kendall lists Bernard Krogh, room R243, 305-237-2128.",
  },
  chef: {
    campus: "Wolfson",
    why: "The culinary labs are at the Miami Culinary Institute on the Wolfson Campus, downtown. That is the
 campus that runs this degree.",
    person: "MDC Works at Wolfson lists Moses Washington, room 4102-06, 305-237-3876.",
  },
  cyber: {
    campus: "North",
    why: "North Campus publishes this associate degree as a program you can start there. Kendall is where th
e later bachelor’s courses were added. For these classes, North is the clearer seat.",
    person: "MDC Works at North lists Maria Sincere, room 1101-04, 305-237-1056.",
  },
  it: {
    campus: "Kendall",
    why: "Kendall houses the School of Engineering, Technology and Design, so the A+ and networking labs are
 more likely to be open there. Padrón also lists this kind of program. Compare both schedules.",
```

```
    person: "MDC Works at Kendall lists Bernard Krogh, room R243, 305-237-2128.",
  },
};

const CAREER_HELP: Record<(typeof ADVISORS)[number][0], string> = {
  Kendall: "MDC Works at Kendall lists Bernard Krogh, room R243, 305-237-2128.",
  Hialeah: "MDC Works at Hialeah lists Kadijah Belle, room 2101, 305-237-8881.",
  Homestead: "MDC Works at Homestead lists Bernard Krogh, room A115, 305-237-2128.",
  Medical: "MDC Works at the Medical Campus lists Kadijah Belle, room 1101-A, 305-237-3959.",
  Meek: "Meek does not list a separate MDC Works person. This email goes to Meek student services.",
  North: "MDC Works at North lists Maria Sincere, room 1101-04, 305-237-1056.",
  Padrón: "MDC Works at Padrón lists Maria Sincere, room 1107-06, 305-237-6508.",
  West: "MDC Works at West lists Yvonne Malavé, room 1332, 305-237-8925.",
  Wolfson: "MDC Works at Wolfson is room 4102, 305-237-9675.",
};

const ACCESS_OFFICES = [
  ["Kendall", "kaccess@mdc.edu"],
  ["Hialeah", "hialeahaccess@mdc.edu"],
  ["Homestead", "haccess@mdc.edu"],
  ["Medical", "maccessservices@mdc.edu"],
  ["Meek", ""],
  ["North", "naccess@mdc.edu"],
  ["Padrón", "paccess@mdc.edu"],
  ["West", "WestAccess@mdc.edu"],
  ["Wolfson", "waccess@mdc.edu"],
] as const;

function Home() {
  const [screen, setScreen] = useState<Screen>("cover");
  const [careerId, setCareerId] = useState<CareerId | null>(null);
  const [usedAccommodations, setUsedAccommodations] = useState(false);
  const [style, setStyle] = useState<Style>("read");
  const [showExplain, setShowExplain] = useState(true);
  const [learnStep, setLearnStep] = useState(0);
  const [prefer, setPrefer] = useState<Exclude<Style, "short">>("read");
  const [stepIndex, setStepIndex] = useState(0);
  const [dyslexic, setDyslexic] = useState(false);
  const [textSize, setTextSize] = useState<TextSize>("regular");
  const [speak, setSpeak] = useState(false);
  const [settingsOpen, setSettingsOpen] = useState(false);
  const [fitStep, setFitStep] = useState(0);
  const [fitFrom, setFitFrom] = useState<"cover" | "careers">("cover");
  const [interest, setInterest] = useState<Interest | null>(null);
  const [school, setSchool] = useState<SchoolPlan | null>(null);
  const [work, setWork] = useState<WorkTiming | null>(null);
  const [doneCourses, setDoneCourses] = useState<string[]>([]);
  const [campus, setCampus] = useState<(typeof ADVISORS)[number][0]>("Kendall");
  const [ready, setReady] = useState(false);
  const device = useDevice();
  const frame = pageFrame(device);

  const career = careerId ? careerById(careerId) : null;
  const steps = career ? (usedAccommodations ? [ACCESS_STEP, ...career.steps] : career.steps) : [];
  const onCourses = Boolean(career && stepIndex === steps.length);
  const step = onCourses ? undefined : steps[stepIndex];
  const total = career ? steps.length + 1 : 0;

  const headline = useMemo(() => {
    if (!step) return "";
    if (style === "short") return step.short;
    return step.action;
  }, [style, step]);

  const matches = interest && school && work ? suggestCareers(interest, school, work) : [];

  useEffect(() => {
    const on = localStorage.getItem("pathbook-reading") === "dyslexic";
    const savedSize = localStorage.getItem("pathbook-size");
    const size: TextSize = savedSize === "small" || savedSize === "large" ? savedSize : "regular";
    const savedSpeak = localStorage.getItem("pathbook-speak") === "on";
    setDyslexic(on);
    setTextSize(size);
    setSpeak(savedSpeak);
    document.documentElement.dataset.reading = on ? "dyslexic" : "book";
    document.documentElement.dataset.size = size;
  }, []);

  useEffect(() => {
    if (!careerId) return;
    try {
      const saved = JSON.parse(localStorage.getItem(`pathbook-courses-${careerId}`) ?? "[]");
      setDoneCourses(Array.isArray(saved) ? saved.filter((item) => typeof item === "string") : []);
    } catch {
      setDoneCourses([]);
    }
  }, [careerId]);

  useEffect(() => {
    const snap = loadLocalProgress();
```

```
  if (snap) {
    for (const [id, list] of Object.entries(snap.doneByCareer)) {
      localStorage.setItem(`pathbook-courses-${id}`, JSON.stringify(list));
    }
    localStorage.setItem("pathbook-pace", snap.pace);
    const campusName = ADVISORS.some(([name]) => name === snap.campus) ? snap.campus : "Kendall";
    setCampus(campusName as (typeof ADVISORS)[number][0]);
    setUsedAccommodations(snap.usedAccommodations);
    setStyle(snap.style as Style);
    setShowExplain(snap.showExplain);
    if (snap.careerId) {
      const picked = careerById(snap.careerId as CareerId);
      setCareerId(picked.id);
      setStepIndex(Math.min(snap.stepIndex, picked.steps.length));
      setDoneCourses(snap.doneByCareer[picked.id] ?? []);
      setScreen("path");
    }
  }
  setReady(true);
}, []);

useEffect(() => {
  if (!ready) return;
  const handle = window.setTimeout(() => {
    const doneByCareer: Record<string, string[]> = {};
    for (const id of ["architect", "mechanical", "chef", "cyber", "it"]) {
      try {
        const raw = JSON.parse(localStorage.getItem(`pathbook-courses-${id}`) ?? "[]");
        if (!Array.isArray(raw)) continue;
        const list = raw.filter((item): item is string => typeof item === "string");
        if (list.length > 0) doneByCareer[id] = list;
      } catch {
        /* ignore a bad saved list */
      }
    }
    if (careerId) doneByCareer[careerId] = doneCourses;
    const snap: PathSnapshot = {
      careerId,
      usedAccommodations,
      style,
      showExplain,
      pace: localStorage.getItem("pathbook-pace") ?? "full",
      campus,
      stepIndex,
      doneByCareer,
    };
    saveLocalProgress(snap);
  }, 500);
  return () => window.clearTimeout(handle);
}, [ready, careerId, usedAccommodations, style, showExplain, campus, stepIndex, doneCourses]);

function replaceCourses(next: string[]) {
  if (!careerId) return;
  setDoneCourses(next);
  localStorage.setItem(`pathbook-courses-${careerId}`, JSON.stringify(next));
}

function openCourses() {
  if (!career) return;
  setStepIndex(steps.length);
  setScreen("path");
  setSettingsOpen(false);
}

function setReading(nextDyslexic: boolean) {
  setDyslexic(nextDyslexic);
  document.documentElement.dataset.reading = nextDyslexic ? "dyslexic" : "book";
  localStorage.setItem("pathbook-reading", nextDyslexic ? "dyslexic" : "book");
}

function setSize(next: TextSize) {
  setTextSize(next);
  document.documentElement.dataset.size = next;
  localStorage.setItem("pathbook-size", next);
}

function setReadAloud(next: boolean) {
  setSpeak(next);
  localStorage.setItem("pathbook-speak", next ? "on" : "off");
  if (!next) stopSpoken();
}

useEffect(() => {
  if (!speak) return;
  let last = "";
  let timer = 0;
  let request = 0;
  function readSelection() {
    window.clearTimeout(timer);
    timer = window.setTimeout(() => {
```

```
      const text = window.getSelection()?.toString().replace(/\s+/g, " ").trim() ?? "";
      if (!text || text === last) return;
      last = text;
      const mine = ++request;
      stopSpoken();
      void speakWithElevenLabs(text, () => mine === request);
    }, 250);
  }
  document.addEventListener("mouseup", readSelection);
  document.addEventListener("touchend", readSelection);
  return () => {
    window.clearTimeout(timer);
    request += 1;
    document.removeEventListener("mouseup", readSelection);
    document.removeEventListener("touchend", readSelection);
    stopSpoken();
  };
}, [speak]);

function startFit(from: "cover" | "careers") {
  setFitFrom(from);
  setFitStep(0);
  setInterest(null);
  setSchool(null);
  setWork(null);
  setScreen("fit");
}

function answerFit(choice: string) {
  if (fitStep === 0) {
    setInterest(choice as Interest);
    setFitStep(1);
    return;
  }
  if (fitStep === 1) {
    setSchool(choice as SchoolPlan);
    setFitStep(2);
    return;
  }
  setWork(choice as WorkTiming);
  setScreen("matches");
}

function pickCareer(id: CareerId) {
  setCareerId(id);
  setCampus(CAMPUS_FIT[id].campus);
  setStepIndex(0);
  setUsedAccommodations(false);
  setStyle("read");
  setScreen("question");
}

function answerAccommodations(yes: boolean) {
  setUsedAccommodations(yes);
  if (yes) {
    setLearnStep(0);
    setScreen("learn");
    return;
  }
  setStyle("read");
  setShowExplain(true);
  setScreen("path");
}

function answerLearn(choice: string) {
  if (learnStep === 0) {
    setPrefer(choice as Exclude<Style, "short">);
    setLearnStep(1);
    return;
  }
  if (learnStep === 1) {
    const nextPrefer = choice === "stop" ? "short" : prefer;
    if (choice === "stop") {
      setStyle("short");
      setShowExplain(false);
      setStepIndex(0);
      setScreen("path");
      return;
    }
    setPrefer(nextPrefer === "short" ? prefer : nextPrefer);
    setLearnStep(2);
    return;
  }
  const explain = choice === "explain";
  const next: Style = explain ? prefer : prefer === "read" ? "short" : prefer;
  setShowExplain(explain && next !== "short");
  setStyle(next);
  setStepIndex(0);
  setScreen("path");
}
```

```
  function cycleStyle() {
    const next = STYLE_ORDER[(STYLE_ORDER.indexOf(style) + 1) % STYLE_ORDER.length];
    setStyle(next);
    if (next === "short") setShowExplain(false);
  }

  function restart() {
    setScreen("cover");
    setCareerId(null);
    setUsedAccommodations(false);
    setStyle("read");
    setShowExplain(true);
    setLearnStep(0);
    setFitStep(0);
    setInterest(null);
    setSchool(null);
    setWork(null);
    setStepIndex(0);
    setSettingsOpen(false);
    window.speechSynthesis?.cancel();
    stopSpoken();
  }

  return (
    <main className={frame.main}>
      <div className={frame.sheet} data-speak-root>
        <div className="absolute top-4 left-4 z-10" data-nospeak>
          <AuthButtons bar />
        </div>
        <button
          type="button"
          aria-label="Settings"
          data-nospeak
          onClick={() => setSettingsOpen((open) => !open)}
          className="absolute top-4 right-4 flex h-10 w-10 items-center justify-center rounded-full border b
order-rule text-ink"
        >
          <Settings size={18} aria-hidden="true" />
        </button>
        {settingsOpen && (
          <div
            data-nospeak
            className="absolute top-16 right-4 left-4 z-10 rounded-2xl border border-rule bg-paper p-4 shado
w-[0_12px_30px_rgba(42,47,147,0.18)]"
          >
            <p className="text-xs tracking-wide text-sage uppercase">Reading</p>
            <p className="mt-3 text-sm font-semibold">Font</p>
            <div className="mt-2 grid grid-cols-2 gap-2">
              <button
                type="button"
                onClick={() => setReading(false)}
                className={`h-11 rounded-full border text-sm font-semibold ${dyslexic ? "border-rule" : "bor
der-neon bg-neon text-paper"}`}
              >
                Book
              </button>
              <button
                type="button"
                onClick={() => setReading(true)}
                className={`h-11 rounded-full border text-sm font-semibold ${dyslexic ? "border-neon bg-neon
 text-paper" : "border-rule"}`}
              >
                Open Sans
              </button>
            </div>
            <p className="mt-4 text-sm font-semibold">Size</p>
            <div className="mt-2 grid grid-cols-3 gap-2">
              {(
                [
                  ["small", "Small"],
                  ["regular", "Regular"],
                  ["large", "Large"],
                ] as const
              ).map(([id, label]) => (
                <button
                  key={id}
                  type="button"
                  onClick={() => setSize(id)}
                  className={`h-11 rounded-full border text-sm font-semibold ${textSize === id ? "border-neo
n bg-neon text-paper" : "border-rule"}`}
                >
                  {label}
                </button>
              ))}
            </div>
            <p className="mt-4 text-sm font-semibold">Read aloud</p>
            <p className="mt-1 text-sm text-muted">Select the words you want heard.</p>
            <div className="mt-2 grid grid-cols-2 gap-2">
              <button
                type="button"
```

```
                onClick={() => setReadAloud(false)}
                className={`h-11 rounded-full border text-sm font-semibold ${speak ? "border-rule" : "border
-neon bg-neon text-paper"}`}
              >
                Off
              </button>
              <button
                type="button"
                onClick={() => setReadAloud(true)}
                className={`h-11 rounded-full border text-sm font-semibold ${speak ? "border-neon bg-neon te
xt-paper" : "border-rule"}`}
              >
                On
              </button>
            </div>
            {career && (
              <div className="mt-4 flex flex-col gap-2 border-t border-rule pt-4">
                <button
                  type="button"
                  onClick={() => {
                    setSettingsOpen(false);
                    setScreen("list");
                  }}
                  className="text-left text-sm font-semibold text-sage"
                >
                  See every step
                </button>
                <button
                  type="button"
                  onClick={() => {
                    setSettingsOpen(false);
                    openCourses();
                  }}
                  className="text-left text-sm font-semibold text-sage"
                >
                  Course timeline
                </button>
              </div>
            )}
          </div>
        )}
        {screen === "cover" && (
          <section className="flex flex-1 flex-col">
            <p className="font-display text-sm tracking-wide text-sage">Miami Dade College</p>
            <h1 className={`mt-10 font-display leading-none font-medium text-ink ${device === "phone" ? "tex
t-5xl" : "text-6xl"}`}>
              Major Spark
            </h1>
            <p className="mt-4 font-display text-xl text-muted italic">notaminor.study</p>
            <div className="mt-8 h-px w-16 bg-rule" />
            <p className={`mt-8 text-lg leading-relaxed text-ink ${device === "phone" ? "max-w-xs" : "max-w-
lg"}`}>
              A clear path from your first class at Miami Dade College to a first job. Pick a career, or ans
wer three questions if you are not sure yet.
            </p>
            <blockquote className={`mt-6 text-lg leading-relaxed text-ink italic font-semibold ${device ===
"phone" ? "max-w-xs" : "max-w-lg"}`}>
              “A spark, no matter how big or small, is the first step to lighting your way to a brighter tom
orrow.”
              <footer className="mt-2 text-sm font-normal not-italic text-muted">Andrea Price</footer>
            </blockquote>
            <div className="mt-auto flex flex-col gap-3 pt-12">
              <button
                type="button"
                onClick={() => setScreen("careers")}
                className="h-12 w-full rounded-full bg-neon text-base font-semibold text-paper shadow-neon"
              >
                I know the career
              </button>
              <button
                type="button"
                onClick={() => startFit("cover")}
                className="h-12 w-full rounded-full border border-neon text-base font-semibold text-sage"
              >
                Help me choose
              </button>
            </div>
          </section>
        )}

        {screen === "careers" && (
          <section className="flex flex-1 flex-col">
            <Header kicker="Choose one" title="Where do you want to go?" />
            <button
              type="button"
              onClick={() => startFit("careers")}
              className="mt-6 h-12 w-full rounded-full border border-neon text-base font-semibold text-sage"
            >
              Not sure yet
            </button>
```

```
            <ul className="mt-6 flex flex-col gap-2">
              {CAREERS.map((item) => (
                <li key={item.id}>
                  <button
                    type="button"
                    onClick={() => pickCareer(item.id)}
                    className="w-full rounded-2xl border border-rule bg-paper px-4 py-4 text-left transition
-colors hover:bg-paper-deep"
                  >
                    <span className="block font-display text-2xl leading-tight">{item.name}</span>
                    <span className="mt-1 block text-sm leading-snug text-muted">{item.line}</span>
                  </button>
                </li>
              ))}
            </ul>
            <TextButton className="mt-6" onClick={() => setScreen("cover")}>
              Back
            </TextButton>
          </section>
        )}

        {screen === "fit" && (
          <section className="flex flex-1 flex-col">
            <Header
              kicker={`Question ${fitStep + 1} of 3`}
              title={
                fitStep === 0
                  ? "What sounds most like you?"
                  : fitStep === 1
                    ? "How long do you want school to last?"
                    : "When do you want to start working?"
              }
            />
            <div className="mt-6 flex flex-col gap-3">
              {(fitStep === 0
                ? [
                    ["make", "Making things I can see"],
                    ["fix", "Figuring out how things work"],
                    ["help", "Helping people use technology"],
                  ]
                : fitStep === 1
                  ? [
                      ["mdc", "Finish at Miami Dade College"],
                      ["transfer", "I can transfer for a longer degree"],
                    ]
                  : [
                      ["during", "While I am still in school"],
                      ["after", "After I finish"],
                    ]
              ).map(([id, label]) => (
                <button
                  key={id}
                  type="button"
                  onClick={() => answerFit(id)}
                  className="rounded-2xl border border-rule px-4 py-4 text-left font-display text-2xl leadin
g-tight"
                >
                  {label}
                </button>
              ))}
            </div>
            <TextButton
              className="mt-auto pt-8"
              onClick={() =>
                fitStep === 0 ? setScreen(fitFrom) : setFitStep((stepNumber) => stepNumber - 1)
              }
            >
              Back
            </TextButton>
          </section>
        )}

        {screen === "matches" && (
          <section className="flex flex-1 flex-col">
            <Header kicker="A place to start" title="This might fit." />
            <p className="mt-4 text-sm leading-relaxed text-muted">
              Not a perfect match. Just a path that lines up with what you said.
            </p>
            <ul className="mt-6 flex flex-col gap-3">
              {matches.map((match) => {
                const item = careerById(match.id);
                return (
                  <li key={match.id}>
                    <button
                      type="button"
                      onClick={() => pickCareer(match.id)}
                      className="w-full rounded-2xl border border-rule px-4 py-4 text-left"
                    >
                      <span className="block font-display text-2xl leading-tight">{item.name}</span>
                      <span className="mt-2 block text-sm leading-relaxed text-muted">{match.reason}</span>
```

```
                </button>
              </li>
            );
          })}
        </ul>
        <TextButton className="mt-6" onClick={() => setScreen("careers")}>
          See all five
        </TextButton>
      </section>
    )}

    {screen === "question" && career && (
      <section className="flex flex-1 flex-col">
        <Header kicker={career.name} title="One question before the steps." />
        <p className="mt-6 text-lg leading-relaxed">
          Have you had, or do you have, accommodations in school?
        </p>
        <div className="mt-8 flex flex-col gap-3">
          <button
            type="button"
            onClick={() => answerAccommodations(true)}
            className="h-12 rounded-full bg-neon text-base font-semibold text-paper shadow-neon"
          >
            Yes
          </button>
          <button
            type="button"
            onClick={() => answerAccommodations(false)}
            className="h-12 rounded-full border border-neon bg-paper text-base font-semibold text-sage"
          >
            No
          </button>
        </div>
        <TextButton className="mt-auto pt-8" onClick={() => setScreen("careers")}>
          Choose a different career
        </TextButton>
      </section>
    )}

    {screen === "learn" && career && (
      <section className="flex flex-1 flex-col">
        <Header
          kicker={`Question ${learnStep + 1} of 3`}
          title={
            learnStep === 0
              ? "When you learn something new, what helps most?"
              : learnStep === 1
                ? "When a page has a lot of words…"
                : "School words, like “internship”…"
          }
        />
        <p className="mt-4 text-sm leading-relaxed text-muted">
          Same path either way. This only changes how the step is written.
        </p>
        <div className="mt-6 flex flex-col gap-3">
          {(learnStep === 0
            ? [
                ["see", "Seeing what finished looks like"],
                ["try", "Trying one small piece myself"],
                ["read", "Reading it in plain words"],
              ]
            : learnStep === 1
              ? [
                  ["stop", "I stop and come back later"],
                  ["stay", "I can stay if the words are clear"],
                ]
              : [
                  ["explain", "Tell me what they mean"],
                  ["skip", "I already know them"],
                ]
          ).map(([id, label]) => (
            <button
              key={id}
              type="button"
              onClick={() => answerLearn(id)}
              className="rounded-2xl border border-rule px-4 py-4 text-left font-display text-2xl leadin
g-tight"
            >
              {label}
            </button>
          ))}
        </div>
        <TextButton
          className="mt-auto pt-8"
          onClick={() =>
            learnStep === 0 ? setScreen("question") : setLearnStep((stepNumber) => stepNumber - 1)
          }
        >
          Back
        </TextButton>
```

```
          </section>
        )}

        {screen === "path" && career && (step || onCourses) && (
          <section className="flex flex-1 flex-col">
            <div className="flex items-start justify-between gap-3">
              <div>
                <p className="text-xs tracking-wide text-sage uppercase">{career.name}</p>
                <p className="mt-1 text-sm text-muted">{career.start}</p>
              </div>
              <button
                type="button"
                onClick={cycleStyle}
                className={`shrink-0 rounded-full border border-rule px-3 py-2 text-xs font-semibold trackin
g-wide text-ink uppercase ${usedAccommodations && !onCourses ? "" : "hidden"}`}
              >
                {STYLE_LABEL[style]}
              </button>
            </div>

            <div className={`flex flex-1 flex-col py-6 ${onCourses ? "" : "justify-center"}`}>
              {step && (
                <>
              <h2 className="font-display text-4xl leading-[1.15] font-medium">{headline}</h2>
              {style === "read" && (
                <p className="mt-6 text-base leading-relaxed">{plainStep(step, true)}</p>
              )}
              {style === "see" && (
                <p className="mt-6 text-base leading-relaxed">{plainStep(step, showExplain)}</p>
              )}
              {style === "try" && (
                <div className="mt-6 space-y-4 text-base leading-relaxed">
                  <p>{step.short}</p>
                  <p>{step.alongside}</p>
                  {showExplain && <p>{step.explain}</p>}
                </div>
              )}
              {style === "short" && <p className="mt-6 text-base leading-relaxed">{asSentence(step.time)}</p
>}
              {style === "read" && (
                <p className="mt-6 text-sm leading-relaxed text-muted">{asSentence(step.alongside)}</p>
              )}
              {style === "see" && (
                <p className="mt-6 text-sm leading-relaxed text-muted">{asSentence(step.alongside)}</p>
              )}
              {usedAccommodations && step.support && step !== ACCESS_STEP && (
                <p className="mt-3 text-sm leading-relaxed text-ink">{step.support}</p>
              )}
              {career && step === career.steps[0] && <RegisterDates />}
              {step === ACCESS_STEP && (
                <div className="mt-6">
                  <label className="block text-sm text-muted">
                    Campus
                    <select
                      value={campus}
                      onChange={(event) => setCampus(event.target.value as (typeof ADVISORS)[number][0])}
                      className="mt-1 h-11 w-full rounded-full border border-rule bg-paper px-4 text-ink"
                    >
                      {ACCESS_OFFICES.map(([name]) => (
                        <option key={name} value={name}>
                          {name}
                        </option>
                      ))}
                    </select>
                  </label>
                  <a
                    href={accessMailto(campus)}
                    className="mt-3 flex h-12 items-center justify-center rounded-full bg-neon text-center t
ext-sm font-semibold text-paper shadow-neon"
                  >
                    {accessMailto(campus).startsWith("mailto:") ? "Email ACCESS" : "See ACCESS offices"}
                  </a>
                </div>
              )}
                </>
              )}
              {onCourses && career && (
                <>
                  <CampusNote careerId={career.id} selected={campus} />
                  <CourseChecklist
                  careerId={career.id}
                  transferring={school === "transfer"}
                  withSupport={usedAccommodations}
                  done={doneCourses}
                  onReplace={replaceCourses}
                  />
                </>
              )}
            </div>
```

```
            <div className="border-t border-rule pt-4">
              <p className="text-center text-sm text-muted">
                Step {stepIndex + 1} of {total}
              </p>
              <div className="mt-4 grid grid-cols-2 gap-3">
                <button
                  type="button"
                  onClick={() => setStepIndex((index) => Math.max(0, index - 1))}
                  disabled={stepIndex === 0}
                  className="h-12 rounded-full border border-rule text-sm font-semibold disabled:opacity-40"
                >
                  Previous
                </button>
                {onCourses && career ? (
                  <a
                    href={advisorMailto(campus, career.name, PLANS[career.id].program)}
                    className="flex h-12 items-center justify-center rounded-full bg-neon text-center text-s
m font-semibold text-paper shadow-neon"
                  >
                    Email an advisor
                  </a>
                ) : (
                  <button
                    type="button"
                    onClick={() => setStepIndex((index) => Math.min(total - 1, index + 1))}
                    className="h-12 rounded-full bg-neon text-sm font-semibold text-paper shadow-neon"
                  >
                    {stepIndex === steps.length - 1 ? "Your classes" : "Next step"}
                  </button>
                )}
              </div>
              {onCourses && career && (
                <>
                <label className="mt-3 block text-sm text-muted">
                  Campus for these classes
                  <select
                    value={campus}
                    onChange={(event) => setCampus(event.target.value as (typeof ADVISORS)[number][0])}
                    className="mt-1 h-11 w-full rounded-full border border-rule bg-paper px-4 text-ink"
                  >
                    {ADVISORS.map(([name]) => (
                      <option key={name} value={name}>
                        {name}
                      </option>
                    ))}
                  </select>
                </label>
                <a
                  href={careerMailto(campus, career.name, PLANS[career.id].program)}
                  className="mt-3 flex h-12 items-center justify-center rounded-full border border-ink text-
center text-sm font-semibold text-ink"
                >
                  Email career help at {campus}
                </a>
                <p className="mt-2 text-sm leading-relaxed text-muted">{careerPerson(campus, career.id)}</p>
                <div className="mt-4 text-center">
                  <TextButton onClick={restart}>Start over</TextButton>
                </div>
                </>
              )}
            </div>
          </section>
        )}

        {screen === "plan" && career && (
          <section className="flex flex-1 flex-col">
            <Header kicker="On-time plan" title={career.name} />
            <p className="mt-3 text-sm leading-relaxed text-ink">{PLANS[career.id].program}</p>
            <p className="mt-2 text-sm leading-relaxed text-muted">{PLANS[career.id].pace}</p>
            <ol className="mt-6 flex flex-col gap-5">
              {PLANS[career.id].terms.map((term) => (
                <li key={term.label}>
                  <p className="text-xs tracking-wide text-sage uppercase">
                    {term.label}
                    <span className="text-rule"> · </span>
                    {term.credits}
                  </p>
                  <ul className="mt-2 flex flex-col gap-2">
                    {term.courses.map((course) => (
                      <li key={course.code + course.name} className="text-sm leading-relaxed">
                        <span className="font-semibold">{course.code}. </span>
                        {course.name}
                      </li>
                    ))}
                  </ul>
                </li>
              ))}
            </ol>
            <h3 className="mt-8 font-display text-2xl">To get into the field</h3>
            <ul className="mt-3 flex flex-col gap-3">
```

```
              {PLANS[career.id].entry.map((item) => (
                <li key={item.name} className="text-sm leading-relaxed">
                  <span className="font-semibold">{item.name}. </span>
                  {item.when}
                </li>
              ))}
            </ul>
            <h3 className="mt-8 font-display text-2xl">During the program</h3>
            <ul className="mt-3 flex flex-col gap-3">
              {PLANS[career.id].credentials.map((item) => (
                <li key={item.name} className="text-sm leading-relaxed">
                  <span className="font-semibold">{item.name}. </span>
                  {item.when}
                </li>
              ))}
            </ul>
            <p className="mt-8 text-xs leading-relaxed text-muted">
              Taken from Miami Dade College’s full-time course sequence guides. A lower math or English plac
ement adds terms. Confirm the current guide with an advisor. This is not a degree audit.
            </p>
            <div className="mt-6">
              <TextButton onClick={() => setScreen("list")}>Back to the steps</TextButton>
            </div>
          </section>
        )}

        {screen === "list" && career && (
          <section className="flex flex-1 flex-col">
            <Header
              kicker={STYLE_LABEL[style]}
              title={career.name}
            />
            <p className="mt-2 text-sm text-muted">{career.start}</p>
            {usedAccommodations && (
              <p className="mt-3 text-sm leading-relaxed text-ink">
                Feasible with support. Extra time or written steps can make a term longer. They do not remov
e the career.
              </p>
            )}
            <ol className="mt-6 flex flex-col gap-5">
              {steps.map((item, index) => (
                <li key={item.action}>
                  <button
                    type="button"
                    onClick={() => {
                      setStepIndex(index);
                      setScreen("path");
                    }}
                    className="w-full text-left"
                  >
                    <span className="font-display text-xl leading-snug">
                      {index + 1}. {item.action}
                    </span>
                    {style === "read" && (
                      <span className="mt-1 block text-sm leading-relaxed text-muted">{item.explain}</span>
                    )}
                    {style === "see" && (
                      <span className="mt-1 block text-sm leading-relaxed text-muted">{item.done}</span>
                    )}
                    {style === "try" && (
                      <span className="mt-1 block text-sm leading-relaxed text-muted">{item.alongside}</span
>
                    )}
                    {style === "short" && (
                      <span className="mt-1 block text-sm leading-relaxed text-muted">{item.short}</span>
                    )}
                    {usedAccommodations && item.support && (
                      <span className="mt-1 block text-sm leading-relaxed text-gold">{item.support}</span>
                    )}
                  </button>
                </li>
              ))}
            </ol>
            <p className="mt-8 text-xs leading-relaxed text-muted">
              A starting map, not an official degree audit. Confirm classes, transfer rules, and timing with
 an MDC advisor.
            </p>
            <div className="mt-6 flex items-center justify-between">
              <TextButton onClick={() => setScreen("path")}>Back to this step</TextButton>
              <TextButton onClick={openCourses}>Course timeline</TextButton>
            </div>
          </section>
        )}
      </div>
    </main>
  );
}

function accessMailto(campus: (typeof ADVISORS)[number][0]) {
  const email = ACCESS_OFFICES.find(([name]) => name === campus)?.[1];
```

```
  if (!email) return "https://www.mdc.edu/access/hours-and-locations/";
  const subject = "Accommodations for my classes";
  const body = `Hello,\n\nI take classes at the ${campus} Campus. I have had accommodations in school. Pleas
e tell me how to get them put in place before the term starts.\n\nThank you.`;
  return `mailto:${email}?subject=${encodeURIComponent(subject)}&body=${encodeURIComponent(body)}`;
}

function careerPerson(campus: (typeof ADVISORS)[number][0], careerId: CareerId) {
  if (campus === "Wolfson" && careerId === "chef") {
    return "MDC Works at Wolfson lists Moses Washington, room 4102-06, 305-237-3876.";
  }
  if (campus === "Wolfson" && (careerId === "cyber" || careerId === "it" || careerId === "mechanical")) {
    return "MDC Works at Wolfson lists Christian Ortega, room 4102-02, 305-237-7864.";
  }
  return CAREER_HELP[campus];
}

function careerMailto(campus: (typeof ADVISORS)[number][0], careerName: string, program: string) {
  const email = ADVISORS.find(([name]) => name === campus)?.[1] ?? "Kadvise@mdc.edu";
  const subject = `Career help for ${careerName}`;
  const body = `Hello,\n\nI am in ${program}. Some of my classes are at the ${campus} Campus. Please help me
 with jobs or internships in this field.\n\nThank you.`;
  return `mailto:${email}?subject=${encodeURIComponent(subject)}&body=${encodeURIComponent(body)}`;
}

function CampusNote({
  careerId,
  selected,
}: {
  careerId: CareerId;
  selected: (typeof ADVISORS)[number][0];
}) {
  const fit = CAMPUS_FIT[careerId];
  return (
    <div className="mb-6">
      <p className="text-base leading-relaxed text-ink">
        The fuller set of classes is more likely at {fit.campus} Campus. {fit.why}
      </p>
      {selected !== fit.campus && (
        <p className="mt-2 text-sm leading-relaxed text-muted">
          You picked {selected}. If a class is offered there, the advisor email and career help both go to {
selected}.
        </p>
      )}
    </div>
  );
}

function advisorMailto(campus: (typeof ADVISORS)[number][0], careerName: string, program: string) {
  const email = ADVISORS.find(([name]) => name === campus)?.[1] ?? "Kadvise@mdc.edu";
  const pace = localStorage.getItem("pathbook-pace");
  const paceLabel = pace === "part" ? "part time" : pace === "three-quarter" ? "3/4 time" : "full time";
  const subject = `Help registering for classes: ${careerName}`;
  const body = `Hello,\n\nI am planning ${program}. I want to study ${paceLabel}. Please help me register fo
r the classes I still need.\n\nThank you.`;
  return `mailto:${email}?subject=${encodeURIComponent(subject)}&body=${encodeURIComponent(body)}`;
}

function CourseChecklist({
  careerId,
  transferring,
  withSupport,
  done,
  onReplace,
}: {
  careerId: CareerId;
  transferring: boolean;
  withSupport: boolean;
  done: string[];
  onReplace: (next: string[]) => void;
}) {
  const plan = PLANS[careerId];
  const [pace, setPace] = useState<StudyPace>("full");

  useEffect(() => {
    const saved = localStorage.getItem("pathbook-pace");
    if (saved === "full" || saved === "three-quarter" || saved === "part") setPace(saved);
  }, []);

  function choosePace(next: StudyPace) {
    setPace(next);
    localStorage.setItem("pathbook-pace", next);
  }

  const terms = termsForPace(plan, pace, transferring);
  const keys = terms.flatMap((term) => term.courses.map((course) => courseKey(course.code, course.name)));
  const onPlan = new Set(keys.map((key) => key.split("|")[0]));
  const finished = keys.filter((key) => done.includes(key)).length;
  const percent = keys.length === 0 ? 0 : Math.round((finished / keys.length) * 100);
  const later = afterDegree(careerId);
```

```
  function withoutDependents(next: string[]) {
    let current = next;
    for (let guard = 0; guard < 12; guard += 1) {
      const drop = new Set(
        current.filter((key) => {
          const course = courseByKey(terms, key);
          return course != null && blockingPrereqs(course, current, onPlan).length > 0;
        }),
      );
      if (drop.size === 0) return current;
      current = current.filter((key) => !drop.has(key));
    }
    return current;
  }

  function toggle(course: { code: string; name: string }) {
    const key = courseKey(course.code, course.name);
    if (done.includes(key)) {
      onReplace(withoutDependents(done.filter((item) => item !== key)));
      return;
    }
    if (blockingPrereqs(course, done, onPlan).length > 0) return;
    onReplace([...done, key]);
  }

  return (
    <div>
      <h2 className="font-display text-4xl leading-[1.15] font-medium">Your classes</h2>
      <p className="mt-3 text-sm leading-relaxed text-muted">{plan.program}</p>
      <div className="mt-4 grid grid-cols-3 gap-2">
        {(
          [
            ["full", "Full time"],
            ["three-quarter", "3/4 time"],
            ["part", "Part time"],
          ] as const
        ).map(([id, label]) => (
          <button
            key={id}
            type="button"
            onClick={() => choosePace(id)}
            className={`h-11 rounded-full border text-sm font-semibold ${pace === id ? "border-neon bg-neon
text-paper" : "border-rule"}`}
          >
            {label}
          </button>
        ))}
      </div>
      <p className="mt-4 text-sm leading-relaxed">
        {graduationLine(terms.length)}
        {pace === "full"
          ? " This matches the full-time sequence guide."
          : pace === "three-quarter"
            ? " Same classes, about 9 credits a term."
            : " Same classes, about 6 credits a term."}{" "}
        Skipping summers, or starting in a lower math or English class, makes it longer.
      </p>
      {later && <p className="mt-2 text-sm leading-relaxed">{later}</p>}
      <h3 className="mt-6 font-display text-2xl">What it takes</h3>
      <p className="mt-2 text-sm leading-relaxed">
        Finish this program, then meet the field requirements below. The job asks for the same degree and th
e same exams either way.
      </p>
      <ul className="mt-3 flex flex-col gap-2">
        {plan.entry.map((item) => (
          <li key={item.name} className="text-sm leading-relaxed">
            <span className="font-semibold">{item.name}. </span>
            {item.when}
          </li>
        ))}
      </ul>
      {plan.credentials.length > 0 && (
        <ul className="mt-3 flex flex-col gap-2">
          {plan.credentials.map((item) => (
            <li key={item.name} className="text-sm leading-relaxed">
              <span className="font-semibold">{item.name}. </span>
              {item.when}
            </li>
          ))}
        </ul>
      )}
      {withSupport && (
        <>
          <p className="mt-4 text-sm leading-relaxed">
            Accommodations can add time. They do not remove a class, a certificate, or the job. Check in wit
h the advisor who sets up your accommodations before the term starts. Ask which of these classes and exams c
an include extra time, a quieter room, or written directions.
          </p>
          <h3 className="mt-6 font-display text-2xl">What can be hard</h3>
          <p className="mt-2 text-sm leading-relaxed">
```

```
            Some neurodivergent students run into these in this field. They are common, not a verdict. The d
egree and the job stay the same.
          </p>
          <ul className="mt-3 flex flex-col gap-2">
            {plan.struggles.map((item) => (
              <li key={item} className="text-sm leading-relaxed">
                {item}
              </li>
            ))}
          </ul>
        </>
      )}
      {transferring && (
        <p className="mt-4 text-sm leading-relaxed">
          You said you may finish at another school. Ask that school which of these credits they accept befo
re you count on them. Finishing your A.A. or A.S. at Miami Dade College first usually makes the transfer smo
other.
        </p>
      )}
      <p className="mt-4 text-sm leading-relaxed">{percent}% of the way through.</p>
      <div
        className="mt-3 h-2 overflow-hidden rounded-full bg-rule"
        role="progressbar"
        aria-valuenow={percent}
        aria-valuemin={0}
        aria-valuemax={100}
        aria-label="Percent of classes finished"
      >
        <div className="h-full rounded-full bg-neon" style={{ width: `${percent}%` }} />
      </div>
      <p className="mt-4 text-sm text-muted">
        Classes stay in the order to take them. A note under a class names what has to come first, or what y
ou take with it.
      </p>
      <ol className="mt-6 flex flex-col gap-5">
        {terms.map((term) => (
          <li key={term.label}>
            <p className="text-xs tracking-wide text-sage uppercase">
              {term.label}
              <span className="text-rule"> · </span>
              {term.credits}
            </p>
            <ul className="mt-2 flex flex-col gap-2">
              {term.courses.map((course) => {
                const key = courseKey(course.code, course.name);
                const checked = done.includes(key);
                const blocked = !checked && blockingPrereqs(course, done, onPlan).length > 0;
                return (
                  <li key={key}>
                    <button
                      type="button"
                      onClick={() => toggle(course)}
                      disabled={blocked}
                      aria-pressed={checked}
                      className={`flex w-full items-start gap-3 text-left ${blocked ? "cursor-not-allowed op
acity-40" : ""}`}
                    >
                      <span
                        className={`mt-0.5 flex h-5 w-5 shrink-0 items-center justify-center rounded border-
2 ${checked ? "border-neon bg-neon text-paper" : "border-ink bg-paper"}`}
                        aria-hidden="true"
                      >
                        {checked && <Check size={12} strokeWidth={3} />}
                      </span>
                      <span className="text-sm leading-relaxed">
                        <span className="font-semibold">{course.code}. </span>
                        {course.name}
                        {courseNeeds(course) && (
                          <span className="mt-1 block text-muted">{courseNeeds(course)}</span>
                        )}
                      </span>
                    </button>
                  </li>
                );
              })}
            </ul>
          </li>
        ))}
      </ol>
    </div>
  );
}

function courseByKey(terms: { courses: { code: string; name: string }[] }[], key: string) {
  for (const term of terms) {
    for (const course of term.courses) {
      if (courseKey(course.code, course.name) === key) return course;
    }
  }
  return null;
```

```
}

function courseKey(code: string, name: string) {
  return `${code}|${name}`;
}

function Header({ kicker, title }: { kicker: string; title: string }) {
  return (
    <header>
      <p className="text-xs tracking-wide text-sage uppercase">{kicker}</p>
      <h2 className="mt-2 font-display text-4xl leading-none font-medium">{title}</h2>
    </header>
  );
}

function TextButton({
  children,
  onClick,
  className = "",
}: {
  children: string;
  onClick: () => void;
  className?: string;
}) {
  return (
    <button
      type="button"
      onClick={onClick}
      className={`text-sm font-semibold text-sage underline decoration-rule underline-offset-4 ${className}`
}
    >
      {children}
    </button>
  );
}

const TERM_DATES = [
  {
    name: "Spring 2027",
    opens: "2026-04-20",
    showUntil: "2027-04-09",
    sessions: [
      { id: "16", label: "16-week", starts: "2027-01-04", ends: "2027-04-30", refund: "2027-01-08", withdraw
: "2027-03-12" },
      { id: "14", label: "14-week", starts: "2027-01-19", ends: "", refund: "2027-01-25", withdraw: "2027-03
-18" },
      { id: "12", label: "12-week", starts: "2027-02-01", ends: "", refund: "2027-02-04", withdraw: "2027-03
-31" },
      { id: "8", label: "First 8-week", starts: "2027-01-04", ends: "", refund: "2027-01-07", withdraw: "202
7-02-05" },
      { id: "8b", label: "Second 8-week", starts: "2027-03-01", ends: "", refund: "2027-03-05", withdraw: "2
027-04-09" },
      { id: "end", label: "Weekend", starts: "", ends: "", refund: "2027-01-12", withdraw: "2027-03-16" },
    ],
  },
  {
    name: "Summer 2027",
    opens: "2026-11-02",
    showUntil: "2027-07-15",
    sessions: [
      { id: "12", label: "12-week", starts: "2027-05-10", ends: "2027-07-30", refund: "2027-05-13", withdraw
: "2027-06-29" },
      { id: "10", label: "10-week", starts: "2027-05-24", ends: "2027-07-30", refund: "2027-05-27", withdraw
: "" },
      { id: "8", label: "First 8-week", starts: "2027-05-10", ends: "2027-07-02", refund: "2027-05-13", with
draw: "2027-06-04" },
      { id: "8b", label: "Second 8-week", starts: "2027-06-07", ends: "2027-07-30", refund: "2027-06-10", wi
thdraw: "2027-07-09" },
      { id: "6", label: "First 6-week", starts: "2027-05-10", ends: "2027-06-18", refund: "2027-05-12", with
draw: "2027-06-03" },
      { id: "6b", label: "Second 6-week", starts: "2027-06-21", ends: "2027-07-30", refund: "2027-06-23", wi
thdraw: "2027-07-15" },
      { id: "end", label: "Weekend", starts: "2027-05-14", ends: "", refund: "2027-05-18", withdraw: "2027-0
6-30" },
    ],
  },
  {
    name: "Fall 2027",
    opens: "2027-04-19",
    showUntil: "2027-08-31",
    sessions: [],
  },
];

function dateOnly(iso: string) {
  const [year, month, day] = iso.split("-").map(Number);
  return new Date(year, month - 1, day);
}

function endOf(iso: string) {
```

```
  const date = dateOnly(iso);
  date.setHours(23, 59, 59, 999);
  return date;
}

function writtenDate(iso: string) {
  return dateOnly(iso).toLocaleDateString("en-US", {
    month: "long",
    day: "numeric",
    year: "numeric",
  });
}

function RegisterDates() {
  const now = new Date();
  const choices = TERM_DATES.filter((term) => now <= endOf(term.showUntil)).flatMap((term) =>
    term.sessions
      .filter((session) => !session.withdraw || now <= endOf(session.withdraw))
      .map((session) => ({
        key: `${term.name}-${session.id}`,
        label: `${term.name}, ${session.label}`,
        session,
      })),
  );
  const [picked, setPicked] = useState(choices[0]?.key ?? "");
  const selected = choices.find((choice) => choice.key === picked) ?? choices[0];
  if (!selected) {
    return <p className="mt-6 text-base">The dates saved here have passed.</p>;
  }
  const { session } = selected;
  return (
    <div className="mt-6 text-base leading-relaxed">
      <label className="block text-sm text-muted">
        Class length
        <select
          value={selected.key}
          onChange={(event) => setPicked(event.target.value)}
          className="mt-1 h-11 w-full rounded-full border border-rule bg-paper px-4 text-base text-ink"
        >
          {choices.map((choice) => (
            <option key={choice.key} value={choice.key}>
              {choice.label}
            </option>
          ))}
        </select>
      </label>
      <p className="mt-3">
        {session.starts ? `Starts ${writtenDate(session.starts)}. ` : ""}
        {session.ends ? `Ends ${writtenDate(session.ends)}.` : "The end date is not posted for this length."
}
      </p>
      <p>
        {now <= endOf(session.refund)
          ? `Full refund by ${writtenDate(session.refund)}.`
          : "The full-refund date has passed."}
      </p>
      <p>
        {session.withdraw ? `A W, instead of a grade, by ${writtenDate(session.withdraw)}.` : "The W date is
 not posted yet."}
      </p>
    </div>
  );
}

let spokenAudio: HTMLAudioElement | null = null;
let spokenUrl: string | null = null;

function stopSpoken() {
  window.speechSynthesis?.cancel();
  spokenAudio?.pause();
  spokenAudio = null;
  if (spokenUrl) URL.revokeObjectURL(spokenUrl);
  spokenUrl = null;
}

async function speakWithElevenLabs(text: string, stillCurrent: () => boolean) {
  try {
    const response = await fetch("/api/speak", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ text }),
    });
    if (!stillCurrent()) return;
    if (!response.ok) throw new Error("fallback");
    const blob = await response.blob();
    if (!stillCurrent()) return;
    spokenUrl = URL.createObjectURL(blob);
    spokenAudio = new Audio(spokenUrl);
    await spokenAudio.play();
  } catch {
```

```
    if (!stillCurrent() || !window.speechSynthesis) return;
    const utter = new SpeechSynthesisUtterance(text);
    utter.rate = 0.95;
    window.speechSynthesis.speak(utter);
  }
}

function plainStep(step: { explain: string; done: string; time: string }, withExplain: boolean) {
  const done = step.done.trim().replace(/\.+$/, "");
  const doneLower = done.charAt(0).toLowerCase() + done.slice(1);
  const time = step.time.trim().replace(/\.+$/, "");
  const timeLine = time.includes(".") ? `${time}.` : `This takes ${time.charAt(0).toLowerCase()}${time.slice
(1)}.`;
  const lead = withExplain ? `${step.explain} ` : "";
  return `${lead}You are finished with this step when ${doneLower}. ${timeLine}`;
}

function asSentence(value: string) {
  const text = value.trim();
  return /[.!?]$/.test(text) ? text : `${text}.`;
}
```

src/routes/auth/$.ts

```
import { auth0Handlers, auth0Server } from "@auth0/auth0-tanstack-start-react/server";
import { createFileRoute } from "@tanstack/react-router";
import { auth0ServerOptions } from "@/lib/auth0-config";

async function handle(request: Request) {
  const handlers = auth0Handlers(auth0Server(auth0ServerOptions()));
  return request.method.toUpperCase() === "POST" ? handlers.POST() : handlers.GET();
}

export const Route = createFileRoute("/auth/$")({
  server: {
    handlers: {
      GET: ({ request }) => handle(request),
      POST: ({ request }) => handle(request),
    },
  },
});
```

src/routes/index.tsx

```
import { useAuth0, useLogin } from "@auth0/auth0-tanstack-start-react/client";
import { createFileRoute, useNavigate } from "@tanstack/react-router";
import { useEffect } from "react";
import { isSchoolEmail } from "@/lib/school-email";

export const Route = createFileRoute("/")({
  validateSearch: (search: Record<string, unknown>) => ({
    denied: search.denied === "school" ? ("school" as const) : undefined,
  }),
  loader: async () => {
    return { safeBuild: true };
  },
  component: Landing,
});

function Landing() {
  const login = useLogin();
  const navigate = useNavigate();
  const { denied } = Route.useSearch();
  const { isAuthenticated, isLoading, user } = useAuth0();
  const schoolAccount = isAuthenticated && isSchoolEmail(user?.email);

  useEffect(() => {
    if (!isLoading && schoolAccount) {
      void navigate({ to: "/app" });
    }
  }, [isLoading, navigate, schoolAccount]);

  return (
    <main className="relative isolate min-h-dvh">
      <img
        src="/landing-book.jpg"
        alt=""
        className="absolute inset-0 -z-10 h-full w-full object-cover"
      />

      <h1 className="absolute top-[9%] right-0 left-0 z-10 text-center font-display text-5xl font-normal tex
t-white uppercase sm:text-7xl">
        Major Spark
      </h1>

      <div className="absolute right-0 bottom-[8%] left-0 z-10 flex flex-col items-center gap-3 px-6">
        {denied ? (
          <p className="max-w-sm text-center font-sans text-sm text-white">Use an @mymdc.net email to enter.
</p>
        ) : null}
        <button
          type="button"
          onClick={() => {
            if (typeof window === "undefined") return;
            login("/app", denied ? { authorizationParams: { prompt: "login" } } : undefined);
          }}
          className="bg-transparent pl-[0.55em] font-sans text-2xl font-medium tracking-[0.55em] text-white
uppercase"
          style={{
            textShadow: "0 0 6px #67e8f9, 0 0 14px #38bdf8, 0 0 28px #2563eb",
          }}
        >
          Enter
        </button>
      </div>
    </main>
  );
}

// Cache Busting Deploy Flag V2
// CACHE_FORCE_CLEAR_HASH_999
```

src/routes/login.tsx

```
import { createFileRoute } from "@tanstack/react-router";
import { AuthButtons } from "@/components/auth-buttons";

export const Route = createFileRoute("/login")({
  component: LoginPage,
});

function LoginPage() {
  return (
    <main className="grid min-h-screen place-items-center bg-indigo-950 p-6">
      <section className="flex w-full max-w-md flex-col rounded-[28px] bg-paper px-8 py-12">
        <p className="font-display text-sm tracking-wide text-sage">Miami Dade College</p>
        <h1 className="mt-8 font-display text-5xl leading-none font-medium text-ink">Major Spark</h1>
        <p className="mt-6 text-lg leading-relaxed text-ink">
          Sign in to keep this path for you. Everyone else gets their own.
        </p>
        <div className="mt-10">
          <AuthButtons />
        </div>
      </section>
    </main>
  );
}
```

src/start.ts

```
import { auth0Middleware } from "@auth0/auth0-tanstack-start-react/server/middleware";
import { createStart } from "@tanstack/react-start";
import { auth0ServerOptions } from "@/lib/auth0-config";

const auth0Options = auth0ServerOptions();
const auth0Ready = Boolean(
  auth0Options.domain && auth0Options.clientId && auth0Options.clientSecret && auth0Options.secret,
);

export const startInstance = createStart(() => ({
  requestMiddleware: auth0Ready ? [auth0Middleware(auth0Options)] : [],
}));
```

src/styles.css

```
@import "tailwindcss";

@theme {
  --color-desk: #2a2f93;
  --color-paper: #f6f1e6;
  --color-paper-deep: #efe6d6;
  --color-ink: #3c3328;
  --color-muted: #6f655b;
  --color-rule: #d5dced;
  --color-sage: #2447d6;
  --color-sage-soft: #e4e9ff;
  --color-neon: #3d6bff;
  --color-gold: #8a6a3b;
  --shadow-neon: 0 0 18px rgb(61 107 255 / 0.7);
  --font-display: "Source Serif 4", "Iowan Old Style", "Palatino Linotype", serif;
  --font-sans: "Source Sans 3", "Segoe UI", sans-serif;
  --radius-page: 1.25rem;
}

html[data-reading="dyslexic"] body,
html[data-reading="dyslexic"] .font-display {
  font-family: "Open Sans", sans-serif;
  letter-spacing: 0.02em;
}

html[data-reading="dyslexic"] .font-display {
  line-height: 1.25;
}

html[data-size="small"] {
  font-size: 15px;
}

html[data-size="regular"] {
  font-size: 16px;
}

html[data-size="large"] {
  font-size: 18px;
}

@layer base {
  button:not(:disabled),
  [role="button"]:not(:disabled) {
    cursor: pointer;
  }

  html,
  body {
    background: var(--color-desk);
    color: var(--color-ink);
    font-family: var(--font-sans);
    min-height: 100%;
  }

  body {
    margin: 0;
  }

  ::selection {
    background: #e4d3b0;
    color: var(--color-ink);
  }
}

@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation: none !important;
    transition: none !important;
  }
}
```